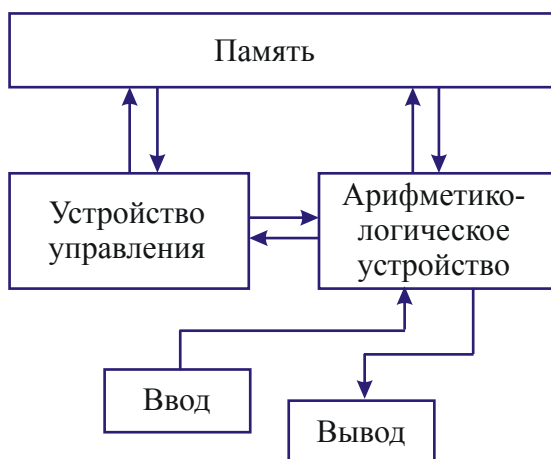


Д.Н. Беклемишев, А.Н. Орлов, А.Л. Переверзев,
М.Г. Попов, А.В. Горячев, А.И. Кононова

Микропроцессорные средства и системы

Курс лекций

Под редакцией
доктора технических наук, профессора **Ю.В. Савченко**



Москва 2013

Министерство образования и науки Российской Федерации
Национальный исследовательский университет «МИЭТ»

**Д. Н. Беклемишев, А. Н. Орлов, А. Л. Переверзев,
М. Г. Попов, А. В. Горячев, А. И. Кононова**

Микропроцессорные средства и системы

Курс лекций

Под редакцией
доктора технических наук, профессора **Ю. В. Савченко**

Утверждено редакционно-издательским советом университета

Москва 2013

УДК 681.32 В75
М59

Рецензенты: докт. техн. наук, проф. *А. Н. Соловьев*;
докт. техн. наук, проф. *С. В. Гаврилов*

**Беклемишев Д. Н., Орлов А. Н., Переверзев А. Л.,
Попов М. Г., Горячев А. В., Кононова А. И.**

М59 Микропроцессорные средства и системы: курс лекций /
Под ред. Ю. В. Савченко. — М.: МИЭТ, 2013. — 288 с.: ил.
ISBN 978-5-7256-0723-9

Представлены материалы курса «Микропроцессорные средства и системы», целью которого является изучение особенностей построения и возможностей применения современной микропроцессорной техники, включающей как простейшие устройства с микропрограммным управлением, так и сложные многопроцессорные параллельные вычислительные системы.

Для студентов факультета МПИТК МИЭТ, обучающихся по специальностям 230100 «Информатика и вычислительная техника», 210100 «Биотехнические системы и технологии», 211000 «Конструирование и технология электронных средств», 231000 «Программная инженерия», 231300 «Прикладная математика». Может быть полезен студентам других специальностей факультета, а также колледжа электроники и информатики.

ISBN 978-5-7256-0723-9

© **МИЭТ, 2013**

Список используемых аббревиатур

DSP	— digital signal processor
FLOPS	— floating point operations per second
MAC	— multiplication with accumulation
MIPS	— million instructions per second
MPA	— message passing architecture
MPI	— message passing interface
PC	— program counter
RAID	— redundant array of independent disks
SMP	— symmetric multiprocessing
АЛУ	— арифметико-логическое устройство
АЦП	— аналого-цифровой преобразователь
БИС	— большая интегральная схема
ВЗУ	— внешнее запоминающее устройство
ЗПДП	— запрос прямого доступа к памяти
ЗУ	— запоминающее устройство
ИП	— интегрированный процессор
КВВ	— контроллер ввода/вывода
КС	— комбинационная схема
КУП	— команда условного перехода
КШ	— контроллер шины
МК	— микроконтроллер
МП	— микропроцессор
МПК	— микропроцессорный комплект
МПУ	— микропроцессорное устройство
ОЗУ	— оперативное запоминающее устройство
ОП	— основная память
ОУ	— операционное устройство
ПА	— периферийный адаптер
ПБПП	— программируемый блок приоритетных прерываний
ПВС	— параллельная вычислительная система
ПДП	— прямой доступ к памяти
ПЗУ	— постоянное запоминающее устройство

ПЛИС	— программируемая логическая интегральная схема
ПЛИМ	— программируемая логическая матрица
ПУ	— периферийное устройство
ПЦОС	— процессор цифровой обработки сигналов
ПЭ	— процессорный элемент
РА	— регистр адреса
РАЛУ	— регистровое арифметико-логическое устройство
РД	— регистр данных
РЗУ	— регистровое запоминающее устройство
РОН	— регистры общего назначения
РПДП	— разрешение прямого доступа к памяти
СОЗУ	— сверхоперативное запоминающее устройство
СУП	— схема ускоренного переноса
УМУ	— устройство микропрограммного управления
УУ	— устройство управления
УФА	— устройство формирования адреса
ЦАП	— цифро-аналоговый преобразователь
ШИМ	— широтно-импульсная модуляция

Предисловие

В настоящем издании представлены материалы курса «Микропроцессорные средства и системы», целью которого является изучение особенностей построения и возможностей применения современной микропроцессорной техники, включающей как простейшие устройства с микропрограммным управлением, так и сложные многопроцессорные параллельные вычислительные системы.

Для достижения указанной цели проанализированы общие тенденции развития и построения микропроцессорной техники, а также архитектуры и структуры микропроцессоров различного назначения, в частности, рассмотрены особенности организации операционных устройств, устройств управления, систем памяти, прерываний, режимов выполнения команд, обмена информацией и т. д.

Курс лекций разбит на пять разделов и имеет иерархическую структуру: каждый последующий раздел базируется на материалах предыдущих.

Раздел 1 состоит из пяти лекций и посвящен особенностям построения операционных устройств микропроцессоров. Первая лекция имеет общий характер и содержит ключевые определения и классификации, описание закономерностей развития микропроцессорной техники. Далее рассмотрены обобщенная структура микропроцессора и процесс синтеза арифметико-логических устройств (АЛУ). Проанализированы способы наращивания разрядности обрабатываемых слов и организации цепей арифметического переноса, приведены примеры использования АЛУ в качестве реконфигурируемого преобразователя информации. Показано функционирование АЛУ с разрядномодульной и однокристалльной организацией.

Раздел 2 включает шесть лекций и посвящен особенностям построения устройств управления, структурным методам повышения производительности, а также режимам функционирования микропроцессорной системы. Рассмотрены общие структуры устройств управления на основе жесткой логики и с применением микропрограммирования, способы формирования управляющих сигналов и адресации

микрокоманд, а также способы адресации операндов. Проанализированы конвейерный принцип выполнения команд, возникающие при этом конфликтные ситуации и методы борьбы с ними. Рассмотрены основные режимы функционирования микропроцессорной системы: выполнение основной программы, вызов подпрограмм, обработка прерываний и исключений.

Раздел 3 состоит из пяти лекций и посвящен принципам организации систем памяти и обмена информацией в микропроцессорных системах. Приведены классификации систем памяти, рассмотрены принципы организации кэш-памяти, режима прямого доступа и виртуальной памяти. Проанализированы способы реализации обмена информацией микропроцессора с внешними устройствами.

Раздел 4 включает четыре лекции и посвящен архитектурам и структурам микропроцессоров и систем на их основе. Рассмотрены особенности архитектур с полным и сокращенным набором команд, суперскалярной, фон Неймана и гарвардской. Особенности архитектурных решений показаны на примерах структур современных микроконтроллеров, процессоров общего назначения и цифровой обработки сигналов.

Раздел 5 состоит из шести лекций и посвящен архитектурам и структурам параллельных вычислительных систем. Рассмотрены их назначение, область применения и классификация. Проанализированы принципы построения и функционирования векторно-конвейерных, матричных, кластерных и систолических систем, а также систем с реконфигурируемой структурой и управляемых потоком данных.

Раздел 1. Микропроцессор: ключевые понятия, классификация, структура, операционные устройства

Лекция 1.1. Микропроцессор: основные определения, классификация, закономерности развития, области применения

Микропроцессор (МП) — программно-управляемое устройство, как правило, выполненное на одной большой интегральной схеме (БИС), осуществляющее цифровую обработку информации и управление этим процессом.

Долгое время процессоры создавались на основе отдельных микросхем малой и средней интеграции, содержащих от нескольких единиц до нескольких сотен транзисторов. Совокупность таких микросхем, совместимых по электрическим, информационным и конструктивным параметрам, называется *микропроцессорным комплектом* (МПК). Примером служит отечественный МПК серии КР580, большинство микросхем которого являются аналогами микросхем серии MCS-85 фирмы Intel. В состав типового МПК входят БИС микропроцессора, оперативных и постоянных запоминающих устройств (ОЗУ и ПЗУ), интерфейсов и контроллеров внешних устройств, служебных устройств (тактовых генераторов, таймеров, регистров, шинных формирователей, контроллеров и арбитров магистрали).

Микропроцессорное устройство (МПУ) — функционально и конструктивно законченное изделие, состоящее из нескольких микросхем, в состав которого входит микропроцессор. МПУ предназначено для выполнения определенного набора функций: получения, обработки, передачи, преобразования информации и управления.

Микроконтроллер (МК) — устройство, конструктивно выполненное на базе одной БИС и содержащее основные функциональные части МПК. Изначально такие устройства называли *однокристальными микроЭВМ*. Первый патент на однокристальную микроЭВМ

был выдан в 1971 году инженерам американской компании Texas Instruments М. Кочрену и Г. Буну. Считается, что именно они впервые предложили разместить на одном кристалле не только процессорное ядро, но и элементы памяти с устройствами ввода/вывода. С появлением однокристальных микроЭВМ связывают начало эры компьютерной автоматизации в области управления. По-видимому, это обстоятельство и определило происхождение термина «микроконтроллер» (от англ. *control* — управление).

В настоящее время существует огромное количество типов и семейств микроконтроллеров, отличающихся архитектурой процессорного модуля, размером и типом встроенной памяти, набором периферийных устройств, типом корпуса и т. д. Это обусловлено тем, что для разных приложений оптимальное соотношение размеров, стоимости, гибкости и производительности может различаться очень сильно. При проектировании микроконтроллеров разработчикам приходится соблюдать баланс между размерами и стоимостью, с одной стороны, и гибкостью и производительностью, с другой.

Интегрированный процессор (ИП) — устройство, занимающее промежуточное положение между микропроцессором и микроконтроллером. Функциональные возможности ИП отражают специфику решаемых с его помощью задач. Примером служит семейство сетевых интегрированных процессоров IXA фирмы Intel.

В целом микропроцессор является универсальным средством цифровой обработки информации, однако отдельные области применения требуют реализации определенных специфических вариантов его структуры и архитектуры.

Архитектурой процессора называется комплекс его аппаратных и программных средств, предоставляемых пользователю, а именно: набор доступных программных регистров и исполнительных (операционных) устройств, система команд и способов адресации, объем и структура адресуемой памяти, виды и способы обработки прерываний.

При описании архитектуры и функционирования микропроцессора обычно используется его представление в виде совокупности программно-доступных регистров, образующих *регистровую*, или *программную модель*. В этих регистрах содержатся обрабатываемые данные (операнды) и управляющая информация. Соответственно, в регистровую модель входит группа *регистров общего назначения* (РОН), предназначенных для хранения операндов, и группа *служеб-*

ных регистров, обеспечивающих управление выполнением программы и режимом работы процессора, организацию обращения к памяти. Регистры общего назначения образуют *регистровое запоминающее устройство* (РЗУ) — внутреннюю регистровую память процессора. Состав служебных регистров определяется конкретной архитектурой микропроцессора, обычно в него входят:

- программный счетчик (Program Counter — PC);
- регистр состояния (State Register — SR);
- регистры управления режимом работы процессора (Control Register — CR).

Функционирование процессора представляется в виде реализации регистровых пересылок — процедур изменения состояния регистров путем чтения и записи их содержимого. Посредством таких пересылок обеспечиваются адресация и выбор команд и операндов, хранение и пересылка результатов, изменение последовательности команд и режимов функционирования процессора в соответствии с поступлением нового содержимого в служебные регистры, а также другие процедуры, реализующие процесс обработки информации согласно заданным условиям.

В общем случае функционирование процессора можно представить как последовательность действий, приведенную на рис. 1.1.1.



Рис. 1.1.1. Общий алгоритм функционирования процессора

На первом этапе очередная команда извлекается из ячейки памяти, адрес которой содержится в программном счетчике. В случае

линейного выполнения программы команды располагаются последовательно в соседних ячейках памяти. Поэтому для определения адреса следующей команды, как правило, необходимо просто увеличить значение счетчика на единицу, что и определило название данного служебного регистра как счетчика команд или программного счетчика. Для изменения порядка следования команд в систему команд вводятся специальные команды переходов, которые вместо инкремента содержимого программного счетчика загружают в него адрес команды, на которую осуществляется переход.

На этапе декодирования команды определяются тип выполняемой операции, адреса операндов и приемника результата, т.е. осуществляется подготовка управляющих воздействий для всех функциональных устройств, участвующих в выполнении данной команды.

После выполнения команды результат заносится в регистр или ячейку памяти.

Выполнение практически любой команды оказывает влияние на содержимое регистра состояния, которое характеризует текущее состояние процессора. Ниже приведены типовые флаги (признаки), входящие в состав регистра состояния микропроцессора практически любой архитектуры:

- N — признак отрицательного результата; при $N = 0$ результат положительный, при $N = 1$ — отрицательный;
- C — признак переноса; если при выполнении команды образовался перенос из старшего разряда, то $C = 1$;
- Z — признак нулевого результата; при $Z = 1$ результат равен нулю.

Обобщая вышеизложенное, можно сказать, что *архитектура микропроцессора* — это совокупность характеристик, описывающих его устройство и функционирование на самом высоком уровне. Одну и ту же архитектуру можно реализовать по-разному, с помощью разных структурных решений, т.е. на основе различных функциональных модулей и связей между ними. Таким образом, *структура микропроцессора* определяет состав и взаимодействие основных функциональных устройств, размещенных на его кристалле. В структуру, как правило, входят:

- центральный процессор или процессорное ядро, состоящее из устройства управления (УУ) и операционных устройств (ОУ);
- внутренняя память (РЗУ, кэш-память, блоки ОЗУ и ПЗУ);

- интерфейсный блок, обеспечивающий выход на системную шину и обмен данными с внешними устройствами через параллельные или последовательные порты ввода/вывода;
- периферийные устройства (таймерные модули, аналого-цифровые преобразователи, специализированные контроллеры);
- различные вспомогательные схемы (генераторы тактовых импульсов, схемы отладки и тестирования, сторожевой таймер и т. д.).

Рассмотрим классификацию МП по функциональному признаку (рис. 1.1.2). Вся совокупность МП можно разделить на два больших класса: микропроцессоры общего назначения и специализированные микропроцессоры.

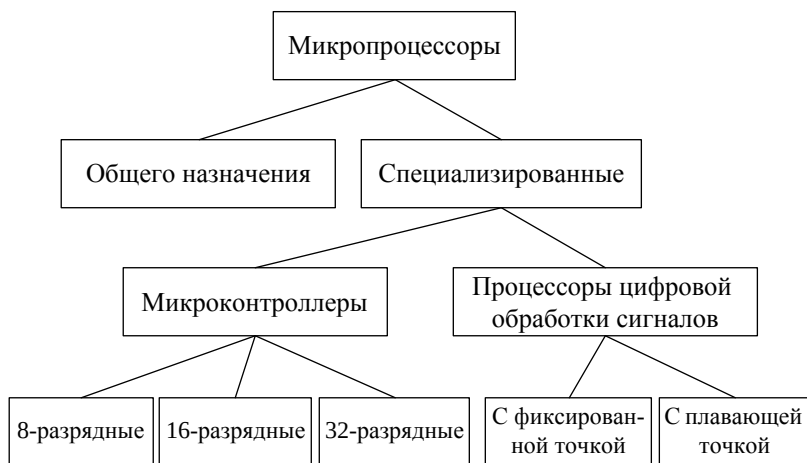


Рис. 1.1.2. Классификация микропроцессоров по функциональному признаку

Микропроцессоры общего назначения служат для решения широкого круга задач обработки разнообразной информации. Их основной областью применения являются персональные компьютеры, рабочие станции, серверы и другие цифровые системы массового использования. К этому классу относят процессоры семейства Pentium компании Intel, K7 компании AMD (Advanced MicroDevices), PowerPC компании Motorola и IBM, SPARC компании Sun Microsystems и ряд других. Расширение области применения таких микропроцессоров достигается главным образом путем роста производительности,

благодаря чему увеличивается круг задач, который можно решать с их использованием.

Среди *специализированных микропроцессоров* наиболее широкое распространение получили микроконтроллеры, предназначенные для выполнения функций управления различными объектами, и процессоры цифровой обработки сигналов (Digital Signal Processor — DSP), ориентированные на реализацию процедур, обеспечивающих преобразование аналоговых сигналов, представленных в цифровой форме.

Микроконтроллеры являются специализированными микропроцессорами, которые ориентированы на реализацию устройств управления, встраиваемых в разнообразную аппаратуру. Ввиду огромного количества объектов, управление которыми обеспечивается с помощью микроконтроллеров, годовой объем их выпуска превышает несколько миллиардов экземпляров, на порядок превосходя объем выпуска микропроцессоров общего назначения. Весьма широкой является также номенклатура выпускаемых микроконтроллеров, которая содержит несколько тысяч типов.

Как уже отмечалось, характерной особенностью структуры микроконтроллеров является размещение на одном кристалле с центральным процессором внутренней памяти и большого набора периферийных устройств. В состав периферийных устройств обычно входят несколько параллельных портов ввода/вывода данных, последовательные интерфейсы, таймерные блоки, аналого-цифровые и цифроаналоговые преобразователи. Кроме того, различные типы микроконтроллеров содержат дополнительные специализированные устройства — блоки формирования сигналов с широтно-импульсной модуляцией, контроллер жидкокристаллического дисплея и т. д. Благодаря использованию внутренней памяти и периферийных устройств реализуемые на базе микроконтроллеров системы управления имеют минимальное количество дополнительных компонентов.

В связи с широким диапазоном решаемых задач управления требования, предъявляемые к производительности МП, объему внутренней памяти команд и данных, набору необходимых периферийных устройств, оказываются весьма разнообразными. Для удовлетворения запросов потребителей выпускается большая номенклатура микроконтроллеров, которые принято подразделять на 8-, 16- и 32-разрядные.

8-разрядные микроконтроллеры представляют наиболее многочисленную группу и характеризуются относительно низкой производительностью, которая, однако, вполне достаточна для решения широкого круга задач управления различными объектами. Это простые и дешевые микроконтроллеры, ориентированные на использование в относительно несложных устройствах массового выпуска. Основными областями их применения являются бытовая и измерительная техника, промышленная автоматика, автомобильная электроника, теле-, видео- и аудиоаппаратура, средства связи. Микроконтроллеры этой группы обычно выполняют относительно небольшой набор команд (50 – 100), использующих наиболее простые способы адресации. Типовыми представителями данного класса устройств являются МК серии AVR фирмы Atmel и серии PIC фирмы Microchip, которые характеризуются малым набором команд, развитой периферией и относительно низкой стоимостью.

Производительность 8-разрядных МК можно оценить по следующему примеру. МК серии AVR использует команды, выполняемые за один машинный такт, таким образом контроллер достигает производительности в 1 MIPS на рабочей частоте 1 МГц.

16-разрядные микроконтроллеры во многих случаях являются усовершенствованной модификацией своих 8-разрядных прототипов. Они характеризуются не только увеличенной разрядностью обрабатываемых данных, но и расширенной системой команд и способов адресации, увеличенным набором регистров и объемом адресуемой памяти, а также рядом других дополнительных возможностей, использование которых позволяет повысить производительность и обеспечить новые области применения.

32-разрядные микроконтроллеры имеют высокопроизводительный процессор, соответствующий по своим возможностям младшим моделям микропроцессоров общего назначения. В последнее время широкое распространение получила 32-разрядная архитектура ARM (Advanced RISC Machine), на основе которой выпускается большое количество семейств МК различными производителями.

Процессоры цифровой обработки сигналов (ПЦОС) представляют класс специализированных микропроцессоров, ориентированных на цифровую обработку поступающих аналоговых сигналов. Специфической особенностью алгоритмов обработки аналоговых сигналов является необходимость последовательного выполнения ряда команд

умножения-сложения с накоплением промежуточного результата в регистре-аккумуляторе. Поэтому архитектура ПЦОС направлена на быстрое выполнение операций такого рода. Набор команд данных процессоров содержит специальные команды МАС (Multiplication with Accumulation), реализующие эти операции.

Значение поступившего сигнала может быть представлено в виде числа с фиксированной или плавающей точкой. Напомним, что число с плавающей точкой — это форма представления действительных чисел, в которой число хранится в виде мантиссы и показателя степени. При этом число имеет фиксированную относительную точность и изменяющуюся абсолютную точность.

В соответствии с этим ПЦОС делятся на процессоры, обрабатывающие числа с фиксированной или плавающей точкой. Более простые и дешевые ПЦОС с фиксированной точкой обычно обрабатывают 16-разрядные операнды, представленные в виде правильной дроби. Однако ограниченная разрядность в ряде случаев не позволяет обеспечить необходимую точность преобразования. Поэтому в ПЦОС с фиксированной точкой, выпускаемых компанией Motorola, принято 24-разрядное представление операндов. Наиболее высокая точность обработки достигается в случае представления данных в формате с плавающей точкой. В ПЦОС, обрабатывающих данные с плавающей точкой, обычно используется 32-разрядный формат их представления.

Если взять за критерий комплексный показатель «количество данных — количество вычислений», то возможны четыре его значения, определяющие области применения и основные характеристики МП (табл. 1.1.1).

Перечисленным четырем группам областей применения соответствуют определенные типы МП, наиболее полно учитывающие в своей архитектуре их специфику, однако установленное соответствие не является строгим. Так, среди 8-разрядных МК есть модели, которые с успехом можно использовать для решения отдельных задач из второй группы. Точно так же 32-разрядные МК в последнее время активно вытесняют 16-разрядные, поскольку разница в цене становится несущественной при более развитой архитектуре.

Основными направлениями развития микропроцессоров является увеличение их производительности и расширение функциональных возможностей, что достигается как повышением уровня микроэлектронной технологии, используемой для производства микропроцессо-

Таблица 1.1.1**Классификация МП по областям применения**

№ п/п	Значение критерия	Области применения	Разрядность и производи- тельность
1	Мало данных — мало вычислений	Логическое управление несложными объектами и процессами	8-разрядные; низкая
2	Мало данных — много вычислений	Системы управления элек- трическими двигателями, подвижными аппаратами, роботами-манипуляторами, станками и т. д.	16-разрядные; средняя
3	Много данных — мало вычислений	Многие сетевые задачи, системы управления пото- ками данных, коммутаторы и т. п.	32-разрядные; высокая
4	Много данных — много вычислений	Системы управления ре- ального времени, обработ- ка сигналов, системы рас- познавания речи, изобра- жений и т. п.	32-разрядные и более; сверхвысокая

ров, так и применением новых архитектурных и структурных вариан-
тов их реализации.

Развитие микроэлектронной технологии обеспечивает непрерыв-
ное уменьшение размеров полупроводниковых компонентов, размеща-
емых на кристалле микропроцессора. При этом уменьшаются пара-
зитные емкости, определяющие задержку переключения логических
элементов, и увеличивается число элементов, размещаемых на кри-
сталле. В настоящее время обеспечивается создание микропроцессо-
ров, работающих с тактовой частотой до 3–4 ГГц и содержащих на
кристалле десятки миллионов транзисторов. В соответствии с эмпи-
рическим правилом, которое в 1975 г. сформулировал Гордон Мур,
один из основателей компании Intel, степень интеграции микросхем
удваивается каждые 1,5–2 года. Это правило выполнялось в тече-

ние более чем 40 лет развития микроэлектроники. Отметим, что в последнее время для получения прироста производительности, соответствующего правилу Мура, разработчики процессоров используют параллельные вычисления вместо увеличения тактовой частоты и параллелизма на уровне исполнения команд.

В рамках курса «Микропроцессорные средства и системы» мы будем изучать архитектурные и структурные методы повышения производительности микропроцессоров и систем на их основе.

Контрольные вопросы

1. Дайте определение микропроцессора.
2. Перечислите основные классы микропроцессоров.
3. В чем заключаются отличия между МП и МК?
4. Дайте определения архитектуры и структуры МП.
5. Назовите основные направления развития микропроцессорной техники.
6. Каковы способы улучшения характеристик МП?

Литература

1. **Бойко В.И.** Схемотехника электронных систем. Микропроцессоры и микроконтроллеры. — СПб.: БХВ-Петербург, 2004. — 464 с.
2. Микропроцессорные системы: учеб. пособие для вузов / **Е.К. Александров, Р.И. Грушвицкий, М.С. Куприянов и др.; под ред. Д.В. Пузанкова.** — СПб.: Политехника, 2002. — 935 с.

Лекция 1.2. Обобщенная структура микропроцессора.

Арифметико-логическое устройство — основа операционных устройств микропроцессора

Несмотря на различную функциональную ориентированность, приводящую к значительной специализации аппаратной части, МП являются универсальными средствами обработки информации в рамках определенного класса задач. Это достигается тем, что структура процессора содержит сложные устройства, позволяющие под программным управлением выполнять над исходными числами логические и арифметические операции. На рис. 1.2.1 представлена обобщенная структура микропроцессора, которая состоит из блока интерфейсной логики, операционного устройства, памяти и устройства управления.

Наиболее характерными узлами МП являются операционное устройство (ОУ) и устройство управления (УУ).

Следует отметить, что в общем случае МП может содержать несколько *операционных устройств* с различной либо идентичной структурой:

- регистровое арифметико-логическое устройство (РАЛУ);
- аппаратные умножители и делители;
- блоки, выполняющие операции с числами, представленными в формате с плавающей точкой;
- блоки, выполняющие специализированные операции, например, умножение с накоплением или операции над комплексными числами.

Устройство управления формирует управляющие сигналы на все блоки процессора, синхронизируя их работу, и обеспечивает выборку команд из памяти в соответствии с заложенным алгоритмом.

Блок интерфейсной логики реализует электрическое и логическое сопряжение МП с внешними устройствами, а также формирует протокол обмена МП с внешними устройствами.

Память предназначена для хранения программ и данных. Всегда состоит из ОЗУ и ПЗУ. В минимальной конфигурации вычислительной системы в ПЗУ находится программа загрузки. Следует отметить,

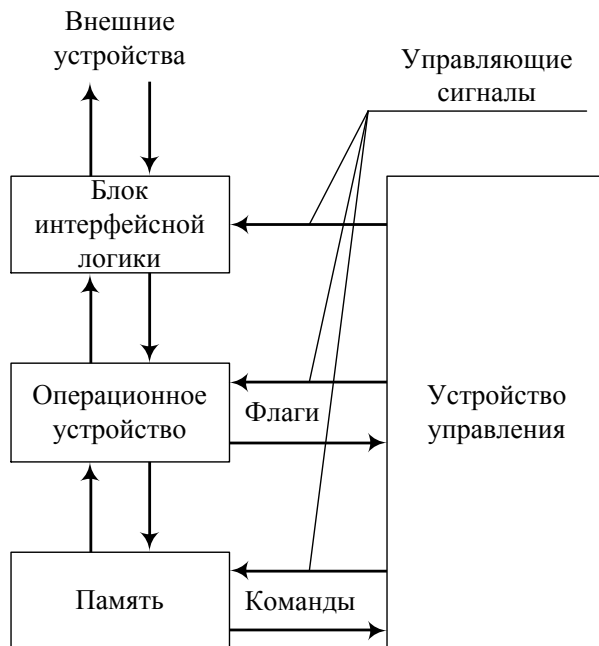


Рис. 1.2.1. Обобщенная структура микропроцессора

что организация системы памяти — это один из ключевых критериев классификации архитектур МП. Существует множество структур, обеспечивающих хранение данных и программ, при этом может использоваться как одношинная, так и многошинная организация обмена с памятью. Таким образом, структура, представленная на рис. 1.2.1, в самом общем виде показывает связи между функциональными модулями МП.

Несмотря на многообразие структур, ОУ в любом случае предназначено для обработки информации в соответствии с поступающим на него управляющим кодом. Наиболее применяемым и традиционным ОУ является РАЛУ, ориентированное на выполнение арифметических и логических операций и хранение промежуточных результатов операций.

Выполняемые в РАЛУ операции можно разделить на четыре группы: арифметические, сдвига, логические и пересылок. В каче-

стве арифметических операций в РАЛУ обычно используются операции сложения и вычитания двух кодов, сложения и вычитания единицы, сложения и вычитания с содержимым признака операции. К операциям сдвига относятся операции арифметического, логического и циклического сдвигов вправо и влево содержимого регистров ОУ. К логическим операциям относятся побитовые операции И, ИЛИ, НЕ, исключающее ИЛИ и т.п. Операции пересылок предназначены для обмена информацией между регистрами и ячейками памяти.

В состав РАЛУ, как правило, входят арифметико-логическое устройство, блок регистров общего назначения, блок формирования содержимого регистра состояния, блок местного управления.

Арифметико-логическое устройство непосредственно выполняет микрооперации над исходными операндами. Формально АЛУ — это комбинационная схема (КС), предназначенная для выполнения арифметических и поразрядно-логических операций над многоразрядными словами. Напомним, что *комбинационной* называется схема, состояние выходных сигналов которой однозначно определяется набором входных сигналов; такая схема не содержит элементов памяти.

Блок РОН обеспечивает хранение операндов и промежуточных результатов вычислений. Характеризуется малым временем обращения и ограниченным количеством регистров.

Блок формирования содержимого регистра состояния записывает в регистр состояния двоичный код, характеризующий арифметические и логические признаки результата операции. Содержимое регистра состояния может быть использовано устройством управления для формирования условных переходов по результатам операций, выполняемых АЛУ.

Блок местного управления обеспечивает выполнение текущей микрокоманды и управляет в соответствии с кодом микрокоманды всеми блоками операционного устройства.

В общем случае к АЛУ предъявляются следующие требования:

- выполнение арифметических и логических операций; разнообразие данных операций обеспечивает набор команд, которые сможет выполнять микропроцессор;
- обеспечение межразрядного переноса при выполнении арифметических операций, от этого во многом зависит быстродействие вычислительной системы;

- обеспечение наращивания разрядности обрабатываемых слов однотипными секциями.

Основой АЛУ является одноразрядный комбинационный сумматор SM , который изменяет конфигурацию в зависимости от управляющего слова S (рис. 1.2.2). Управляющее слово S входит в состав микрокоманды и определяет тип выполняемой операции. Сигнал M обеспечивает разделение арифметических и логических операций за счет запрета передачи арифметического переноса от секции к секции: при $M = 1$ выполняются арифметические операции, при $M = 0$ — логические. P_i — бит арифметического переноса, формируемый i -й секцией АЛУ; a_i , b_i — разряды многоразрядных операндов A и B ; R_i — разряды результата R .

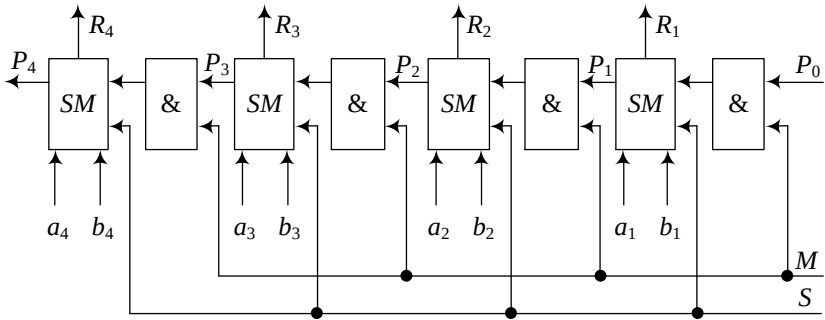


Рис. 1.2.2. Структура АЛУ на примере 4-разрядной секции

Рассмотрим особенности синтеза и функционирования АЛУ на примере, для чего сформулируем конкретные требования. Предположим, что нам необходимо реализовать:

- арифметическую операцию сложения ($A + B$);
- арифметическую операцию вычитания ($A - B$);
- логическую операцию ИЛИ ($A \vee B$);
- логическую операцию И ($A \cdot B$).

Работу одноразрядного комбинационного сумматора можно описать выражением

$$R_i^+ = a_i \oplus b_i \oplus MP_{i-1}. \quad (1.2.1)$$

После тождественных преобразований получим

$$R_i^+ = (\overline{a_i} \vee \overline{b_i})(a_i \vee b_i) \oplus MP_{i-1} = \overline{a_i b_i}(a_i \vee b_i) \oplus MP_{i-1}. \quad (1.2.2)$$

Анализ (1.2.2) показывает, что для реализации арифметической операции сложения необходимо полностью сохранить данное выражение, а для реализации логической операции ИЛИ следует исключить операции $a_i \overline{b_i}$ и MP_{i-1} .

Дополним (1.2.2) управляющим сигналом S_3 , который в зависимости от определенных условий может принимать значения 0 или 1:

$$R_i' = \overline{S_3 a_i b_i}(a_i \vee b_i) \oplus MP_{i-1}. \quad (1.2.3)$$

При $M = S_3 = 0$ результатом преобразования является побитовая логическая операция ИЛИ над многоразрядными операндами A и B . При $M = S_3 = 1$ результатом преобразования является арифметическая операция сложения многоразрядных операндов A и B .

Таким образом, выражение (1.2.3) описывает работу АЛУ, которое может выполнять две из четырех требуемых операций.

Для реализации арифметической операции вычитания в вычислительных машинах используются обратный и дополнительный коды:

$$R_i^- = a_i \oplus \overline{b_i} \oplus MP_{i-1}. \quad (1.2.4)$$

При $P_0 = 1$ получаем арифметическое сложение операнда A с операндом B , представленным в дополнительном коде, т.е. арифметическую операцию вычитания.

Выполним тождественные преобразования аналогично (1.2.2):

$$R_i^- = (\overline{a_i} \vee b_i)(a_i \vee \overline{b_i}) \oplus MP_{i-1} = \overline{a_i \overline{b_i}}(a_i \vee \overline{b_i}) \oplus MP_{i-1}. \quad (1.2.5)$$

Введем управляющий сигнал S_2 , который аналогично сигналу S_3 позволяет при необходимости исключать из выражения (1.2.5) первый множитель:

$$R_i'' = \overline{S_2 a_i \overline{b_i}}(a_i \vee \overline{b_i}) \oplus MP_{i-1}. \quad (1.2.6)$$

При $M = S_2 = P_0 = 1$ результатом преобразования будет арифметическая операция вычитания многоразрядных операндов A и B .

Объединим (1.2.3) и (1.2.6) в одно выражение, добавив управляющие сигналы S_1 и S_0 :

$$R_i = \overline{S_3 a_i b_i} \vee \overline{S_2 a_i \overline{b_i}} \cdot (a_i \vee S_1 \overline{b_i} \vee S_0 b_i) \oplus MP_{i-1}. \quad (1.2.7)$$

При $M = S_3 = S_1 = S_0$ и $S_2 = 1$ результатом преобразования является побитовая логическая операция И над многоразрядными операндами A и B .

Таким образом, мы получили выражение, которое реализуют все четыре заданные операции.

Выражение удобно использовать при анализе логических операций, в случае анализа арифметических операций (1.2.7) можно представить тождественным выражением

$$R_i = (S_3 a_i b_i \vee S_2 a_i \bar{b}_i) \oplus (a_i \vee S_1 \bar{b}_i \vee S_0 b_i) \oplus M P_{i-1}. \quad (1.2.8)$$

Убедимся в тождественности выражений (1.2.7) и (1.2.8). Введем следующие обозначения:

$$\begin{aligned} D_i &= S_3 a_i b_i \vee S_2 a_i \bar{b}_i; \\ F_i &= a_i \vee S_1 \bar{b}_i \vee S_0 b_i, \end{aligned}$$

где D_i, F_i — подготовительные функции нулевого порядка. Эти функции используются для реализации арифметического переноса в АЛУ и имеют важную особенность: значения D_i, F_i не зависят от переноса из предыдущей секции.

Известно, что $D_i \oplus F_i = \overline{D_i F_i} \vee D_i \overline{F_i}$, т. е. для доказательства тождества нам необходимо доказать, что слагаемое $D_i \overline{F_i}$ всегда равно нулю:

$$\begin{aligned} D_i \overline{F_i} &= (S_3 a_i b_i \vee S_2 a_i \bar{b}_i) \cdot \overline{a_i \vee S_1 \bar{b}_i \vee S_0 b_i} = \\ &= (S_3 a_i b_i \vee S_2 a_i \bar{b}_i) \cdot \bar{a}_i \cdot \overline{S_1 \bar{b}_i} \cdot \overline{S_0 b_i} = \\ &= S_3 a_i b_i \cdot \bar{a}_i \cdot \overline{S_1 \bar{b}_i} \cdot \overline{S_0 b_i} \vee S_2 a_i \bar{b}_i \cdot \bar{a}_i \cdot \overline{S_1 \bar{b}_i} \cdot \overline{S_0 b_i} = 0. \end{aligned}$$

Заметим, что помимо заданных двух логических и двух арифметических операций с помощью выражений (1.2.7) и (1.2.8) можно реализовать большее количество операций. Данные выражения содержат четыре управляющих переменных, посредством которых можно определить шестнадцать различных операций. Количество операций удваивается, поскольку с помощью сигнала M мы разделяем шестнадцать логических и шестнадцать арифметических операций. Результат арифметической операции зависит от значения P_0 , т. е. существуют два разных результата арифметической операции (при $P_0 = 0$

и $P_0 = 1$). Таким образом, синтезированное нами выражение позволяет построить АЛУ, выполняющее 48 операций.

Рассмотрим методику определения выполняемой АЛУ операции по заданным управляющим сигналам S , M , P_0 и составим таблицу, отражающую все возможные операции (табл. 1.2.1).

Таблица 1.2.1

Операции, выполняемые АЛУ, согласно (1.2.7) и (1.2.8)

№ п/п	Управляющее слово				R		
					При	При $M = 1$	
					$M = 0$	При $P_0 = 0$	При $P_0 = 1$
1	0	0	0	0	A	A	$A + 1$
2	0	0	0	1	$A \vee B$	$A \vee B$	$A \vee B + 1$
3	0	0	1	0	$A \vee \overline{B}$	$A \vee \overline{B}$	$A \vee \overline{B} + 1$
4	0	0	1	1	-1	-1	0
5	0	1	0	0	AB	$A\overline{B} + A$	$A\overline{B} + A + 1$
6	0	1	0	1	B	$A\overline{B} + (A \vee B)$	$A\overline{B} + (A \vee B) + 1$
7	0	1	1	0	$A \oplus \overline{B}$	$A - B - 1$	$A - B$
8	0	1	1	1	$\overline{A} \vee B$	$A\overline{B} - 1$	$A\overline{B}$
9	1	0	0	0	$A\overline{B}$	$AB + A$	$AB + A + 1$
10	1	0	0	1	$A \oplus B$	$A + B$	$A + B + 1$
11	1	0	1	0	$\overline{B}$	$AB + (A \vee \overline{B})$	$AB + (A \vee \overline{B}) + 1$
12	1	0	1	1	$\overline{AB}$	$\overline{AB} - 1$	$\overline{AB}$
13	1	1	0	0	0	$A + A$	$A + A + 1$
14	1	1	0	1	$\overline{AB}$	$A + (A \vee B)$	$A + (A \vee B) + 1$
15	1	1	1	0	$\overline{A \vee B}$	$A + (A \vee \overline{B})$	$A + (A \vee \overline{B}) + 1$
16	1	1	1	1	$\overline{A}$	$A - 1$	A

При $M = 0$, т.е. в случае логической операции, требуется упростить выражение (1.2.7) с учетом конкретных значений $S_3 - S_0$ и записать результат в таблицу. Поясним на примере. Пусть $S_3 = 0$,

$S_2 = 1, S_1 = 0, S_0 = 1$. Подставив значения S , получим:

$$\begin{aligned} R_i &= \overline{0 \cdot a_i b_i \vee 1 \cdot a_i \bar{b}_i \cdot (a_i \vee 0 \cdot \bar{b}_i \vee 1 \cdot b_i)} \oplus 0 \cdot P_{i-1} = \\ &= \overline{a_i \bar{b}_i (a_i \vee b_i)} = (\bar{a}_i \vee b_i)(a_i \vee b_i) = \\ &= a_i \bar{a}_i \vee \bar{a}_i b_i \vee a_i b_i \vee b_i = b_i(\bar{a}_i \vee a_i \vee 1) = b_i. \end{aligned}$$

Таким образом, при $S_3 = 0, S_2 = 1, S_1 = 0, S_0 = 1$ АЛУ транслирует на свой выход операнд B без изменений.

При $M = 1$, т.е. в случае арифметической операции, используем для анализа выражение (1.2.8). Любую операцию будем трактовать как арифметическое сложение операндов D и F . В данном случае $D_i = a_i \bar{b}_i, F_i = a_i \vee b_i$. При $P_0 = 0$ получим:

$$R = A\bar{B} + (A \vee B).$$

При $P_0 = 1$ к этому результату добавляется единица младшего разряда:

$$R = A\bar{B} + (A \vee B) + 1.$$

Аналогичным образом заполним остальные элементы таблицы.

Контрольные вопросы

1. Приведите обобщенную структуру микропроцессора, укажите назначение его составных частей.
2. Опишите общий алгоритм функционирования микропроцессора.
3. Каким образом выполняются команды условного перехода?
4. Опишите типовой состав операционного устройства.
5. Дайте определение арифметико-логического устройства.
6. Поясните на примере, каким образом выполняются операции в АЛУ.

Литература

1. Микропроцессоры. В 3-х кн. Кн. 1. Архитектура и проектирование микроЭВМ: учебник для вузов / **Под ред. Л.Н. Преснухина**. — М.: Высшая школа, 1986. — 495 с.

Лекция 1.3. Организация цепей переноса в пределах секции АЛУ. Нарращивание разрядности обрабатываемых слов. Примеры использования АЛУ

Организация цепей переноса в пределах секции АЛУ

Многообразие решаемых с помощью вычислительных систем задач определяет многообразие форм представления входных данных. В первую очередь это отражается на разрядности операндов. Поскольку разработчики аппаратуры, как правило, стараются построить универсальную аппаратную платформу хотя бы для узкого класса задач, АЛУ принято делить на секции определенной разрядности и на основе таких секций компоновать операционные устройства с учетом конкретных требований. При дальнейшем рассмотрении будем полагать, что каждая секция АЛУ обрабатывает 4-разрядные операнды.

Рассмотрим структуру секции АЛУ, представленную на рис. 1.3.1 и функционирующую согласно выражению

$$R_i = D_i \oplus F_i \oplus MP_{i-1},$$

где $D_i = S_3 a_i b_i \vee S_2 a_i \bar{b}_i$; $F_i = a_i \vee S_1 \bar{b}_i \vee S_0 b_i$. На информационные входы секции АЛУ поступают разряды операндов a_i , b_i и бит переноса P_{i-1} . На управляющие входы подаются коды $S_3 - S_0$ и M выполняемой операции. Каждая секция формирует: подготовительные функции D_i , F_i , которые могут использоваться при параллельной организации арифметического переноса, биты арифметического переноса P_i и результата R_i .

Объединение нескольких подобных секций в операционном устройстве требует организации цепей переноса. Быстродействие расширенного по разрядности АЛУ в большой степени зависит от способа организации цепей переноса внутри секции и между секциями. Внутри секции могут быть предложены два способа переноса: последовательный (сквозной) и параллельный.

При *последовательном (сквозном) переносе* в каждой секции АЛУ формируется сигнал переноса на основании подготовительных

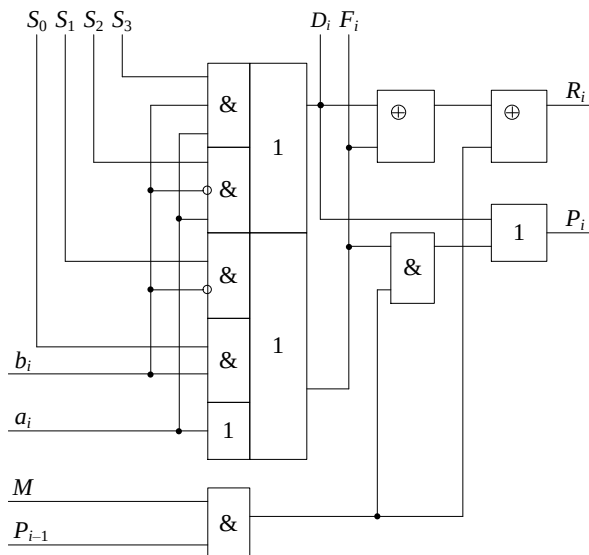


Рис. 1.3.1. Структура секции АЛУ

функций и сигнала переноса из предыдущей секции в соответствии с выражением

$$P_i = D_i \vee F_i P_{i-1}.$$

Время распространения переноса по всей схеме равно nt_n , где n — число разрядов в секции; t_n — время распространения переноса через одну секцию (разряд). Таким образом, схема с последовательным переносом характеризуется относительной простотой (см. рис. 1.3.1). Использовать последовательный перенос целесообразно, когда нет необходимости в высоком быстродействии, но предъявляются жесткие требования к аппаратным затратам.

Параллельный перенос обеспечивает более высокое быстродействие, но требует больших аппаратных затрат. При этом все сигналы переноса формируются параллельно на основании подготовительных функций в соответствии с выражением

$$P_i = D_i \vee D_{i-1} F_i \vee D_{i-2} F_{i-1} F_i \vee \dots \vee P_0 F_1 F_2 F_3 \dots F_i, \quad (1.3.1)$$

где $i = 1 \dots n$ — номер разряда.

Для 4-разрядной секции АЛУ имеем следующую систему выражений:

$$\begin{aligned}
 P_1 &= D_1 + P_0 F_1; \\
 P_2 &= D_2 + P_1 F_2 = D_2 + D_1 F_2 + P_0 F_1 F_2; \\
 P_3 &= D_3 + P_2 F_3 = D_3 + D_2 F_3 + D_1 F_2 F_3 + P_0 F_1 F_2 F_3; \\
 P_4 &= D_4 + P_3 F_4 = \\
 &= \underbrace{D_4 + D_3 F_4 + D_2 F_3 F_4 + D_1 F_2 F_3 F_4}_{D^1} + \underbrace{P_0 F_1 F_2 F_3 F_4}_{F^1},
 \end{aligned}$$

где D^1, F^1 — подготовительные функции первого порядка, используемые для реализации межсекционного переноса. Из данной системы выражений видно, что каждый из сигналов зависит только от операндов и нулевого бита переноса, т.е. все они могут вычисляться одновременно, за счет чего и достигается увеличение производительности.

Нарращивание разрядности обрабатываемых слов

При наращивании разрядности между секциями обычно используют два способа организации цепей переноса: последовательный (сквозной) и групповой.

Последовательный (сквозной) перенос между секциями применим при малом числе связанных секций (рис. 1.3.2). В этом случае внутри секций, как правило, реализуют параллельный перенос. Суммарная задержка появления старшего бита переноса результата с момента поступления операндов и управляющих сигналов определяется как $T_{\Sigma} = nt_{\text{АЛУ}}$, где $t_{\text{АЛУ}}$ — время формирования бита по переносу одной секцией АЛУ; n — число секций в системе.

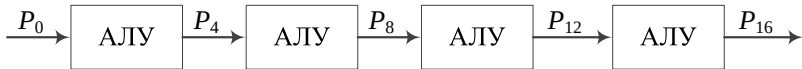


Рис. 1.3.2. Организация последовательного (сквозного) переноса

Групповой перенос с помощью внешней схемы ускоренного переноса (СУП) целесообразно использовать при числе секций больше

четырёх. В этом случае в каждой секции формируются подготовительные функции первого порядка, поступающие на вход СУП, как показано на рис. 1.3.3. На основании этих функций согласно выражению (1.3.1) в СУП формируются входные биты переноса для каждой секции. Суммарная задержка появления старшего бита переноса составляет $T_{\Sigma} = t_{\text{АЛУ}} + t_{\text{СУП}}$, где $t_{\text{СУП}}$ — время формирования результата СУП.

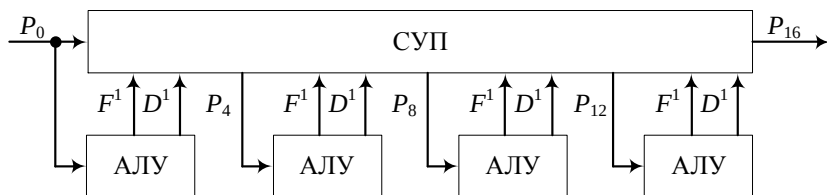


Рис. 1.3.3. Организация группового переноса

Примеры использования АЛУ в качестве реконфигурируемого преобразователя информации

Благодаря многофункциональным возможностям АЛУ находит применение не только в составе микропроцессоров, но и в составе специализированных преобразователей информации. Такой подход оказывается эффективным, когда применение полноценного процессора неоправданно усложняет схемотехническое решение или повышает стоимость устройства. Считается, что использование АЛУ в качестве решающего устройства в составе специализированных преобразователей рационально в тех случаях, когда количество реконфигураций не превышает восьми. В иных случаях целесообразней применять микропроцессор, поскольку резко возрастают аппаратные затраты на реализацию реконфигурируемой структуры или устройства управления.

Рассмотрим несколько примеров использования АЛУ в качестве преобразователя прямых, обратных и дополнительных двоичных кодов, когда на устройство поступает 5-разрядное знаковое число:

$$\underbrace{N_5}_{\text{знак}} \underbrace{N_4 N_3 N_2 N_1}_{\text{мантисса}}$$

Пример 1.3.1. Разработка преобразователя прямого кода в обратный. Из определения обратного кода следует, что данный преобразователь должен инвертировать все разряды, кроме знакового, если число отрицательное, и оставить число без изменений, если оно положительное.

Из анализа выражений, описывающих функционирование АЛУ, разработанного в предыдущей лекции, известно, что синтезированная нами структура АЛУ способна выполнять операции передачи операнда B на выход схемы как без изменений, так и в инвертированном виде. Это обеспечивается выполнением логических операций, задаваемых управляющими кодами $S_3 = 0, S_2 = 1, S_1 = 0, S_0 = 1$ и $S_3 = 1, S_2 = 0, S_1 = 1, S_0 = 0$ соответственно. Результат i -й секции определяется выражением:

- в первом случае:

$$\begin{aligned} R_i &= \overline{0 \cdot a_i b_i \vee 1 \cdot a_i \bar{b}_i} \cdot (a_i \vee 0 \cdot \bar{b}_i \vee 1 \cdot b_i) \oplus 0 \cdot P_{i-1} = \\ &= \overline{a_i \bar{b}_i} (a_i \vee b_i) = (\bar{a}_i \vee b_i) (a_i \vee b_i) = \\ &= \bar{a}_i b_i \vee a_i b_i \vee b_i = b_i (\bar{a}_i \vee a_i \vee 1) = b_i; \end{aligned}$$

- во втором случае:

$$\begin{aligned} R_i &= \overline{1 \cdot a_i b_i \vee 0 \cdot a_i \bar{b}_i} \cdot (a_i \vee 1 \cdot \bar{b}_i \vee 0 \cdot b_i) \oplus 0 \cdot P_{i-1} = \\ &= \overline{a_i b_i} (a_i \vee \bar{b}_i) = (\bar{a}_i \vee \bar{b}_i) (a_i \vee \bar{b}_i) = \\ &= \bar{a}_i \bar{b}_i \vee a_i \bar{b}_i \vee \bar{b}_i = \bar{b}_i (\bar{a}_i \vee a_i \vee 1) = \bar{b}_i. \end{aligned}$$

Заметим, что в обоих случаях результат не зависит от значения операнда A .

Составим таблицу, устанавливающую зависимость между знаком числа N_5 и управляющими кодами АЛУ (табл. 1.3.1).

Из таблицы видно, что управление АЛУ сводится к прямому и инверсному подключению сигнала знака N_5 к парам управляющих сигналов S_3, S_1 и S_2, S_0 соответственно. На неиспользованные входы АЛУ подадим код нуля.

Схема полученного преобразователя будет иметь вид, представленный на рис. 1.3.4.

Таблица 1.3.1

Зависимость между знаком числа и управляющими кодами АЛУ

N_5	Результат	S_3	S_2	S_1	S_0	M	P_0
0	$\{N_5, N_4, N_3, N_2, N_1\}$	0	1	0	1	0	X
1	$\{N_5, \overline{N_4}, \overline{N_3}, \overline{N_2}, \overline{N_1}\}$	1	0	1	0	0	X

Примечание: X — состояние сигналов, не влияющее на выполнение текущей операции.

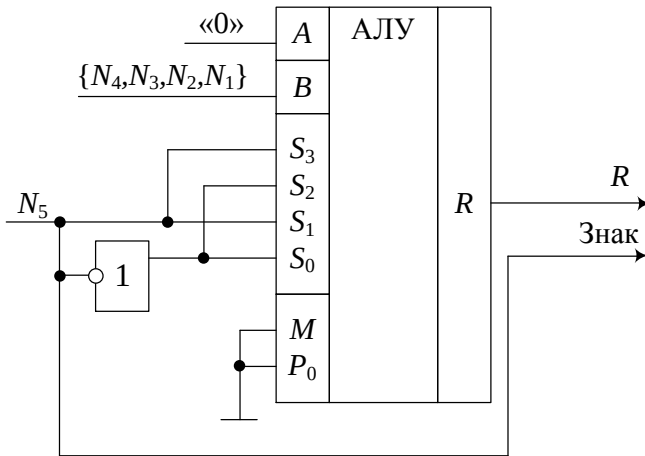


Рис. 1.3.4. Схема преобразователя прямого кода в обратный

Пример 1.3.2. Разработка преобразователя прямого кода в дополнительный. Из определения дополнительного кода следует, что данный преобразователь должен инвертировать все разряды, кроме знакового, и прибавить единицу младшего разряда, если число отрицательное, и оставить число без изменений, если оно положительное. Заметим, что этот преобразователь отличается от предыдущего только тем, что в случае отрицательного числа к результату необходимо прибавить единицу. Это можно обеспечить, выполнив арифметическую операцию $A - B$ при $A = 0$.

Составим таблицу, определяющую зависимость между знаком числа N_5 и управляющими кодами АЛУ (табл. 1.3.2).

Таблица 1.3.2

Зависимость между знаком числа и управляющими кодами АЛУ

N_5	Результат	S_3	S_2	S_1	S_0	M	P_0
0	$\{N_5, N_4, N_3, N_2, N_1\}$	0	1	0	1	0	X
1	$\{\overline{N_5}, \overline{N_4}, \overline{N_3}, \overline{N_2}, \overline{N_1}\} + 1$	0	1	1	0	1	1

Из таблицы видно, что управление АЛУ сводится к прямому и инверсному подключению сигнала знака N_5 к управляющим сигналам S_1 и S_0 соответственно, помимо этого для добавления единицы младшего разряда необходимо подавать сигнал знака N_5 на входы M и P_0 . Сигналы S_3 и S_2 задаются статически логическим «0» и «1» соответственно. На неиспользованные входы АЛУ подадим код нуля.

Схема полученного преобразователя будет иметь вид, представленный на рис. 1.3.5.

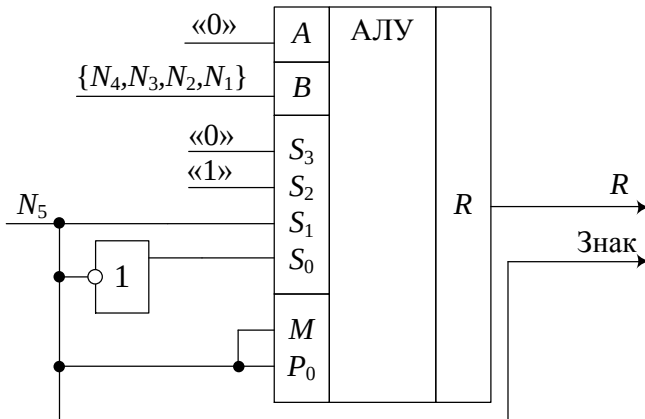


Рис. 1.3.5. Схема преобразователя прямого кода в дополнительный

Пример 1.3.3. Разработка преобразователя дополнительного кода в обратный. Данный преобразователь должен выполнять следующие действия: если число положительное, оставить его без изменений; если число отрицательное, вычесть единицу младшего разряда. Это можно реализовать двумя способами: 1) непосредственным

вычитанием $A - B$ при $B = 1$; 2) прибавлением единицы к каждому разряду исходного числа, что соответствует прибавлению дополнительного кода единицы, т. е. вычитанию единицы, но без использования операнда B . Рассмотрим способ 2.

Анализ выражений, описывающих функционирование АЛУ, показывает, что синтезированная нами структура АЛУ способна выполнять операции передачи операнда A на выход схемы как без изменений, так и за вычетом единицы младшего разряда.

Операция вычитания единицы (декремента) обеспечивается выполнением арифметической операции, задаваемой управляющим кодом $S_3 = 1$, $S_2 = 1$, $S_1 = 1$, $S_0 = 1$ при $P_0 = 0$:

$$R_i = (1 \cdot a_i b_i \vee 1 \cdot a_i \bar{b}_i) \oplus (a_i \vee 1 \cdot \bar{b}_i \vee 1 \cdot b_i) \oplus 1 \cdot P_{i-1} = a_i \oplus 1 \oplus P_{i-1}.$$

Передача операнда A на выход АЛУ без изменений может быть реализована несколькими способами. Например, это достигается при выполнении логической операции, задаваемой управляющим кодом $S_3 = 0$, $S_2 = 0$, $S_1 = 0$, $S_0 = 0$:

$$R_i = \overline{0 \cdot a_i b_i \vee 0 \cdot a_i \bar{b}_i} \cdot (a_i \vee 0 \cdot \bar{b}_i \vee 0 \cdot b_i) \oplus 0 \cdot P_{i-1} = a_i.$$

В обоих рассмотренных случаях результат не зависит от значения операнда B .

Составим таблицу, определяющую зависимость между знаком числа N_5 и управляющими кодами АЛУ (табл. 1.3.3).

Таблица 1.3.3

Зависимость между знаком числа и управляющими кодами АЛУ

N_5	Результат	S_3	S_2	S_1	S_0	M	P_0
0	$\{N_4, N_3, N_2, N_1\}$	0	0	0	0	0	X
1	$\{N_4, N_3, N_2, N_1\} - 1$	1	1	1	1	1	0

Схема полученного преобразователя будет иметь вид, представленный на рис. 1.3.6.

Рассмотренные примеры показывают, каким образом должно управляться АЛУ для реализации того или иного конкретного преобразования, однако при построении многофункциональных устройств

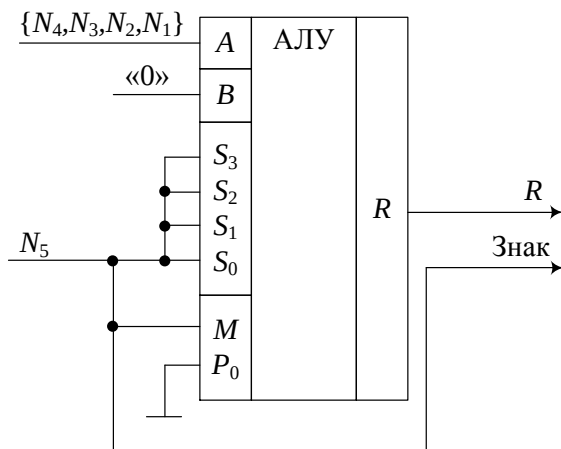


Рис. 1.3.6. Схема преобразователя дополнительного кода в обратный

необходимо устройство управления, обеспечивающее формирование управляющих сигналов в соответствии с запрашиваемым преобразованием.

Контрольные вопросы

1. Каким образом организуется арифметический перенос в пределах одной секции АЛУ?
2. Каким образом организуется арифметический перенос между секциями АЛУ?
3. За счет чего происходит повышение быстродействия при использовании СУП?

Литература

1. Микропроцессоры. В 3-х кн. Кн. 1. Архитектура и проектирование микроЭВМ: учебник для вузов / **Под ред. Л.Н. Преснухина**. — М.: Высшая школа, 1986. — 495 с.
2. **Угрюмов Е.П.** Цифровая схемотехника: учеб. пособие для вузов. — 2-е изд., перераб. и доп. — СПб.: БХВ-Петербург, 2005. — 800 с.

Лекция 1.4. Структуры операционных устройств. Регистровое арифметико-логическое устройство

В общем случае операционные устройства процессора — это не только регистровые арифметико-логические устройства, выполняющие логические и целочисленные арифметические операции, но и блоки, реализующие операции над десятичными числами или числами с плавающей точкой и т. д. Специализация и расширение аппаратуры ОУ происходят по мере расширения технологических возможностей, поскольку это обеспечивает повышение быстродействия.

В минимальной конфигурации ОУ может содержать аппаратуру только для выполнения основных логических операций, сдвигов, арифметических операций сложения и вычитания над числами с фиксированной точкой. На основе этого набора можно программным способом реализовать остальные логические и арифметические операции над числами, представленными как в форме с фиксированной точкой, так и в других формах.

Выполнение команды в ОУ можно представить в виде следующей последовательности действий:

- размещение операндов A и B на входах ОУ;
- выполнение операции в ОУ;
- запись результата операции в элемент памяти.

Каждое действие в данной последовательности невозможно разделить на более простые подоперации. Такие элементарные преобразования, не разложимые на более простые и выполняющиеся в течение одного такта сигнала синхронизации, называют *микрооперациями*.

Основываясь на приведенной последовательности действий, составим обобщенную структуру ОУ (рис. 1.4.1), в которую входят:

- регистры, обеспечивающие кратковременное хранение слов данных;
- управляемые шины, предназначенные для передачи слов данных;
- КС, реализующая вычисление различных функций от операндов в зависимости от управляющих сигналов.

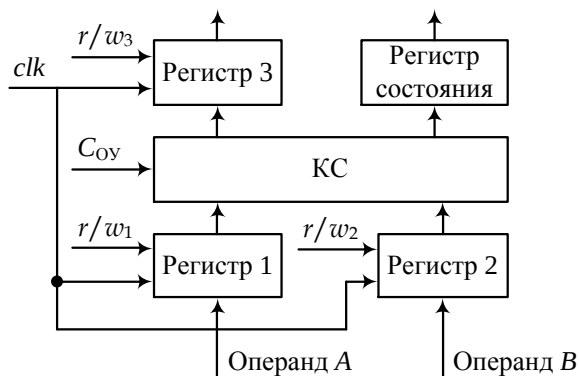


Рис. 1.4.1. Обобщенная структура операционного устройства

Для того чтобы структура, представленная на рис. 1.4.1, была работоспособна, необходимо дополнительно решить две задачи: синхронизации и управления. *Задача синхронизации* состоит в том, чтобы сформировать тактовые импульсы clk таким образом, чтобы в регистрах фиксировалась достоверная информация. *Задача управления* в минимальном объеме заключается в том, чтобы обеспечить формирование управляющих сигналов, задающих тип операции, выполняемой ОУ ($C_{ОУ}$), и определяющих режимы работы регистров (r/w_1 , r/w_2 , r/w_3).

Для того чтобы использовать данную структуру при построении вычислительных систем, необходимо сформировать набор микроопераций, каждая из которых содержит информацию о типе операции, источниках операндов, приемнике результата, способе определения следующей микрокоманды.

Различают ОУ с жесткой и магистральной структурами. Рассмотрим особенности каждой из них.

Операционные устройства с жесткой структурой

ОУ с жесткой структурой содержит несколько комбинационных схем, реализующих определенные микрооперации, с каждой комбинационной схемой связан набор регистров.

На рис. 1.4.2 приведен пример ОУ с жесткой структурой, обеспечивающего выполнение операции арифметического сложения. В со-

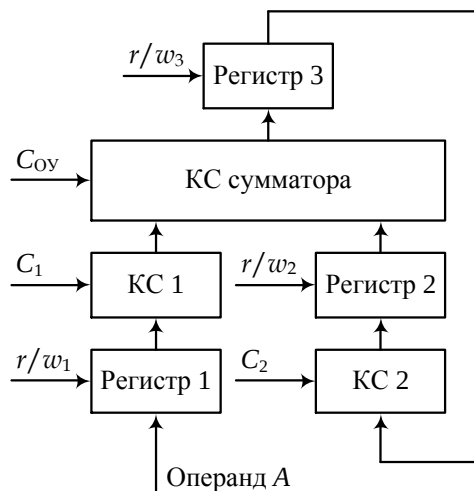


Рис. 1.4.2. Пример ОУ с жесткой структурой

став ОУ входят три регистра и три комбинационные схемы (КС 1, КС 2 и КС сумматора).

КС 1 обеспечивает передачу первого слагаемого из регистра 1 на один из входов сумматора в прямом или инверсном коде со сдвигом на разряд влево или вправо в зависимости от совокупности управляющих сигналов C_1 .

КС 2 реализует передачу результата из регистра 3 в регистр 2 в прямом или инверсном коде со сдвигом на разряд влево или вправо в зависимости от совокупности управляющих сигналов C_2 .

Достоинством ОУ с жесткой структурой является высокое быстродействие, недостатком — низкая регулярность, что затрудняет реализацию таких ОУ в виде БИС.

Операционные устройства с магистральной структурой; регистровое АЛУ

В ОУ с магистральной структурой (рис. 1.4.3) внутренние регистры объединены в отдельный блок РОН, а комбинационные схемы — в операционный блок (ОБ), как правило, называемый арифметическим устройством.

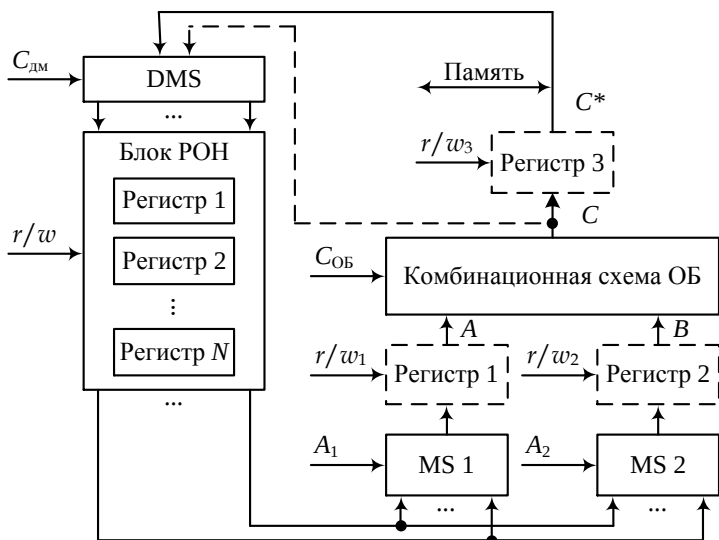


Рис. 1.4.3. Пример ОУ с магистральной структурой

В состав блока РОН входят N регистров общего назначения, подключаемых к магистралям через мультиплексоры MS 1 и MS 2, каждый из которых является управляемым коммутатором, соединяющим выход одного регистра с соответствующей магистралью. Номер подключаемого регистра определяется адресом A_1 и A_2 соответственно. Результат операции, выполненной комбинационной схемой ОБ, заносится в один из регистров блока РОН через управляемый демультиплексор DMS.

Организация блока РОН может обеспечивать одноканальный или двухканальный доступ как по записи, так и по чтению. В первом случае ко входу блока подключается один демультиплексор, а к выходу — один мультиплексор; во втором случае — соответственно два демультиплексора и (или) два мультиплексора. Двухканальный доступ позволяет повысить быстродействие за счет параллельных обращений к парам регистров.

С учетом необязательных элементов, отмеченных на рис. 1.4.3 пунктиром, можно выделить четыре варианта структуры, применяемых на практике:

- 1) регистры входят в состав блока РОН;
- 2) регистры расположены на входах ОУ;
- 3) регистры расположены на входах и выходе ОБ, память и демультиплексор соединены с выходом регистра 3;
- 4) регистры расположены на входах и выходе ОБ, память соединена с выходом регистра 3, вход демультиплексора соединен с выходом комбинационной схемы ОБ.

Сравним эти варианты структуры с точки зрения аппаратных затрат и быстродействия.

Интеграция всех регистров в блоке РОН (структура 1) требует наименьших аппаратных затрат, что снижает быстродействие: во-первых, нельзя изменять содержимое регистров, подключенных ко входам ОБ, до завершения текущей микрооперации; во-вторых, при записи результата в память значения на входе ОБ должны быть достоверными в течении всего цикла записи.

Введение регистров на входах ОБ (структура 2) решает первую проблему. Однако теперь для выполнения полного цикла, состоящего из чтения операндов, выполнения операции и записи результата в блок РОН, требуется два такта вместо одного (рис. 1.4.4, а, б).

Проблема, связанная с организацией обмена с внешней памятью, решается введением в структуру ОБ регистра 3 (структуры 3 и 4). В этом случае для выполнения полного цикла с сохранением операции в блоке РОН требуется три такта.

Поясним особенности рассмотренных структур с помощью временных диаграмм.

На первом такте устанавливаются все управляющие сигналы. Через время, определяемое динамическими характеристиками мультиплексоров, на выходах мультиплексоров появляются актуальные значения операндов. Через время, определяемое динамическими характеристиками комбинационной схемы ОБ, на выходе ОБ структуры 1 появляется результат операции. На втором такте этот результат фиксируется в РОН и меняется состояние управляющих сигналов. Прежде всего это относится к сигналам, управляющим режимами элементов памяти. Структуры, содержащие регистры 1, 2, на втором такте фиксируют значения операндов в этих регистрах. Далее выполняется операция в ОБ. Структура второго типа фиксирует результат операции в РОН на третьем такте, а третьего и четвертого типов — на четвертом такте.

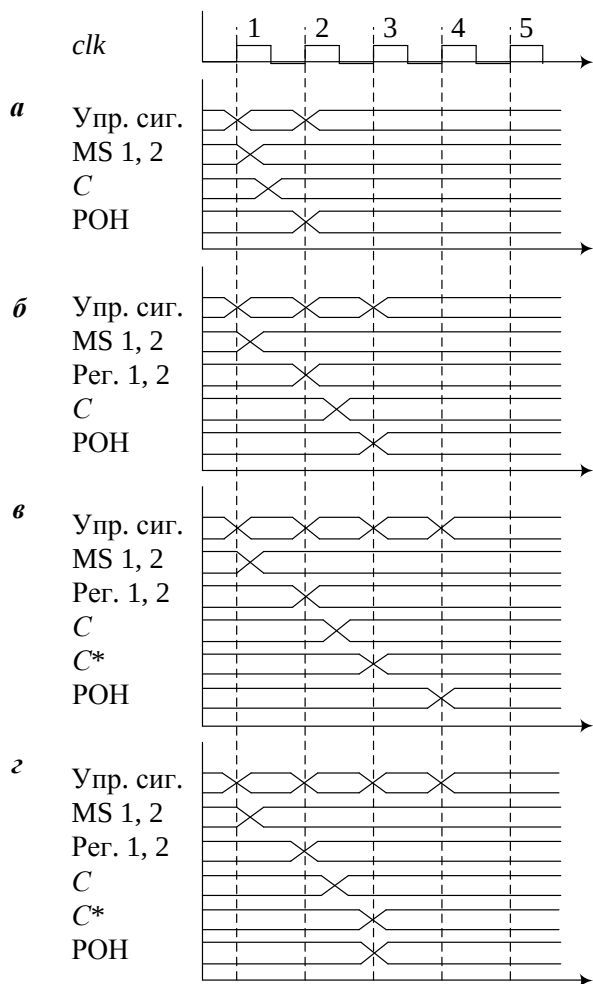


Рис. 1.4.4. Временные диаграммы, поясняющие работу четырех вариантов структуры операционных устройств: *а*, *б*, *в*, *г*, соответственно

Итак, проведенное сравнение вариантов структуры ОУ позволяет сделать следующие выводы:

- 1) структура 1 обеспечивает возможность реализации большого количества последовательных вычислений с малым количеством обращений к памяти;
- 2) структура 2 позволяет совмещать во времени операции чтения данных из внешней памяти в блок РОН с выполнением операций в АЛУ;
- 3) структура 3 позволяет совмещать во времени операции загрузки операндов из блока РОН в регистры 1, 2 с операцией записи результата в память;
- 4) структура 4 является универсальной и сочетает достоинства трех рассмотренных ранее структур. Платой за универсальность являются наибольшие аппаратные затраты. Данная структура обеспечивает параллельную работу как с внешней памятью, так и с блоком РОН.

Варианты организации операционного блока

Комбинационная схема ОБ может выполнять операции как последовательно, так и параллельно.

В ОБ с параллельной организацией все разряды операндов обрабатываются одновременно, как это было показано на примере АЛУ в предыдущих лекциях.

В ОБ с последовательной организацией (рис. 1.4.5) операции выполняются побитово, разряд за разрядом. Бит переноса, возникающий при обработке i -х разрядов операндов, подается на вход ОБ и учитывается при обработке $i+1$ -х разрядов. На каждой итерации осуществляется сдвиг операндов и результата в регистрах на один разряд.

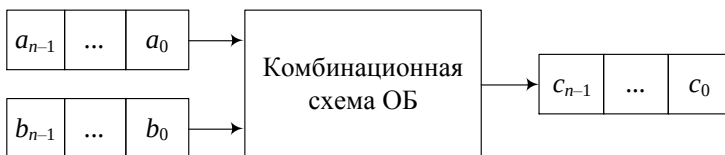


Рис. 1.4.5. ОБ с последовательной организацией

ОУ на основе АЛУ — регистровое арифметико-логическое устройство

На рис. 1.4.6 представлен пример структуры ОУ на основе рассмотренной ранее комбинационной схемы АЛУ. Подобная структура ОУ применяется в составе МПК КР580.

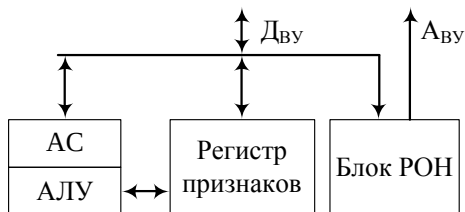


Рис. 1.4.6. Пример структуры РАЛУ

Данное ОУ построено на одной двунаправленной магистрали $D_{ВУ}$. Источником операндов для АЛУ является внешнее устройство либо блок РОН. Результаты вычислений по той же магистрали могут передаваться либо во внешние устройства, либо в РОН. Результат каждого выполненного действия записывается в специальный регистр — аккумулятор АС. В регистре признаков формируется двоичный код, характеризующий признаки результата выполненной операции. Информация блока РОН поступает на однонаправленную адресную магистраль $A_{ВУ}$ для передачи кода адреса внешнего устройства.

Особенность ОУ с одной магистралью данных — необходимость последовательного выполнения текущей микрокоманды. Каждая команда выполняется в течение определенного количества тактов в зависимости от ее сложности. Последовательное управление блоками ОУ осуществляется устройством управления.

Контрольные вопросы

1. Поясните механизм выполнения операций в операционных устройствах.
2. Какие вам известны варианты структуры ОБ?
3. В чем заключаются задачи управления и синхронизации?

4. Охарактеризуйте ОУ с жесткой структурой, приведите пример.
5. Охарактеризуйте ОУ с магистральной структурой, приведите пример.

Литература

1. Микропроцессоры. В 3-х кн. Кн. 1. Архитектура и проектирование микроЭВМ: учебник для вузов / ***Под ред. Л.Н. Преснухина.*** — М.: Высшая школа, 1986. — 495 с.
2. ***Угрюмов Е.П.*** Цифровая схемотехника: учеб. пособие для вузов. — 2-е изд., перераб. и доп. — СПб.: БХВ-Петербург, 2005. — 800 с.

Лекция 1.5. Разрядно-модульные и однокристалльные регистровые арифметико-логические устройства

По способу наращивания разрядности различают разрядно-модульные и однокристалльные РАЛУ. В разрядно-модульных РАЛУ разрядность наращивают аппаратно посредством добавления в структуру однотипных секций. В однокристалльных РАЛУ наращивание разрядности производят программным путем. Следствием этих отличий является более высокое быстродействие разрядно-модульных РАЛУ, поскольку они обеспечивают параллельное выполнение операций. Однако разрядно-модульные РАЛУ требуют больших аппаратных затрат.

Рассмотрим структурные и функциональные особенности РАЛУ обоих типов на примерах. Условимся, что все элементы приводимых далее структур — 4-разрядные.

Регистровое арифметико-логическое устройство разрядно-модульного типа

Основу структуры разрядно-модульного РАЛУ (рис. 1.5.1) составляет комбинационная схема АЛУ, на вход которой подаются два операнда, представляющие собой содержимое регистра 1 (Рг. 1) и регистра 2 (Рг. 2). Рг. 1 имеет два режима (записи и чтения) и способен принимать данные как из блока РОН, так и от внешних устройств (ВУ), что реализуется с помощью мультиплексора. Рг. 2 имеет четыре режима (записи, чтения, сдвига влево и сдвига вправо) и принимает данные только из блока РОН. Результат операции может быть записан в блок РОН и в регистр АС, с выхода которого информация попадает к внешним устройствам.

Определим состав и назначение управляющих сигналов для данной структуры РАЛУ и рассмотрим примеры наращивания разрядности и формирования микрокоманд и микропрограмм.

Сигналы w/r , v_0 , v_3 предназначены для управления режимами чтения/записи блока РОН, Рг. 1 и АС соответственно: 0 — чтение информации; 1 — запись.

Мультиплексор управляется сигналом А: при $A = 0$ на вход Рг. 1 подаются данные из блока РОН, при $A = 1$ — от внешних устройств.

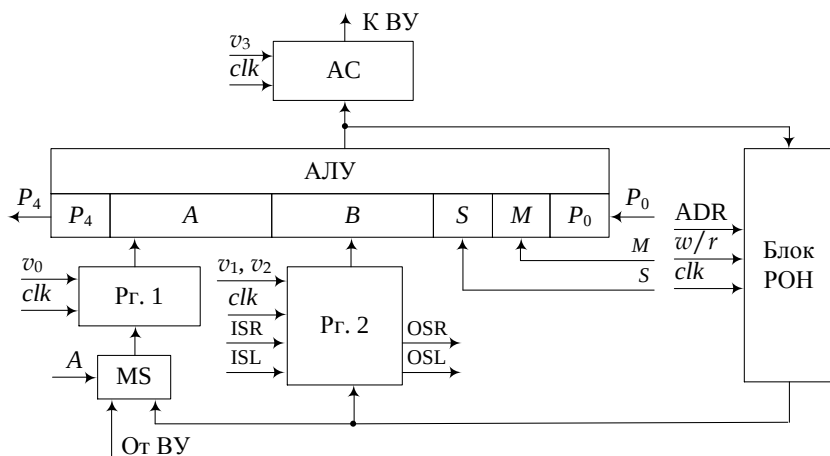


Рис. 1.5.1. Пример структуры разрядно-модульного РАЛУ

Блок РОН представляет собой синхронное ОЗУ статического типа, содержащее набор регистров. Для выбора нужного регистра используются линии адреса ADR.

Для выбора режима работы Рг.2 используется пара управляющих сигналов:

$$\{v_1, v_2\} = \begin{cases} 00 & \text{— чтение;} \\ 01 & \text{— сдвиг влево;} \\ 10 & \text{— сдвиг вправо;} \\ 11 & \text{— запись.} \end{cases}$$

При сдвигах применяются две пары сигналов:

ISR — значение разряда, подаваемого на вход регистра при сдвиге вправо;

ISL — значение разряда, подаваемого на вход регистра при сдвиге влево;

OSR — значение разряда, появляющееся на выходе регистра при сдвиге вправо;

OSL — значение разряда, появляющееся на выходе регистра при сдвиге влево.

При выполнении арифметических операций используются бит переноса из предыдущей секции РАЛУ P_0 , бит переноса в следующую секцию P_4 и подготовительные функции первого порядка D^1, F^1 .

Все описанные сигналы можно разделить на три категории: информационные, управляющие и сигналы синхронизации.

Информационные сигналы: шина от ВУ, P_0 , ISL, ISR (входные), шина к ВУ, P_4 , D^1, F^1 , OSR, OSL (выходные).

Сигналы управления: $v_0, v_1, v_2, v_3, w/r, \text{ADR}, M, S$.

Сигнал синхронизации: clk .

На рис. 1.5.2 приведен пример наращивания разрядности с помощью СУП (показаны только те сигналы, которые задействованы при увеличении разрядности). Управляющие и информационные сигналы поступают на все секции параллельно. Блок РОН, Рг. 1, Рг. 2, АС увеличивают разрядность пропорционально количеству использованных секций. При выполнении арифметических операций каждая секция РАЛУ формирует подготовительные функции первого порядка F^1, D^1 , которые поступают на входы СУП. Для организации операций сдвига требуются связи между соответствующими парами сигналов OSR, ISR и OSL, ISL.

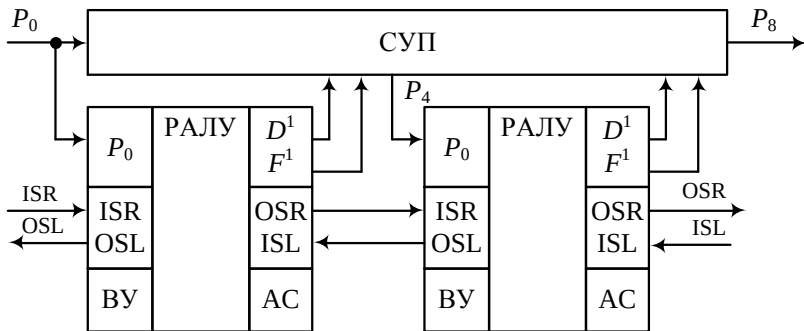


Рис. 1.5.2. Наращивание разрядности разрядно-модульного РАЛУ с помощью СУП

В табл. 1.5.1 представлены примеры формирования микрокоманд и микропрограмм для рассмотренной структуры РАЛУ.

Пример 1.5.1. Передача содержимого Рг.1 в РОН с адресом A_k . В этом случае АЛУ не выполняет преобразований данных,

Таблица 1.5.1
Примеры формирования микрокоманд и микропрограмм для разрядно-модульного РАЛУ

Пример	Операция	Обозначение	ADR	A	w/r	$S_3 \dots S_0$	M	v_0	v_1	v_2	v_3	P_0
1.5.1	Передача содержимого Рг. 1 в РОН с адресом A_k	$\text{Рг. 1} \rightarrow \text{РОН}(A_k)$	A_k	X	1	0000	0	0	0	0	0	X
1.5.2	Выполнение арифметических или логических операций над содержимым Рг. 1 и Рг. 2 с размещением результата в АС и РОН с адресом A_k	$\text{Рг. 1 } [S; M; P_0] \text{ Рг. 2} \rightarrow \text{РОН}(A_k), \text{ АС}$	A_k	X	1	$S_3 \dots S_0$	M	0	0	0	1	P_0
1.5.3	Передача информации от ВУ в Рг. 1	$\text{ВУ} \rightarrow \text{Рг. 1}$	X	1	0	XXXX	X	1	0	0	0	X
1.5.4	Запись информации из РОН с адресом A_k в Рг. 1 и Рг. 2	$\text{РОН}(A_k) \rightarrow \text{Рг. 1}, \text{Рг. 2}$	A_k	0	0	XXXX	X	1	1	1	0	X
1.5.5	Сложение содержимого РОН с адресами A_n и A_m с размещением результата в РОН с адресом A_k	$\text{РОН}(A_n) \rightarrow \text{Рг. 1}$ $\text{РОН}(A_m) \rightarrow \text{Рг. 2}$ $\text{Рг. 1} + \text{Рг. 2} \rightarrow \text{РОН}(A_k)$	A_n A_m A_k	0 X X	0 0 1	XXXX XXXX 1001	X X 1	1 0 1	0 1 0	0 1 0	0 0 0	X X P_0

вместо этого происходит трансляция операнда A на выход комбинационной схемы. Это действие обеспечивается выполнением логической операции, задаваемой управляющим сигналом $S_3 = 0, S_2 = 0, S_1 = 0, S_0 = 0$. Значение бита переноса из предыдущей секции не влияет на результат. Все регистры, не задействованные в данной операции, должны хранить свое текущее значение, поэтому $v_0 = v_1 = v_2 = v_3 = 0$. На управляющие входы блока РОН подаются адрес A_k и сигнал записи $w/r = 1$. Состояние сигнала, управляющего мультиплексором, не оказывает влияния на результат, поскольку для выполнения данной операции неважно входное значение Рг. 1.

Пример 1.5.2. Выполнение арифметических или логических операций над содержимым Рг. 1 и Рг. 2 с размещением результата в АС и РОН с адресом A_k . Рг. 1 и Рг. 2 находятся в режиме чтения $v_0 = v_1 = v_2 = 0$. Блок РОН и регистр АС находятся в режиме записи $w/r = v_3 = 1$. Запись в блок РОН осуществляется по адресу A_k , поэтому $ADR = A_k$. Сигналы S, M, P_0 задают тип выполняемой операции. Состояние сигнала, управляющего мультиплексором, не оказывает влияния на результат.

Пример 1.5.3. Передача информации от ВУ в Рг. 1. В этой операции активными являются Рг. 1 и мультиплексор: $v_0 = 1, A = 1$. Сигналы управления АЛУ не влияют на результат. Все элементы памяти, за исключением Рг. 1, находятся в режиме чтения $w/r = v_1 = v_2 = v_3 = 0$.

Пример 1.5.4. Запись информации из РОН с адресом A_k в Рг. 1 и Рг. 2. Эта операция аналогична предыдущей, за исключением режима мультиплексора и Рг. 2: $A = 0, v_1 = v_2 = 1$. Данные из РОН выбираются по адресу A_k , поэтому $ADR = A_k$.

Пример 1.5.5. Сложение содержимого РОН с адресами A_n и A_m с размещением результата в РОН с адресом A_k . Для выполнения этой операции необходимо составить микропрограмму из трех микрокоманд: размещение первого операнда в Рг. 1; размещение второго операнда в Рг. 2; выполнение операции сложения с сохранением результата в блоке РОН.

Анализ приведенных примеров показывает, что при составлении микрокоманд и микропрограмм необходимо следовать основному правилу: содержимое регистров, не участвующих в выполнении микрокоманды, должно быть сохранено или восстановлено без искажения данных.

Регистровое арифметико-логическое устройство однокристалльного типа

Наращивание разрядности однокристалльного РАЛУ производится только программным способом, для чего необходима определенная аппаратная поддержка. Поясним это на примере сложения двух 8-разрядных чисел с помощью 4-разрядного однокристалльного РАЛУ (рис. 1.5.3). Исходные операнды представляются в виде двух пар: 4-разрядных младших и старших частей чисел, то есть каждое число занимает две ячейки РОН. Сначала выполняется сложение младших частей чисел с сохранением результата и значения бита переноса. Затем выполняется сложение старших частей чисел с учетом сохраненного значения бита переноса.

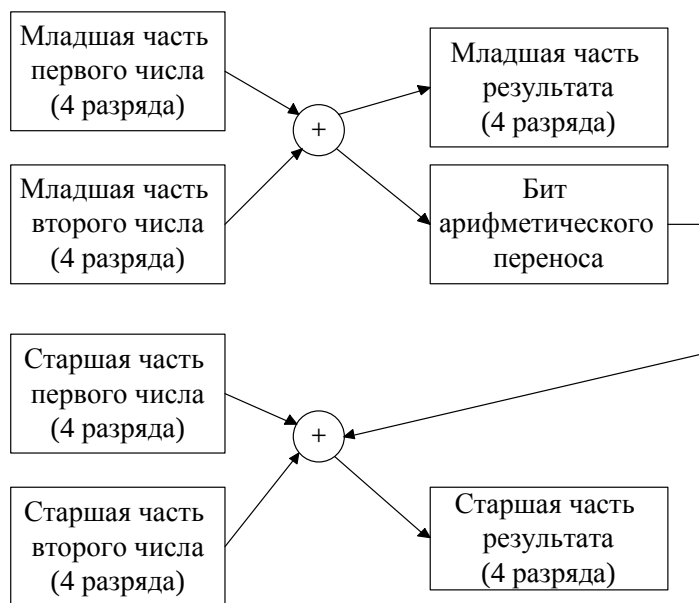


Рис. 1.5.3. Нарастивание разрядности однокристалльного РАЛУ программным способом

Из приведенного примера видно, что для поддержки арифметических операций над числами с расширенной разрядностью в структуру

РАЛУ следует вводить элемент памяти для сохранения значения бита переноса.

Аналогичные рассуждения приводят нас к выводу о необходимости вводить элемент памяти и для сохранения битов сдвига. Помимо ввода элементов памяти нужна организация мультиплексирования сигналов ранее сохраненного арифметического переноса и значения P_0 , подаваемого извне.

На рис. 1.5.4 представлен пример структуры однокристалльного РАЛУ. По сравнению с разрядно-модульным РАЛУ в данной структуре появились дополнительные элементы, цепи и управляющие сигналы.

Цепи переноса включают триггер $T1$ и логические элементы 1, 2. Сигнал P обеспечивает фиксацию значения разряда переноса в триггере $T1$.

Цепи сдвига включают триггер $T2$ и логические элементы 3, 4, 5. Сигнал L_2 реализует выбор сдвигаемого разряда (сдвиг влево, или сдвиг вправо). Сигнал L_3 фиксирует сдвигаемый разряд в триггере $T2$. Сигнал L_4 выбирает источник сдвига (внутренний или поданный извне).

В табл. 1.5.2 приведены примеры формирования микрокоманд и микропрограмм для однокристалльного РАЛУ.

Пример 1.5.6. Сдвиг влево содержимого Рг.2 на один разряд с запоминанием результата сдвига и использование ранее запомненного разряда. При выполнении данной операции активными элементами являются Рг.2, триггер 2 и логические вентили 3, 4, 5: $v_1 = 0$, $v_2 = 1$, $L_2 = L_3 = L_4 = 1$. Элементы памяти, не задействованные в выполнении операции, находятся в режиме чтения.

Пример 1.5.7. Сложение двух операндов A и B , младшие части которых находятся в РОН с адресами A_1 и A_2 , а старшие части — в РОН с адресами A_3 и A_4 соответственно, с размещением результата в РОН A_1 и A_2 . Данная операция требует составления микропрограммы, реализующей следующую последовательность действий: размещение младших частей операндов в Рг.1 и Рг.2; сложение содержимого Рг.1 и Рг.2 с сохранением результата в РОН с сохранением значения бита переноса в триггере $T1$; размещение старших частей операндов в Рг.1 и Рг.2; сложение содержимого Рг.1 и Рг.2 с использованием ранее сохраненного значения бита переноса и сохранением результата в РОН.

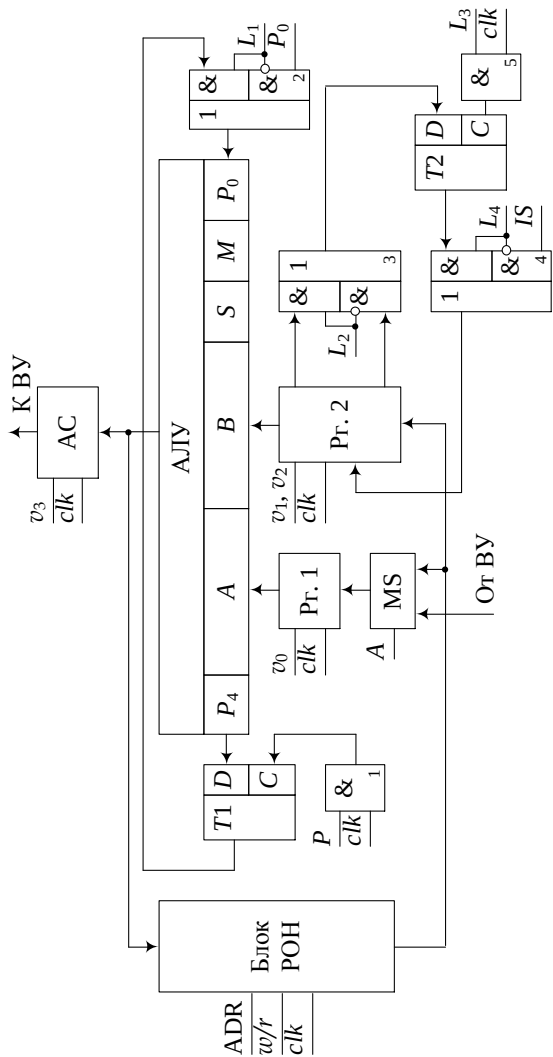


Рис. 1.5.4. Пример структуры однокристального РАЛУ

Таблица 1.5.2
Примеры формирования микрокоманд и микропрограмм для однокристалльного РАЛУ

Пример	Операция	Обозначение	$S_3 \dots S_0$	M	P_0	P	A	ADR	w/r	v_0	v_1	v_2	v_3	L_1	L_2	L_3	L_4
1.5.6	Сдвиг влево содержимого Рг. 2 на один разряд с запоминанием результата сдвига и использование ранее за- помненного разряда	$(\text{Рг. 2})^3 \rightarrow {}^3(\text{Рг. 2})$	XXXX	X	X	0	X	X	0	0	0	1	0	X	1	1	1
1.5.7	Сложение двух операндов A и B , младшие части которых находятся в РОН с адресом A_1 и РОН с адресом A_2 , а старшие части — в РОН с адресом A_3 и РОН с адресом A_4 соответственно, с размещением результата в РОН A_1 и РОН A_2	$\text{РОН}(A_1) \rightarrow \text{Рг. 1}$ $\text{РОН}(A_2) \rightarrow \text{Рг. 2}$ $\text{Рг. 1} + \text{Рг. 2} \rightarrow \text{РОН}(A_1)$ $\text{РОН}(A_3) \rightarrow \text{Рг. 1}$ $\text{РОН}(A_4) \rightarrow \text{Рг. 2}$ $\text{Рг. 1} + \text{Рг. 2} \rightarrow \text{РОН}(A_2)$	XXXX XXXX 1001 XXXX XXXX 1001	X X 1 X X 1	X X P_0 X X X	0 0 1 0 0 1	A_1 A_2 A_1 A_3 A_4 A_2	0 0 1 0 0 1	0 1 0 0 0 0	0 0 0 0 0 0	0 1 0 0 1 0	0 0 0 0 0 0	0 0 0 0 0 1	0 0 0 0 0 0	0 0 0 0 0 0	0 0 0 0 0 0	

Контрольные вопросы

1. Охарактеризуйте РАЛУ с однокристалльной организацией.
2. Охарактеризуйте РАЛУ с разрядно-модульной организацией.
3. Составьте последовательность микроопераций для выполнения арифметической операции вычитания двух восьмиразрядных слов при помощи четырехразрядного однокристалльного РАЛУ.
4. Приведите пример структуры РАЛУ, укажите ее достоинства и недостатки.

Литература

1. Микропроцессоры. В 3-х кн. Кн. 1. Архитектура и проектирование микроЭВМ: учебник для вузов / *Под ред. Л.Н. Преснухина*. — М.: Высшая школа, 1986. — 495 с.
2. *Угрюмов Е.П.* Цифровая схемотехника: учеб. пособие для вузов. — 2-е изд., перераб. и доп. — СПб.: БХВ-Петербург, 2005. — 800 с.

Раздел 2. Устройства управления.

Конвейерный принцип выполнения команд.

Основные режимы функционирования микропроцессора

Лекция 2.1. Устройство управления: структура, способы формирования управляющих сигналов и адресации микрокоманд

Устройство управления в том или ином виде присутствует в любом современном микропроцессоре, микроконтроллере и даже в графических процессорах видеокарт. Устройство управления может быть распределенным, но, тем не менее, четко логически различимым, с определенным набором функций. Из обобщенной структуры МП, представленной на рис. 1.2.1, видно, что УУ имеет связи со всеми основными блоками, т.е. занимает значимое место в этой структуре.

Впервые УУ было выделено в отдельный функциональный блок в 1946 году при описании архитектуры фон Неймана. С тех пор его роль и важность в процессорных системах не изменились.

Устройство управления — это законченный цифровой автомат, организующий выборку команд программы и обеспечивающий выполнение последовательности микроопераций в соответствии с кодом выполняемой команды. Напомним, что *микрооперацией* называют элементарное преобразование данных, не разложимое на более простые и выполняющееся в течение одного такта сигналов синхронизации.

Иногда говорят, что устройство управления является «мозгом мозга», потому что оно управляет процессором, который в свою очередь управляет всей вычислительной системой в целом.

В современных микропроцессорах принято выделять следующие основные функции УУ:

- синхронизация и координирование всех исполнительных устройств микропроцессора;
- выборка и декодирование команд из памяти;
- управление исполнительными блоками;
- управление ресурсами вычислительной системы;
- распределение потоков данных между процессором и внешними устройствами.

УУ устанавливает однозначное соответствие между множеством управляющих кодов и внешних условий $\{Q, U\}$ и множеством управляющих сигналов Y . Взаимодействие УУ с остальными блоками микропроцессорной системы представлено на рис. 2.1.1.

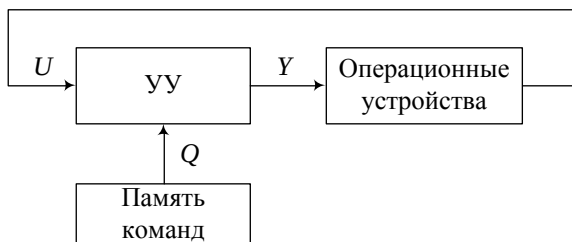


Рис. 2.1.1. Взаимодействие УУ с блоками микропроцессорной системы

Сформулируем главные требования, предъявляемые к УУ:

- УУ должно формировать полную совокупность управляющих сигналов Y ;
- число состояний автомата, моделирующего работу УУ, должно быть не меньше числа команд, подлежащих реализации;
- должна быть обеспечена возможность произвольной выборки команд в зависимости от управляющих кодов Q ;
- должна быть обеспечена возможность модификации последовательности команд в зависимости от внешних условий U .

Существуют два способа реализации устройств управления:

- на жесткой логике (hardwired control unit);
- с микропрограммным управлением (microprogrammed control unit).

Устройство управления на жесткой логике — это цифровой автомат, закон функционирования которого определяется коммутацией внутренних цепей, условия перехода из одного состояния в другое

реализуются с помощью КС, а состояния автомата хранятся в регистрах. Термин «жесткая логика» означает, что поведение УУ не может быть изменено без физической модификации схемы устройства, будь то печатная плата или микросхема. Это делает невозможным быстрое устранение найденных ошибок, а также добавление новых или изменение имеющихся функций процессора.

Рассмотрим пример реализации устройства управления на жесткой логике (рис. 2.1.2). Результатом работы УУ являются сигналы Y , каждый из которых есть функция от дешифрованного кода команды Q , флагов состояний процессора U и метки времени T :

$$Y = f(Q, U, T).$$

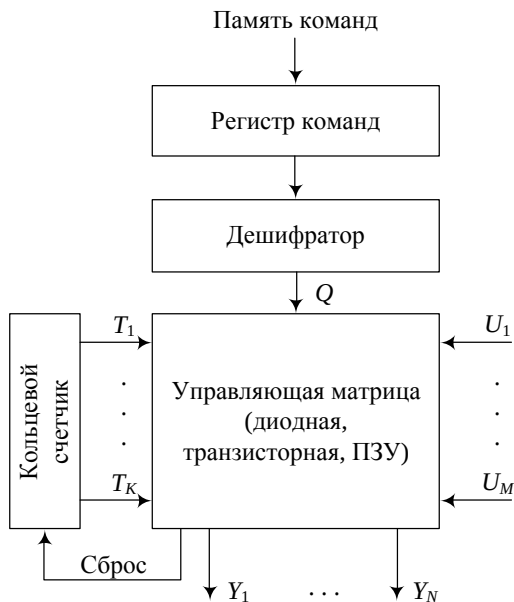


Рис. 2.1.2. Пример схемы УУ на жесткой логике

Текущая выполняемая команда поступает из памяти команд в регистр команд, откуда выбирается код операции и подается на дешифратор. Управляющая матрица объединяет все входящие сигналы и

выдает управляющие сигналы Y . Применение диодной, транзисторной матрицы или ПЗУ позволяет сделать УУ более регулярным, упростить топологию и уменьшить время разработки процессора в целом.

В общем случае управляющая память в устройстве управления может быть реализована с помощью программируемой логической матрицы (ПЛМ), ПЗУ или даже ОЗУ. В современных процессорах общего назначения фирм Intel и AMD память микропрограмм можно изменять с помощью специального программного обеспечения. Это позволяет исправлять технические ошибки в процессорах без замены всего устройства.

До конца 1970-х годов подавляющее большинство программ писались на ассемблере. При этом вся тяжесть управления процессором ложилась на программиста, что, в свою очередь, повышало требования к его квалификации и увеличивало время разработки программы. Для облегчения процесса написания программ производители МП вводили в них все более сложные команды, такие как, например, обработка текстовой строки одной инструкцией. Очевидно, что эти команды увеличивали сложность УУ, которое могло занимать до 70% площади всего кристалла процессора. В качестве примера можно привести компьютеры IBM System/360 и Digital Equipment Corporation VAX, обладающие весьма развитым набором сложных машинных команд. Вскоре поддерживать все эти команды стало трудно, и разработчикам пришла мысль заменить жесткую логику микропрограммированием.

Впервые конструкция УУ с микропрограммным управлением была предложена американским ученым Моррисом Вилкесом в 1951 году. Идея состояла в том, что каждой инструкции машинного кода соответствует некоторый набор микрокоманд, которые хранятся в специальной памяти, называемой *управляющей памятью*. Таким образом, *устройство управления с микропрограммированием* — это цифровой автомат, закон функционирования которого определяется управляющей памятью.

Микропрограммирование имеет следующие преимущества перед жесткой логикой:

- простота разработки УУ;
- возможность построения сколь угодно сложных команд;
- меньшее количество логических вентилях, а следовательно, уменьшение площади кристалла и тепловыделения;

- возможность быстрого исправления ошибок проектирования процессора: нет необходимости в физической модификации микросхемы, достаточно запрограммировать в управляющую память новый набор микрокоманд;

- возможность специализации МП под конкретную задачу.

Микропрограмма состоит из последовательности микроинструкций. Каждая инструкция формирует набор низкоуровневых управляющих сигналов. Например, типовая операция сложения двух операндов может быть представлена следующим набором микроопераций:

- 1) перенести в регистр *A* РАЛУ значение из РОН 1;
- 2) перенести в регистр *B* РАЛУ значение из РОН 4;
- 3) выполнить операцию сложения в РАЛУ;
- 4) сохранить результат РАЛУ в РОН 2;
- 5) сохранить флаги состояния РАЛУ (нулевой результат, переполнение и т. д.) в регистре состояния;
- 6) перейти к следующей машинной команде.

Упрощенная схема устройства микропрограммного управления (УМУ) представлена на рис. 2.1.3.

Устройство формирования адреса (УФА) вырабатывает адрес следующей микрокоманды в зависимости от внешних сигналов, поступающих от ОУ и от текущей микрокоманды. Код микрокоманды из управляющей памяти подается в регистр микрокоманд. Регистр состоит из двух частей: операционной, в которой содержатся управляющие сигналы, и адресной, которая поступает в УФА для управления загрузкой следующей микрокоманды (рис. 2.1.4).

УМУ можно классифицировать по нескольким признакам. Один из них — по способу представления операционной части регистра микрокоманд. По этому признаку различают УМУ с горизонтальным, вертикальным и смешанным микропрограммированием.

При *горизонтальном микропрограммировании* каждому разряду операционной части регистра микрокоманд соответствует один определенный управляющий сигнал из набора Y (рис. 2.1.5). Если N — количество управляющих сигналов, которое необходимо реализовать, то разрядность операционной части регистра микрокоманд будет также равна N . Для этого типа микропрограммирования характерна очень большая разрядность управляющей памяти (более сотни битов) и, соответственно, большой объем микрокода. В то же время такая реализация проще со схемотехнической точки зрения.

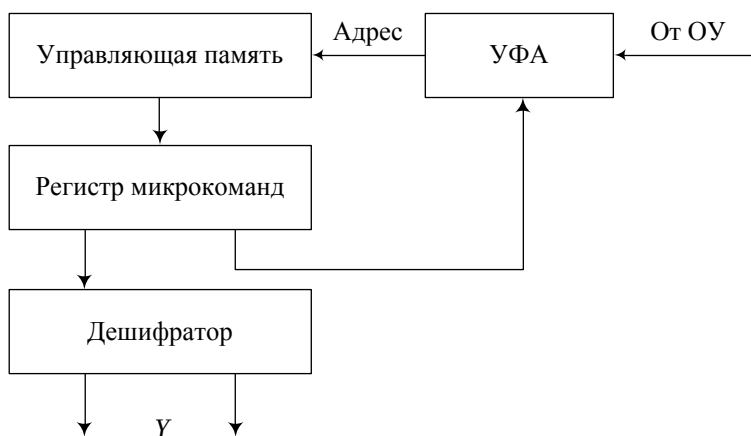


Рис. 2.1.3. Упрощенная схема устройства микропрограммного управления

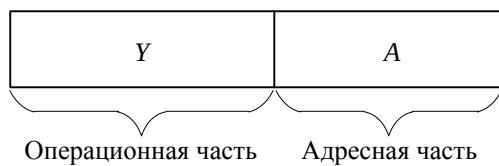


Рис. 2.1.4. Регистр микрокоманд

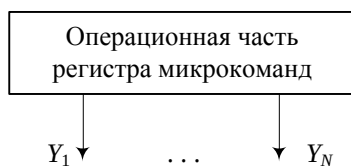


Рис. 2.1.5. Горизонтальное микропрограммирование

При *вертикальном микропрограммировании* операционная часть регистра микрокоманд хранится в памяти в закодированном виде и подвергается дешифрации перед подачей в операционные устройства МП (рис. 2.1.6). Достоинством этого типа микропрограммирования является уменьшение требований к объему памяти для микрокода. При этом сложность микропроцессора увеличивается, а быстродействие уменьшается в связи с появлением дополнительных комбинационных схем — дешифраторов.

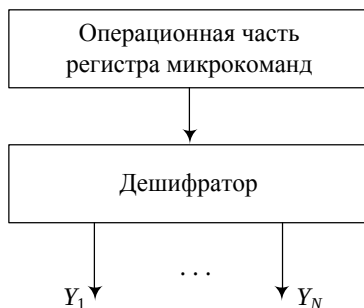


Рис. 2.1.6. Вертикальное микропрограммирование

Смешанное микропрограммирование сочетает достоинства горизонтального и вертикального. При этом набор управляющих сигналов Y разбивается на несколько поднаборов. Каждый поднабор управляет отдельным ОУ либо устройствами, которые совместно используются редко. Далее на каждый поднабор устанавливается свой дешифратор (рис. 2.1.7). Как результат, вследствие некоторого допустимого усложнения схемы УУ уменьшается объем микрокода.

Развитие технологий производства микросхем, удешевление изготовления внутренней памяти, а также необходимость увеличения частот МП привели к тому, что в настоящее время вертикальное микропрограммирование практически вытеснено горизонтальным и смешанным микропрограммированием.

Другой распространенный признак классификации УМУ — по способу формирования адреса следующей микрокоманды. Различают УМУ с принудительной адресацией и с естественной.

В УМУ с *принудительной адресацией* (рис. 2.1.8) каждая микрокоманда содержит адресную часть, которая управляет порядком

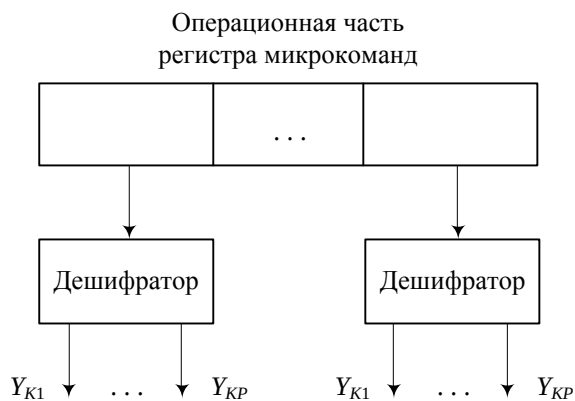


Рис. 2.1.7. Смешанное микропрограммирование

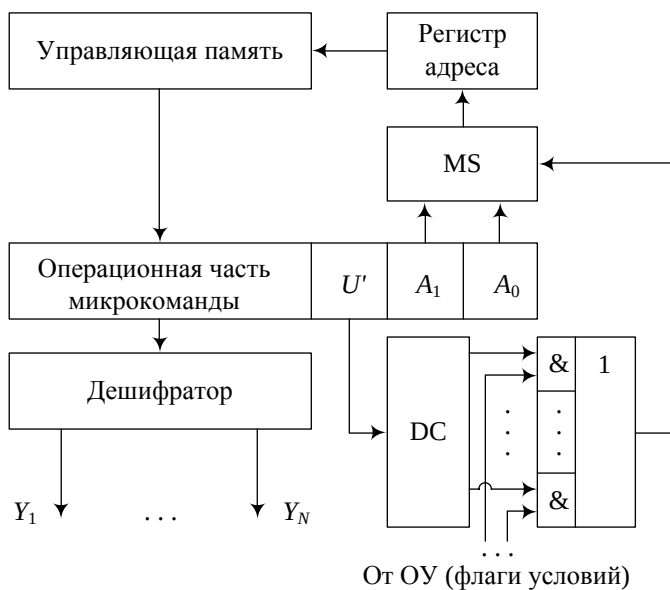


Рис. 2.1.8. Схема УМУ с принудительной адресацией

следования микрокоманд. Адресная часть в свою очередь содержит поле логических условий U' , служащее маской для набора внешних логических условий U , и два поля адреса A_0 и A_1 следующей микрокоманды. Поле U' совместно с флагами от операционных устройств управляет мультиплексором выбора адреса следующей микрокоманды. На мультиплексор будет подана единица, только если в позиции соответствующих коду условий будет единица, и при этом условия были выполнены. В таком случае выполняется ветвление, и в регистр адреса будет загружен A_1 . Если условия не выполнены, то ветвления нет, и используется поле A_0 .

В УМУ с естественной адресацией (рис. 2.1.9) применяется инкрементирующий счетчик адреса следующей микрокоманды. Если содержащийся в микрокоманде признак P равен нулю, то линейный порядок следования микрокоманд не изменяется и адрес просто инкрементируется в счетчике. Если P равен единице, то операционная часть микрокоманды выступает в роли нового адреса, который загружается в счетчик. Таким образом реализуется механизм ветвлений.

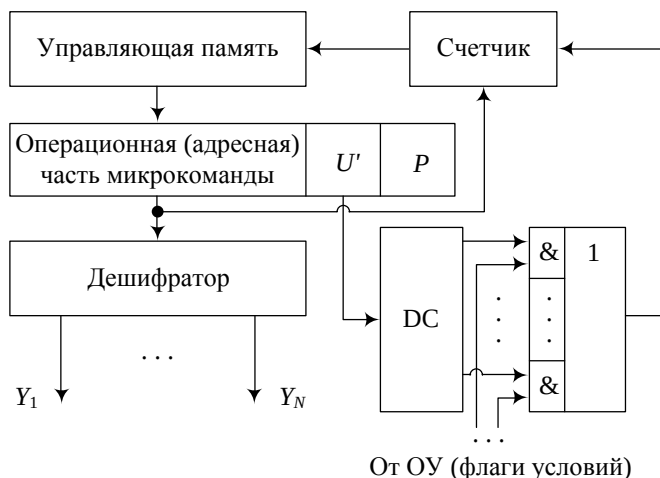


Рис. 2.1.9. Схема УМУ с естественной адресацией

С появлением языков программирования высокого уровня, таких как С, С++ и др., проблема сложности системы команд процессо-

ра утратила свою остроту, поскольку теперь компилятор занимается непосредственной генерацией машинного кода. В связи с этим команды процессоров стали упрощать и УУ на жесткой логике снова оказались востребованными. На сегодняшний день большинство специализированных микропроцессоров, сигнальных процессоров, микроконтроллеров и синтезируемых процессоров используют УУ на жесткой логике. В то же время высокопроизводительные процессоры, в том числе процессоры общего назначения, с большим количеством сложных команд применяют УУ с микропрограммным управлением.

Контрольные вопросы

1. Дайте определение устройства управления.
2. Какие бывают устройства управления?
3. Перечислите достоинства и недостатки УУ с микропрограммным управлением.
4. Расскажите о вертикальном, горизонтальном и смешанном микропрограммировании.
5. Какие функции выполняет устройство формирования адреса?

Литература

1. Микропроцессоры. В 3-х кн. Кн. 1. Архитектура и проектирование микроЭВМ: учебник для вузов / **Под ред. Л.Н. Преснухина**. — М.: Высшая школа, 1986. — 495 с.
2. Микропроцессорные системы: учеб. пособие для вузов / **Е.К. Александров, Р.И. Грушвицкий, М.С. Куприянов и др.; под ред. Д.В. Пузанкова**. — СПб.: Политехника, 2002. — 935 с.

Лекция 2.2. Система команд и способы адресации операндов. Конвейерный принцип выполнения команд

Система команд и способы адресации операндов

Система команд (instruction set) — это совокупность команд, или инструкций, выполнение которых на аппаратном уровне поддерживает процессор. Понятие «система команд» не следует путать с более широким понятием «архитектура системы команд». *Архитектура системы команд* — это не только система команд, но и средства для их выполнения, такие как форматы данных, системы регистров, способы адресации, модели памяти.

Отметим, что различные процессоры могут использовать одну систему команд, но при этом иметь разное внутреннее устройство. В качестве примера можно привести МП общего назначения фирм Intel и AMD. Программы, разработанные для одного из них, могут быть выполнены и на другом, несмотря на то, что оба процессора созданы независимыми группами людей, имеют разные архитектуры и наборы технических решений.

В настоящее время существует множество систем команд, отличающихся по сложности и функциональным возможностям. Некоторые системы команд известны уже не один десяток лет и постоянно расширяются. Проследим историю наиболее популярной из них — системы команд x86 на примере процессоров фирмы Intel:

- 8086 — 16-разрядные регистры и подсистема памяти, 80 команд (1978 г.);
- 80186 — добавлены 7 команд (1982 г.);
- 80286 — добавлены 17 команд (1982 г.);
- 80386 — регистры и подсистема памяти расширены до 32 разрядов, введены 33 новые команды (1985 г.);
- 80486 — добавлены 6 команд, а также команды для работы с блоком вычислений с плавающей точкой (1989 г.);
- Pentium — добавлены 6 команд, всего их более 150 (1993 г.);

- x86-64 — регистры и подсистема памяти расширены до 64 разрядов, введены 14 новых команд (2003 г.).

Команды, входящие в систему команд, можно классифицировать по нескольким признакам (рис. 2.2.1): по функциональному назначению; числу адресов, к которым обращается команда; способу адресации.

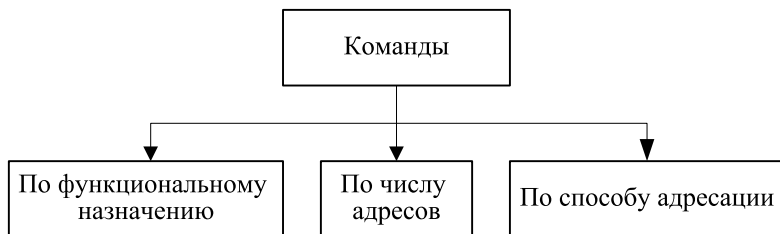


Рис. 2.2.1. Классификация команд процессора

По *функциональному назначению* команды разделяются на следующие основные группы операций:

- пересылки;
- арифметические;
- логические;
- сдвига;
- сравнения;
- управления программой;
- управления процессором.

Обычно вместо машинных кодов используются близкие к ним машинно-ориентированные языки — ассемблеры, которые позволяют применять mnemonic представление команд. При дальнейшем рассмотрении будем использовать mnemonic ассемблерные команды условного процессора, не привязываясь к конкретной системе команд. Тем не менее, нижеприведенные обозначения широко распространены и встречаются в составе ассемблеров многих современных процессоров.

Операции пересылки обеспечивают передачу данных между двумя регистрами или регистром и ячейкой памяти. Некоторые процессоры предполагают использование отдельных команд для общения

с портами ввода/вывода, пересылки между ячейками памяти и групповой пересылки, когда содержимое нескольких регистров пересылается в память или загружается из нее. Формат команд этой группы: MOV <приемник>, <источник> — пересылка между регистрами; LD — загрузка данных из памяти в регистр; ST — сохранение данных из регистра в память.

Арифметические операции включают такие команды, как сложение, вычитание, умножение и деление. Различают команды, которые выполняют операции непосредственно с операндами, и команды, которые учитывают сформированный предыдущими командами бит переноса. Команды, учитывающие бит переноса, используются при обработке слов с разрядностью, превышающей разрядность процессора. Так, в простейших системах операции умножения и деления не входят в систему команд процессора и реализуются программно. При умножении разрядность произведения в два раза больше разрядности операндов, поэтому для хранения результата используются два регистра (младшая и старшая части произведения). При делении делимое тоже имеет удвоенную разрядность и размещается в двух регистрах. В этом случае в качестве результата в двух регистрах сохраняются частное и остаток. Формат арифметических команд: ADD <операнд 1/приемник результата>, <операнд 2>, SUB, MUL, DIV — сложение, вычитание, умножение и деление соответственно.

Логические операции включают операции И (AND), ИЛИ (OR), НЕ (NOT), Исключающее ИЛИ (XOR). Логические операции выполняются поразрядно над содержимым двух регистров, регистра и ячейки памяти или с использованием операнда, который размещен в самой команде. Для этого применяют двухадресные команды AND (OR, XOR) <операнд 1/приемник результата>, <операнд 2>. Одноадресная команда NOT инвертирует каждый разряд операнда.

Микропроцессоры могут включать в свою систему команд логические, арифметические и циклические сдвиги адресуемых операндов на один или несколько разрядов (рис. 2.2.2). Сдвигаемый операнд может находиться в регистре или ячейке памяти, а число разрядов, на которое сдвигается операнд может либо задаваться с помощью операнда, содержащегося в команде при непосредственной адресации, либо определяться содержимым заданного регистра *C*. Мнемоника таких команд выглядит следующим образом:

- SHL (SHR) <операнд/приемник результата> — логический сдвиг влево (вправо);
- SAL (SAR) — арифметический сдвиг влево (вправо);
- ROL (ROR) — циклический сдвиг влево (вправо);
- RCL (RCR) — сдвиг влево (вправо) на заданное число разрядов.

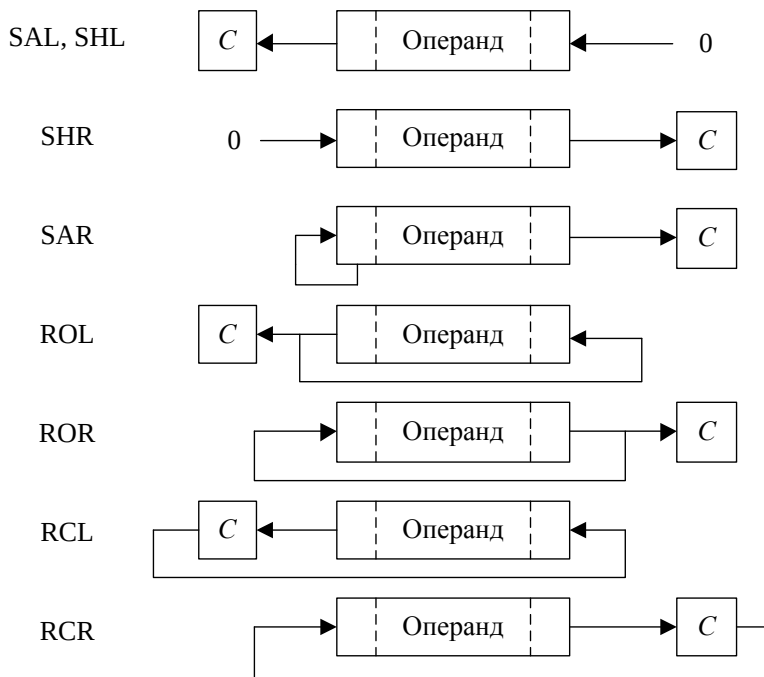


Рис. 2.2.2. Реализация команд сдвига

Операции сравнения в общем виде можно представить следующим образом: CMP <операнд 1>, <операнд 2>. Для выполнения данной команды производится вычитание двух указанных операндов, при этом результат вычитания не сохраняется, операнды не изменяются, а результатом операции является изменение содержимого регистра состояний процессора.

Операции управления программой делятся на команды безусловной передачи управления (CALL — вызов подпрограммы, JMP — переход по указанному адресу), команды возврата (RET — возврат из подпрограммы, IRET — возврат из прерывания) и команды условного перехода (JCC), которые выполняются в зависимости от состояния флагов (нулевого результата, переноса и других) процессора.

Операции управления процессором включают команды:

- HALT — остановка выполнения текущей программы;
- RST (reset) — сброс всех регистров, в том числе регистров состояния процессора;
- NOP (no operation) — нет операции, цикл ожидания.

Отметим, что при проектировании процессора и выборе набора команд для него чаще всего применяют проблемно-ориентированный подход. Для универсальных процессоров используют большой набор команд. В набор команд специализированных процессоров, например обработки сигналов, включают команду умножения с накоплением, которая выполняется за один такт и позволяет повысить производительность процессора в задачах фильтрации и обработки сигналов. Недорогие микроконтроллеры содержат минимальный набор необходимых инструкций (что упрощает их разработку), дающий возможность, однако, реализовать сложные функции через последовательность простых. Конечно, при этом приходится жертвовать быстродействием, но для ряда задач важно минимальное энергопотребление в ущерб производительности.

В соответствии с концепцией проблемно-ориентированного подхода широкое распространение сейчас получили «проблемно-ориентированные процессоры» — компоненты систем на кристалле, спроектированные для выполнения специфических операций (например, векторные сопроцессоры, процессорные ядра цифровой обработки сигналов), позволяющие достичь компромисса между универсальностью МП общего назначения и производительностью специализированной заказной микросхемы.

Команда микропроцессора состоит из двух полей (рис. 2.2.3): поля кода операции (КОП) и поля адреса, которое может включать несколько кодов адреса (КАД).

КОП определяет действие, выполняемое процессором, а КАД — операнды. КАД может отсутствовать, если команда является безадресной, например, команды проверки флагов процессора или управ-

Поле КОП	КАД 1	КАД 2	...	КАД n
	Поле адреса			

Рис. 2.2.3. Формат команды микропроцессора

ляющие его работой. Команды, работающие с данными (пересылки, арифметические, логические и сдвига), могут иметь один и более операндов и, соответственно, полей КАД (см. рис. 2.2.3). Так, операция побитовой инверсии является одноадресной, команды сложения и вычитания — двухадресными. В более сложных системах команд встречаются трехадресные инструкции, например, чтение из регистров двух операндов, их сложение и размещение результата в третьем ($C = A + B$) или инструкция умножения с накоплением ($C = A \cdot B + C$).

Различают шесть основных способов адресации операндов:

- непосредственная;
- прямая;
- регистровая;
- косвенно-регистровая;
- косвенно-регистровая со смещением;
- относительная.

При *непосредственной* адресации операнд содержится в поступившей команде и используется для введения различных констант. Такой способ вызывает увеличение размера команды на число байтов заданного операнда.

Разновидностью такой адресации является адресация, при которой операнд подразумевается, а не указывается явным образом. Примером может служить операция сравнения с нулем, в которой нуль не задается явным образом, но подразумевается.

При *прямой* адресации операнд выбирается из ячейки памяти, адрес которой указан в команде (рис. 2.2.4). Достоинство данного способа — простота, недостаток — большая разрядность адресного поля, зависящая от размера адресного пространства.

При *регистровой* адресации операнд выбирают из ячейки регистрового запоминающего устройства (РЗУ), номер (или имя) которой указан в команде (рис. 2.2.5). Регистровая адресация отличается

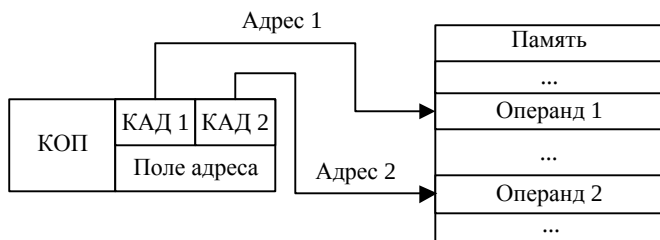


Рис. 2.2.4. Прямая адресация операндов

простотой и быстродействием и весьма распространена. Такой способ адресации не требует обращения к внешней памяти из-за того, что РЗУ находится на кристалле процессора. Как следствие, выполнение операций при регистровой адресации занимает минимальное время, однако необходимо периодическое обращение к памяти для сохранения результатов и записи операндов в РЗУ, что несколько уменьшает выгоду от использования такого способа адресации. Дешифрация адреса также относительно проста: объем РЗУ ограничен, а для задания номера регистра требуется всего несколько битов (3 – 8).

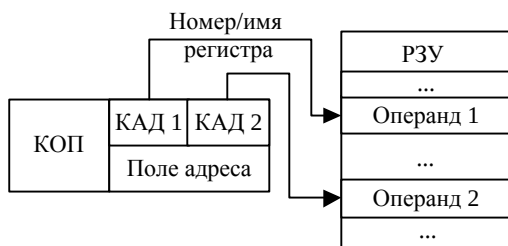


Рис. 2.2.5. Регистровая адресация операндов

При *косвенно-регистровой* адресации операнд выбирается из ячейки памяти, адрес которой содержится в регистре, указанном в команде (рис. 2.2.6).

При *косвенно-регистровой адресации со смещением* операнд выбирают из ячейки памяти, адрес которой является суммой содержимого регистра, адрес которого указан в команде, и операнда смещения (смещение может быть положительным или отрицательным).

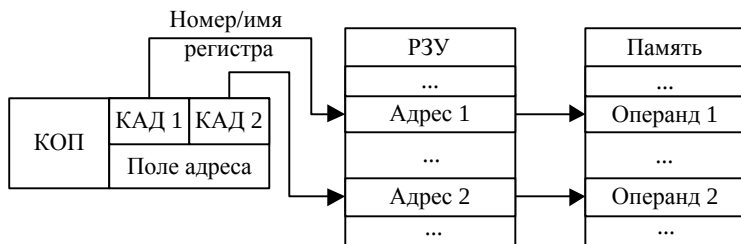


Рис. 2.2.6. Косвенно-регистровая адресация операндов

Разновидностями косвенно-регистровой адресации со смещением являются автоинкрементная и автодекрементная адресации. Как и при косвенно-регистровой адресации, адрес ячейки операнда хранится в регистре, но при каждом обращении к регистру его значение увеличивается или уменьшается на единицу. Эти способы адресации удобны при обработке массивов данных в цикле, поскольку нет необходимости каждый раз загружать адрес обрабатываемого значения массива в регистр.

При *относительной* адресации операнд выбирается из ячейки памяти, адрес которой является суммой текущего содержимого программного счетчика и заданного в команде смещения — базового адреса (рис. 2.2.7). В большинстве случаев такой способ адресации используется не для адресации операнда, а для формирования адреса команды, к которой переходит программа при выполнении команд ветвления. Сформированный адрес загружается в счетчик программ, обеспечивая выборку требуемой команды. Достоинством данного способа является то, что адресное поле команды может быть меньше шины адреса процессора.

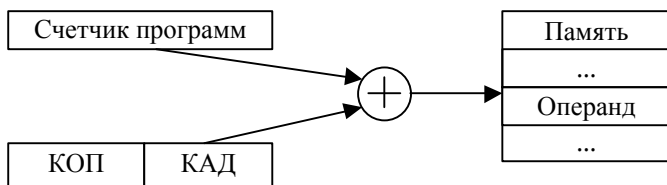


Рис. 2.2.7. Относительная адресация операндов

Отметим, что наиболее распространенными являются различные варианты косвенно-регистровой адресации. При этом в команде указывается только номер регистра, используемого в качестве адресного, следовательно, размер команды оказывается небольшим (как при регистровой адресации). Однако выборка операнда из ОЗУ требует выполнения циклов передачи данных по системной шине, что снижает производительность МП.

Принцип совмещения выполнения операций во времени известен давно и широко применяется на практике. Этот общий принцип включает два понятия: *параллелизм* и *конвейеризация*. Хотя у них много общего и их зачастую трудно различить, эти термины отражают два различных подхода: при параллелизме совмещение операций достигается воспроизведением их в нескольких копиях аппаратной структуры, этот подход рассмотрен в разделе 5, далее подробнее рассмотрим конвейеризацию.

Конвейерный принцип выполнения команд

Конвейеризация (instruction pipelining) в общем случае основана на разделении выполняемой операции на части, называемые *ступенями*, и выделении для каждой из них отдельного блока аппаратуры. Таким образом, обработку любой машинной команды можно разделить на несколько этапов, организовав передачу данных от одного этапа к следующему. При этом конвейерную обработку можно использовать для совмещения этапов выполнения разных команд. Производительность процессора в данном случае возрастает благодаря тому, что одновременно на различных ступенях конвейера выполняются несколько команд. Конвейерная обработка широко применяется во всех современных быстродействующих процессорах.

Конвейеризация увеличивает количество команд, завершающихся в единицу времени, следовательно, программа будет выполняться быстрее. Отметим, что при этом конвейеризация не сокращает время выполнения отдельной команды. Производительность процессора прямо пропорциональна его тактовой частоте и обратно пропорциональна длительности самой медленной ступени конвейера.

Если в процессоре без применения конвейера одна команда выполняется за N тактов, то говорят о производительности $1/N$ команд за такт. Если в этот процессор ввести конвейер длиной N ступеней, то

при полной загрузке конвейера производительность процессора увеличится в N раз. Таким образом, эффективнее разбивать команды на большее количество ступеней и делать конвейер более длинным, повышая общую производительность процессора.

Конвейер позволяет увеличить производительность процессора в N раз только тогда, когда его загрузка близка к полной, а скорость подачи новых команд и операндов соответствует максимальной производительности конвейера. Если произойдет задержка, то параллельно будет выполняться меньшее количество команд, и суммарная производительность конвейера, а следовательно, и всего МП снизится.

Кроме того, введение конвейерной обработки возможно для больших комбинационных схем. В качестве примера можно привести умножитель 32×32 бита, в котором самый старший бит результата является функцией от всех 64 битов операндов, поэтому расчет произведения занимает значительное время. Такой умножитель можно реализовать, объединив четыре умножителя 16×16 битов, работающих последовательно, что позволяет увеличить частоту работы схемы.

Однако в процессе конвейерной обработки иногда возникают ситуации, которые препятствуют выполнению очередной команды из потока команд в предназначенном для нее такте. Такие ситуации называются *конфликтами*. Конфликты приводят к необходимости приостановки выполнения команд и снижают производительность конвейера. В самом простом случае, если приостанавливается какая-либо команда, то все следующие за ней команды также приостанавливаются. Команды, предшествующие приостановленной, могут продолжать выполняться, но во время приостановки не выбирается ни одна новая команда.

Состояние ожидания (ОЖ) — состояние, при котором конвейер пропускает один или несколько тактов из-за того, что не готовы операнды.

Состояние простоя (ПР) — состояние, при котором конвейер пропускает один или несколько тактов потому, что данный этап конвейера не используется в данной команде.

Рассмотрим пример разбиения команды на ступени:

- 1) ВК — выборка команды;
- 2) ДК — дешифрация команды;
- 3) ФА — формирование адреса;

- 4) ПО — получение операндов;
- 5) ВО — выполнение операции;
- 6) РР — размещение результата в РЗУ или памяти.

На рис. 2.2.8, *а* представлены временные диаграммы функционирования конвейера при идеальной загрузке на примере выполнения трех команд: инструкция K_1 находится на стадии РР, в то время как последняя команда K_3 находится на стадии получения операндов.

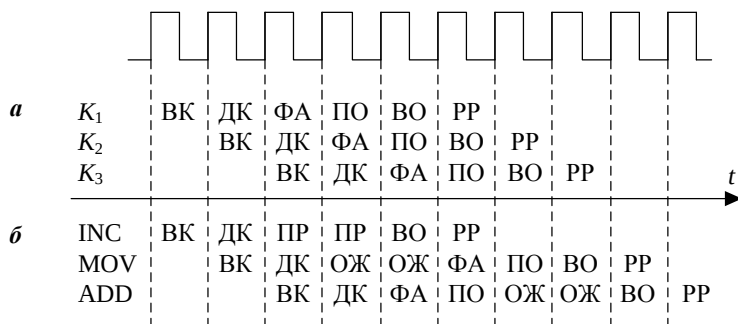


Рис. 2.2.8. Временные диаграммы функционирования 6-ступенчатого конвейера команд: *а* — при идеальной загрузке; *б* — при наличии зависимостей по данным

На рис. 2.2.8, *б* показано, как снижается производительность конвейерной обработки команд при выполнении трех команд, имеющих зависимости по данным:

- 1) INC R2 — увеличение содержимого регистра R2 на единицу;
- 2) MOV R3, (R2) — пересылка содержимого ячейки памяти, адрес которой находится в R2, в регистр R3;
- 3) ADD R3, (R4) — сложение содержимого регистра R3 с содержимым ячейки памяти, адрес которой находится в R4, сохранение результата в R3.

На рис. 2.2.8, *б* показано также, что при выполнении реальной программы возникают состояния ожидания и простоя конвейера. Первая команда вызывает состояние простоя конвейера, потому что команда инкремента не проходит стадии ПО. Вторая и третья команды вызывают состояние ожидания конвейера из-за того, что неизвестен результат выполнения предыдущей операции.

По причинам возникновения различают следующие типы конфликтов:

1) структурные — когда аппаратные средства не могут поддерживать все возможные комбинации команд в режиме одновременного выполнения с совмещением;

2) конфликты по данным — когда выполнение текущей команды зависит от результата выполнения предыдущей команды;

3) конфликты по управлению — когда команда переходов либо какая-то другая изменяет значение программного счетчика.

Контрольные вопросы

1. Дайте определение системы команд процессора.
2. По каким признакам можно классифицировать команды микропроцессора?
3. Какие способы адресации вы знаете?
4. В чем состоит конвейерный принцип выполнения команд?
5. Какие типы конфликтов выделяют в конвейере?

Литература

1. Микропроцессоры. В 3-х кн. Кн. 1. Архитектура и проектирование микроЭВМ: учебник для вузов / **Под ред. Л.Н. Преснухина**. — М.: Высшая школа, 1986. — 495 с.

2. Микропроцессорные системы: учеб. пособие для вузов / **Е.К. Александров, Р.И. Грушвицкий, М.С. Куприянов и др.; под ред. Д.В. Пузанкова**. — СПб.: Политехника, 2002. — 935 с.

Лекция 2.3. Структурные конфликты и конфликты по данным. Методы их минимизации

В конвейерных системах все команды разделяются на несколько ступеней, каждая из которых выполняется отдельным блоком аппаратуры, причем эти блоки функционируют параллельно. Если на выполнение каждой ступени расходуется одинаковое время (один такт), то на выходе конвейера на каждом такте появляется результат очередной команды.

В реальных системах из-за возникновения конфликтов при работе конвейера среднее количество тактов $N_{\text{конв}}$ для выполнения команды, измеряемое в CPI (Cycles Per Instruction — циклов на инструкцию), определяется как:

$$N_{\text{конв}} = N_{\text{ид. конв}} + N_{\text{стр}} + N_{\text{дан}} + N_{\text{упр}},$$

где $N_{\text{конв}}$ — CPI конвейера; $N_{\text{ид. конв}}$ — CPI идеального конвейера; $N_{\text{стр}}$, $N_{\text{дан}}$, $N_{\text{упр}}$ — количество циклов на разрешение структурных конфликтов, конфликтов по данным и конфликтов по управлению соответственно.

CPI идеального конвейера есть не что иное как максимальная пропускная способность конвейера при идеальной загрузке. Для увеличения пропускной способности, а следовательно, уменьшения общего CPI конвейера необходимо минимизировать влияние конфликтов на работу конвейера.

Классификация методов минимизации структурных конфликтов и конфликтов по данным представлена на рис. 2.3.1.

Структурные конфликты

Совмещенный режим выполнения команд в общем случае требует конвейеризации функциональных устройств и дублирования ресурсов для разрешения всех возможных комбинаций команд в конвейере. Если какая-нибудь комбинация команд не может быть принята из-за конфликта по ресурсам, то говорят, что в процессоре имеется *структурный конфликт*. Типичным примером вычислительных

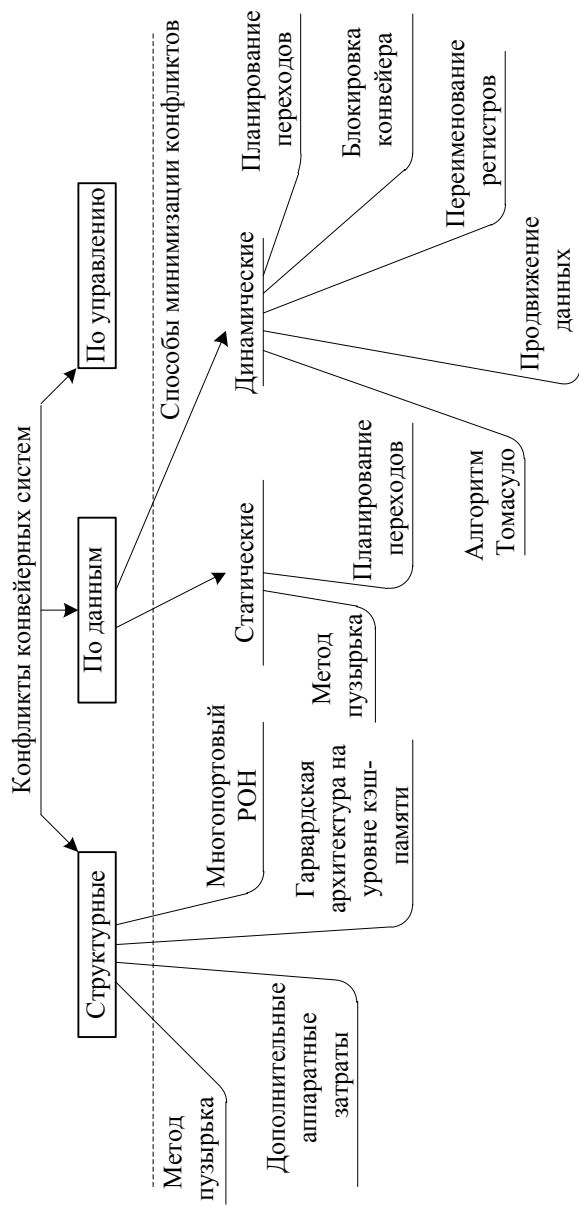


Рис. 2.3.1. Классификация основных методов минимизации структурных конфликтов и конфликтов по данным

систем, в которых возможно появление структурных конфликтов, являются машины с частично конвейеризированными функциональными устройствами. Время работы такого устройства может составлять несколько тактов синхронизации конвейера. В этом случае команды, которые используют данное функциональное устройство, не могут поступать в него в каждом такте.

Возникновение структурных конфликтов может быть связано с недостаточным дублированием некоторых ресурсов, что препятствует выполнению произвольной последовательности команд в конвейере без его приостановки. Например, процессор имеет только один порт записи в регистровый файл, а конвейеру при определенных обстоятельствах требуется выполнить две записи в регистровый файл в одном такте. Когда последовательность команд наталкивается на такой конфликт, конвейер приостанавливает выполнение одной из команд до тех пор, пока не станет доступным необходимое устройство.

Конвейеризация всех функциональных устройств может оказаться слишком дорогой. Процессоры, допускающие два обращения к памяти в одном такте, должны иметь удвоенную пропускную способность памяти, например, путем организации кэш-памяти отдельно для команд и данных. Аналогично, полностью конвейеризированное устройство деления с плавающей точкой требует большого количества вентиляей. Поэтому разработчики следуют принципу: если структурные конфликты не будут возникать слишком часто, то не стоит платить за то, чтобы их обойти. В целом влияние структурного конфликта на производительность конвейера относительно невелико.

Конфликты по данным

Одним из факторов, влияющих на производительность конвейерных систем, являются межкомандные логические зависимости. Такие зависимости существенно ограничивают потенциальный параллелизм смежных операций, обеспечиваемый соответствующими аппаратными средствами обработки. Степень влияния этих зависимостей определяется как архитектурой процессора (в основном структурой устройства управления, конвейером команд и параметрами функциональных устройств), так и характеристиками программ.

Конфликты по данным возникают в том случае, когда применение конвейерной обработки может изменить порядок обращений

к операндам так, что он будет отличаться от порядка, который наблюдается при последовательном выполнении команд на неконвейерной машине.

Конфликты по данным возникают, когда имеет место зависимость между командами, и они расположены по отношению друг к другу достаточно близко, так что совмещение операций, происходящее при конвейеризации, может привести к изменению порядка обращения к операндам.

Известны три возможных типа конфликтов по данным в зависимости от порядка операций чтения и записи:

- 1) чтение-после-записи;
- 2) запись-после-чтения;
- 3) запись-после-записи.

Рассмотрим две команды K_1 и K_2 , при этом команде K_1 предшествует K_2 .

Чтение-после-записи (Read-After-Write — RAW) — команда K_2 пытается прочитать операнд-источник данных прежде чем его запишет команда K_1 , так что команда K_2 может некорректно получить старое значение. Это наиболее распространенный тип конфликтов; способ их преодоления с помощью механизма «обходов» будет рассмотрен позже.

Запись-после-чтения (Write-After-Read — WAR) — команда K_2 пытается записать результат в приемник прежде чем он считывается командой K_1 , так что команда K_1 может некорректно получить новое значение. Этот тип конфликтов, как правило, не возникает в системах с централизованным управлением потоком команд, обеспечивающим выполнение команд в порядке их поступления, так что последующая запись всегда выполняется позже, чем предшествующее считывание. Особенно часто конфликты такого типа возникают в системах, допускающих выполнение команд не в порядке их расположения в программном коде.

Запись-после-записи (Write-After-Write — WAW) — команда K_2 пытается записать операнд прежде чем будет записан результат команды K_1 , т. е. записи заканчиваются в неверном порядке, оставляя в приемнике значение, записанное командой K_1 , а не K_2 . Этот тип конфликтов присутствует только в конвейерах, которые выполняют запись с многих ступеней (или позволяют команде выполняться даже в случае, когда предыдущая команда приостановлена).

В борьбе с конфликтами по данным выделяют два основных аспекта: своевременное обнаружение потенциального конфликта и его устранение.

Признаком возникновения конфликта по данным между двумя командами K_1 и K_2 служит невыполнение хотя бы одного из трех условий Бернштейна:

- 1) пересечение $W(K_1)$ с $W(K_2)$ пусто;
- 2) пересечение $W(K_1)$ с $R(K_2)$ пусто;
- 3) пересечение $R(K_1)$ с $W(K_2)$ пусто,

где $W(K_1)$ — набор выходных данных команды K_1 ; $R(K_1)$ — набор входных данных команды K_1 . Если все условия выполняются, то операторы K_1 и K_2 могут быть выполнены одновременно разными исполнителями в параллельной вычислительной системе.

Для борьбы с конфликтами по данным применяются как статические, так и динамические методы.

Статические методы ориентированы на устранение самой возможности конфликтов еще на стадии составления или компиляции программы. Компилятор пытается создать такой код, чтобы между командами, склонными к конфликтам, находилось достаточное количество нейтральных в их отношении команд. Если это невозможно, то между конфликтующими командами компилятор вставляет необходимое количество команд типа «Нет операции» (NOP). Такой подход называют *методом пузыря*.

Многие типы приостановок конвейера могут происходить достаточно часто. Например, для оператора $A = B + C$ компилятор, скорее всего, сгенерирует последовательность команд, представленную на рис. 2.3.2.

MOV R1, B	БК	ДК	ФА	ПО	ПР	РР				
MOV R2, C		БК	ДК	ФА	ПО	ПР	РР			
ADD R2, R1			БК	ДК	ПР	ПР	ОЖ	ВО	РР	
MOV A, R2				БК	ДК	ФА	ПР	ПР	ОЖ	РР

Рис. 2.3.2. Конвейерное выполнение оператора $A = B + C$

Очевидно, что выполнение команды ADD должно быть приостановлено до тех пор, пока не станет доступным поступающий из памяти

ти операнд C . Дополнительной задержки выполнения команды MOV не произойдет в случае применения цепей обхода для пересылки результата операции АЛУ непосредственно в регистр данных памяти для последующей записи.

В данном простом примере компилятор никак не может улучшить ситуацию, однако в ряде более общих случаев он может реорганизовать последовательность команд так, чтобы избежать приостановок конвейера. Этот метод, называемый *планированием загрузки конвейера* (pipeline scheduling), или *планированием потока команд* (instruction scheduling), использовался начиная с 1960-х годов, но особую актуальность приобрел в 1980-х годах, когда конвейерные системы получили широкое распространение.

Рассмотрим фрагмент программы, составленной на языке высокого уровня и выполняющей арифметические операции над переменными. Положим для примера следующую последовательность вычислений:

$$A = B + C;$$

$$D = E - F.$$

(Напомним, что *переменная* — это поименованная область памяти, адрес которой можно использовать для осуществления доступа к данным.) Чтобы сгенерировать код, не вызывающий остановок конвейера, допустим, что задержка загрузки из памяти составляет один такт.

На рис. 2.3.3 приведен пример устранения конфликтов компилятором, символом (*) отмечены команды, вызывающие блокировку.

В результате оптимизации устранены обе блокировки (командой загрузки MOV R_C , $AdrC$ команды ADD R_B , R_C и командой MOV R_F , $AdrF$ команды SUB R_E , R_F). Заметим, что использование разных регистров для первого и второго операторов было достаточно важным для реализации такого правильного планирования. В частности, если бы переменная E была загружена в тот же регистр, что B или C , то планирование не было бы корректным. В общем случае планирование конвейера может требовать увеличенного количества регистров. Такое увеличение может оказаться особенно существенным для машин, которые способны выдавать на выполнение несколько команд в одном такте.

Многие современные компиляторы используют метод планирования потока команд для улучшения производительности конвейера.

а	MOV R _B , AdrB	б	MOV R _B , AdrB
	MOV R _C , AdrC (*)		MOV R _C , AdrC
	ADD R _B , R _C		MOV R _E , AdrE
	MOV AdrA, R _B		ADD R _B , R _C
	MOV R _E , AdrE		MOV R _F , AdrF
	MOV R _F , AdrF (*)		MOV AdrA, R _B
	SUB R _E , R _F		SUB R _E , R _F
	MOV AdrD, R _E		MOV AdrD, R _E

Рис. 2.3.3. Пример устранения конфликтов компилятором: *а* — неоптимизированная последовательность команд; *б* — оптимизированная последовательность команд

ра. В простейшем алгоритме компилятор планирует распределение команд в одном и том же базовом блоке. Базовый блок представляет собой линейный участок последовательности программного кода, в котором отсутствуют команды перехода, за исключением начала и конца участка (переходы внутрь этого участка тоже должны отсутствовать). Планирование такой последовательности команд осуществляется достаточно просто, поскольку компилятор знает, что каждая команда в блоке будет выполняться, если выполняется первая из них, и можно просто построить граф зависимостей этих команд и упорядочить их так, чтобы минимизировать приостановки конвейера. Для простых конвейеров стратегия планирования на основе базовых блоков вполне удовлетворительна. Однако когда конвейеризация становится более интенсивной и действительные задержки конвейера растут, требуются более сложные алгоритмы планирования.

Существуют аппаратные методы, позволяющие изменить порядок выполнения команд программы так, чтобы минимизировать приостановки конвейера. Эти методы получили общее название *методов динамической оптимизации* (в англоязычной литературе в последнее время часто применяются также термины «out-of-order execution» — «неупорядоченное выполнение» и «out-of-order issue» — «неупорядоченная выдача»).

Основными средствами методов динамической оптимизации являются:

1) размещение схемы обнаружения конфликтов в возможно более низкой точке конвейера команд так, чтобы позволить команде продвигаться по конвейеру до тех пор, пока ей реально не потребуются операнд, являющийся также результатом логически более ранней, но еще не завершившейся команды. Альтернативный подход — это централизованное обнаружение конфликтов на одной из ранних ступеней конвейера;

2) буферизация команд, ожидающих разрешения конфликта, и выдача последующих логически не связанных команд в «обход» буфера; в этом случае команды могут выдаваться на выполнение не в том порядке, в котором они расположены в программе, однако аппаратура обнаружения и устранения конфликтов между логически связанными командами обеспечивает получение результатов согласно заданной программе;

3) соответствующая организация коммутирующих магистралей, реализующая засылку результата операции непосредственно в буфер, хранящий логически зависимую команду, задержанную из-за конфликта, или непосредственно на вход функционального устройства до того, как этот результат будет записан в регистровый файл или в память (short-circuiting, data forwarding или data bypassing — метод, который будет рассмотрен далее).

В примере, представленном на рис. 2.3.4, все команды, следующие за командой ADD, используют результат ее выполнения. Команда ADD записывает результат в регистр R1, а команда SUB читает это значение. Если не предпринять никаких мер для того, чтобы предотвратить этот конфликт (например, введением цикла ожидания, как и показано на рисунке), команда SUB прочтет неправильное значение и попытается его использовать. На самом деле значение, используемое командой SUB, является даже неопределенным (хотя логично предположить, что SUB всегда будет использовать значение R1, которое было присвоено какой-либо командой, предшествовавшей ADD, это не всегда так). Если произойдет прерывание между командами ADD и SUB, то команда ADD завершится, и значение R1 в этой точке будет соответствовать результату ADD. Такое непрогнозируемое поведение очевидно неприемлемо.

Проблема, поставленная в этом примере, может быть решена с помощью достаточно простой организации коммутирующих магистра-

ADD R1, R2	БК	ДК	ПР	ПР	ВО	ПР					
SUB R3, R1		БК	ДК	ПР	ПР	ОЖ	ВО	ПР			
AND R4, R1			БК	ДК	ПР	ПР	ОЖ	ВО	ПР		
OR R5, R1				БК	ДК	ПР	ПР	ОЖ	ВО	ПР	
XOR R6, R1					БК	ДК	ПР	ПР	ОЖ	ВО	ПР

Рис. 2.3.4. Пример последовательности команд в конвейере, имеющих зависимость по данным

лей, которая называется пересылкой или продвижением данных (data forwarding), «обходом» (data bypassing), иногда «закороткой» (short-circuiting). Эта организация состоит в следующем (рис. 2.3.5). Результат операции АЛУ с его выходного регистра всегда снова подается на входы АЛУ. Если аппаратура обнаруживает, что предыдущая операция АЛУ записывает результат в регистр, соответствующий источнику операнда для следующей операции АЛУ, то логические схемы управления выбирают в качестве входа для АЛУ результат, поступающий по цепи «обхода», а не значение, прочитанное из регистрового файла.

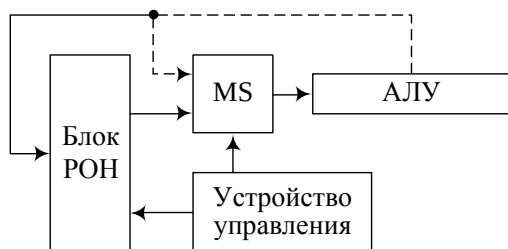


Рис. 2.3.5. Схема продвижения данных

Эта организация «обхода» может быть обобщена для того, чтобы включить передачу результата прямо в то функциональное устройство, которое в нем нуждается: результат с выхода одного устройства «пересылается» на вход другого, а не с выхода некоторого устройства только на его вход.

Не все потенциальные конфликты по данным могут обрабатываться с помощью организации «обхода». Рассмотрим последовательность команд, представленную на рис. 2.3.6.

MOV R1, (R6)	БК	ДК	ФА	ПО	ПР	ПР				
ADD R7, R1		БК	ДК	ПР	ПР	ОЖ	ВО	ПР		
SUB R5, R1			БК	ДК	ПР	ПР	ОЖ	ОЖ	ВО	ПР
AND R6, R1				БК	ДК	ПР	ПР	ОЖ	ОЖ	ОЖ

Рис. 2.3.6. Пример последовательности команд с приостановкой конвейера

Эта последовательность отличается от последовательности подряд идущих команд АЛУ. Команда загрузки (MOV) регистра R1 из памяти по адресу, содержащемуся в регистре R6 (косвенная адресация), имеет задержку, которая не может быть устранена обычным обходным путем. В этом случае, чтобы обеспечить корректное выполнение программы, применяют *метод блокировки конвейера* (pipeline interlock). Аппаратура обнаруживает конфликт и приостанавливает конвейер до тех пор, пока он существует. Приостановка начинается с команды, которая хочет использовать данные в то время, когда предыдущая команда, результат которой является операндом для нашей, вырабатывает этот результат. Аппаратура вызывает приостановку конвейера или появление «пузырька» точно так же, как и в случае структурных конфликтов.

Еще одним динамическим методом минимизации конфликтов по данным является *метод переименования регистров* (register renaming), который служит для устранения конфликтов по данным типа запись-после-чтения (WAR) и запись-после-записи (WAW). Поясним суть метода на примере возникновения конфликта по данным типа запись-после-чтения (рис. 2.3.7).

$$\begin{aligned}
 K_1 &: \text{ADD R1, R2} \\
 K_2 &: \text{MUL R2, R5} \\
 &\dots \\
 K'_2 &: \text{MUL R33, R5}
 \end{aligned}$$

Рис. 2.3.7. Пример переименования регистров

Команда K_1 использует регистр R2 в качестве операнда, следующая за ней команда использует этот же регистр в качестве приемника результата. Для разрешения конфликтной ситуации при одновременном обращении к регистру R2 результат выполнения команды K_2 записывается в R33 вместо R2, т. е. происходит переименование регистра. Отметим, что при выполнении последующих команд все обращения к переименованным регистрам должны перенаправляться до тех пор, пока это переименование остается актуальным.

При аппаратной реализации метода переименования регистров выделяются логические регистры, обращение к которым выполняется с помощью соответствующих полей команды (специальных тегов), и физические регистры, которые размещаются в регистровом файле процессора. Номера логических регистров динамически отображаются на номера физических регистров посредством таблиц отображения, которые обновляются после декодирования каждой команды. Новый результат записывается в новый физический регистр. Однако предыдущее значение каждого логического регистра сохраняется и может быть восстановлено в случае, если выполнение команды будет прервано из-за возникновения исключительной ситуации или неправильного предсказания направления условного перехода.

В процессе выполнения программы генерируется множество временных регистровых результатов. Эти временные значения записываются в регистровые файлы вместе с постоянными значениями. Временное значение становится новым постоянным значением, когда завершается выполнение команды (фиксируется ее результат). В свою очередь, завершение выполнения команды происходит, когда все предыдущие команды успешно завершились в заданном программой порядке.

Программист имеет дело только с логическими регистрами. Реализация физических регистров от него скрыта.

Метод переименования регистров упрощает контроль зависимостей по данным. В машине, которая способна выполнять команды не в порядке их расположения в программе, номера логических регистров могут стать двусмысленными, поскольку один и тот же регистр может быть назначен последовательно для хранения различных значений. Однако поскольку номера физических регистров уникально идентифицируют каждый результат, все неоднозначности устраняются.

Контрольные вопросы

1. Какие существуют типы конфликтов в конвейерных системах?
2. Расскажите об основных методах борьбы со структурными конфликтами.
3. Приведите классификацию конфликтов по данным.
4. Расскажите об основных методах борьбы с конфликтами по данным.
5. В чем суть метода планирования загрузки конвейера? Приведите пример.
6. В чем суть метода переименования регистров? Приведите пример.

Литература

1. Микропроцессоры. В 3-х кн. Кн. 1. Архитектура и проектирование микроЭВМ: учебник для вузов / *Под ред. Л.Н. Преснухина*. — М.: Высшая школа, 1986. — 495 с.
2. *Карпов В.Е.* Введение в распараллеливание алгоритмов и программ // Компьютерные исследования и моделирование. — 2010. — Т. 1, № 3. — С. 231–272.
3. *Цилькер Б.Я., Орлов С.А.* Организация ЭВМ и систем: учебник для вузов. — СПб.: Питер, 2004. — 654 с.

Лекция 2.4. Минимизация конфликтов по управлению. Сокращение потерь на выполнение команд условного перехода

Конфликты по управлению могут приводить к бóльшим потерям производительности конвейера, чем конфликты по данным и структурные.

Команда условного перехода (КУП) в ходе выполнения может либо изменить, либо не изменить значение счетчика команд. Если КУП изменяет значение счетчика команд на адрес, вычисленный в команде, то переход называется *выполняемым*; в противном случае он называется *невыполняемым*. Таким образом, пока КУП не выполнена (не определен целевой адрес перехода), неизвестно, какую операцию необходимо загрузить в конвейер следующей. Для устранения или частичного сокращения издержек, связанных с указанной неопределенностью, предложены различные методы, которые условно можно разделить на четыре группы: буферы предвыборки, множественные потоки, задержанный переход, предсказание переходов. Классификация методов минимизации конфликтов по управлению представлена на рис. 2.4.1.

Наиболее эффективной группой методов минимизации конфликтов по управлению является предсказание переходов, которая, в свою очередь, подразделяется на статические и динамические методы.

Простейший метод работы с условными переходами заключается в приостановке конвейера (рис. 2.4.2). Как только обнаруживается КУП, конвейер приостанавливается до тех пор, пока она не достигнет ступени конвейера, которая вычисляет новое значение счетчика команд.

Приостановки конвейера из-за конфликтов по управлению должны реализовываться иначе, чем приостановки из-за конфликтов по данным, поскольку выборка команды, следующей за КУП, должна быть выполнена как можно быстрее, как только мы узнаем окончательное направление КУП.

Например, если конвейер будет приостановлен на три такта на каждой КУП, то это может существенно отразиться на производи-

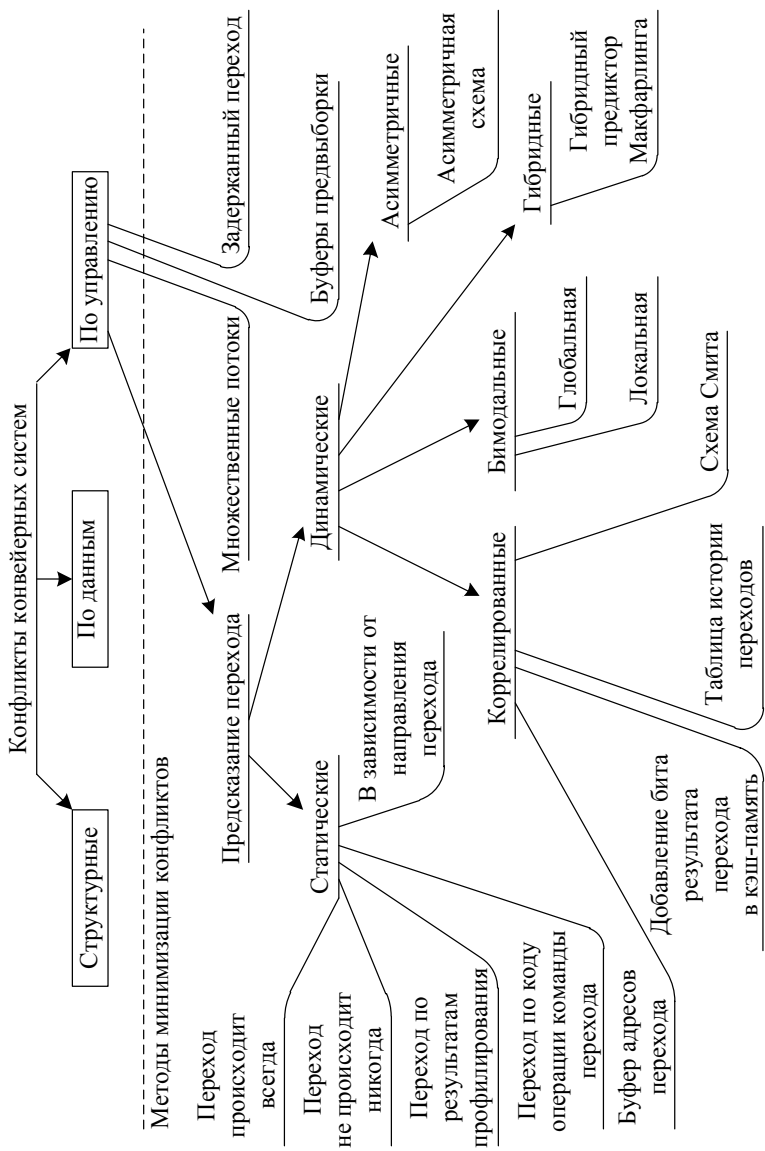


Рис. 2.4.1. Классификация основных методов минимизации конфликтов по управлению

КУП	ВК	ДК	ПР	ПР	ВО	РР						
K_i		ВК	ОЖ	ОЖ	ОЖ	ДК	ФА	ПО	ВО	РР		
K_{i+1}			ОЖ	ОЖ	ОЖ	ОЖ	ВК	ДК	ФА	ПО	ВО	РР
K_{i+2}				ОЖ	ОЖ	ОЖ	ОЖ	ВК	ДК	ФА	ПО	ВО

Рис. 2.4.2. Приостановка конвейера при выполнении КУП

тельности машины. При частоте КУП в программах, равной 30%, машина с приостановками условных переходов достигает только половины ускорения, получаемого за счет конвейерной организации. Таким образом, снижение потерь от условных переходов становится критическим вопросом.

Число тактов, теряемых при приостановках из-за условных переходов, может быть уменьшено двумя способами:

- выявлением характера условного перехода на более ранних ступенях конвейера;
- более ранним вычислением значения счетчика команд для выполняемого перехода (т. е. вычислением целевого адреса перехода).

В статических методах предсказания переходов прогнозируемое направление перехода фиксируется для каждой КУП на все время выполнения программы, т. е. такие методы применяются на уровне компилятора. В динамических методах предсказания переходов схемы прогнозирования условных переходов реализуются аппаратно.

Статическое предсказание переходов

Рассмотрим три распространенных статических метода предсказания переходов: метод пузырька, метод выжидания, задержанный переход.

Метод пузырька. Тривиальная схема обработки КУП заключается в выполнении команды «Нет операции» (NOP) до тех пор, пока не станет известным направление перехода. Привлекательность такого решения заключается в его простоте.

Метод выжидания. Более эффективный и относительно несложный метод состоит в том, чтобы прогнозировать условный переход как невыполняемый. При этом аппаратура должна продолжать выполнять программу, как если бы условного перехода вовсе не было. В данном

случае необходимо позаботиться о том, чтобы не изменить состояние машины до тех пор, пока направление перехода не станет окончательно известным. В некоторых машинах эта схема с невыполняемыми по прогнозу условными переходами реализована путем продолжения выборки команд, как если бы условный переход был обычной командой. Однако если условный переход на самом деле выполняется, конвейер очищается от команд, выбранных за КУП, и заново повторяется выборка команд по целевому адресу перехода. Альтернативная схема прогнозирует переход как выполняемый.

Задержанный переход. Идея метода заключается в том, что между моментом загрузки в конвейер КУП и моментом собственно перехода по выбранной ветке в конвейер загружаются команды, выполнение которых не связано с данной КУП. Этот процесс поясняет рис. 2.4.3. В таком случае n команд находится во временном интервале задержанного перехода. Задача программного обеспечения состоит в том, чтобы сделать команды, следующие за командой перехода, действительными и полезными. Аппаратура гарантирует реальное выполнение этих команд перед выполнением собственно перехода. Здесь используются несколько приемов оптимизации.

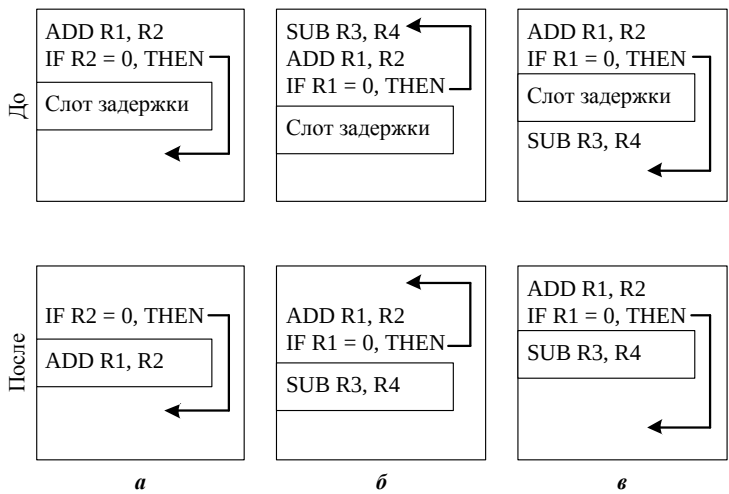


Рис. 2.4.3. Планирование задержанного перехода

На рис. 2.4.3 представлены три стратегии (*a*, *b*, *в*) планирования задержанного перехода. В верхней части рисунка для каждой стратегии показана исходная последовательность команд, а в нижней части — последовательность команд, полученная в результате планирования, при условии, что в интервал надо поместить одну команду.

В стратегии *a* интервал задержки заполняется независимой командой, находящейся перед КУП. Это наилучший выбор. Стратегии *b* и *в* используются, если применение стратегии *a* невозможно.

В последовательностях команд для стратегий *b* и *в* использование содержимого регистра R1 в качестве условия перехода препятствует перемещению команды ADD (которая записывает результат в регистр R1) за команду перехода. В стратегии *b* интервал задержки заполняется командой, находящейся по целевому адресу команды перехода. Обычно такую команду приходится копировать, поскольку к ней возможны обращения и из других частей программы. Стратегии *b* отдается предпочтение, когда с высокой вероятностью переход является выполняемым, например, если это переход на начало цикла.

Наконец, интервал задержки может заполняться командой, находящейся между командой невыполняемого перехода и командой, расположенной по целевому адресу, как в стратегии *в*. Чтобы подобная оптимизация была законной, надо иметь возможность все-таки выполнить команду SUB, если переход пойдет не по прогнозируемому направлению. При этом предполагается, что команда SUB выполнит ненужную работу, но вся программа будет выполняться корректно. Это может быть в случае, если регистр R4 используется только для временного хранения промежуточных результатов вычислений, когда переход выполняется не по прогнозируемому направлению.

Покажем различные ограничения для рассмотренных стратегий планирования условного перехода, а также ситуации, в которых они дают выигрыш.

Компилятор должен соблюдать определенные требования при подборе подходящей команды для заполнения интервала задержки. Если такой команды не находится, слот задержки должен заполняться пустой операцией.

Для стратегии *a* КУП не должна зависеть от переставляемой команды, и тогда производительность будет увеличена во всех случаях. В стратегии *b* требуется, чтобы выполнение переставляемой команды было корректным, даже если переход не выполняется. Для этой

стратегии повышение производительности будет происходить, когда переход выполняется. Стратегия *в* требует корректного выполнения команды, даже если переход выполняется, в таком случае производительность поднимется, когда переход не выполнится.

Динамическое предсказание переходов

Чтобы снижение производительности конвейера из-за его остановок по причине конфликтов по управлению не превысило 10%, точность предсказания переходов должна быть выше 97,7%, поэтому необходимо применять максимально эффективные методы работы такой системы.

В динамических методах решение о наиболее вероятном исходе КУП принимается в процессе вычислений на основе информации о предшествующих переходах, собираемой при выполнении программы. В целом динамические стратегии по сравнению со статическими обеспечивают более высокую точность предсказания.

Существует множество реализаций динамического предсказания переходов (см. рис. 2.4.1), рассмотрим наиболее интересные из них.

Динамическое предсказание переходов предполагает накопление информации о предшествующих результатах выполнения КУП, которые фиксируются в форме таблицы, имеющей общепринятое название — *таблица истории переходов* (Pattern History Table — PHT). Каждый элемент таблицы, состоящий из m битов, является результатом работы автомата.

В простейшем случае элемент таблицы состоит из одного бита информации, который устанавливается в лог. «1», если переход состоялся, и дается предсказание, что в будущем переход состоится; в противном случае элементу таблицы присваивается лог. «0», и дается предсказание, что переход не состоится. После обработки очередной команды содержимое элемента таблицы корректируется.

Другие автоматы предполагают большее число состояний, поэтому в них используется таблица с многоразрядными элементами. Чаще всего ограничиваются двумя разрядами ($m = 2$) и, соответственно, автоматами с четырьмя состояниями: переход состоится; переход скорее всего состоится; переход скорее всего не состоится и переход не состоится. В зависимости от того, состоялся переход или нет, значение элемента таблицы инкрементируется или декрементируется.

Динамические методы предсказания переходов обычно подразделяют на бимодальные, коррелированные, гибридные и асимметричные.

Первый и наиболее простой подход, относящийся к *бимодальным методам*, заключается в том, что процессор фиксирует результат выполнения предыдущих команд ветвления по данному адресу и считает, что следующая команда с обращением по этому адресу даст аналогичный результат. Этот метод предполагает более высокую вероятность повторного обращения к определенной команде, задаваемой данным условием ветвления. Для реализации метода используется специальная память ВТВ (Branch Target Buffer), где хранятся адреса ранее выполненных условных переходов. При поступлении аналогичной команды ветвления предсказывается переход к ветви, которая была выбрана в предыдущем случае, и производится загрузка в конвейер команд из соответствующей ветви. При правильном предсказании не требуется перезагрузка конвейера, и эффективность его использования не снижается. Эффективность такого метода предсказания зависит от объема памяти и оказывается достаточно высокой: вероятность правильного предсказания составляет 80% и более.

Второй подход предполагает наличие дополнительного разряда в каждой ячейке кэш-памяти, который используется только применительно к КУП. Состояние разряда отражает исход предыдущего выполнения команды (1 — переход был, 0 — перехода не было), а новое предсказание совпадает с результатом предшествующего выполнения данной команды.

Бимодальные методы предсказания ориентированы на КУП, очередной исход которых существенно зависит от их собственных предыдущих исходов. В то же время для многих команд программы наблюдается сильная зависимость не от собственных исходов, а от результатов выполнения предшествующих им КУП. Это явление учитывается в коррелированных методах предсказания переходов.

В *коррелированных методах* предсказания переходов выделяют таблицы двух уровней.

В роли таблицы первого уровня выступает регистр глобальной истории (Global History Register — GHR), представляющий собой сдвиговый регистр. После выполнения очередной КУП содержимое регистра сдвигается на один разряд, а в освободившуюся позицию заносится единица, если исходом команды был переход, или нуль —

в противном случае. GHR отражает историю выполнения последних КУП.

Каждый элемент таблицы второго уровня служит для хранения истории переходов отдельной КУП. Обычно таблица второго уровня состоит из двухразрядных счетчиков и организована в виде матрицы. Содержимое счетчика команд определяет один из регистров в таблице первого уровня и одну строку в таблице второго уровня. В свою очередь, кодовая комбинация (шаблон), хранящаяся в выбранном регистре таблицы первого уровня, определяет нужный счетчик в указанном ряду таблицы второго уровня. Выбранный счетчик используется для формирования предсказания. После выполнения команды содержимое регистра и счетчика обновляется. Точность предсказания повышается с увеличением глубины предыстории переходов.

Как видно, подобный подход учитывает историю предшествующих переходов не только для этой КУП, но и для последних выполненных КУП.

Точность предсказания всех рассмотренных методов существенно зависит от особенностей программ, в рамках которых эти методы реализуются. Один и тот же метод может проявлять себя положительно с одними программными продуктами и показывать неудовлетворительные результаты с другими. Кроме того, на точность предсказания влияет глубина истории переходов: требуется определенное время (называемое временем «разогрева») для накопления соответствующей информации. Пока идет «разогрев», точность предсказания весьма низкая.

Гибридные методы объединяют несколько различных механизмов предсказания переходов — элементарных предикторов, и в каждой конкретной ситуации задействуется тот элементарный предиктор, от которого можно ожидать наибольшей точности предсказания.

Гибридный предиктор Макфарлинга (рис. 2.4.4) содержит два элементарных предиктора, отличающихся по своим характеристикам (размером таблиц истории и временем «разогрева») и работающих независимо друг от друга. Селектор обеспечивает выбор наиболее подходящего в данной ситуации предиктора и представляет собой таблицу двухразрядных счетчиков.

Счетчики адресуются в таблице k младшими разрядами адреса КУП, для которой осуществляется предсказание. Обновление таблиц истории в каждом из предикторов производится обычным об-

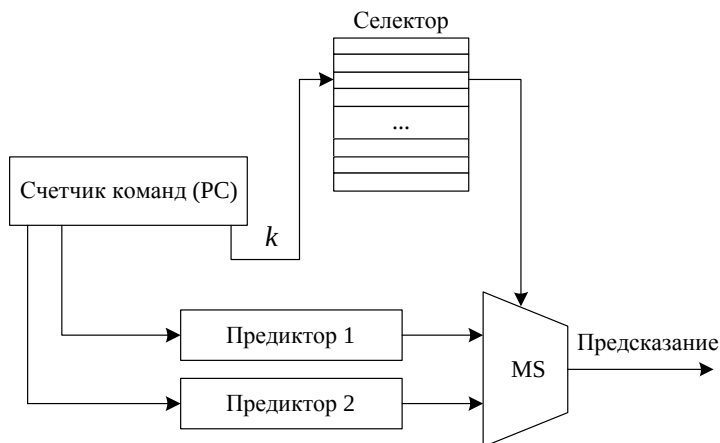


Рис. 2.4.4. Гибридный предиктор Макфарлинга

разом, как при их автономном использовании. Если оба предиктора одновременно дали одинаковое предсказание, содержимое счетчика не изменяется. При правильном предсказании от первого предиктора и неверном от второго содержимое счетчика увеличивается на единицу, в противоположном случае — уменьшается на единицу. Выбор предиктора, на основании которого делается результирующая оценка, реализуется с помощью мультиплексора, управляемого старшим разрядом соответствующего счетчика селектора.

По имеющимся оценкам, точность предсказания переходов с помощью гибридных методов в среднем составляет 97,13%, что существенно выше по сравнению с другими вариантами.

Асимметричные методы сочетают особенности гибридных и коррелированных методов предсказания переходов. От гибридных методов они «переняли» одновременное срабатывание нескольких различных элементарных предикторов. В асимметричных методах таких предикторов три, и каждый из них использует собственную таблицу РНТ. Для доступа к таблицам аналогично коррелированным методам используется как адрес КУП, так и содержимое регистра глобальной истории (рис. 2.4.5).

Шаблон для обращения к каждой из трех РНТ формируется по-разному (применены различные функции хэширования). При выпол-

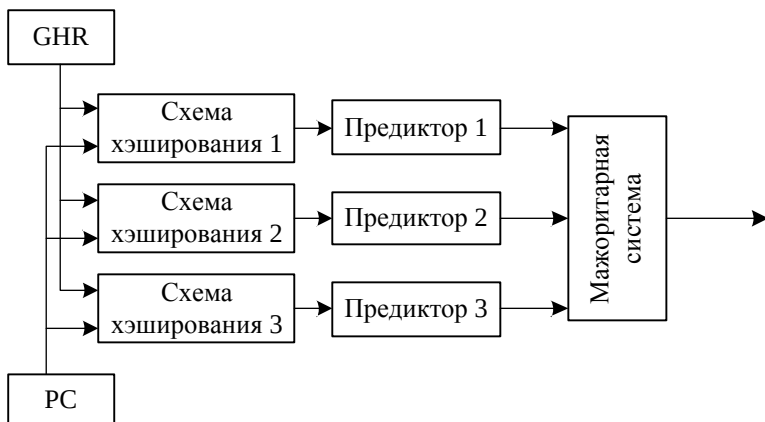


Рис. 2.4.5. Асимметричный метод предсказания переходов

нении КУП каждый из трех предикторов выдвигает свое предположение, но окончательное решение принимается по мажоритарной системе. После завершения КУП содержимое всех трех таблиц обновляется.

Контрольные вопросы

1. Дайте определение конфликта по управлению.
2. Классифицируйте основные методы борьбы с конфликтами по управлению.
3. Сформулируйте принципы работы метода задержанных переходов.
4. Какие методы динамического предсказания переходов существуют?
5. Сформулируйте основные принципы работы гибридного предиктора Макфарлинга.

Литература

1. Микропроцессоры. В 3-х кн. Кн. 1. Архитектура и проектирование микроЭВМ: учебник для вузов / *Под ред. Л.Н. Преснухина*. — М.: Высшая школа, 1986. — 495 с.

2. **Цилькер Б.Я., Орлов С.А.** Организация ЭВМ и систем: учебник для вузов. — СПб.: Питер, 2004. — 654 с.

3. Branch Classification: a New Mechanism for Improving Branch Predictor Performance / **P.Y. Chang, E. Hao, T.Y. Yeh, Y. Patt** // Proceedings of the 27 ACM/IEEE International Symposium on Microarchitecture. — Dec. 1994. — P. 22–31.

Лекция 2.5. Режимы функционирования микропроцессорной системы: выполнение основной программы, вызов подпрограмм

Микропроцессорная система может находиться в одном из следующих режимов работы:

- выполнение основной программы;
- вызов подпрограмм;
- обработка прерываний и исключений;
- прямой доступ к памяти.

Выполнение основной программы. В этом режиме микропроцессор последовательно загружает из памяти команды и выполняет их. Для загрузки следующей команды микропроцессор хранит указатель (адрес) на нее в специальном регистре — *программном счетчике* (Program Counter — PC, или Instruction Pointer — IP). Содержимое этого регистра увеличивается на единицу каждый раз после выборки из памяти очередного слова* основной программы. При выполнении команд ветвления (безусловный и условный переходы, циклы) в счетчик загружается новое значение из кода адресации операндов, в результате процессор переходит к выполнению другой части программы. При использовании механизмов относительной адресации регистр PC является базовым регистром, к которому прибавляется адресная часть команды.

Очередная команда, загруженная из памяти команд, поступает в регистр команд в устройстве управления. Затем она дешифрируется, т. е. расшифровывается КОП, и вычисляются адреса операндов согласно КАД. Далее устройство управления в соответствии с операндом команды генерирует последовательность управляющих сигналов, которая в конечном счете приводит к выполнению поступившей команды и обеспечивает получение нужного результата.

Выборка каждой команды требует определенного количества тактов сигнала частоты. Если в системе команд процессора предусмотрены команды различной разрядности, то для выборки всей команды

*Разрядность слова зависит от разрядности шины данных общения с памятью. В самом простом случае слово равно одному байту.

необходимо знать ее общую длину. Поэтому в такой системе устройство управления считывает из памяти сначала первое слово команды, в котором содержится длина команды (непосредственно или косвенно в КОП), а затем остальную команду и помещает в регистр устройства управления. Применение системы команд с фиксированной разрядностью упрощает разработку процессора, поскольку отпадает необходимость в реализации механизмов определения длины команды.

Машинным (процессорным) тактом в микропроцессорных системах называют длительность тактовых сигналов T , которая задается тактовой частотой F микропроцессора. При выполнении операций, не требующих обращений к системной шине, эта частота определяет производительность микропроцессора.

Вызов подпрограмм. Для эффективного использования памяти дублирующиеся участки программы выделяют в отдельные подпрограммы. Для обращения к таким подпрограммам в систему команд процессора вводятся специальные команды вызова подпрограмм и возвращений из них (обычно обозначаемые CALL и RET соответственно). Команда CALL в коде адреса содержит указатель на первый байт подпрограммы (или смещение, если используется относительная адресация), который загружается в регистр РС, обеспечивая на следующем такте выборку первой команды из подпрограммы. Предварительно текущее значение программного счетчика сохраняется для последующего возврата в основную программу. При поступлении команды RET процессор восстанавливает предыдущее значение регистра РС и продолжает выполнение основной программы с команды, следующей за командой CALL.

Основная проблема при реализации механизма вызова подпрограмм — необходимость обеспечить возможность последовательного вызова вложенных подпрограмм (рис. 2.5.1). При вложенном вызове подпрограмм процессор должен запоминать адрес возврата каждой подпрограммы, при этом количество вложений не должно быть ограничено.

Для реализации такой возможности существует несколько подходов, в частности, использование стека. *Стек* (от англ. *stack* — стопка) — это структура данных, работающая по принципу «последним пришел — первым вышел» (Last In, First Out — LIFO).

Стековая память может различаться по способу реализации: регистровый стек или стек на основе ОЗУ.

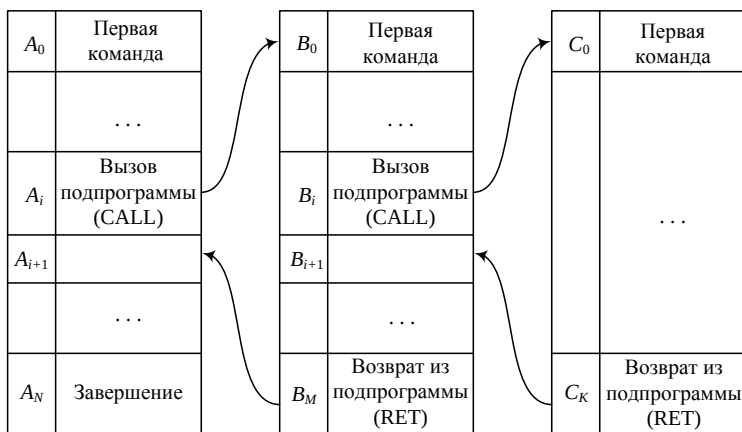


Рис. 2.5.1. Последовательный вызов вложенных подпрограмм

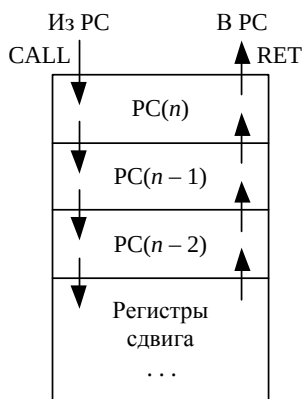


Рис. 2.5.2. Регистровый стек

Регистровый стек состоит из набора регистров (рис. 2.5.2), данные по которым могут перемещаться как в одну, так и в другую сторону. Команда CALL вызывает запись содержимого регистра PC в вершину стека, одновременно сдвигая остальные данные вниз. Команда RET, напротив, считывает содержимое самого верхнего регистра в PC, сдвигая остальные данные вверх. Количество регистров в стеке (называемое *глубиной стека*) определяет максимальное количество вложенных подпрограмм. Если все реги-

стры заняты и при этом снова происходит вызов подпрограммы, то последняя точка возврата теряется, и процессор не может вернуться к основной программе. Обычно в микропроцессорных системах с такой организацией стека при исчерпании всех ячеек генерируется внутреннее исключение, обработчик которого может выгрузить дан-

ные из стека в ОЗУ. Такой подход снимает ограничение на количество вложенных подпрограмм, но требует от программиста написания дополнительных секций программы, отвечающих за обработку исключения. Если число вложений подпрограмм ограничено на уровне проектирования программы и не превышает глубину стека, то никаких дополнительных мер предпринимать не нужно.

Стек на основе ОЗУ (рис. 2.5.3) — более совершенное решение стековой памяти. В этом случае в ОЗУ выделяется отдельный участок для работы в качестве стека. Адресация к ячейкам стека производится с помощью специального регистра — указателя стека (Stack Pointer — SP), который входит в состав УУ либо является одним из регистров общего назначения. Регистр SP содержит адрес последней (самой верхней) заполненной ячейки стека, в которой хранится значение PC, записанное командой CALL. При переходе на новую подпрограмму значение SP увеличивается на единицу, указывая на новую еще не заполненную ячейку памяти. После этого в нее записывается текущее значение PC. Таким образом происходит последовательное заполнение стека «снизу-вверх», при этом регистр SP всегда указывает на вершину стека. При выходе из подпрограммы с помощью оператора RET в регистр PC записывается значение, хранящееся в вершине стека, а регистр SP декрементируется.

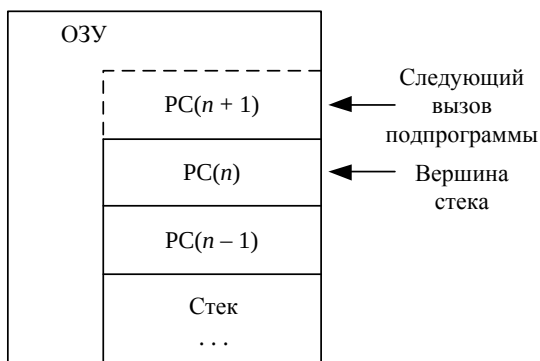


Рис. 2.5.3. Стек на основе ОЗУ

Поскольку ОЗУ, как правило, имеет значительный объем, то для стека можно выделить достаточно большое количество ячеек, обес-

печивая необходимый уровень сложения подпрограмм. Вместе с тем этот способ реализации стековой памяти имеет недостаток, связанный с необходимостью доступа к памяти всякий раз при входе в подпрограмму или выходе из нее. Это замедляет работу программы, особенно при большом количестве маленьких подпрограмм.

Для снижения накладных расходов на работу со стеком при вызове подпрограмм в некоторых системах получил распространение *метод регистрового окна*, разработанный в университете Беркли (США) и примененный в архитектуре системы команд SPARC, а позднее в процессоре Intel i960.

Предположим, что блок РОН состоит из K регистров, расположенных по кругу (рис. 2.5.4). Программе доступна лишь часть регистров L , составляющих текущее регистровое окно. Окно делится на три части: L_1 — входные регистры, L_2 — локальные регистры, L_3 — выходные регистры. При переходе в подпрограмму, регистровое окно сдвигается таким образом, что выходные регистры предыдущей активной программы становятся входными регистрами текущей. Как следствие, сохраняется в неизменном виде текущее состояние программы, а частичное перекрытие окон дает возможность передачи входных параметров запускаемой подпрограмме.

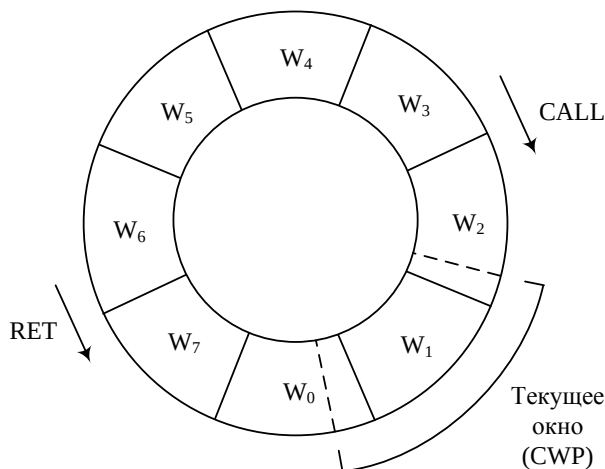


Рис. 2.5.4. Блок РОН с восемью регистровыми окнами

Указатель на начало текущего окна хранится в специальном регистре CWP (Current Window Pointer). В случае исчерпания лимита окон генерируется исключение и необходимо выгрузить содержимое первого регистрового окна в ОЗУ (так же, как это делается в регистровом стеке).

Обработка прерываний и исключений. При работе микропроцессорной системы появляются события (например ввод данных от ВУ, или сработавший системный таймер), которые требуют немедленной реакции на них, для чего необходимо прервать текущую выполняемую программу и запустить подпрограмму обработки возникшего события. Такие события называются *прерываниями* (interrupt), или исключениями.

Механизм обработки прерываний похож на механизм перехода в подпрограмму. При появлении сигнала прерывания процессор должен сохранить текущее состояние (*контекст*):

- регистр РС для обеспечения возможности возврата к выполнению основной программы;
- регистры состояния процессора (флаги АЛУ и т.п.);
- регистры общего назначения;
- специальные регистры.

После этого процессор передает управление обработчику прерывания, который производит необходимые действия для обеспечения реакции системы на прерывание и возвращает управление основной программе. Затем процессор возвращает контекст основной программы и продолжает выполнение с точки останова.

Прямой доступ к памяти. Для выполнения рутинных операций пересылки массивов данных между ОЗУ и каким-либо внешним устройством применяется специальный режим прямого доступа к памяти (ПДП). Для этого в современные процессоры встраивают отдельный контроллер ПДП, связанный общей шиной с ОЗУ и периферийными устройствами.

Когда процессору необходимо переслать некоторый объем данных, он программирует контроллер ПДП на выполнение этой задачи. Процесс программирования состоит в передаче контроллеру трех значений: адреса—источника данных, адреса—приемника данных и количества передаваемых слов. Затем контроллер сам начинает передачу данных, а процессор в это время может выполнять основную программу.

Контроллер ПДП может иметь внешние сигналы запросов от периферийных устройств. При появлении такого сигнала на входе контроллер начинает копировать данные по заложенной процессором заранее программе. После завершения копирования контроллер ПДП генерирует сигнал прерывания, сообщая процессору, что данные скопированы и с ними можно работать.

Применение режима ПДП позволяет избавить процессор от выполнения задач пересылки данных, которые могут быть весьма длительными, особенно при общении с медленными периферийными устройствами.

Учитывая материал предыдущих лекций, рассмотрим типовую структуру современной микропроцессорной системы, построенной по магистрально-модульному принципу (рис. 2.5.5). Все отдельные устройства (модули), входящие в состав такой системы, обмениваются данными по общей системной магистрали.

Основным модулем такой системы является микропроцессор, содержащий устройство управления, одно или несколько операционных устройств и блок РОН. Модуль ОЗУ необходим для хранения выполняемой программы (или ее части) и обрабатываемых данных. ПЗУ предназначено для хранения выполняемой программы и констант. В процессорах общего назначения в ПЗУ хранится *загрузчик* — небольшая программа, которая загружает из внешнего запоминающего устройства в ОЗУ основную программу. В процессорах цифровой обработки сигналов в ПЗУ могут храниться таблицы часто используемых арифметических функций, например синусов.

Внешние устройства подключаются к общей шине посредством *периферийных адаптеров* (ПА), реализующих протоколы параллельного или последовательного обмена. В однокристальных микропроцессорных системах внешними устройствами могут быть различные таймеры, встроенные блоки АЦП и ЦАП, контроллеры внешней памяти, интерфейсы USB и др. Многие внешние устройства могут генерировать сигналы прерываний и запросы ПДП. Для реализации всех возможностей таких устройств к общей шине подключаются также контроллер прерываний и контроллер ПДП.

Структура системы с общей шиной упрощает разработку и программирование процессора. Доступ к памяти и внешним устройствам осуществляется с помощью одних и тех же команд пересылки. С другой стороны, такая реализация является достаточно медленной, по-

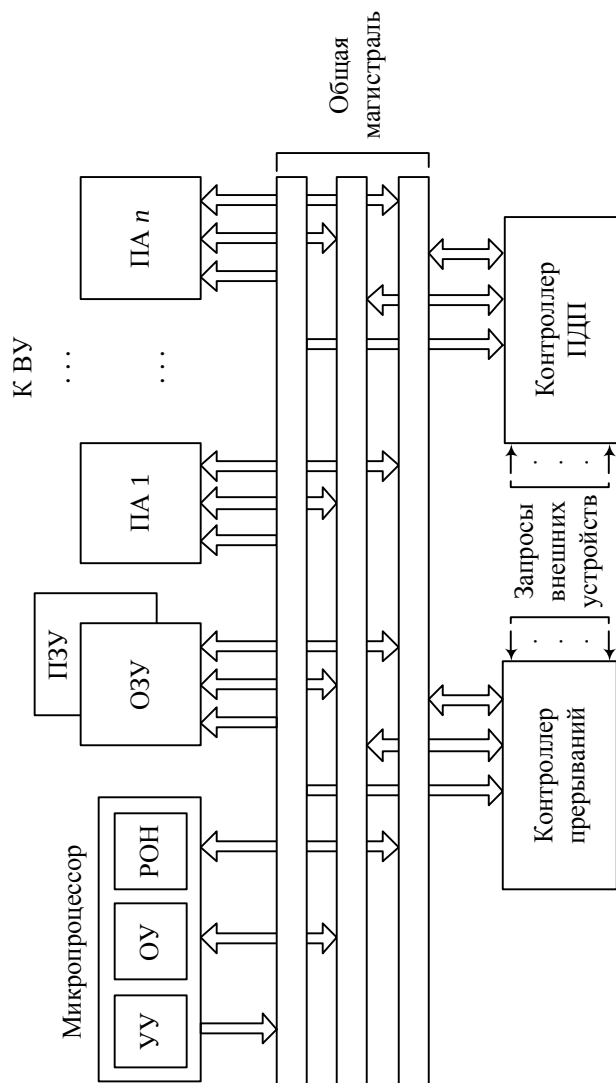


Рис. 2.5.5. Типовая структура микропроцессорной системы

сколько обмен по магистрали происходит в несколько тактов, в течение которых процессор вынужден простаивать в ожидании ответа устройства.

Контрольные вопросы

1. Перечислите основные режимы функционирования микропроцессорной системы.
2. Опишите режим выполнения основной программы.
3. Какие шаги предпринимает процессор для перехода к выполнению подпрограммы?
4. Что такое стек? Какие существуют варианты его реализации?
5. Что такое прерывание?
6. В чем суть режима прямого доступа к памяти?

Литература

1. Микропроцессоры. В 3-х кн. Кн. 1. Архитектура и проектирование микроЭВМ: учебник для вузов / **Под ред. Л.Н. Преснухина**. — М.: Высшая школа, 1986. — 495 с.
2. Микропроцессорные системы: учеб. пособие для вузов / **Е.К. Александров, Р.И. Грушвицкий, М.С. Куприянов и др.; под ред. Д.В. Пузанкова**. — СПб.: Политехника, 2002. — 935 с.

Лекция 2.6. Обработка прерываний и исключений.

Примеры построения систем прерывания

При работе микропроцессорной системы часто возникают события, которые требуют прервать выполнение текущей программы и перейти к подпрограмме, обеспечивающей обработку возникших при работе системы событий. Таким образом, *событие-прерывание* — это автоматическое изменение хода выполнения программы, вызванное событием или совокупностью событий, возникших в некоторой части вычислительной системы. Под собственно *прерыванием* в дальнейшем будем понимать ситуации, возникающие при поступлении соответствующих команд (программные прерывания) или сигналов от внешних устройств (аппаратные прерывания).

Важной особенностью прерываний является то, что по завершению выполнения подпрограммы требуется обеспечить возврат к основной программе и продолжить ее выполнение с точки прерывания. Подпрограмма, которую выполняет процессор после выполнения прерывания, называется *обработчиком прерывания*. Запросы прерывания могут генерироваться несколькими параллельно развивающимися процессами, что позволяет реализовать многозадачность операционных систем.

На рис. 2.6.1 приведена классификация событий, требующих обработки процессором. В зависимости от причин возникновения такие события можно разделить на прерывания и исключения.

Программные прерывания реализуются с помощью специальных команд, фактически это переход по фиксированному адресу памяти с сохранением текущего состояния прерываемой программы. Программные прерывания могут создаваться пользователем, и их обработка очень похожа на вызов подпрограммы.

Причинами возникновения *аппаратных прерываний* являются запросы от внешних устройств системы. Эти запросы поступают на внешние выводы процессора или формируются устройствами, размещенными на одном кристалле с процессором. Примерами аппаратных прерываний служат прерывания, поступающие от устройств ввода/вывода, аппаратных таймеров, контроллера ПДП.

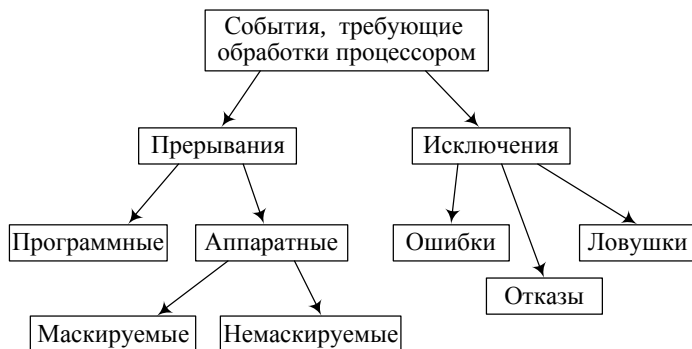


Рис. 2.6.1. Классификация событий, требующих обработки процессором

В состав процессора входит блок, отвечающий за обработку запросов прерываний. Каждый из входов внешних прерываний подключен к этому блоку. По признаку появления прерываний на входе блока прерываний их можно разделить на два типа: по фронту и по логическому уровню.

Прерывание по фронту возникает, когда появляется перепад сигнала прерывания на входе обработчика прерывания. Если происходит перепад из состояния лог. «1» в лог. «0», то говорят, что прерывание выполняется по заднему фронту; если, наоборот, из лог. «0» в лог. «1» — по переднему фронту.

Срабатывание внешнего прерывания может быть настроено на низкий или высокий логический уровень. Если на соответствующем входе обработчика прерывания появляется логический уровень, по которому происходит прерывание, оно будет обработано. Если после этого логический уровень на этом входе не будет сброшен, прерывание будет выполнено еще раз.

При одновременном поступлении нескольких запросов или возникновении еще одного запроса во время выполнения прерывания обслуживание реализуется в соответствии с их приоритетами. Для обеспечения приоритетного обслуживания прерываний в структуру процессора вводят контроллер прерываний, который позволяет программно назначать уровень приоритета для всех прерываний в системе.

Аппаратные прерывания подразделяют на маскируемые и немаскируемые.

Маскируемые прерывания обслуживаются в том случае, если установлен соответствующий бит управления (рис. 2.6.2).



Рис. 2.6.2. Блок прерываний на основе маски прерываний

Маска прерывания представляет собой двоичный код, разряды которого поставлены в соответствие запросам прерывания. Маска прерывания загружается командой программы в регистр маски RG 2. Состояние лог. «1» в данном разряде регистра маски разрешает, а состояние лог. «0» — запрещает (маскирует) прерывание текущей программы от соответствующего запроса. Замаскированные прерывания, которые записаны в регистр RG 1, могут игнорироваться, если сохранение этого прерывания будет оказывать влияние на часть программы, к которой это прерывание не относится (например, прерывание переполнения). Замаскированные прерывания следует сохранить, если, например, прерывание было вызвано окончанием операции в периферийном устройстве.

Немаскируемые прерывания обслуживаются в первую очередь и не могут быть запрещены. Обычно процессор имеет один вход немаскируемого прерывания, на него подают запросы, возникающие при аварийных ситуациях. Например, если напряжение питания выходит за допустимые пределы, то специальный датчик (монитор питания) вырабатывает запрос на немаскируемое прерывание. По этому запросу процессор должен сохранить все критически важные данные и подготовить систему к отключению питания.

Исключениями называются нештатные ситуации, возникающие при ошибках в работе процессора либо ошибках при работе с памятью. При выявлении таких ошибок блоки, контролирующие работу процессора, вырабатывают внутренние запросы на прерывания для вызова необходимой программы обслуживания. Исключения, в свою очередь, можно разделить на ошибки, ловушки и отказы.

Ошибки (faults), как правило, выявляются и обслуживаются до выполнения команды, которая является причиной их возникновения. В качестве примера можно привести ошибки при обращении очередной команды по адресу, отсутствующему в адресуемой оперативной памяти.

Типичной причиной возникновения *ловушки* (traps) является использование нулевого делителя при выполнении команды деления (деления на ноль).

Ошибки и ловушки предусматривают продолжение нормального выполнения программы после завершения их обслуживания.

Отказы (aborts) свидетельствуют о возникновении серьезных нарушений в работе системы. Примером таких нарушений может быть неисправность аппаратуры или возникновение исключения при обслуживании ранее поступившего запроса. Отказ не предусматривает продолжения выполнения прерванной программы, поэтому при его возникновении обычно происходит перезапуск процессора.

При поступлении запроса на прерывание процессор должен выполнить следующую последовательность действий:

- сформировать код номера запроса на прерывание;
- определить приоритет поступившего запроса;
- завершить выполнение текущей команды;
- сохранить текущее содержимое программного счетчика и регистров состояний;
- перейти к выполнению подпрограммы обработчика прерывания, для этого в программный счетчик загрузить вектор прерывания — начальный адрес подпрограммы обработчика прерывания;
- по окончании работы подпрограммы обработчика прерывания восстановить содержимое программного счетчика и регистров состояния для обеспечения возврата к прерванной программе.

Основной характеристикой процесса обработки прерывания является время реакции на прерывание — время между поступившим запросом на прерывание и выполнением первой полезной команды об-

работчика прерывания. Это время в основном определяется скоростью сохранения вектора состояния процессора.

Сохранение состояния выполняемой программы и возврат к ней осуществляются с помощью специальной памяти, называемой *стеком вызовов* (call stack) (рис. 2.6.3). Как правило, стек вызовов реализуется в оперативной памяти, однако он может быть выполнен и в регистровой памяти процессора или даже в специализированном стековом регистре.

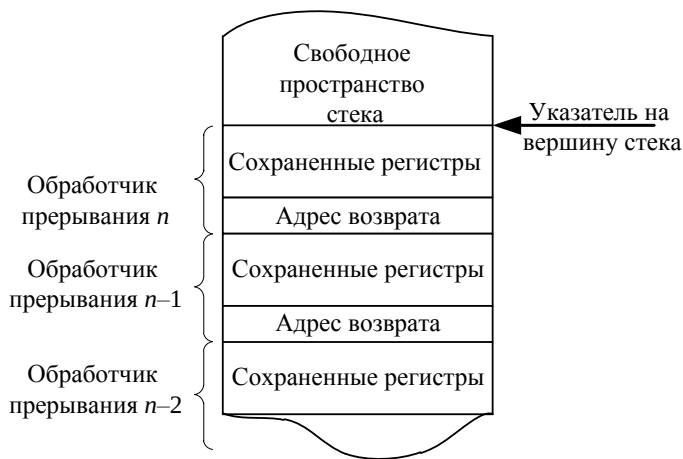


Рис. 2.6.3. Организация сохранения состояния выполняемой программы с помощью стека вызовов

В случае использования механизма «быстрых прерываний» стек вызовов может быть реализован как в теневых регистрах (shadow registers), значения из которых копируются в основные после выполнения подпрограммы обработчика прерывания, так и в оперативной памяти.

Векторы прерываний являются точками входа в подпрограммы обслуживания прерываний и хранятся в таблицах векторов прерываний, местоположение которых зависит от конкретной реализации системы, но обычно записываются в ОЗУ. Размер таблицы зависит от числа обрабатываемых процессором прерываний и исключений и является индивидуальным для каждой архитектуры команд.

При вызове обработчика прерывания, приоритет которого выше выполняемого, в стек сохраняются регистры и флаги процессора, а также адрес, по которому можно будет продолжить выполнение прерванной программы—обработчика прерывания. Возврат к выполнению прерванной программы осуществляется с помощью специального регистра, называемого *указателем на вершину стека*.

Примеры построения систем прерывания

Система прерывания должна сформировать сигнал общего запроса на прерывание и код номера запроса на прерывание, по которому можно определить, каким образом обрабатывать данное прерывание. Если в вычислительной системе находится несколько устройств, которым требуется работа с процессором, то можно организовать *систему с циклическим опросом*. В такой системе все линии, по которым в процессор поступают запросы на прерывание, опрашиваются по очереди с помощью специальной аппаратуры (рис. 2.6.4).

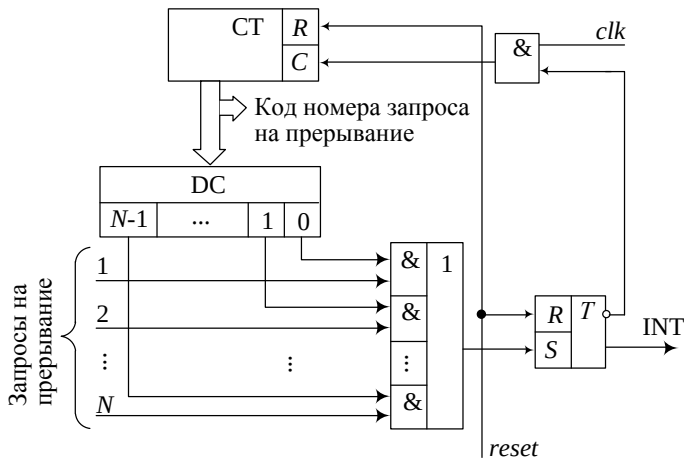


Рис. 2.6.4. Система с циклическим опросом

По сигналу reset выход триггера Т и счетчик СТ обнуляются, счетчик начинает считать. Кодовое слово с него поступает на дешифратор, на выходе которого присутствует только один активный уро-

вень, соответствующий коду. Если данный активный уровень совпадает с запросом на прерывание, то на триггер поступает сигнал, который переключает его в единичное состояние. Счетчик перестает считать, и на его выходе формируется код, соответствующий номеру запроса на прерывание.

Таким образом, данная система формирует общий запрос на прерывание INT и код номера запроса на прерывание, который, как правило, является номером регистра, содержимое которого используется для формирования начального адреса обработчика прерывания. После передачи управления обработчику прерывания счетчик и триггер сбрасываются в нуль, и процедура опроса запросов возобновляется, начиная с первого входа.

Достоинствами этой системы являются простота реализации и минимальные аппаратные затраты на нее. Однако при этом имеется ряд недостатков, которые могут быть критичными для систем, работающих в режиме реального времени. Основной недостаток системы заключается в том, что все прерывания обслуживаются по очереди, поэтому время реакции на прерывание линейно зависит от количества линий прерываний и может стать недопустимо большим. Кроме того, при увеличении количества входов для прерываний резко возрастают аппаратные затраты.

Для устранения недостатка, связанного с большим временем реакции на прерывание, используется цепочечная одноктактная система определения приоритетного запроса (*дейзи-цепочка*), которая состоит из n элементов И, n элементов И-НЕ и элемента ИЛИ (рис. 2.6.5).

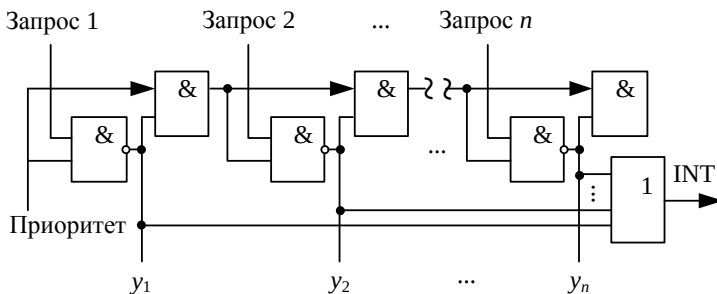


Рис. 2.6.5. Реализация цепочечной одноктактной системы определения приоритетного запроса

Процедура определения приоритетного запроса инициируется сигналом «Приоритет», поступающим на первый элемент И. При отсутствии запросов этот сигнал проходит через цепочку и сигнал общего запроса прерывания не формируется. Если среди выставленных запросов прерывания наибольший приоритет имеет i -й запрос, то распространение сигнала «Приоритет» блокируется. На i -м выходе цепочечной системы будет сигнал $y_i = 1$, на всех остальных — нуль.

Отметим, что ранжирование поступающих запросов на прерывание по важности происходит аппаратно, приоритет запросов прерывания уменьшается с возрастанием его номера, высший приоритет имеет линия первого запроса на прерывание.

Существенным недостатком обеих систем является то, что во время работы программы невозможно поменять приоритет того или иного запроса на прерывание.

Система прерываний, в которой отсутствует программное ранжирование, называется *одноуровневой*. Для устранения этого недостатка используется программируемый блок приоритетных прерываний, который позволяет обеспечить ранжирование запросов на прерывание по их важности или уровню.

На рис. 2.6.6 представлен программируемый блок приоритетных прерываний (ПБПП) на основе *уровня прерывания*.

Ключевыми элементами ПБПП являются:

- цепочечная одноканальная схема определения приоритетного запроса;
- шифратор номера приоритетного запроса и формирователь кода номера запроса на прерывание;
- цифровой компаратор СМР.

На выходе цепочечной схемы определения приоритетного запроса формируются запрос на прерывание INT 1 и номер приоритетного запроса, который попадает в шифратор. Шифратор формирует код номера запроса, принятого к обслуживанию. Порог прерывания задается командой выполняемой программы, устанавливающей в регистре порога прерывания код уровня прерывания. Если код номера запроса, поступившего на обслуживание, больше кода уровня прерывания, то формируется общий сигнал прерывания INT 2; если нет, то поступивший запрос откладывается до тех пор, пока не будет выполнена текущая программа.

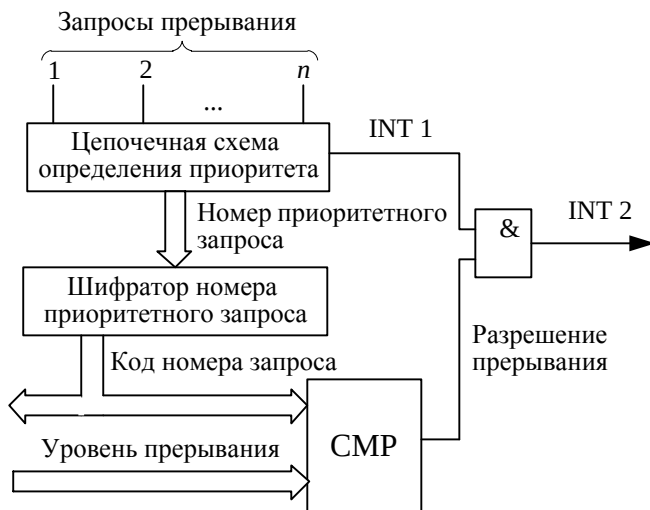


Рис. 2.6.6. Программируемый блок приоритетных прерываний

ПБПП имеет два режима работы: активный и неактивный.

Активный режим характеризуется тем, что шифратор занимается поисками очередного запроса на прерывание, при этом вычислительная система может обрабатывать какой-либо запрос на прерывание. При отыскании очередного запроса на прерывание шифратор формирует соответствующий код номера запроса на прерывание, который поступает на компаратор.

В неактивном режиме ПБПП останавливается и ждет, когда будет обработан вновь поступивший запрос, если он по уровню ниже текущего.

Контрольные вопросы

1. В чем различие между прерыванием и исключением?
2. Каковы недостатки системы с циклическим опросом?
3. Какие задачи выполняет программируемый блок приоритетных прерываний?

Литература

1. Микропроцессоры. В 3-х кн. Кн. 1. Архитектура и проектирование микроЭВМ: учебник для втузов / ***Под ред. Л.Н. Преснухина.*** — М.: Высшая школа, 1986. — 495 с.
2. Микропроцессорные системы: учеб. пособие для вузов / ***Е.К. Александров, Р.И. Грушвицкий, М.С. Куприянов и др.; под ред. Д.В. Пузанкова.*** — СПб.: Политехника, 2002. — 935 с.
3. ***Каган Б.М.*** Электронные вычислительные машины и системы: учеб. пособие для вузов. — 3-е изд., перераб. и доп. — М.: Энергоатомиздат, 1991. — 592 с.

Раздел 3. Системы памяти. Обмен информацией в микропроцессорных системах

Лекция 3.1. Классификация систем памяти. Организация систем памяти в микропроцессорных системах

Система памяти — функциональный блок микропроцессорной системы, предназначенный для временного или постоянного хранения данных и команд, состоящий из запоминающего устройства (ЗУ) и устройства управления. ЗУ служит для фиксации, хранения и выдачи информации во время работы микропроцессорной системы. Процесс фиксации информации в ЗУ называется *записью*, процесс выдачи информации — *чтением*, а совместно их определяют как *процессы обращения к ЗУ*.

Системы памяти (в дальнейшем просто *память*) имеют большое количество характеристик и параметров, которые необходимо учитывать при проектировании систем: емкость, единица пересылки, местоположение, быстродействие, технология, физические особенности, стоимость и т. д. Рассмотрим основные из них.

Емкость — это количество битов данных, которые запоминающее устройство может хранить одновременно.

Единица пересылки — количество битов данных, передаваемых по линиям шины параллельно.

По *местоположению* различают память:

- внешнюю;
- внутреннюю;
- процессорную.

Внешняя память является местом длительного хранения данных, не используемых и не требуемых в определенный момент времени в основной памяти компьютера, также она является энергонезависимой. К *внутренней памяти* относятся основная память,

кэш-память второго и других уровней. *Процессорная память* включает служебные регистры, блок РОН, кэш-память первого уровня.

Память различают по *быстродействию*, выделяя следующие характеристики:

- время доступа;
- период обращения;
- скорость передачи.

Время доступа — временной интервал с момента выставления процессором на адресную шину адреса требуемой ячейки памяти до момента осуществления связи адресуемой ячейки с шиной данных. *Период обращения* — минимальное время между двумя последовательными обращениями в память. *Скорость передачи* — это скорость, с которой происходят чтение и запись в память.

По типу *технологии*, используемой для реализации запоминающего устройства, память разделяют на:

- полупроводниковую;
- оптическую;
- магнитную.

Запоминающие устройства обладают также набором *физических особенностей*, таких как энергонезависимость (т. е. будет ли хранить запоминающее устройство информацию после отключения питания), потребляемая энергия, плотность интеграции компонентов, допустимая температура окружающей среды, допустимая влажность, тип корпуса и т. д.

Важной характеристикой памяти является ее *стоимость*.

По методу доступа к данным различают память:

- последовательную;
- прямую;
- произвольную;
- ассоциативную.

При *последовательном доступе* к данным информация хранится в памяти в виде последовательно расположенных блоков. Чтобы прочитать требуемый элемент (слово, байт, бит), необходимо прочитать все предшествующие ему данные.

Прямой доступ подразумевает, что каждая запись имеет уникальный адрес в памяти. Обращение осуществляется как адресный доступ к началу записи с дальнейшим последовательным доступом к определенной единице информации внутри записи.

Произвольный доступ к данным означает, что каждая ячейка памяти имеет уникальный адрес.

При *ассоциативном доступе к данным* выполняется параллельный поиск одновременно среди всех ячеек памяти, содержащих такую информацию, в которой значение отдельных битов совпадает с состоянием одноименных битов в заданном образце, другими словами, находится элемент по заданному признаку.

Иерархическая организация памяти представлена на рис. 3.1.1. Четыре верхних уровня образуют внутреннюю память, остальные можно отнести к внешней памяти.

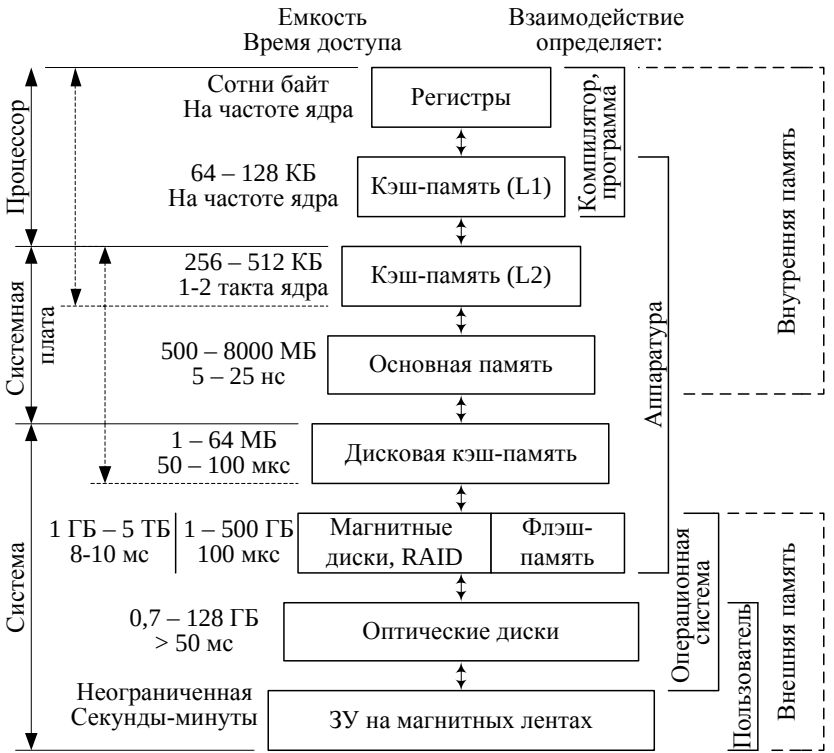


Рис. 3.1.1. Иерархическая организация памяти

В приведенной иерархии можно проследить некоторые закономерности, при большей удаленности памяти от АЛУ:

- уменьшается соотношение стоимость/бит;
- возрастает емкость;
- увеличивается время доступа;
- уменьшается частота обращения к памяти со стороны МП.

На основе вышеизложенного можно сделать вывод, что иерархическая организация памяти ведет к уменьшению общей стоимости системы при заданном уровне производительности.

Закономерность обращений программы к памяти обычно выражается в том, что обращения к памяти обладают *временной* и *пространственной* локальностью. Если рассмотреть процесс выполнения большинства программ, то можно заметить, что с очень высокой вероятностью адрес очередной команды программы либо следует непосредственно за адресом, по которому была считана текущая команда, либо расположен вблизи него. Такое расположение адресов называется *пространственной локальностью программы*. Обработываемые данные, как правило, структурированы, и такие структуры обычно хранятся в последовательных ячейках памяти. Данная особенность программ называется *пространственной локальностью данных*. Кроме того, программы содержат множество небольших циклов и подпрограмм. Если произошло обращение к ячейке памяти, то велика вероятность, что эта ячейка памяти вскоре понадобится снова. Это означает, что небольшие наборы команд могут многократно повторяться в течение некоторого интервала времени, т. е. имеет место *временная локальность*.

Оба вида локальности объединяет понятие «локальность по обращению». Принцип локальности часто облачают в численную форму и представляют в виде так называемого «правила 90/10»: 90% времени работы программы связано с доступом к 10% адресного пространства этой программы.

Несмотря на то, что иерархия памяти обычно состоит из многих уровней, в каждый момент времени мы имеем дело только с двумя близлежащими уровнями. Минимальная единица информации, которая может либо присутствовать, либо отсутствовать в двухуровневой иерархии, называется *блоком*. Размер блока может быть фиксированным либо переменным. Если этот размер зафиксирован, то объем памяти является кратным размеру блока.

Самая быстрая, но обладающая наименьшим объемом память — это внутренние регистры процессора, которые иногда обобщенно называют *сверхоперативным запоминающим устройством* (СОЗУ). Как правило, число регистров невелико, хотя в архитектурах с сокращенным набором команд оно может достигать до нескольких сотен.

По способу доставки содержимого ячейки памяти на шину данных различают следующие методы:

- последовательный;
- конвейерный;
- регистровый;
- страничный;
- быстрый страничный;
- пакетный;
- метод удвоенной скорости.

Последовательный метод (flow through mode) состоит в том, что адрес и управляющие сигналы подаются на шину до синхроимпульса и в момент его поступления вся входная информация запоминается во внутренних регистрах и начинается цикл чтения.

Конвейерный метод (pipelined mode) доступа к данным — метод при котором можно продолжать операцию чтения по предыдущему адресу в процессе запроса по следующему.

Регистровый метод (register to latch) отличается наличием регистра на выходе микросхемы.

Страничный метод (page mode) основан на принципе локальности при доступе к ячейкам со смежными адресами, причем к таким, где все запоминающие элементы расположены в одной строке матрицы, доступ ко второй и последующим ячейкам можно производить значительно быстрее.

Быстрый страничный метод (fast page mode) — модификация страничного метода, в котором полный адрес передается только при первом обращении к строке.

Пакетный метод (burst mode) состоит в том, что на запрос по конкретному адресу память возвращает пакет данных, хранящихся не только по этому адресу, но и по нескольким последующим адресам.

Метод удвоенной скорости (Double Data Rate — DDR) заключается в том, что передача данных осуществляется по обоим фронтам импульса синхронизации, т. е. дважды за период.

По функциональному назначению память можно разделить на следующие типы:

- основная (ОЗУ, ПЗУ);
- внешняя (магнитные диски, оптические диски, различные полупроводниковые накопители);
- кэш-память (промежуточный буфер с быстрым доступом, содержащий информацию, которая может быть запрошена с наибольшей вероятностью, и работающий на частоте ядра);
- ассоциативная (осуществляет поиск данных не по адресу, а по какому-то характерному признаку);
- стековая, или LIFO (обеспечивает такой режим работы, при котором информация записывается и считывается по принципу «последним записан — первым считан»);
- виртуальная (способ управления памятью, при котором для каждой программы используются независимые схемы адресации памяти, отображающиеся тем или иным способом на физические адреса в памяти).

Основная память в свою очередь делится на ОЗУ и ПЗУ. ОЗУ, являясь полупроводниковым устройством, делится на статическую (SRAM) и динамическую (DRAM) память. В статических ОЗУ запоминающий элемент может хранить записанную информацию неограниченно долго (при наличии напряжения питания). Запоминающий элемент динамического ОЗУ способен хранить информацию только в течение достаточно короткого промежутка времени, после которого информацию надо записывать заново, иначе она будет потеряна. Классификация статической и динамической памяти представлена на рис. 3.1.2 и 3.1.3.

Поясним некоторые аспекты из представленной классификации. В микросхемах, где реализован синхронный принцип, процессы чтения и записи (если это ОЗУ) выполняются одновременно с тактовыми сигналами контроллера памяти. Асинхронный принцип предполагает, что момент начала очередного действия определяется только моментом завершения предшествующей операции.

Важное значение в статическом ОЗУ имеет технология записи: стандартная или запаздывающая. В стандартной технологии адрес и данные выставляются на соответствующие шины в одном и том же такте. В запаздывающей технологии данные передаются в следующем такте после выбора адреса нужной ячейки.



Рис. 3.1.2. Классификация статической памяти

Статическая память с чередованием адреса разделяется на два независимых банка ячеек. Поскольку оба банка могут быть задействованы одновременно, непрерывный поток данных может обеспечиваться простым переключением между банками. Конвейерная адресация позволяет осуществлять доступ ко вторым запрошенным данным до завершения обработки первых запрошенных данных.

К энергонезависимым ОЗУ относят несколько типов памяти. От перепрограммируемых ПЗУ их отличает отсутствие этапа стирания,

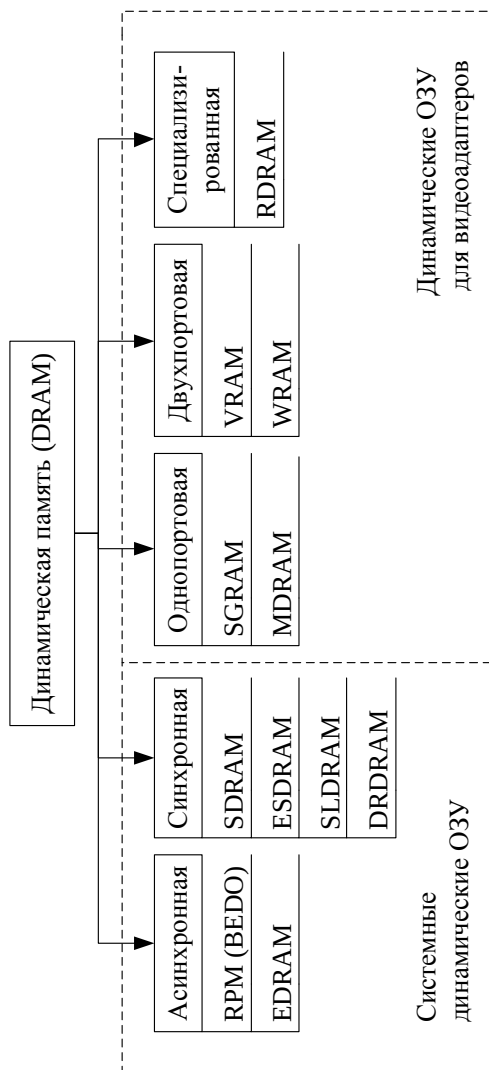


Рис. 3.1.3. Классификация динамической памяти

предваряющего запись новой информации, поэтому вместо термина «программирование» для них употребляют стандартный термин «запись».

BBSRAM (Battery Backed SRAM) — статические ОЗУ со встроенным литиевым аккумулятором и усиленной защитой от искажения информации в момент включения и отключения внешнего питания.

FRAM (Ferroelectric RAM) — статические ОЗУ, использующие слой сегнетоэлектрика вместо диэлектрического слоя для обеспечения энергонезависимости. По быстродействию данные ЗУ несколько уступают динамическим ОЗУ и в настоящее время рассматриваются лишь как альтернатива флэш-памяти. Причисление FRAM к ОЗУ обусловлено отсутствием перед записью явно выраженного цикла стирания информации.

NVRAM (Non Volatile RAM) — энергонезависимая память с произвольным доступом. Особенность ее состоит в том, что в одном корпусе объединены статическое ОЗУ и перепрограммируемая постоянная память типа EEPROM. При включении питания данные копируются из EEPROM в SRAM, а при выключении автоматически перезаписываются из SRAM в EEPROM. Благодаря такому приему данный тип памяти можно считать энергонезависимым.

BEDO (Burst Extended Data Out DRAM) — пакетная память с усовершенствованным выходом. Содержит регистр-защелку выходных данных, что обеспечивает некоторую конвейеризацию работы для повышения производительности при чтении.

EDRAM (Embedded DRAM) — встраиваемая DRAM, обычно используется в качестве кэш-памяти.

SDRAM (Synchronous DRAM) — синхронная динамическая память с произвольным доступом.

ESDRAM (Enhanced SDRAM) — SDRAM, улучшенная за счет использования на чипе небольшого количества SRAM в качестве кэш-памяти.

SLDRAM — внутрикристалльная конфигурация множественных независимых логических банков, обеспечивающая быстрые циклы обращения шины при проведении операций чтения/записи и способность работать в полностью конвейеризированном пакетном режиме. Адресация SLDRAM происходит согласно всем основным требованиям, предъявляемым к динамической памяти в целом, однако имеет довольно значимые особенности, основанные на полностью пакетном

протоколе, что исключает совместимость с любыми другими архитектурами, делая данную уникальной.

DRDRAM (Direct Rambus DRAM) имеет внутреннюю многобанковую структуру с чередованием (16 банков), чем обеспечивается ее высокая пропускная способность.

Внешняя память является важным звеном в иерархии запоминающих устройств. Она наиболее удалена от АЛУ, имеет наибольшее время доступа, но одновременно с этим наибольшие объемы хранимых данных по сравнению с остальными звеньями иерархии памяти.

Контрольные вопросы

1. Каковы основные характеристики памяти?
2. Воспроизведите иерархию запоминающих устройств микропроцессорных систем по скорости обмена данными с АЛУ.
3. Классифицируйте память по функциональному назначению.
4. Расскажите об основных отличиях статической и динамической памяти.
5. Классифицируйте память по методу доступа к данным.

Литература

1. **Цилькер Б.Я., Орлов С.А.** Организация ЭВМ и систем: учебник для вузов. — СПб.: Питер, 2004. — 654 с.
2. **Denning P.** The Working Set Model For Program Behaviour // Communications of the ACM. — May 1968. — P. 323–333.

Лекция 3.2. Принципы организации кэш-памяти. Схема обнаружения и исправления ошибок

Принципы организации кэш-памяти

Поскольку обмен информацией между внешними устройствами и ОЗУ происходит на частоте шины или даже на более низкой частоте, то при выполнении программы процессор вынужден простаивать несколько тактовых периодов, пока информация из памяти установится на шине данных. Эту проблему решают введением в структуру системы быстродействующей буферной памяти малого объема, работающей на тактовой частоте процессора. Данная буферная или кэш-память реализуется на базе ОЗУ статического типа.

В основу работы кэш-памяти положены два принципа: временной и пространственной локальности.

Принцип временной локальности заключается в том, что существует высокая вероятность того, что при считывании данных из памяти программа обратится к этим данным в течение некоторого небольшого интервала времени несколько раз.

Принцип пространственной локальности состоит в том, что есть высокая вероятность того, что программа обратится к нескольким последовательно расположенным ячейкам памяти.

Таким образом, в соответствии с принципом временной локальности информация хранится в кэш-памяти в течение небольших интервалов времени, а согласно принципу пространственной локальности в кэш-память целесообразно загружать несколько соседних ячеек. Есть статистика, что линейные участки программ в большинстве случаев не превышают трех—пяти команд. Следовательно, нет смысла хранить в кэш-памяти данные, объем которых превышает объем данных, необходимых для выполнения трех—пяти команд. Как правило, блоки информации в кэш-памяти состоят из двух—четырех слов.

При обращении процессора к памяти прежде всего проверяется наличие необходимых данных в кэш-памяти, если их нет, то эти данные переписываются из ОЗУ.

В общем случае кэш-память работает следующим образом. Когда процессор пытается прочитать слово из основной памяти, осуществляется поиск копии этого слова в кэш-памяти. Если копия обнаружена, то обращение к основной памяти не производится, а в процессор передается слово, полученное из кэш-памяти. Данную ситуацию принято называть *успешным обращением*, или *попаданием* (hit). При отсутствии слова в кэш-памяти, или *промахе* (miss) требуемое слово передается в процессор из основной памяти, но одновременно из основной памяти в кэш-память пересылается блок данных, содержащий это слово, для использования принципа временной локальности.

По способу отображения информации из основной памяти в кэш-память различают следующие типы кэш-памяти:

- с прямым отображением;
- полностью ассоциативная;
- множественно-ассоциативная.

На рис. 3.2.1 представлена обобщенная структура системы с основной и кэш-памятью. Основная память состоит из 2^L адресуемых слов, где каждое слово имеет уникальный L -разрядный адрес. При взаимодействии с кэш-памятью эта память рассматривается как M блоков фиксированной длины по K слов в каждом ($M = 2^L/K$). Кэш-память состоит из C блоков аналогичного размера (блоки в кэш-памяти принято называть *строками*), причем их число значительно меньше числа блоков основной памяти, т. е. $C \ll M$. При считывании слова из какого-либо блока основной памяти этот блок копируется в одну из строк кэш-памяти (в рассмотренном примере в некоторую i -ю строку). Поскольку число блоков основной памяти больше числа строк, отдельная строка не может быть выделена постоянно одному и тому же блоку. По этой причине кэш-память состоит из двух частей: памяти данных и памяти признаков (тегов). Каждой строке кэш-памяти соответствует тег, содержащий сведения о том, копия какого блока основной памяти в данный момент хранится в данной строке. В качестве тега обычно используется старшая часть двоичного представления адреса основной памяти.

Кэш-память с прямым отображением (рис. 3.2.2) является наиболее простым типом кэш-памяти. Основная память делится на блоки размером соответствующим размеру строк кэш-памяти, и представляется в виде двумерного массива, в котором число строк равно числу строк кэш-памяти. Согласно принципу пространственной локальности

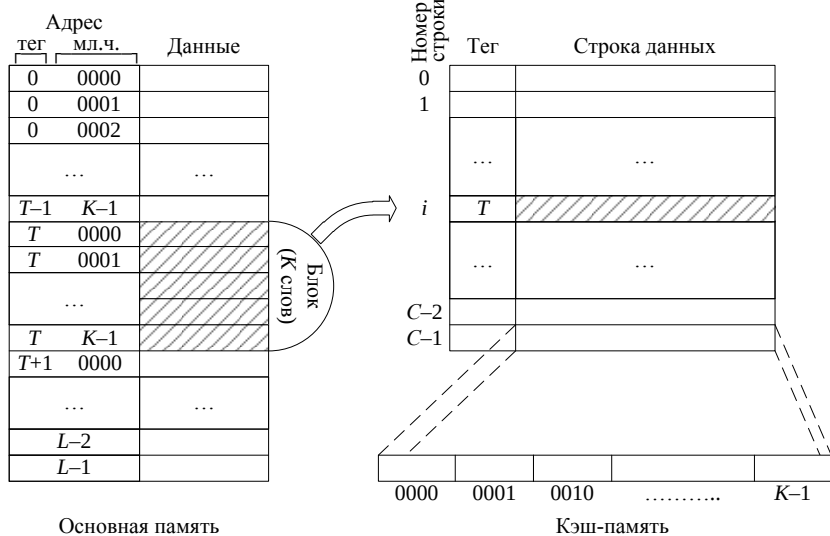


Рис. 3.2.1. Обобщенная структура системы с основной и кэш-памятью

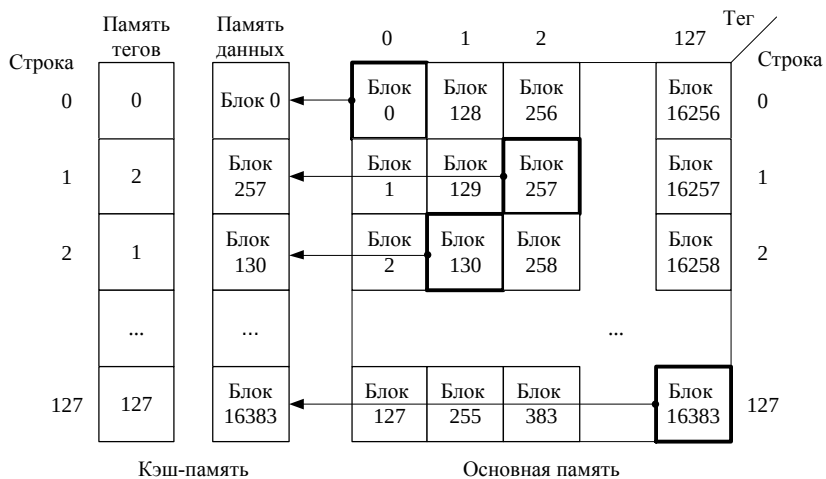


Рис. 3.2.2. Структура кэш-памяти с прямым отображением

кэш-память может содержать блоки с последовательными номерами, поэтому блоки в массиве основной памяти нумеруются по столбцам. Иными словами, в i -ю строку кэш-памяти отображается каждый 128-й блок (для рассмотренного примера) основной памяти, если отсчет начинать с блока, номер которого равен i .

При реализации такого отображения адрес блока в основной памяти условно разбивается на две части: старшая является его тегом, а младшая — полем строки. В память тегов кэш-памяти помещается тег того блока, который в данный момент заносится в кэш-память, чтобы отличить его от других блоков, которые могут быть загружены в ту же строку кэш-памяти.

При считывании данных из памяти процессор обращается сначала к кэш-памяти и проверяет, содержит ли она нужные данные. Этот процесс происходит путем сравнения признака, записанного в кэш-памяти, со старшими разрядами адреса, которые выставляет процессор. Совпадение этих величин означает, что в кэш-памяти хранятся необходимые данные, которые затем и считываются из кэш-памяти. Если величины не совпадают, то выполняется копирование соответствующих данных из основной памяти в кэш-память.

Достоинством такого типа кэш-памяти является относительно высокое быстродействие, что объясняется необходимостью выполнения лишь одного сравнения признака со старшими разрядами адреса ОЗУ.

Из-за жесткого закрепления за определенными блоками основной памяти одной строки кэш-памяти, если программа поочередно обращается к словам из двух различных блоков, отображаемых на одну и ту же строку кэш-памяти, постоянно станет происходить обновление данной строки и вероятность попадания будет низкой. В таком случае содержимое строки кэш-памяти данных пересылается обратно в ОЗУ, а в строку пересылается требуемый блок. В результате увеличиваются количество пересылок данных и время обмена информацией.

Для устранения указанного недостатка кэш-памяти с прямым отображением используют *полностью ассоциативную кэш-память* (рис. 3.2.3). В этом случае любой блок ОЗУ может отображаться в любой строке кэш-памяти. Память тегов представляет собой ассоциативную память, в которую записывается признак, т. е. старшие биты разрядов адреса, и адрес блока в кэш-памяти. Для проверки наличия

копии блока в кэш-памяти логика управления кэш-памяти должна одновременно проверить теги всех строк на совпадение с полем тега адреса. Если кэш-память не заполнена, блок записывается в любую свободную строку. Если кэш-память заполнена, блок из ОЗУ записывается в строку, к которой было наименьшее число обращений. Недостаток кэш-памяти с прямым отображением устраняется за счет определения блока, к которому было наименьшее число обращений. При этом увеличивается время обработки запросов из-за необходимости сравнения адреса и признака.

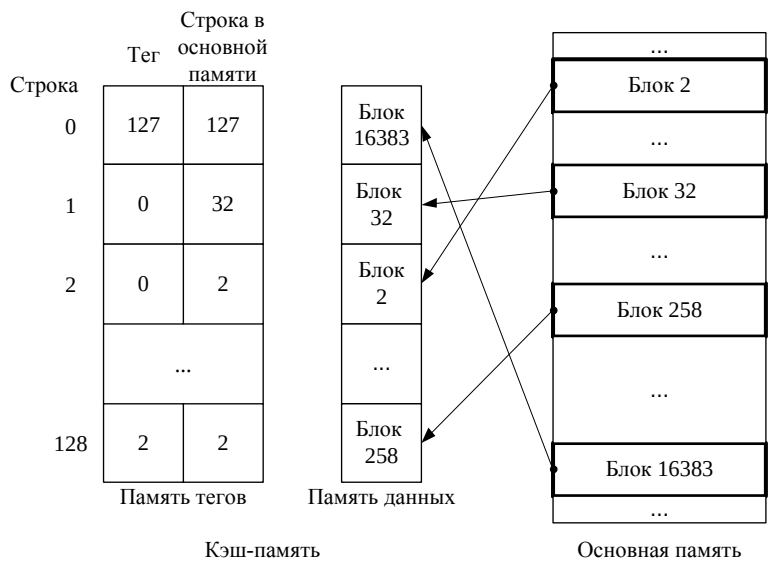


Рис. 3.2.3. Структура полностью ассоциативной кэш-памяти

Множественно-ассоциативная кэш-память (рис. 3.2.4) объединяет достоинства обоих рассмотренных типов. Кэш-память (как тегов, так и данных) разбивается на v подмножеств (в дальнейшем будем называть такие подмножества *модулями*), каждое из которых содержит v строк (принято говорить, что модуль имеет v входов, соответственно различают 2-, 4- и более входную ассоциативную кэш-память). Зависимость между модулем и блоками основной памяти такая же, как и при прямом отображении: на строки, входящие в модуль i ,

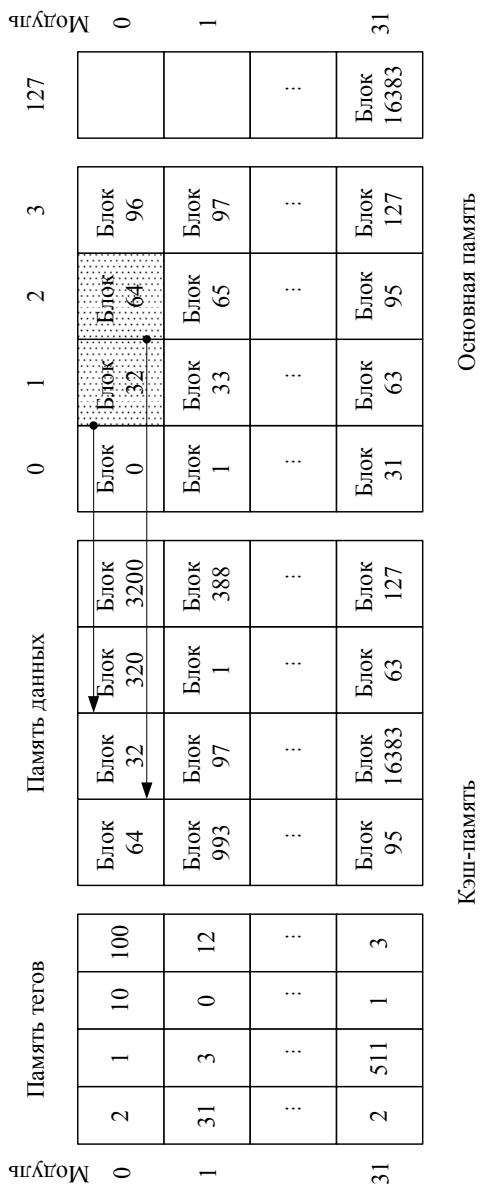


Рис. 3.2.4. Структура 4-входовой множественно-ассоциативной кэш-памяти

могут быть отображены только вполне определенные блоки основной памяти. В то же время размещение блоков по строкам модуля произвольное, и для поиска нужной строки в пределах модуля используется ассоциативный принцип, в соответствии с которым если одна из строк группы занята некоторым блоком, то следующий блок с таким же индексом будет размещен в свободной строке. Внутри группы кэш-память является полностью ассоциативной.

Увеличение количества строк в группе повышает эффективность ее работы, однако при этом возрастает количество сравнений адресов, т.е. увеличивается время обработки запроса ячейки ОЗУ.

Эффективность работы кэш-памяти характеризуется коэффициентом успешных обращений. Кэш-память с прямым отображением имеет такой же коэффициент успешных обращений, что и 2-входная множественно-ассоциативная кэш-память.

При использовании кэш-памяти может нарушаться целостность данных, т.е. содержимое кэш-памяти не будет соответствовать содержимому ОЗУ. Это происходит после записи в кэш-память. Существует несколько способов решения проблемы, основными являются способ сквозной записи и способ обратной записи.

Способ сквозной записи заключается в том, что данные одновременно записываются и в кэш-память, и в ОЗУ.

Способ обратной записи состоит в том, что данные в ОЗУ записываются только в том случае, если содержимое кэш-памяти изменилось. Для этого каждой строке кэш-памяти ставится в соответствие специальный бит, указывающий на изменение содержимого строки. При замещении строки кэш-памяти новым блоком из ОЗУ проверяется состояние этого бита, и если он установлен, то выполняется перезапись блока из кэш-памяти в ОЗУ. Только после этого в кэш-памяти размещается новый блок из ОЗУ. Данный способ более эффективен по сравнению со способом сквозной записи, поскольку сокращает число обращений к ОЗУ.

Схема обнаружения и исправления ошибок

При работе с полупроводниковой памятью не исключено возникновение различного рода отказов и сбоев. Причиной отказов могут быть производственные дефекты, повреждение микросхем или их физический износ. Отказы проявляются в том, что в отдельных раз-

рядах одной или нескольких ячеек постоянно считывается 0 или 1 вне зависимости от реально записанной информации. *Сбой* — это случайное событие, выражающееся в неверном считывании (или записи) информации в отдельных разрядах одной или нескольких ячеек, не связанное с дефектами микросхемы. Сбои обычно обусловлены проблемами с источником питания или с воздействием альфа-частиц, возникающих в результате распада радиоактивных элементов, которые в небольших количествах присутствуют практически в любых материалах. Как отказы, так и сбои крайне нежелательны, поэтому в большинстве систем основной памяти содержатся схемы, предназначенные для обнаружения и исправления ошибок.

Вне зависимости от того, как именно реализуются контроль и исправление ошибок, в основе их всегда лежит введение избыточности. Это означает, что контролируемые данные дополняются контрольными разрядами, благодаря которым и возможно обнаружение ошибок, а в ряде методов — их коррекция.

Общая схема обнаружения и исправления ошибок представлена на рис. 3.2.5. Перед записью M -разрядных данных в память производится их обработка, обозначенная на рисунке функцией f , в результате которой формируется добавочный K -разрядный код. В память заносятся как данные, так и этот вычисленный код, т. е. $(M + K)$ -разрядная информация. При чтении информации повторно формируется K -разрядный код, который сравнивается с аналогичным кодом, считанным из ячейки. Сравнение приводит к одному из трех результатов:

- не обнаружено ни одной ошибки; извлеченные из ячейки данные подаются на выход памяти;
- обнаружена ошибка, которая может быть исправлена; биты данных и добавочного кода подаются на схему коррекции, после исправления ошибки в M -разрядных данных они поступают на выход памяти; коды, используемые для подобных операций, называют *корректирующими*, или *кодами с исправлением ошибок*;
- обнаружена ошибка, и она не может быть исправлена; выдается сообщение о неисправимой ошибке.

Код с обнаружением ошибки основан на добавлении к каждому байту информации одного бита паритета. *Битом паритета* называют дополнительный бит, значение которого устанавливается таким, чтобы суммарное число единиц в данных с учетом этого дополни-

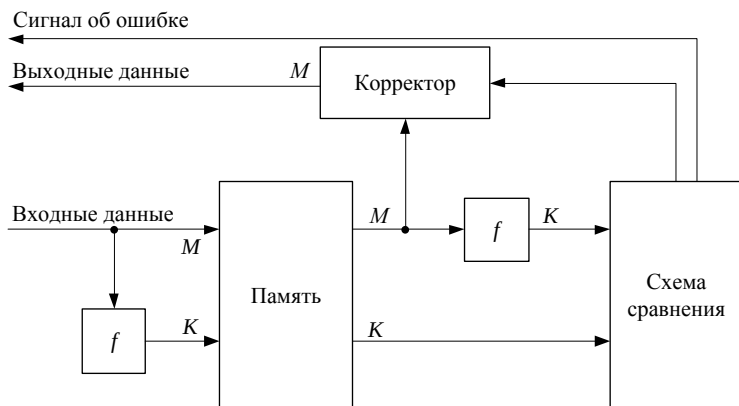


Рис. 3.2.5. Общая схема обнаружения и исправления ошибок

тельного бита было четным или нечетным (в ряде систем за основу берется четность, в иных — нечетность). Для 32-разрядного слова требуется четыре бита паритета, т. е. ячейка памяти должна хранить 36 разрядов. При записи слова в память для каждого байта формируется бит паритета. Это может быть сделано с помощью схемы в виде дерева, составленного из схем сложения по модулю 2. При чтении из памяти выполняется аналогичная операция над считанными информационными битами, а ее результат сравнивается с битом паритета, вычисленным при записи и хранившимся в памяти. Такой способ кодирования позволяет обнаружить ошибку, если исказилось нечетное число битов. При четном числе ошибок способ неработоспособен. К сожалению, фиксируя ошибку, данный способ кодирования не может указать ее местоположение, что позволило бы внести исправления, в силу чего его и называют кодом с обнаружением ошибки.

В основе корректирующих кодов лежит достаточно простая идея. Для контроля двоичной кодовой последовательности длиной M битов к ней добавляют K дополнительных контрольных разрядов так, чтобы общая длина последовательности составляла $M + K$ разрядов. В этом случае из возможных $N = 2^{M+K}$ комбинаций интерес представляют только $L = 2^M$ последовательностей, которые называют *разрешенными*. Оставшиеся $N - L$ последовательности называют *запрещенными*. Если при обработке (записи в память, считывании

или передаче) разрешенной кодовой последовательности происходят ошибки и возникает одна из запрещенных последовательностей, то тем самым эти ошибки обнаруживаются. Если же ошибки превращают одну разрешенную последовательность в другую, то такие ошибки не обнаруживаются.

Для исправления ошибок необходимо выполнить разбиение множества запрещенных последовательностей на L непересекающихся подмножеств и каждому подмножеству поставить в соответствие одну из разрешенных последовательностей. Тогда, если была принята некоторая запрещенная последовательность, входящая в одно из подмножеств, считается, что передана разрешенная последовательность, соответствующая этому подмножеству, производится замена, чем и исправляется возникшая ошибка.

Контрольные вопросы

1. Перечислите основные характеристики кэш-памяти.
2. Каковы принципы организации кэш-памяти?
3. Какие типы кэш-памяти существуют?
4. Сформулируйте основные принципы обнаружения и исправления ошибок.
5. Какие существуют способы записи в кэш-память?

Литература

1. **Цилькер Б.Я., Орлов С.А.** Организация ЭВМ и систем: учебник для вузов. — СПб.: Питер, 2004. — 654 с.
2. Микропроцессорные системы: учеб. пособие для вузов / **Е.К. Александров, Р.И. Грушвицкий, М.С. Куприянов и др.; под ред. Д.В. Пузанкова.** — СПб.: Политехника, 2002. — 935 с.
3. **Антошина И.В., Котов Ю.Т.** Микропроцессоры и микропроцессорные системы (аналитический обзор): учеб. пособие. — М.: МГУЛ, 2005. — 432 с.

Лекция 3.3. Обмен информацией между микропроцессором и внешними устройствами. Арбитр магистрали

Обмен информацией между микропроцессором и внешними устройствами

Информационная связь между устройствами вычислительной системы осуществляется через магистраль. *Магистраль* — это набор электронных линий, связывающих воедино процессор, память и периферийные устройства (ПУ). Учитывая разнообразие сопрягаемых устройств, системная шина обычно подключается не непосредственно к периферийному устройству, а через периферийный адаптер (ПА). Известно множество периферийных адаптеров, реализующих различные интерфейсы сопряжения устройств. В широком смысле под *интерфейсом* понимают совокупность механических, электрических (аппаратных) и программных средств, с помощью которых компоненты системы объединяются для решения задачи обмена информацией. Поскольку в рамках данного курса рассматриваются аппаратные средства, в дальнейшем термин «интерфейс» будем использовать для обозначения именно аппаратных средств сопряжения.

В зависимости от функционального назначения все множество интерфейсов можно разделить на три типа: машинные, системно-модульные и системно-приборные.

Машинные интерфейсы решают задачу соединения процессора с другими функциональными блоками, а также подключения периферийных устройств.

Системно-модульные интерфейсы унифицируют сопряжение модулей (функциональных блоков), предназначенных для работы в системе, что и определяет их конструктивные особенности. Модули, выполненные с учетом применения подобного интерфейса, как правило, не рассчитаны на использование в качестве автономных приборов, которые могут работать вне системы.

Системно-приборные интерфейсы осуществляют объединение в систему модулей, которые могут работать автономно и для которых

характерны широкие функциональные возможности (измерение ряда параметров, различные режимы работы, программируемость и т.п.).

Стандартные интерфейсы можно классифицировать в зависимости от схемы соединения модулей между собой и с центральным модулем вычислительной системы (устройством обработки). Существуют три основные типовые структуры микропроцессорных систем: магистральная, магистрально-каскадная и магистрально-радиальная.

Магистральная структура основана на следующем принципе действия. Если число каналов у центрального модуля меньше требуемого числа, то отдельные модули связывают с центральным модулем через общую магистраль с последовательным во времени адресным обращением. При каждом обращении (опросе) к центральному модулю подключается только тот модуль, адрес которого вызывается программой.

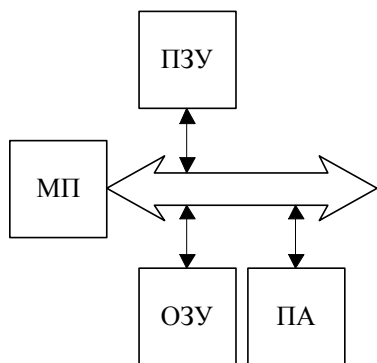


Рис. 3.3.1. Магистральная структура

Магистральная структура строится из заранее определенных периферийных устройств: оперативно-запоминающего устройства, постоянного запоминающего устройства и периферийных адаптеров (рис. 3.3.1). *Периферийный адаптер* — устройство, позволяющее подключить к магистрали блок, не приспособленный к непосредственной работе с магистралью. Данную структуру иногда называют структурой закрытого типа, так как любое изменение в ее составе может привести к неработоспособности системы.

В *магистрально-каскадной структуре* (рис. 3.3.2) контроллер шины (КШ) разделяет адресное пространство и позволяет подключить к микропроцессорной системе дополнительные внешние устройства. Выход из строя дополнительного внешнего устройства не приводит к неработоспособности всей системы.

В *магистрально-радиальной структуре* (рис. 3.3.3) контроллер ввода/вывода (КВВ) служит для преобразования одного вида инфор-

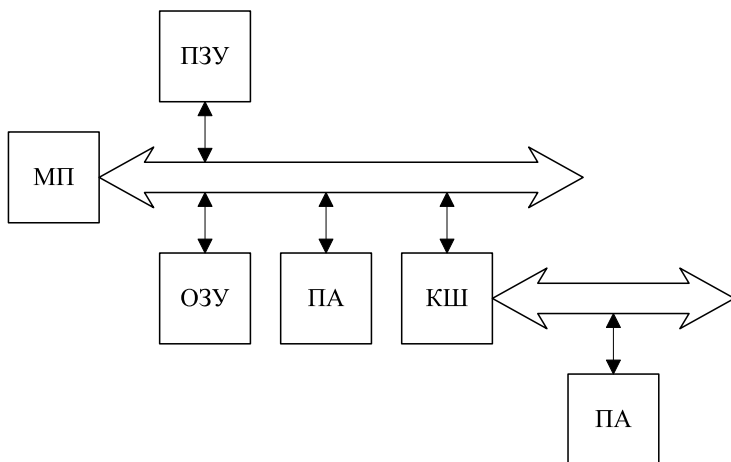


Рис. 3.3.2. Магистрально-каскадная структура

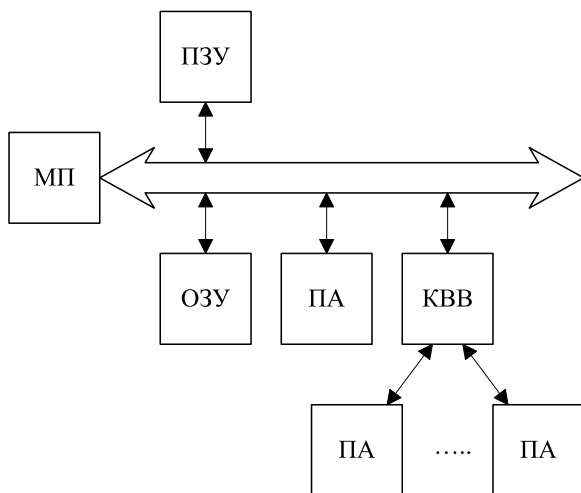


Рис. 3.3.3. Магистрально-радиальная структура

мации в другой (например, из магнитного поля на магнитном диске в параллельную последовательность электрических сигналов для обработки в процессорном элементе).

Для соединения функциональных блоков или приборов между собой служат цепи, называемые *линиями интерфейса*. Группы линий, предназначенных для выполнения определенной функции в программно-управляемом процессе передачи данных, называют *шиной*. Назначение отдельных шин и линий, их взаимное расположение (топология) играют основополагающую роль в работе интерфейса.

Магистраль включает в себя три многоразрядные шины:

- шину данных;
- шину адреса;
- шину управления.

Шина данных (Data Bus — DB) — это основная шина, которая используется для передачи информационных кодов между всеми устройствами микропроцессорной системы. Обычно в пересылке информации участвует процессор, который передает код данных в какое-то устройство или ячейку памяти либо принимает код данных из какого-то устройства или ячейки памяти. Возможна передача информации между устройствами без участия процессора. Шина данных всегда двунаправленная.

Количество разрядов (линий связи) шины определяет скорость и эффективность информационного обмена, а также максимально возможное количество команд. Обычно шина данных имеет 8, 16, 32 или 64 разряда. Понятно, что за один цикл обмена по 64-разрядной шине может передаваться 8 байтов информации, а по 8-разрядной — только 1 байт. Разрядность шины данных определяет и разрядность всей магистрали. Например, когда говорят о 32-разрядной системной магистрали, подразумевается, что она имеет 32-разрядную шину данных.

Шина адреса (Address Bus — AB) служит для установки уникального адреса (ячейки памяти или устройства), с которым процессор обменивается информацией в настоящий момент. При выставлении адреса на шину устройство, соответствующее этому адресу, получает информацию о предстоящем обмене данными. Шина адреса определяет максимальный объем памяти, а следовательно, максимально возможные размер программы и объем запоминаемых данных. Количество адресов, обеспечиваемых этой шиной, определяется как 2^N , где N — количество разрядов.

Шина управления (Control Bus — CB), в отличие от шины адреса и шины данных, состоит из отдельных управляющих сигналов. Каждый из этих сигналов во время обмена информацией выполняет заданную функцию. Некоторые сигналы служат для стробирования передаваемых или принимаемых данных (т.е. определяют моменты времени, когда информационный код выставлен на шину данных). Другие управляющие сигналы используются для подтверждения приема данных, сброса всех устройств в исходное состояние, синхронизации устройств и т.д. Кроме того, управляющие сигналы обеспечивают согласованную работу процессора (или другого хозяина магистрали) и памяти или устройства ввода/вывода. Управляющие сигналы также обслуживают запрос и предоставление прерываний, запрос и предоставление прямого доступа к памяти. Шина управления может быть как однонаправленной, так и двунаправленной.

Магистрали такого типа называют трехшинными с различными шинами адреса и данных (рис. 3.3.4).

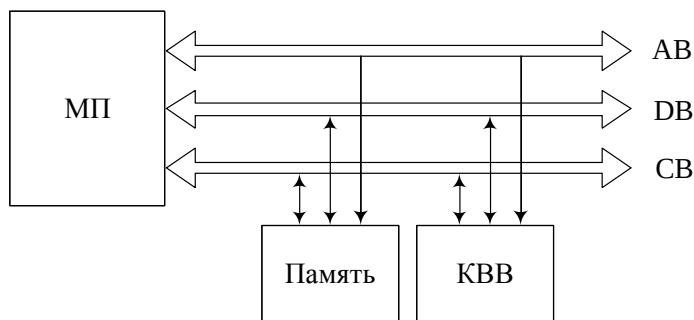


Рис. 3.3.4. Трехшинная магистраль

В некоторых системах для сокращения количества физических линий магистрали применяется совмещенная шина адрес—данные (Address Data — AD). На этой шине адрес и данные разделены во времени дополнительными сигналами: адрес сопровождается сигналом ALE (Address Latch Enable), а данные — сигналом WR (рис. 3.3.5).

Существует два способа разделения адреса и данных: первый подразумевает двухшинную организацию магистрали, а разделение шин обеспечивается уже в элементах вычислительной техники, а вто-

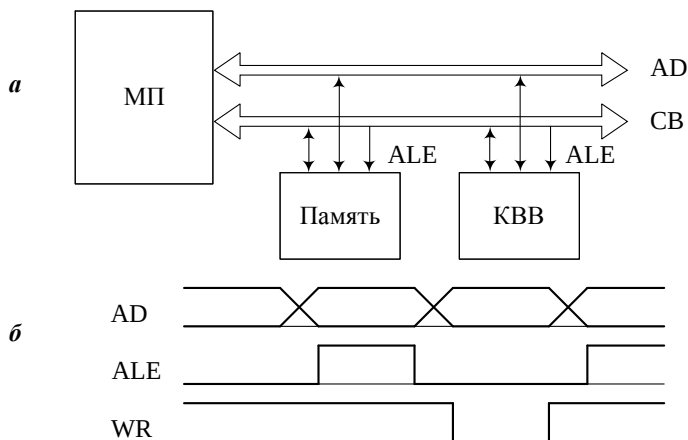


Рис. 3.3.5. Двухшинная магистраль (а) и временная диаграмма ее работы (б)

рой предполагает разделение шины сразу после выхода из процессора и по всей остальной вычислительной системе проводится трехшинная магистраль.

Обмен данными в микропроцессорной системе может быть параллельным или последовательным.

Параллельный обмен может быть реализован синхронным или асинхронным способом.

Синхронная передача данных подразумевает присутствие синхронизирующего сигнала, с помощью которого четко определяются моменты времени чтения и записи данных. Простейшая подсистема синхронного параллельного обмена должна обеспечить лишь дешифрацию адреса ВУ и электрическое подключение его данных к системной шине данных по соответствующим управляющим сигналам.

На рис. 3.3.6 показано устройство *асинхронного параллельного ввода/вывода* информации в составе микропроцессорной системы на базе буферных регистров.

При обращении процессора (он в подобных циклах играет роль активного устройства) к устройству ввода адрес соответствующего регистра помещается процессором на шину адреса и формируется управляющий сигнал IOR. Дешифратор адреса, включающий и ли-

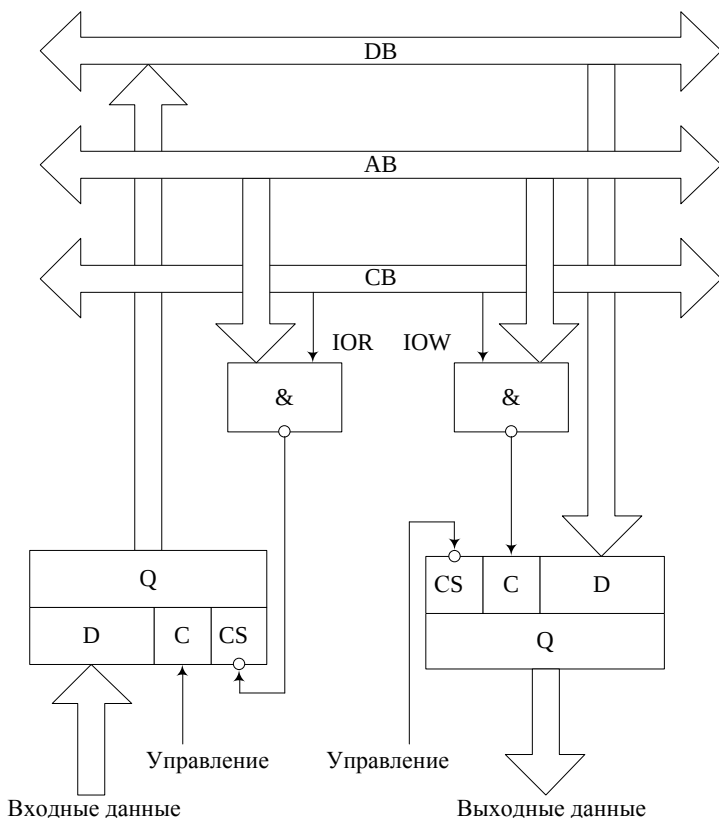


Рис. 3.3.6. Параллельный асинхронный обмен на базе буферных регистров

нию IOR, при совпадении адреса и управляющего сигнала активизирует выходные линии регистра и его содержимое поступает по шине данных в процессор.

Аналогично идет обращение процессора к устройству вывода. Совпадение адреса устройства на шине адреса с активным уровнем сигнала IOW обеспечивает «защелкивание» состояния шины данных в регистре вывода.

Характерно, что при асинхронном параллельном обмене процессор не анализирует готовность ВУ к обмену, а длительности адреса,

данных и управляющего сигнала целиком определяются тактовой системой процессора и принятым алгоритмом командного цикла.

Последовательный обмен также может быть реализован синхронным или асинхронным способом.

При синхронном способе передатчик генерирует две последовательности — информационную TxD и синхрои импульсы CLK, которые передаются на приемник по разным линиям. Синхрои импульсы обеспечивают синхронизацию передаваемых битов, а начало передачи отмечается по-разному.

При организации внешней синхронизации (рис. 3.3.7) сигнал начала передачи BD генерируется передатчиком и передается на приемник по специальной линии.

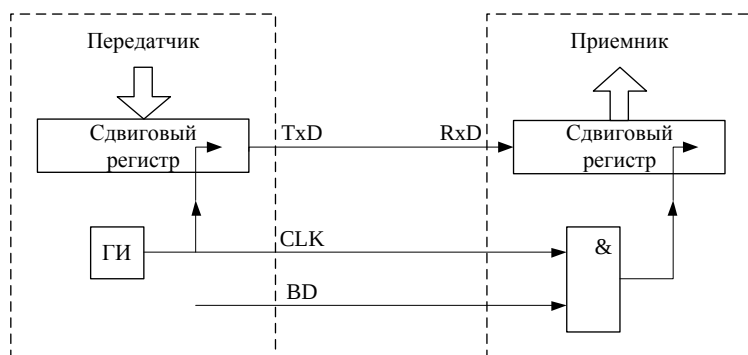


Рис. 3.3.7. Последовательный синхронный обмен с внешней синхронизацией (ГИ — генератор импульсов)

При организации внутренней синхронизации системы генерируют на линию данных специальные коды длиной 1 – 2 байта — символы синхронизации. Для каждого приемника предварительно определяются конкретные синхросимволы, таким образом можно осуществлять адресацию конкретного абонента из нескольких, работающих на одной линии. Каждый приемник постоянно принимает биты с линии RxD, формирует символы и сравнивает с собственными синхросимволами. При совпадении принятых символов с заданными для этого приемника синхросимволами последующие биты поступают в канал данных приемника. В случае реализации внутренней синхронизации

между приемником и передатчиком «прокладывают» только две линии — данных и синхروимпульсов.

При асинхронном способе можно ограничиться одной линией — данных.

Последовательный асинхронный обмен по единственной линии представлен на рис. 3.3.8. Пока передачи нет, передатчик удерживает на линии высокий уровень H . Передача начинается с выдачи на линию старт-бита низкого уровня (длительности всех битов одинаковы и определяются частотой передатчика f_T).

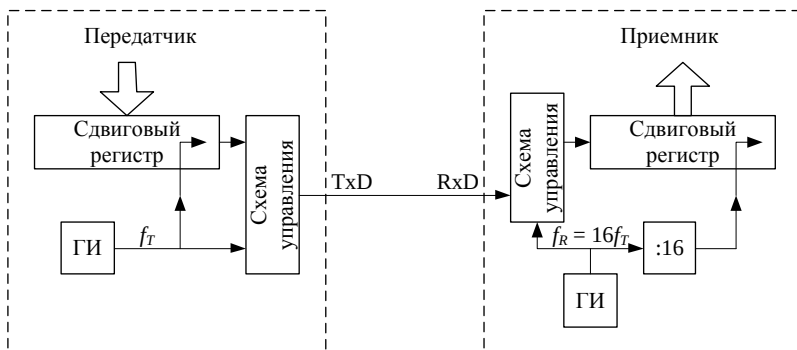


Рис. 3.3.8. Пример последовательного асинхронного обмена данными

Чтобы избежать ошибок синхронизации, частота приемника f_R устанавливается в несколько раз больше f_T (в рассмотренном примере используется частота, равная $16f_T$). Когда приемник обнаруживает на линии перепад $H \rightarrow L$, он включает счетчик тактов до 16, причем еще дважды за период T проверяет состояние линии. Если низкий уровень L подтверждается, приемник считает, что принял старт-бит, и включает счетчик принимаемых битов. Если во второй и третьей проверках на линии определяется уровень H , то перепад считается помехой и старт-бит не фиксируется.

Каждый последующий (информационный) бит принимается таким образом, что за период T трижды проверяется состояние линии (например, в 3-, 8- и 11-м тактах приемника) и значение принимаемого бита определяется по мажоритарному принципу. Принятый бит помещается слева в сдвиговой регистр приемника. После принятия

последнего информационного бита (количество битов в посылке определяется протоколом обмена и составляет обычно от 5 до 9) обязательно должен последовать стоп-бит *H*-уровня. Во время поступления стоп-бита содержимое сдвигового регистра приемника передается в память, а в регистр передатчика может загружаться новая порция информации для передачи. Отсутствие стоп-бита воспринимается приемником как ошибка передачи посылки.

После стоп-бита можно формировать старт-бит новой посылки или «держат паузу» произвольной длительности, при которой на линии присутствует *H*-уровень.

Наличие старт-бита позволяет в начале каждой посылки синхронизировать фазы приемника и передатчика, компенсировав неизбежный уход фаз передатчика и приемника. Короткие посылки и относительно низкая частота передачи позволяют надеяться, что неизбежное рассогласование частот передатчика и приемника не приведет к ошибкам при передаче посылки.

Рассмотрим возможные режимы обмена информацией между микропроцессором и внешними устройствами.

Обмен информацией между МП и ВУ по инициативе МП (рис. 3.3.9). По мере обработки информации МП обращается к ВУ за получением следующего блока данных. ВУ выставляет подтверждение запроса, сообщая тем самым, что информация готова.



Рис. 3.3.9. Обмен информацией между МП и ВУ по инициативе МП

Этот режим целесообразен в тех случаях, когда динамические характеристики МП и ВУ близки между собой, т. е. отсутствует циклическое ожидание процессорного элемента.

Обмен информацией между МП и ВУ по инициативе ВУ (рис. 3.3.10). ВУ по мере готовности данных выставляет запрос прерывания, МП входит в режим прерывания, сохраняет регистры в стековой области памяти, забирает необходимую информацию из ВУ, записывает ее в память и возвращается к прерванной программе. В дан-

ном случае непроизводительные потери времени связаны с тем, что МП необходимо обеспечить вход и выход из режима прерывания.

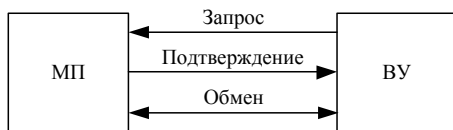


Рис. 3.3.10. Обмен информацией между МП и ВУ по инициативе ВУ

Этот режим используется, когда динамические характеристики МП значительно выше динамических характеристик ВУ.

Обмен информацией между двумя ВУ по инициативе одного из них (рис. 3.3.11). Данный режим подразумевает наличие активного ВУ. По мере готовности информации или через заранее определенный промежуток времени ВУ формирует запрос магистрали. Получив его, МП завершает обмен информацией по магистрали, переводит свои шины в третье состояние (состояние высокого импеданса) и выставляет подтверждение запроса. По данному сигналу ВУ формирует полную совокупность управляющих сигналов для магистрали. Завершив обмен, ВУ снимает запрос шины, и МП выходит на магистраль.

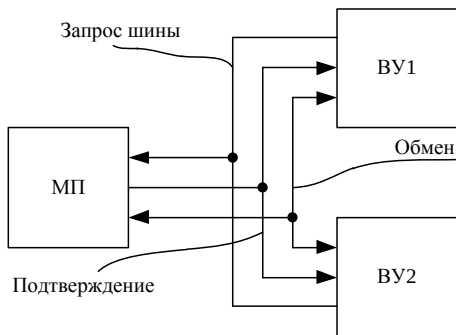


Рис. 3.3.11. Обмен информацией между двумя ВУ по инициативе одного из них

Данный режим называется *режимом захвата магистрали*. Если ВУ представляет собой память, то такой режим называется режимом *прямого доступа к памяти*.

Арбитр магистрали

В каждый момент времени на магистрали допускается присутствие только одного активного устройства. В простейших системах в качестве активного устройства используется процессор, который и организует все управление магистралью. В сложных системах со многими активными модулями магистраль распределяется между ними в соответствии с приоритетными соглашениями. В современных вычислительных системах задача распределения магистрали возлагается на арбитра магистрали.

Арбитра магистрали характеризуют следующие параметры:

- число сигнальных линий;
- число обрабатываемых запросов;
- время реакции на запрос;
- тип синхронизации;
- алгоритмы выбора приоритета;
- способ реализации.

Рассмотрим подробнее некоторые параметры арбитра магистрали.

Алгоритмы выбора приоритета. В реальных системах на роль ведущего вправе претендовать сразу несколько из подключенных к шине устройств, однако управлять шиной в каждый момент времени может только одно из них. Чтобы исключить конфликты, шина должна предусматривать определенные механизмы арбитража запросов и правила предоставления шины одному из запросивших устройств. Решение принимается на основе приоритетов.

Каждому потенциальному ведущему устройству присваивается определенный уровень приоритета, который может оставаться неизменным (статический или фиксированный приоритет) либо изменяться по какому-либо алгоритму (динамический приоритет).

Основной недостаток статических приоритетов в том, что устройства с высоким приоритетом в состоянии полностью блокировать доступ к шине устройств с низким приоритетом. Системы с динамическими приоритетами дают шанс каждому из запросивших устройств рано или поздно получить право на управление шиной, т. е. в таких системах реализуется принцип равнодоступности.

Наибольшее распространение получили следующие алгоритмы динамических приоритетов:

- простая циклическая смена приоритетов;

- циклическая смена приоритетов с учетом последнего запроса;
- смена приоритетов по случайному закону;
- алгоритм равных приоритетов;
- алгоритм наиболее давнего использования.

В *алгоритме простой циклической смены приоритетов* после каждого цикла арбитража все приоритеты понижаются на один уровень, при этом устройство, имевшее ранее низший уровень приоритета, получает наивысший приоритет.

В *алгоритме циклической смены приоритетов с учетом последнего запроса* все возможные запросы упорядочиваются в виде циклического списка. После обработки очередного запроса обслуживаемому ведущему устройству назначается низший уровень приоритета. Следующее в списке устройство получает наивысший приоритет, а остальным устройствам приоритеты назначаются в убывающем порядке, согласно их следованию в циклическом списке.

В обоих алгоритмах циклической смены приоритетов каждое ведущее устройство имеет шанс получить шину в свое распоряжение, однако большее распространение получил второй алгоритм.

При *смене приоритетов по случайному закону* после очередного цикла арбитража с помощью генератора псевдослучайных чисел каждому ведущему устройству присваивается случайное значение уровня приоритета.

В *алгоритме равных приоритетов* при поступлении к арбитру нескольких запросов каждый из них имеет равные шансы на обслуживание. Возможный конфликт разрешается арбитром. Такой алгоритм принят в асинхронных системах.

В *алгоритме наиболее давнего использования* после каждого цикла арбитража наивысший приоритет присваивается ведущему устройству, которое дольше других не использовал шину.

Способ реализации арбитража магистрали. Арбитраж магистрали делится на централизованный и децентрализованный.

При *централизованном арбитраже* в системе имеется специальное устройство — центральный арбитр, или центральный контроллер шины, — ответственное за предоставление доступа к шине только одному из запросивших устройств. Это устройство может быть самостоятельным модулем или частью процессора. Наличие на шине только одного арбитра означает, что в централизованной схеме существует единственная точка отказа. Интерфейсы с централизованным

арбитражем отличаются пониженной надежностью, требуют отдельных линий запроса и разрешения доступа к магистрали для каждого модуля системы, но для их реализации не нужны большие аппаратные затраты.

При *децентрализованном*, или *распределенном арбитраже* единый арбитр отсутствует. Каждое устройство содержит блок управления доступом к шине, и при совместном использовании шины такие блоки взаимодействуют друг с другом, разделяя между собой ответственность за доступ к шине. По сравнению с централизованным децентрализованный арбитраж менее чувствителен к отказам претендующих на шину устройств.

Децентрализованный арбитраж обеспечивает высокую отказоустойчивость системы, простое изменение ее конфигурации при повреждениях или отказах отдельных модулей, имеет гибкую систему приоритетов, высокую скорость обслуживания запросов. Однако аппаратные затраты растут пропорционально числу абонентов, имеются сложности в организации системной синхронизации при арбитраже.

Рассмотрим пример работы арбитра магистрали (рис. 3.3.12). Активные устройства (обозначены кружочками) независимо друг от дру-

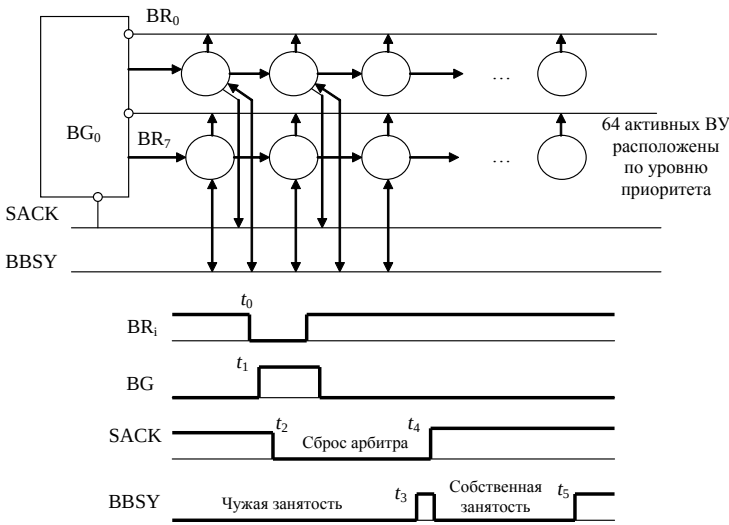


Рис. 3.3.12. Пример работы арбитра магистрали с временными диаграммами

га генерируют запрос на захват магистрали BR_i . В момент времени t_1 арбитр принимает запрос, формирует сигнал BG_i , который распространяется по линии ВУ, одно из которых сформировало сигнал запроса магистрали. В момент времени t_2 ВУ, поглотившее сигнал BG_i , формирует сигнал сброса арбитра. Данный сигнал является признаком для арбитра, что необходимо прекратить поиски других активных ВУ до того времени, пока не будет предоставлена магистраль активному ВУ, поглотившему BG_i . В момент времени t_3 ВУ, занимавшее магистраль, освобождает ее, и магистраль занимает активное ВУ. В момент времени t_5 , завершив обмен информацией, активное ВУ освобождает магистраль.

Контрольные вопросы

1. Какие основные типовые структуры микропроцессорных систем вы знаете?
2. Каково назначение шин данных, адреса и управления?
3. Какими характеристиками обладает арбитр магистрали?
4. Расскажите об основных алгоритмах выбора приоритета при арбитраже магистрали.
5. Поясните, как происходит асинхронный обмен информацией.

Литература

1. **Угрюмов Е.П.** Цифровая схемотехника: учеб. пособие для вузов. — 2-е изд., перераб. и доп. — СПб.: БХВ-Петербург, 2005. — 800 с.
2. **Ливенцов С.Н., Вильнин А.Д., Горюнов А.Г.** Основы микропроцессорной техники: учеб. пособие. — Томск: Изд-во Томского политехнического ун-та, 2007. — 118 с.
3. **Цилькер Б.Я., Орлов С.А.** Организация ЭВМ и систем: учебник для вузов. — СПб.: Питер, 2004. — 654 с.

Лекция 3.4. Режим прямого доступа к памяти

Режим прямого доступа к памяти (ПДП, или DMA — Direct Memory Access) используется, если необходимо произвести пересылку значительного массива данных между ОЗУ и каким-либо внешним устройством. В этом режиме обмен данными между устройствами и основной памятью происходит без участия процессора. Реализация такой пересылки с помощью соответствующей программы обмена потребовала бы выполнения отдельной команды пересылки для передачи каждого байта или слова. При этом необходим определенный объем памяти для хранения программы, а также значительное время для ее выполнения.

Обменом информацией в режиме ПДП управляет не программа, выполняемая процессором, а электронные схемы, внешние по отношению к ядру процессора. Обычно схемы, управляющие обменом данными в режиме ПДП, размещаются или в специальном контроллере, который называется *контроллером прямого доступа к памяти*, или в контроллере самого внешнего устройства.

Обмен данными в режиме ПДП позволяет использовать в микропроцессорном устройстве быстродействующие внешние запоминающие устройства, поскольку ПДП может обеспечить время обмена одним байтом данных между памятью и внешним запоминающим устройством, равное циклу обращения к памяти, в отличие от процессора, которому может потребоваться больше времени, в частности, из-за подготовки и дешифрации команд.

Для реализации режима ПДП необходимо обеспечить непосредственную связь контроллера ПДП и памяти микропроцессорного устройства. Для этой цели можно было бы использовать специально выделенные шины адреса и данных, связывающие контроллер ПДП с основной памятью, но такое решение приведет к значительному усложнению устройства в целом, особенно при подключении нескольких внешних запоминающих устройств. Для сокращения количества линий в шинах микропроцессорного устройства контроллер ПДП подключается к памяти посредством уже имеющегося системного интерфейса. При этом возникает проблема совместного использования шин

системного интерфейса процессором и контроллером ПДП. Можно выделить два основных способа ее решения: реализация обмена в режиме ПДП *с захватом цикла* и *с блокировкой процессора*.

Существуют две разновидности ПДП с захватом цикла:

1) применяющий шину данных в те моменты, когда процессор ее не использует;

2) применяющий дополнительные управляющие сигналы контроля обмена данными.

Первый способ организации ПДП состоит в том, что для обмена используются те машинные циклы процессора, в которых он не обменивается данными с памятью. В такие циклы контроллер ПДП может обмениваться данными с памятью, не мешая работе процессора. Однако возникает необходимость выделения таких циклов, чтобы не произошло временного перекрытия обмена ПДП с обменом, инициируемым процессором. В некоторых процессорах формируется специальный управляющий сигнал, указывающий циклы, в которых процессор не обращается к системному интерфейсу. При использовании других процессоров для выделения таких циклов необходимо применение в контроллерах ПДП специальных селектирующих схем, что усложняет их конструкцию.

Применение рассмотренного способа организации ПДП не снижает производительности микропроцессорной системы, но при этом обмен в режиме ПДП возможен только в случайные по отношению к устройству, формирующему запрос, моменты времени одиночными байтами или словами.

Второй способ организации ПДП с захватом цикла — принудительное отключение процессора от шин системного интерфейса. В этом случае системный интерфейс микропроцессорной системы дополняется двумя линиями для передачи управляющих сигналов запроса прямого доступа к памяти (ЗПДП) и разрешения прямого доступа к памяти (РПДП).

Управляющий сигнал ЗПДП формируется контроллером ПДП (см. рис. 3.4.1). Процессор, получив этот сигнал, приостанавливает выполнение очередной команды, выдает на системный интерфейс управляющий сигнал РПДП и отключается от шин системного интерфейса. Во время передачи информации по каналу ПДП процессор переводит схемы управления магистралями в высокоомное состояние и тем самым изолируется от остальной части системы. Состояния

внутренних регистров замораживаются, т. е. процессор в режиме ожидания сохраняет то информационное состояние, которое было в нем к моменту удовлетворения запроса канала ПДП. С этого момента все шины системного интерфейса управляются контроллером ПДП. Контроллер ПДП, используя шины системного интерфейса, осуществляет обмен одним байтом или словом данных с памятью микропроцессорной системы и затем, сняв сигнал ЗПДП, возвращает управление системным интерфейсом процессору. Как только контроллер ПДП готов к обмену следующим байтом, он вновь «захватывает» цикл процессора и т. д. В интервалах между сигналами ЗПДП процессор продолжает выполнять команды программы. Тем самым выполнение программы замедляется, но в меньшей степени, чем при обмене в режиме прерываний (см. лекцию 2.6).

Применение в микропроцессорной системе обмена данными с ВУ в режиме ПДП всегда требует предварительной подготовки, а именно: для каждого ВУ необходимо выделить область памяти, используемую при обмене, и указать ее размер, т. е. количество записываемых в память или читаемых из памяти байтов (слов) информации. Следовательно, контроллер ПДП обязательно должен иметь в своем составе регистр адреса (РА) и счетчик байтов (слов). Перед началом обмена с ВУ в режиме ПДП процессор должен выполнить программу загрузки. Эта программа обеспечивает запись в указанные регистры контроллера ПДП начального адреса выделенной ВУ памяти и ее размера в байтах или словах в зависимости от того, какими порциями информации ведется обмен.

Вышеизложенное не относится к начальной загрузке программ в память в режиме ПДП. В этом случае содержимое РА и счетчика байтов (слов) устанавливается переключателями или перемычками непосредственно на плате контроллера.

Блок-схема контроллера ПДП, обеспечивающего ввод данных в память микропроцессорной системы по инициативе ВУ в режиме ПДП с захватом цикла, приведена на рис. 3.4.1.

Перед началом очередного сеанса ввода данных из ВУ процессор загружает в регистры его контроллера следующую информацию: в счетчик байтов — количество принимаемых байтов данных; в РА — начальный адрес области памяти для вводимых данных. Тем самым контроллер подготавливается к выполнению операции ввода данных из ВУ в память микропроцессорной системы в режиме ПДП.

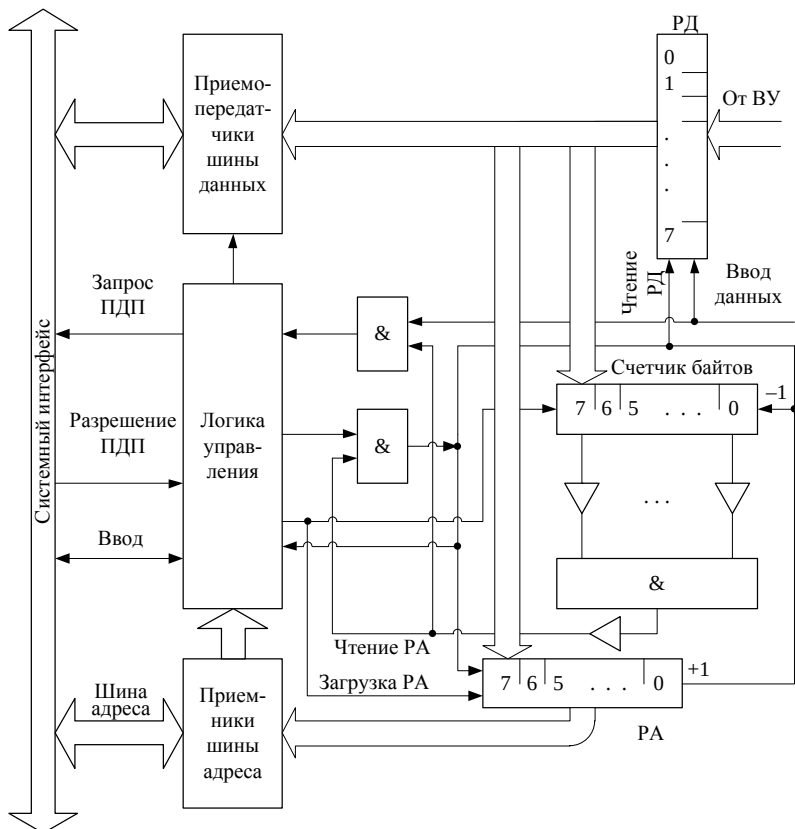


Рис. 3.4.1. Контроллер ПДП для ввода данных из ВУ в режиме с захватом цикла и отключением процессора от шин системного интерфейса

Байты данных из ВУ поступают в регистр данных (РД) контроллера в постоянном темпе. При этом каждый байт сопровождается управляющим сигналом из ВУ «Ввод данных», который обеспечивает запись байта данных в РД контроллера. По этому же сигналу и при ненулевом состоянии счетчика байтов контроллер формирует сигнал ЗПДП. По ответному сигналу процессора РПДП контроллер выставляет на шины адреса и данных системного интерфейса содержимое своих РА и РД соответственно. Формируя управляющий сиг-

нал «Вывод», контроллер ПДП обеспечивает запись байта данных из своего РД в общую память. Сигнал РПДП используется в контроллере и для модификации счетчика байтов и РА. По каждому сигналу РПДП из содержимого счетчика байтов вычитается единица, и как только содержимое счетчика становится равным нулю, контроллер прекращает формирование сигналов ЗПДП.

На примере простого контроллера ПДП мы рассмотрели только процесс подготовки контроллера и непосредственно передачу данных в режиме ПДП. На практике любой сеанс обмена данными с ВУ в режиме ПДП всегда инициируется программой, выполняемой процессором, и включает два следующих этапа:

1) этап подготовки ВУ к очередному сеансу обмена, когда процессор опрашивает состояние ВУ (проверяет его готовность к обмену) и посылает в ВУ команды, обеспечивающие подготовку ВУ к обмену. Затем выполняется загрузка регистров контроллера ПДП. На этом подготовка к обмену в режиме ПДП завершается, и процессор переключается на выполнение другой программы;

2) этап обмена данными в режиме ПДП, который начинается после завершения подготовительных операций в ВУ по инициативе либо ВУ, как это было рассмотрено выше, либо процессора. В этом случае контроллер ПДП необходимо дополнить регистром состояния и управления, содержимое которого будет определять режим работы контроллера ПДП. Один из разрядов этого регистра будет инициировать обмен данными с ВУ. Загрузка информации в регистр состояния и управления контроллера ПДП производится программным путем.

Наиболее распространенным является обмен в режиме ПДП с блокировкой процессора. В этом случае управление системным интерфейсом передается контроллеру ПДП не на время обмена одним байтом, а на время обмена блоком данных. Такой режим ПДП применяется в тех случаях, когда время обмена одним байтом с ВУ сопоставимо с циклом системной шины.

В микропроцессорной системе можно использовать несколько ВУ, работающих в режиме ПДП. Предоставление таким ВУ шин системного интерфейса для обмена данными производится на приоритетной основе (рис. 3.4.2).

Что случается после получения контроллером ПДП контроля над шиной памяти, зависит от того, какой режим работы установлен для контроллера ПДП. Помимо вышеописанных, существует режим, ко-

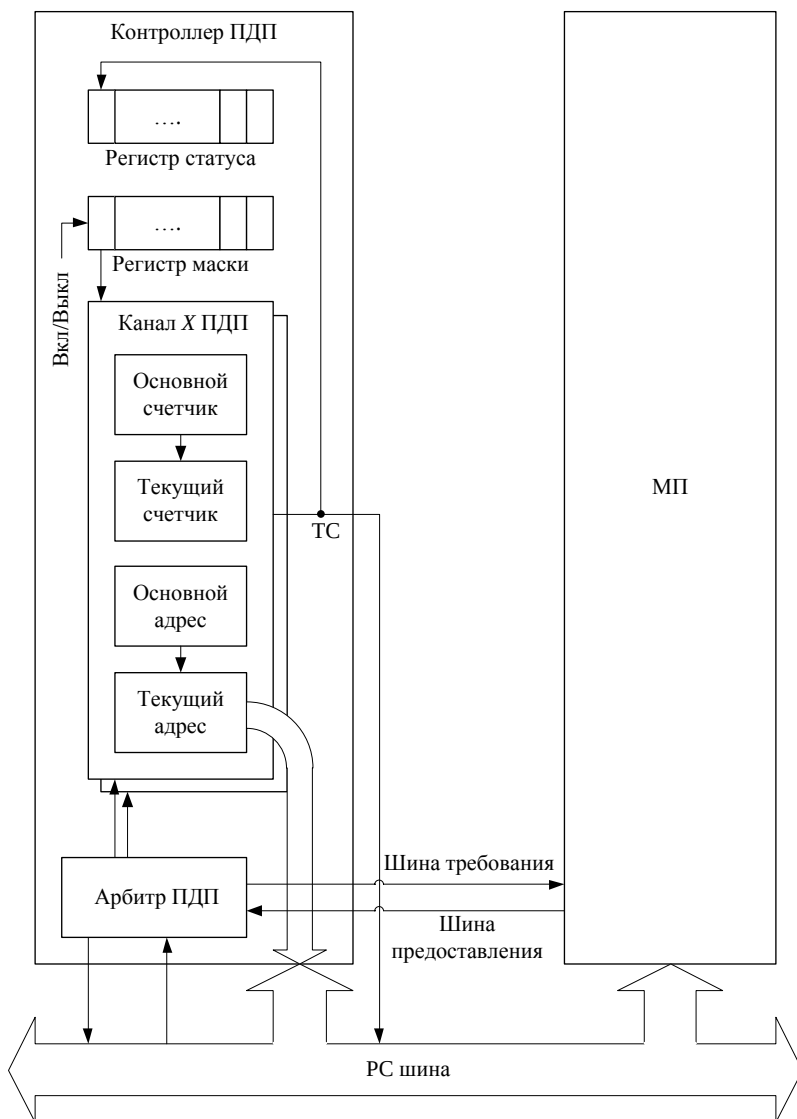


Рис. 3.4.2. Организация многоканального ПДП

торый называется *взрывным* (burst). Когда контроллер ПДП работает в этом режиме, он сохраняет контроль над шиной памяти в течение всего процесса передачи данных. Недостатком такого режима является то, что МП не может работать с шиной памяти, пока контроллер ПДП не завершит передачу данных. Другими словами, процессор может работать только со своей кэш-памятью первого и второго уровней. Это ограничивает возможности МП на время ожидания процессором завершения операции и возвращения контроля над шиной памяти контроллером ПДП.

Режим ПДП с блокировкой процессора используется наиболее часто, хотя большинство контроллеров ПДП могут работать в различных режимах.

Каков оптимальный размер блока, который будет передаваться перед возвращением контроля над шиной памяти и повторением запроса, — вопрос довольно сложный. Огромное значение в этом случае имеет наблюдаемая частота ошибок. Когда обнаруживаются ошибки при передаче данных, необходимо повторить передачу данных. Поэтому, если ошибки случаются часто, оптимальными являются блоки небольшой длины. Однако если ошибок немного, малый размер блоков приведет к увеличению количества циклов запросов и ответов между контроллером ПДП и МП, что увеличит общее время передачи данных. Как именно реализуются эти решения, зависит от конкретного производителя контроллеров ПДП.

Один из важных режимов ПДП — *цепочечный режим*, обеспечивающий, в частности, реализацию циклических буферов на основе ОЗУ. В этом режиме инициализация заключается в подготовке последовательности конфигурационных слов, связанных между собой ссылками. По завершению очередного цикла контроллер ПДП автоматически инициализируется конфигурационным словом из подготовленной последовательности. При этом каждое такое слово является уникальным и может определять различные источники, приемники и объемы передаваемой информации (рис. 3.4.3).

Поле «Режим» предназначено для установки различных параметров цикла ПДП, например, формируется или нет сигнал прерывания по завершению обмена и т. п.

Когда контроллер ПДП завершает передачу блока и возвращает контроль над шиной памяти МП, процессор может использовать шину для своих целей. Это дает процессору возможность обновлять

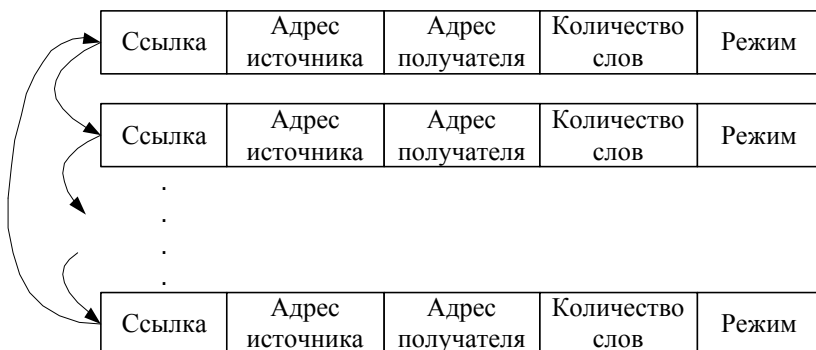


Рис. 3.4.3. Последовательность конфигурационных слов цепочечного режима ПДП

индикаторы прогресса, а также файловую систему информацией, относящейся к осуществляемым операциям ПДП.

Использование контроллера ПДП может привести к проблеме, возникающей при операциях ПДП, связанных с когерентностью кэш-памяти. Когда процессор обращается к некоторому месту в памяти, содержимое этого места записывается в кэш-память процессора. Если реализуются операции ПДП, содержимое кэш-памяти процессора может отличаться от содержимого памяти.

У этой проблемы существует решение. Например, системы с полной когерентностью кэш-памяти реализуют аппаратное решение, в котором контроллер ПДП отправляет сигнал контроллеру кэш-памяти, сообщающий о месте в памяти, с которым он работает. Если при прямом доступе требуется произвести запись в это место, контроллер кэш-памяти аннулирует содержимое кэш-памяти процессора. Если производится чтение, контроллер кэш-памяти обновит содержимое кэш-памяти процессора, чтобы оно содержало текущее значение (которое и должно храниться в кэш-памяти МП).

Такой метод работы требует некоторых накладных расходов, но при этом гарантирует когерентность кэш-памяти процессора.

Контрольные вопросы

1. Каково функциональное назначение устройства ПДП?
2. Какие существуют режимы ПДП?
3. Расскажите о ПДП с захватом шины.
4. Что происходит, если контроллер ПДП занял шину памяти, а процессор начал к ней обращаться?
5. Расскажите о ПДП с блокировкой процессора.

Литература

1. **Цилькер Б.Я., Орлов С.А.** Организация ЭВМ и систем: учебник для вузов. — СПб.: Питер, 2004. — 654 с.
2. Микропроцессорные системы: учеб. пособие для вузов / **Е.К. Александров, Р.И. Грушвицкий, М.С. Куприянов и др.; под ред. Д.В. Пузанкова.** — СПб.: Политехника, 2002. — 935 с.
3. **Антошина И.В., Котов Ю.Т.** Микропроцессоры и микропроцессорные системы (аналитический обзор): учеб. пособие. — М.: МГУЛ, 2005. — 432 с.

Лекция 3.5. Виртуальная память. Устройство управления памятью

В микропроцессорных системах может возникать ситуация, когда размещение программы и данных, с которыми она работает, в основной памяти (ОП) невозможно из-за их большого объема. Так как в каждый момент времени процессор обрабатывает относительно небольшие участки кода, в основной памяти достаточно хранить только используемые в определенный период времени части программы, а остальные части можно располагать во внешних запоминающих устройствах (ВЗУ). Проблема состоит в том, что время обращения к ВЗУ существенно больше времени обращения к основной памяти, что усложняет задачу программиста, которому придется «вручную» контролировать расположение частей программы на разных уровнях иерархии памяти, в разные моменты времени.

Решением данной проблемы стало использование концепции *виртуальной памяти*, под которой понимается автоматическое управление иерархической памятью, при котором программист имеет дело с единой памятью большой емкости и высокого быстродействия. По своей сути виртуализация памяти представляет собой способ аппаратно-программной реализации концепции иерархической памяти.

Основная память представляет собой линейное пространство из N адресов и является физическим пространством памяти. Если появляется задача, требующая более N ячеек, то в рамках идеи виртуализации памяти предоставляется значительно большее адресное пространство, обычно равное общей емкости всех видов памяти и называемое *виртуальным пространством*.

Адреса виртуального пространства называют *виртуальными*, а адреса физического пространства — *физическими*. Программа использует виртуальные адреса, но поскольку для ее выполнения надо, чтобы обрабатываемые команды и данные находились в основной памяти, требуется, чтобы каждому виртуальному адресу соответствовал физический адрес. Таким образом, в процессе вычислений необходимо прежде всего переписать из ВЗУ в ОП ту часть информации, на которую указывает виртуальный адрес (отобразить виртуальное про-

странство на физическое), а затем преобразовать виртуальный адрес в физический (рис. 3.5.1).

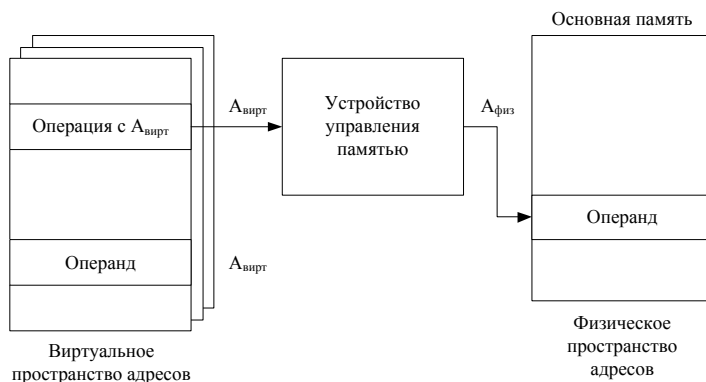


Рис. 3.5.1. Соответствие виртуального адреса физическому адресу

Среди систем виртуальной памяти можно выделить два класса: системы с фиксированным размером блоков (страничная организация) и системы с переменным размером блоков (сегментная организация). Оба класса обычно совмещают, образуя сегментно-страничную организацию виртуальной памяти.

Страничная организация виртуальной памяти

Суть *страничной организации виртуальной памяти* заключается в следующем: все адресное пространство, включая ОП и ВЗУ, разделяется на одинаковые по размеру части (кратные степени двойки), называемые *страничными кадрами*, или *фрэймами* (page frame). Выполняемые программы делятся на блоки такого же размера, т. е. постранично. Страницам виртуальной и физической памяти присваивают номера (адрес).

Для получения данных из памяти процессор посылает устройству управления памятью виртуальный адрес ячейки, состоящий из номера виртуальной страницы и смещения относительно ее начала, с целью преобразования его в физический адрес. Поскольку смещения в виртуальном и физическом адресах одинаковы, преобразованию подвергается лишь номер страницы. Если система преобразования адресов

обнаруживает, что необходимая физическая страница отсутствует в ОП (т.е. произошел промах или страничный сбой), то нужная страница считывается из внешней памяти в ОП.

Преобразование осуществляется с помощью специальной *страничной таблицы* (СТ). При отсутствии нужной страницы в ОП преобразователь адресов вырабатывает признак страничного сбоя, по которому процессор приостанавливает вычисления, пока нужная страница не будет считана из вторичной памяти в ОП.

На рис. 3.5.2 приведен пример страничной организации виртуальной памяти. Из рисунка видно, что виртуальная память является расширением физической памяти и предоставляет одно адресное пространство независимо от того, где находятся страницы в данный момент: в ОП или в ВЗУ.

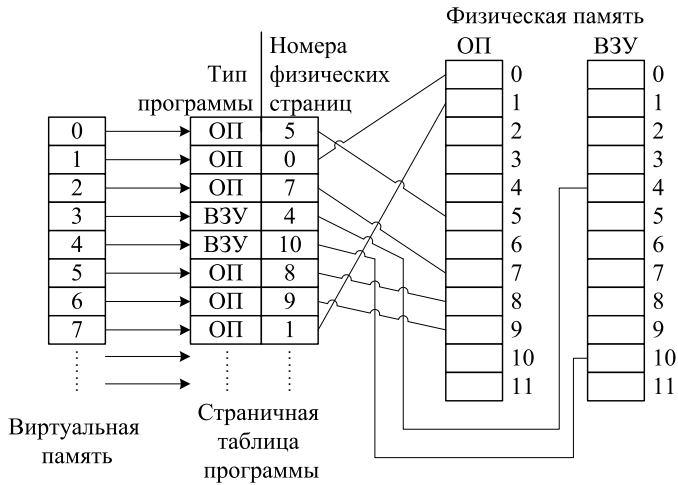


Рис. 3.5.2. Страничная организация виртуальной памяти

Страничная таблица полностью описывает виртуальное пространство. Она определяет, какие виртуальные страницы находятся в ОП и в каких физических кадрах (рис. 3.5.3).

В общем случае число записей в СТ равно числу виртуальных страниц. Каждая запись содержит поле номера физической страницы и четыре признака:

Номер виртуальной страницы	Страничная таблица					Карта ВЗУ	
	<i>V</i>	<i>R</i>	<i>M</i>	<i>A</i>	Номер физической страницы	Адрес ячейки во внешней памяти	
0							
1							
...	...					...	
$N - 1$							

Рис. 3.5.3. Структура страничной таблицы

- 1) признак присутствия *V*;
- 2) признак использования *R*;
- 3) признак модификации *M*;
- 4) признак прав доступа *A*.

*Признак присутствия *V** равен единице, если виртуальная страница находится в данный момент в ОП. В этом случае в поле номера физической страницы располагается соответствующий физический адрес. Если же в поле *V* содержится нуль, то при попытке обратиться к данной виртуальной странице преобразователь адреса генерирует сигнал страничного сбоя (page fault), и предпринимаются действия по загрузке страницы из внешней памяти в ОП с помощью карты ВЗУ. В карте указан адрес расположения страницы в ВЗУ. Загрузка страницы из внешней памяти в ОП сопровождается записью в соответствующую строку СТ (указывается номер физической страницы, куда была загружена виртуальная страница).

*Признак использования страницы *R** устанавливается при обращении к данной странице. Данный признак применяется в алгоритмах замещения информации для выбора страницы, которая с наибольшей вероятностью не будет использована в ближайший промежуток времени, чтобы освободить место для новой. Проблемы замещения информации в ОП решаются так же, как и в кэш-памяти.

Поскольку в ОП находятся лишь копии страниц ВЗУ, то при изменении данных в ОП необходимо обеспечить идентичность подлинников и копий. За выполнением этой операции следит *признак модификации *M**. При удалении страницы из ОП проверяется состояние признака *M*. Если $M = 1$, то перед удалением страницы из ОП ее необходимо переписать в ВЗУ, а при $M = 0$ этого можно не делать.

Признак прав доступа А служит целям защиты информации и определяет, какой вид доступа к странице разрешен: только для чтения, только для записи, для чтения и для записи.

Ввиду того, что благодаря страничной организации страницу не нужно загружать в ОП до тех пор, пока она действительно не понадобится, сокращается объем пересылаемой информации. Данные, загружаемые из ВЗУ, могут быть помещены в любые свободные в данный момент страничные кадры.

Способ реализации СТ очень важен для эффективности работы виртуальной памяти, поскольку каждое обращение к ней предполагает обращение к СТ. Наиболее быстрый способ — хранение таблицы в специально выделенных для этого регистрах, но от него приходится отказываться при большом объеме СТ. Можно выделять пространство для СТ непосредственно в основной памяти, но это приводит к двукратному увеличению времени доступа к информации, а следовательно, к замедлению всей микропроцессорной системы.

Чтобы этого избежать, в состав вычислительной системы вводят специальное быстродействующее запоминающее устройство, называемое *буфером быстрого преобразования адресов* (Translation Lookaside Buffer — TLB), или *буфером опережающей выборки* и представляющее собой кэш-память. При каждом преобразовании номера виртуальной страницы в номер физической страницы результат заносится в TLB: номер физической страницы — в память данных, а виртуальной — в память тегов. Таким образом, в TLB попадают результаты нескольких последних операций трансляции адресов. При каждом обращении к ОП преобразователь адресов сначала производит поиск в памяти тегов TLB номера требуемой виртуальной страницы. При попадании адрес соответствующей физической страницы берется из памяти данных TLB. Если в TLB зафиксирован промах, то процедура преобразования адресов производится с помощью СТ, после чего осуществляется запись новой пары «номер виртуальной страницы — номер физической страницы» в TLB. Структура TLB представлена на рис. 3.5.4.

Буфер быстрого преобразования адресов обычно реализуется в виде полностью ассоциативной или множественно-ассоциативной кэш-памяти с высокой степенью ассоциативности и временем доступа, сопоставимым с аналогичным показателем для кэш-памяти первого уровня.

Номер виртуальной страницы	<i>V</i>	<i>R</i>	<i>M</i>	<i>A</i>	Номер физической страницы
...	...				

Память теговПамять данных

Рис. 3.5.4. Структура буфера быстрого преобразования адресов

На рис. 3.5.5 приведен алгоритм преобразования виртуального адреса в физический с помощью буфера быстрого преобразования адресов.

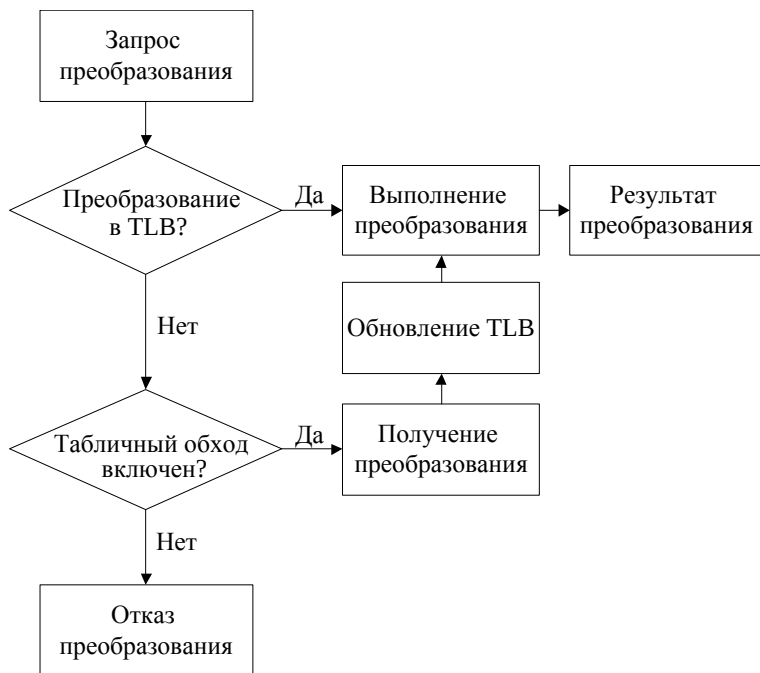


Рис. 3.5.5. Алгоритм преобразования виртуального адреса в физический с помощью буфера быстрого преобразования адресов

Серьезной проблемой, возникающей при реализации системы с виртуальной памятью, является большой объем СТ, который пропорционален числу виртуальных страниц — таблица занимает значительную часть ОП.

Один из способов сокращения объема СТ основан на введении многоуровневой иерархии организации таблиц. В этом случае информация оформляется в виде нескольких СТ сравнительно небольшого объема, которые образуют второй уровень. Первый уровень представлен таблицей с каталогом, где указано местоположение каждой из СТ (адрес начала таблицы в памяти) второго уровня. Сначала в каталоге определяется расположение нужной СТ и лишь затем производится обращение к ней.

Сегментно-страничная организация виртуальной памяти

При страничной организации предполагается, что виртуальная память — это непрерывный массив со сквозной нумерацией слов, что не всегда можно признать наилучшим решением. Обычно программа состоит из нескольких частей — кодовой, информационной и стековой. Так как заранее неизвестны длины этих составляющих, то удобно, чтобы при программировании каждая из них имела собственную нумерацию слов, отсчитываемую с нуля. Для этого организуют систему сегментированной памяти, выделяя в виртуальном пространстве независимые линейные пространства переменной длины, называемые *сегментами*. Каждый сегмент представляет собой отдельную логическую единицу информации, содержащую совокупность данных или программный код и расположенную в адресном пространстве пользователя. В каждом сегменте устанавливается собственная нумерация слов, начиная с нуля. Виртуальная память также разбивается на сегменты с независимой адресацией слов внутри них. Каждой составляющей программы выделяется сегмент памяти. Виртуальный адрес определяется номером сегмента и адресом внутри сегмента. Для преобразования виртуального адреса в физический используется специальная сегментная таблица.

Недостатком такого подхода является то, что неодинаковый размер сегментов приводит к неэффективному использованию ОП. Так, если ОП заполнена, то при замещении одного из сегментов требуется вытеснить другой, размер которого равен или больше размера нового.

При многократном повторе подобных действий в ОП остается множество свободных участков, недостаточных по размеру для загрузки полного сегмента. Решением проблемы служит сегментно-страничная организация памяти. В ней размер сегмента выбирается не произвольно, а задается кратным размеру страницы. Сегмент может содержать то или иное, но обязательно целое число страниц, даже если одна из страниц заполнена частично. Возникает определенная иерархия в организации доступа к данным, состоящая из трех ступеней: сегмент → страница → слово. Этой структуре соответствует иерархия таблиц, служащих для перевода виртуальных адресов в физические. В сегментной таблице перечисляются все сегменты данной программы с указанием начальных адресов СТ, относящихся к каждому сегменту. Число СТ равно числу сегментов, и любая из них определяет расположение каждой из страниц сегмента в памяти, которые могут располагаться не подряд — часть страниц может находиться в ОП, остальные — во внешней памяти.

Процесс преобразования виртуального адреса в физический адрес представлен на рис. 3.5.6. Для получения физического адреса

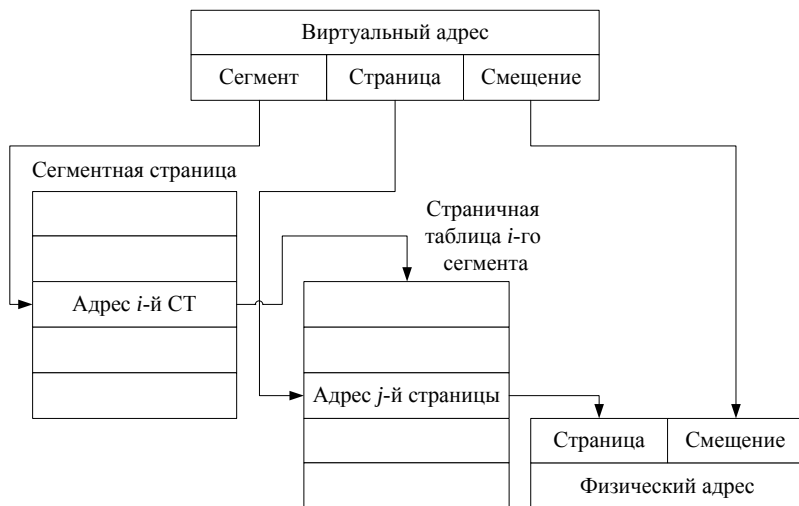


Рис. 3.5.6. Преобразование виртуального адреса в физический с помощью сегментно-страничной организации памяти

необходим доступ к сегментной и одной из страничных таблиц, поэтому преобразование адреса может занимать много времени.

Контрольные вопросы

1. Каково назначение виртуальной памяти?
2. Каково назначение устройства управления памятью?
3. Как достигается увеличение скорости преобразования адресов?
4. Какие способы организации виртуальной памяти вы знаете?
5. В чем суть сегментно-страничной организации виртуальной памяти?

Литература

1. Микропроцессорные системы: учеб. пособие для вузов / **Е.К. Александров, Р.И. Грушвицкий, М.С. Куприянов и др.; под ред. Д.В. Пузанкова.** — СПб.: Политехника, 2002. — 935 с.
2. **Цилькер Б.Я., Орлов С.А.** Организация ЭВМ и систем: учебник для вузов. — СПб.: Питер, 2004. — 654 с.
3. **Антошина И.В., Котов Ю.Т.** Микропроцессоры и микропроцессорные системы (аналитический обзор): учеб. пособие. — М.: МГУЛ, 2005. — 432 с.

Раздел 4. Архитектуры и структуры микропроцессоров и систем на их основе

Лекция 4.1. Классификация архитектур современных микропроцессоров

Одной из важных задач, возникающих при создании средств проектирования микропроцессорных систем, является разработка компиляторов, обеспечивающих эффективный перевод программы с языка высокого уровня на машинно-ориентированный язык, называемый *ассемблером* (assembler). Известно, что существующие компиляторы увеличивают количество команд машинно-ориентированного языка по сравнению с эталонной программой в 1,2 – 2 раза, что приводит к ряду нежелательных эффектов, таких как:

- неэффективное использование команд;
- чрезмерный объем памяти для хранения программы;
- сложность компилятора.

На архитектурном уровне перечисленные проблемы решают посредством создания процессоров, имеющих архитектуры как с полным (Complex Instruction Set Computer — CISC), так и с сокращенным (Reduced Instruction Set Computer — RISC) набором команд.

На практике указанные архитектуры могут дополнять друг друга. Например, современные процессоры общего назначения фирмы Intel имеют все внешние признаки CISC-архитектуры, однако их структура содержит преобразователь CISC-команд в последовательности операций для RISC-ядра, которое непосредственно выполняет все действия над операндами.

CISC-архитектура появилась в начале 1960-х годов как результат усилий разработчиков по созданию универсальной системы команд, обеспечивающей на аппаратном уровне выполнение команд, заданных естественным языком. Это направление было приоритетным по двум основным причинам:

- компиляторы языков высокого уровня только начали появляться, программирование осуществлялось преимущественно с использованием ассемблера;

- многообразие команд и способов адресации позволяло уменьшить объем памяти и количество обращений к ней, что, в свою очередь, давало выигрыш в быстродействии и стоимости.

В целом CISC-архитектура характеризуется большим набором разноформатных команд с использованием многочисленных способов адресации. Основное достоинство такой системы состоит в возможности создавать эффективные алгоритмы для решения широкого круга задач. Поэтому CISC-процессоры называют универсальными, или общего назначения.

Типичным представителем CISC-архитектуры является семейство Pentium фирмы Intel. Первый процессор семейства Pentium имеет архитектуру Intel P5 и появился в 1993 году. Его система команд вместе с расширениями включает около 1000 различных команд, длина командного слова от 1 до 15 байтов, при этом можно использовать более 10 способов адресации.

Одним из существенных недостатков практической реализации архитектуры с полным набором команд является сложная структура устройства управления, реализующего декодирование CISC-команд. Это приводит к увеличению площади и стоимости кристалла, а также снижению производительности. Анализ результатов выполнения различных программ процессором с CISC-архитектурой показал, что в отдельно взятой задаче большинство команд и способов адресации не используются. При этом устройство управления занимает 70 – 80% площади кристалла микропроцессора.

Недостатки CISC-систем способствовали развитию процессоров с сокращенным набором команд (RISC), для которых характерно использование команд фиксированного формата. Современные RISC-процессоры реализуют порядка 100 команд длиной 4 байта, с минимальным числом способов адресации памяти.

Термин «RISC» как название программы исследований в университете Беркли (США) появился в 1980 году, однако первым RISC-компьютером можно назвать суперкомпьютер CDC 6600, который был разработан в 1964 году Сеймуром Креем, реализовывал два способа адресации (регистровый и непосредственный) и мог выполнять 74 операции.

Вычислительная машина CDC 6600 обладала характерной особенностью, которая и сейчас присутствует в RISC-системах и заключается в том, что и операнды, и результаты выполнения любой операции размещаются в регистровой памяти процессора. Данная особенность следует из того, что оператор присвоения является одним из самых распространенных операторов, поэтому операнды целесообразно хранить в непосредственной близости к операционному устройству в памяти, обладающей максимальным быстродействием, т.е. в РОН. При этом возникает задача эффективного распределения операндов по регистрам.

Указанная задача в RISC-процессорах может решаться как программно, так и аппаратно. Программный подход основан на использовании компилятора, который обеспечивает загрузку регистров теми переменными, которые в течение определенного периода времени будут использоваться наиболее часто. Аппаратный подход заключается в наращивании количества РОН для того, чтобы поместить в них наибольшее количество операндов, а также в использовании конвейера. RISC-процессоры имеют увеличенный объем регистровой памяти — от 32 до нескольких сотен РОН.

Важной особенностью RISC-процессоров является использование принципа «load-store». Он обеспечивает сокращение количества обращений к памяти за счет эффективной загрузки операндов и последующей записи результатов выполнения операций в РОН. Данный принцип позволяет выполнять большинство инструкций за один машинный цикл, поскольку не требуется обращения к памяти, работающей на меньшей частоте, чем РОН процессора.

Перечисленные особенности RISC-архитектуры позволили упростить структуру процессоров, что привело к уменьшению площади и стоимости кристалла. С применением конвейерного принципа появилась возможность выполнять несколько команд одновременно. Однако различие между машинно-ориентированным и языком высокого уровня стало еще больше, чем у CISC-процессоров. Как следствие, усложнился компилятор, возникли конфликты, связанные с конвейером команд.

В целом RISC-процессоры эффективны в тех областях применения, в которых можно использовать структурные способы уменьшения времени доступа к оперативной памяти. Однако если программа генерирует произвольные последовательности адресов обращения

к памяти и каждая единица данных применяется только для выполнения одной команды, то производительность процессора определяется временем обращения к основной памяти. В этом случае использование сокращенного набора команд даже уменьшит производительность МП, поскольку требует пересылки «память—РОН» вместо пересылки «память—память».

Дальнейшее повышение производительности процессоров RISC-архитектуры основано:

- на увеличении тактовой частоты и совершенствовании конвейерной обработки данных;
- уменьшении времени доступа к памяти;
- параллельной обработке данных за счет использования ресурсов кристалла.

Современные микропроцессоры могут содержать десять и более операционных устройств, каждое из которых представляет собой конвейер. Увеличение тактовой частоты возможно в результате совершенствования конвейерной обработки и технологии производства. Совершенствование конвейерной обработки заключается в увеличении количества ступеней и эффективном разделении поступающей инструкции на микрокоманды. Это позволяет сократить время выполнения каждой ступени конвейера.

Уменьшение времени доступа к памяти возможно за счет увеличения объема памяти, размещенной на кристалле процессора. С совершенствованием технологии производства объем памяти, размещенный на кристалле, может вырасти, однако память представляет собой ресурс вычислительной системы, который непосредственно не производит вычислений. В связи с этим привлекательным является использование кристалла процессора для построения совокупности функциональных устройств — микропроцессорных ядер, размещенных на кристалле процессора. Основное препятствие на пути повышения производительности в результате увеличения количества функциональных устройств — организация загрузки всех функциональных устройств полезной работой, для этого необходимо обеспечить распараллеливание команд.

Можно выделить два основных подхода к распараллеливанию команд на архитектурном уровне: аппаратный и программный.

Аппаратный подход заключается в том, что параллельная обработка достигается с помощью специальной аппаратуры, которая вхо-

дит в состав микропроцессора и разделяет поступающую команду на микрокоманды для каждого исполнительного устройства. При этом система команд процессора явным образом не ориентирована на параллельную обработку. Такая архитектура называется *суперскалярной*. Достоинства такой системы заключаются в простоте компилятора и возможности миграции программ, написанных под другие архитектуры. Однако при этом не всегда достигается оптимальная загрузка исполнительных устройств. В качестве примера можно привести МП общего назначения с архитектурой Intel P6, ядро которого является суперскалярным: входящая в состав МП аппаратура выполняет трансляцию поступающих сложных CISC-команд на микрокоманды для RISC-исполнительных устройств. Такая архитектура позволяет обеспечить миграцию программ, написанных под CISC-архитектуру, но приводит к дополнительным аппаратным затратам.

Программный подход состоит в использовании специализированного компилятора, разделяющего микрокоманды для каждого исполнительного устройства уже в программе. Этот подход предоставляет программисту все возможности параллельной обработки. В специально отведенных полях макрокоманды каждому из исполнительных устройств микропроцессора указывается своя микрокоманда. Такая архитектура носит название *архитектуры с длинным (очень) командным словом* (Very Long Instruction Word — VLIW). Длина командного слова составляет 128 и более бит.

Отметим преимущества этой архитектуры перед суперскалярной:

- открытость архитектуры для программиста;
- простота аппаратной части и, как следствие, уменьшение площади кристалла и энергопотребления.

В то же время архитектура с длинным командным словом имеет ряд недостатков:

- большой объем программного кода;
- невозможность миграции программ, написанных для других архитектур;
- сложность отладки;
- сложность компилятора.

Из-за указанных недостатков архитектура VLIW не получила широкого распространения среди процессоров общего назначения. Тем не менее, она активно применяется в процессорах цифровой обработки сигналов и в шейдерных ядрах видеопроцессоров AMD.

В настоящее время применяются комбинированные подходы с использованием векторных сопроцессоров, размещенных на одном кристалле с МП, например, технологии MMX и SSE в процессорах Intel.

В связи с возрастающим объемом информации, обрабатываемой современными микропроцессорами, отдельно стоит задача организации памяти в вычислительных системах.

В 1946 году группа ученых под руководством фон Неймана опубликовала статью, в которой рассматривались принципы, по которым должны строиться компьютеры, а именно:

- двоичное кодирование;
- последовательное программное управление;
- однородность памяти;
- адресуемость памяти.

Принцип двоичного кодирования заключается в использовании двоичной системы счисления для построения цифровых вычислительных машин.

Принцип последовательного программного управления состоит в том, что арифметические устройства выполняют последовательно поступающие из памяти команды.

Принцип однородности памяти заключается в том, что программы и данные хранятся в одной и той же памяти, и неважно, что хранится в данной ячейке памяти – число или команда.

Принцип адресуемости памяти заключается в том, что память должна состоять из пронумерованных ячеек, причем к каждой из них должен быть обеспечен мгновенный доступ.

Архитектуру компьютеров, отвечающих этим принципам, называют *принстонской*, или *фон Неймана*. Однако обычно, когда говорят о принстонской архитектуре, речь идет о способе организации памяти, потому что большинство современных компьютеров отвечают принципам фон Неймана.

Согласно принципу адресуемости, память микросистемы представляет собой упорядоченный набор K -разрядных ячеек с произвольным доступом. Такая память называется *линейной*. Совокупность адресов памяти от 0 до $2^n - 1$, где n — количество двоичных разрядов адреса, называется *адресным пространством*. Обычно адресное пространство разделяется на два подмножества: пространство ввода/вывода и адресное пространство памяти. В этом случае ввод/вывод называют *изолированным*, в противном случае — *совместным*.

Как правило, система ввода/вывода представляет собой набор адресуемых буферных схем или регистров (портов), через которые осуществляется связь с внешними и внутренними аппаратными средствами микросистемы. Система ввода/вывода использует механизм распределения портов, размещая их в специальном адресном пространстве ввода/вывода (Input/Output Segment — IOSEG).

Область POH (Register Segment — RSEG) может быть полностью изолирована от пространства данных или частично пересекаться с ним. Адресные пространства команд (Code Segment — CSEG) и данных (Data Segment — DSEG) могут быть как отдельными, так и совместными.

Характерной особенностью *принстонской архитектуры* является то, что в ее состав входит отделенная от исполнительных устройств общая память команд и данных. В системе с такой архитектурой исполнительные устройства и основная память соединены одним коммутационным трактом, передачи команд и данных разделены во времени. Достоинствами принстонской архитектуры стали гибкость вычислительной системы, простота реализации и отладки.

Гибкость заключается в том, что неважно, к какому типу относится содержимое ячейки: к командам или к данным.

Простота реализации обусловлена структурной простотой: исполнительные устройства, устройство управления исполнительными устройствами, память — структура, ставшая классической (рис. 4.1.1).

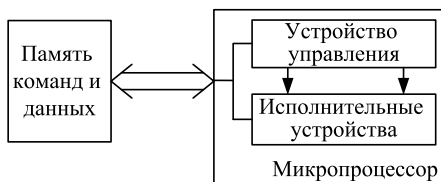


Рис. 4.1.1. Структура вычислительной системы с принстонской архитектурой

Простота отладки определяется наличием только одной системной шины. Однако в связи с увеличением производительности исполнительных устройств и ростом объема передаваемых данных единая системная шина стала ограничивать производительность таких систем.

В *гарвардской архитектуре* память команд и память данных физически разделены. Шины, соединяющие их с процессором, могут иметь как разную разрядность, так и разный объем памяти под команды и данные. Такое архитектурное решение (рис. 4.1.2) дает выигрыш в быстродействии по сравнению с принстонской архитектурой, поскольку за один машинный цикл процессор может получить команды и данные.

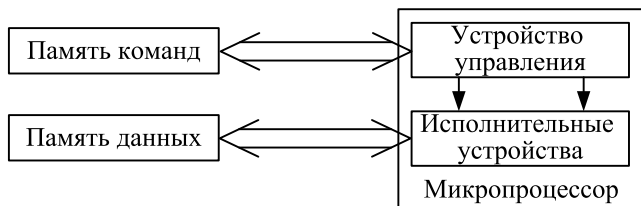


Рис. 4.1.2. Структура вычислительной системы с гарвардской архитектурой

Основным недостатком гарвардской архитектуры является сложность технической реализации двух независимых шин, требующая дополнительных аппаратных затрат. Другим недостатком является то, что объемы памяти команд и памяти данных фиксированы, нет возможности динамически перераспределять информацию между ними.

В *модифицированной гарвардской архитектуре* эти недостатки устранены за счет использования общей памяти и общих шин данных и адреса для всей внешней информации. Внутри процессора применяются два модуля памяти с независимыми шинами адреса, данных и команд. Таким образом, сокращается количество физических линий, а также используется внешняя память для хранения как данных, так и команд (рис. 4.1.3).

Дальнейшим развитием гарвардской архитектуры является применение отдельной внутрикристальной кэш-памяти для команд и данных. Такая архитектура получила название *расширенной гарвардской архитектуры* (Super HArvard ARchitecture Computer — SHARC).

Сегментацию адресного пространства памяти в принстонской и гарвардской архитектурах поясним с помощью рис. 4.1.4. В принстонской архитектуре сегменты команд и данных (CSEG и DSEG) нахо-

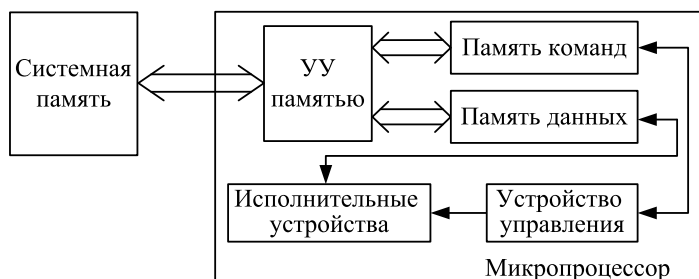


Рис. 4.1.3. Структура вычислительной системы с модифицированной гарвардской архитектурой

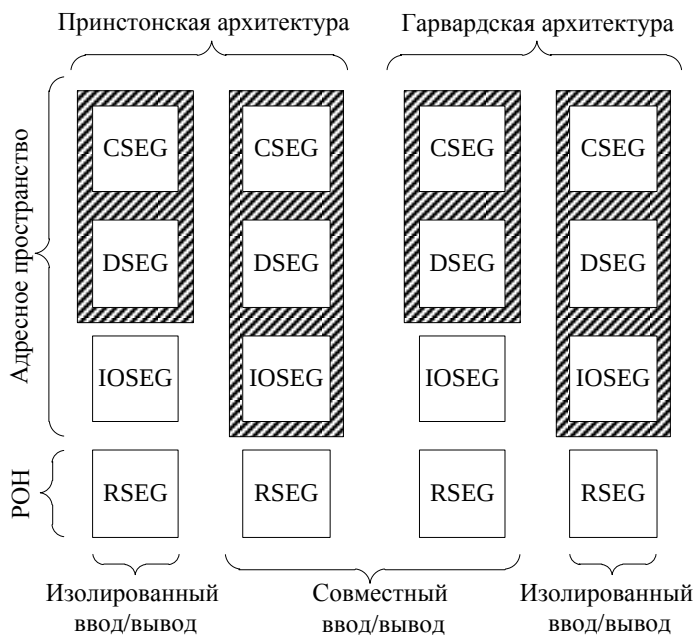


Рис. 4.1.4. Распределение адресного пространства в системах с принстонской и гарвардской архитектурами памяти

дятся в одном адресном пространстве и к ним добавляется сегмент ввода/вывода IOSEG, если ввод/вывод совместный. В гарвардской архитектуре CSEG и DSEG находятся в разных адресных пространствах. Если ввод/вывод изолированный, то DSEG и IOSEG также находятся в разных адресных пространствах.

Контрольные вопросы

1. По каким признакам принято классифицировать архитектуры вычислительных систем?
2. Что означает термин «архитектура с сокращенным набором команд»?
3. В чем заключаются отличия между суперскалярной и VLIW-архитектурами?
4. Сформулируйте принципы фон Неймана.

Литература

1. Микропроцессорные системы: учеб. пособие для вузов / **Е.К. Александров, Р.И. Грушвицкий, М.С. Куприянов и др.**; под ред. **Д.В. Пузанкова**. — СПб.: Политехника, 2002. — 935 с.
2. **Антошина И.В., Котов Ю.Т.** Микропроцессоры и микропроцессорные системы (аналитический обзор): учеб. пособие. — М.: МГУЛ, 2005. — 432 с.
3. **Burks A.W., Goldstine H.H., Neumann J.** Preliminary discussion of the logical design of an electronic computing instrument. — Institute for Advanced Study, Princeton, N.J., 1946.

Лекция 4.2. Структура современных 32-разрядных микроконтроллеров с RISC-архитектурой

Микроконтроллеры, называемые также однокристальными микроЭВМ, являются специализированными микропроцессорами, которые сочетают на одном кристалле функции процессора и периферийных устройств и в основном ориентированы на реализацию устройств управления.

Ввиду большого количества объектов, управление которыми осуществляется с помощью микроконтроллеров, годовой объем их выпуска на порядок превышает объем выпуска процессоров общего назначения. Выгода от использования микроконтроллеров очевидна: вычислительное устройство, построенное на одной микросхеме вместо целого набора дискретных компонентов, значительно снижает размеры, энергопотребление и стоимость устройств на его базе. Использование микроконтроллеров для построения устройств управления позволяет в кратчайшие сроки производить разработку новых и модернизацию существующих систем, корректируя программу микроконтроллера.

Список периферийных устройств, которые расширяют функциональные возможности микроконтроллера и располагаются на кристалле, наряду с процессорным ядром, может включать универсальные цифровые порты ввода/вывода, обеспечивающие обмен информацией между ядром процессора и разнообразными внешними устройствами; различные интерфейсы ввода/вывода, такие как UART, I²C, SPI, CAN, USB, IEEE 1394, Ethernet; АЦП и ЦАП; компараторы; широтно-импульсные модуляторы (ШИМ); таймеры; контроллеры дисплеев и клавиатур; радиочастотные приемники и передатчики; ПЗУ; встроенный тактовый генератор и сторожевой таймер и т. п.

Разработчикам микроконтроллеров приходится соблюдать баланс между размерами и стоимостью, с одной стороны, и гибкостью и производительностью, с другой. Для разных приложений оптимальное соотношение этих и других параметров может сильно различаться. Поэтому существует огромное количество типов микроконтроллеров, отличающихся архитектурой процессорного модуля, размером и типом встроенной памяти, набором периферийных устройств, типом

корпуса и т. д. Тем не менее, основу современных микроконтроллеров составляют гарвардская и RISC-архитектуры. Напомним, что гарвардская архитектура подразумевает хранение данных и команд в двух физически разных модулях памяти, а в основе RISC-архитектуры лежит идея сокращенного набора команд и четыре принципа:

1) каждая команда независимо от ее типа выполняется за один машинный цикл с максимально короткой длительностью; стандартом для RISC-процессоров считается длительность машинного цикла, равная времени сложения двух целых чисел;

2) все команды имеют одинаковую длину и используют минимум адресных форматов, что резко упрощает логику центрального управления процессором и позволяет RISC-процессору выбирать очередную команду в темпе обработки, т. е. одну команду за один цикл;

3) обращение к памяти происходит только при выполнении операций записи и чтения, модификация операндов в памяти возможна лишь с помощью команды «Запись», вся обработка данных осуществляется исключительно в регистровой структуре процессора;

4) система команд должна обеспечивать поддержку компиляции с конкретного языка программирования (компиляторы для RISC на порядок сложнее, чем компиляторы для CISC).

Считается, что первый микроконтроллер появился в 1976 году, спустя 5 лет после создания первого микропроцессора. Это была микросхема фирмы Intel, названная 8048. Следующий микроконтроллер — Intel MCS 51 — стал классическим образцом устройств данного типа. Большинство фирм-производителей микроконтроллеров и в настоящее время выпускают устройства, основанные на этой архитектуре.

Значимым событием в развитии микроконтроллеров стало появление в конце 1980-х годов PIC-контроллеров (Peripheral Interface Controller) фирмы Microchip. Микросхемы предлагались по рекордно низким ценам, к тому же производительность PIC-контроллеров не уступала, а нередко и превосходила производительность микроконтроллеров MCS 51. Эти контроллеры отличались усовершенствованными портами ввода/вывода. Несмотря на то, что система команд крайне ограничена, PIC-контроллеры и по сей день остаются востребованными в тех случаях, когда необходимо создать недорогую интегрированную систему, не предъявляющую высоких требований по быстродействию.

В 1995 году два студента из Норвегии предложили американской корпорации Atmel, которая была известна своими микросхемами с флэш-памятью, выпустить новый 8-битный RISC-микроконтроллер и снабдить его флэш-памятью программ на одном кристалле с вычислительным ядром. Уже в 1996 году фирма Atmel представила семейство микросхем на новом прогрессивном ядре AVR. Быстродействие больше, чем у контроллеров Microchip, и привлекательная ценовая политика сделали эти микросхемы достаточно популярными.

На данный момент существует целый ряд различных 32-разрядных микроконтроллеров с RISC-архитектурой, наиболее распространенными среди них являются микроконтроллеры с архитектурой ARM (Advanced RISC Machine — усовершенствованная RISC-машина), разработанной компанией ARM Limited. Эти процессоры имеют низкое энергопотребление и относительно малые габариты, поэтому находят широкое применение во встраиваемых системах и могут рассматриваться как альтернатива существующим 8-разрядным контроллерам.

По состоянию на 2009 год на процессоры ARM приходилось до 90% всех встроенных 32-разрядных процессоров. Их популярность можно также объяснить, например, использованием технологий, не присущих другим микроконтроллерам, таких как Jazelle (технология, предназначенная для аппаратного ускорения выполнения приложений, написанных на языке Java), Thumb (технология повышения плотности кода), аппаратные реализации декодирования видеопотока и т. п.

Сегодня известно большое количество производителей микроконтроллеров с ARM-архитектурой, а именно: Analog Devices, Atmel, Apple, Intel, NXP, STMicroelectronics, Samsung, Qualcomm, Sony Ericsson, Texas Instruments, nVidia, Freescale, Миландр, HiSilicon и многие другие.

К настоящему моменту разработано не одно семейство ядер процессоров ARM. К наиболее значимым можно отнести: ARM7, ARM9, ARM11 и Cortex.

Рассмотрим структуру современных 32-разрядных микроконтроллеров с RISC-архитектурой на примере микроконтроллера LPC2300 с архитектурой ARM7 (рис. 4.2.1).

С точки зрения программиста, память любого из устройств LPC2300 представляет собой непрерывное 32-битное адресное про-

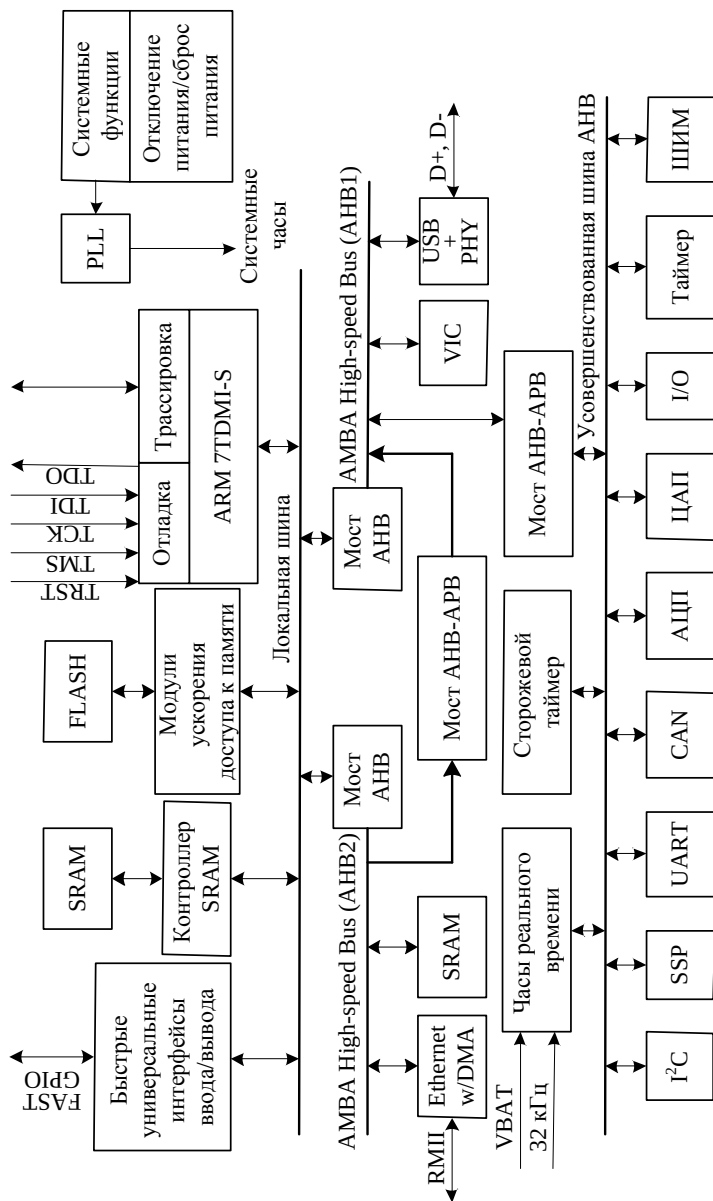


Рис. 4.2.1. Структурная схема 32-разрядного микроконтроллера LPC2300 с архитектурой ARM7

странство. На самом деле внутри устройства имеется по меньшей мере две стандартные шины: усовершенствованная высокоскоростная шина АНВ (Advanced High performance Bus) для наиболее быстродействующих периферийных устройств и усовершенствованная периферийная шина АРВ (Advanced Peripheral Bus) для остальных модулей. Шина АРВ подключена к АНВ посредством моста, в состав которого входит делитель частоты тактового сигнала. Благодаря этому делителю при запуске пользовательской периферии на частоте ниже частоты ядра уменьшается потребляемая мощность.

Кроме того, в состав LPC2300 входит ряд быстродействующих периферийных устройств, каждое из которых имеет собственный модуль ПДП. Чтобы минимизировать конфликты захвата шины между ядром и ПДП, микроконтроллер включает в себя две шины АНВ, одну шину АРВ и дополнительную локальную шину для связи с ядром.

Основной элемент ядра ARM7 — трехступенчатый конвейер команд, который не подвержен возникновению конфликтов, таких как «чтение-после-записи», встречающихся в конвейерах с большим числом ступеней. Конвейер имеет три аппаратно-независимые ступени, благодаря которым одновременно с выполнением первой команды осуществляются декодирование второй и выборка третьей команд. Эффективно ускоряется прохождение команд через ОУ так, что большинство команд ARM может выполняться за один такт. Конвейер наиболее эффективен при выполнении линейного кода. При обнаружении перехода конвейер сбрасывается, и для возобновления выполнения программы с максимальной скоростью он должен сначала заполниться.

Процессор ARM7 имеет архитектуру «загрузка-хранение» (load-and-store), т. е. для выполнения любой обработки данных необходимо сначала перенести эти данные из памяти в определенные регистры, выполнить команду обработки данных и затем записать полученные значения обратно в память.

Основной регистровый файл состоит из 16 пользовательских 32-битных регистров R0–R15. Регистры R0–R12 предназначены исключительно для нужд пользователя и не выполняют никаких дополнительных функций. Регистр R13 используется в качестве указателя стека (Stack Pointer — SP). При вызове подпрограммы адрес возврата автоматически запоминается в регистре связи R14 (Link Register — LR), откуда затем считывается при возврате, что позволяет быстро

переходить к «концевым» функциям (функции, которые не вызывают других функций) и возвращаться из них. Если же функция входит в состав ветви, т.е. вызывает другие функции, содержимое регистра связи необходимо сохранять в стеке (R13). Регистр R15 выполняет функции счетчика команд (Program Counter — PC). Многие команды могут работать с регистрами R13–R15, как с обычными пользовательскими регистрами.

Наряду с банком регистров в ядре ARM7 имеется дополнительный 32-битный регистр, называемый *регистром текущего состояния программы* (Current Program Status Register — CPSR), который содержит набор флагов, управляющих функционированием МП ARM7 и отображающих его состояние.

В старших четырех битах регистра CPSR хранятся флаги условий, значения которых определяются процессором. Благодаря им можно узнать, не было ли получено в результате выполнения команды отрицательное или нулевое значение, не произошел ли перенос или переполнение. Младшие восемь битов содержат флаги, значения которых прикладная программа может изменять. Среди них флаги для разрешения и запрещения двух линий прерываний, являющихся внешними по отношению к МП ARM7, флаг THUMB, показывающий, какой из наборов команд используется (16- или 32-битный), и пять битов, отвечающих за режим работы процессора. В общей сложности процессор ARM7 поддерживает семь режимов работы.

При возникновении исключительной ситуации изменяется режим работы МП, и в регистр программного счетчика загружается адрес соответствующего вектора прерывания. Процессор выполняет следующие действия: сохраняет в регистре связи адрес следующей выполняемой команды, копирует CPSR в специальный регистр, а в PC заносит адрес вектора прерывания. Режим работы процессора меняется, в результате чего регистры R13 и R14 заменяются соответствующими регистрами этого режима.

Ядро ARM7 разработано таким образом, чтобы его можно было использовать как в качестве процессора с обратным порядком битов (big-endian processor), так и в качестве процессора с прямым порядком битов (little-endian processor). В первом случае старший бит (Most Significant Bit — MSB) 32-битного слова располагается в начале слова, а во втором случае — в конце. В семействе LPC2300 используется только прямой порядок битов.

Одна из наиболее интересных особенностей набора команд ARM заключается в том, что каждая команда поддерживает условное выполнение. В традиционных микроконтроллерах условными командами являются команды условных переходов и иногда команды проверки либо изменения состояния отдельных битов. В наборе команд ARM старшие четыре бита кода команды всегда сравниваются с флагами условий в регистре CPSR. Если их значения не совпадают, команда не выполняется и проходит через конвейер как команда NOP (No Operation). Таким образом можно выполнить какую-либо команду обработки данных, изменяющую флаги условий в регистре CPSR. В зависимости от результата следующая команда может быть либо выполнена, либо нет.

Целью такого условного выполнения команд является обеспечение непрерывности потока команд через конвейер, так как при каждом выполнении команд перехода конвейер сбрасывается, и на его повторное заполнение требуется время, что резко снижает общую производительность. На практике существует некоторый порог, при котором принудительное «проталкивание» команд NOP через конвейер оказывается эффективнее выполнения традиционных команд условного перехода и связанного с этим повторного заполнения буфера.

Все множество команд ARM можно разбить на шесть основных групп: команды ветвления, команды обработки данных, команды передачи данных, команды передачи блоков данных, команды умножения и команда программного прерывания.

Команды ветвления. Базовая команда перехода B (от англ. Branch) позволяет выполнять переход в диапазоне до 32 МБ как вперед, так и назад. Команда перехода с сохранением адреса (BL) выполняет ту же операцию, однако при этом сохраняет в регистре связи текущее значение PC и используется в качестве команды вызова подпрограмм.

Существует еще две команды перехода: переход со сменой состояния (BX) и переход со сменой состояния и сохранением адреса (BLX). Эти команды выполняют те же операции, что и предыдущие, но при этом переключают с набора команд ARM на THUMB и обратно.

Команды обработки данных. Обобщенный формат всех команд обработки данных приведен на рис. 4.2.2. В каждой команде имеется регистр результата и два операнда. Первый операнд обязатель-

но должен быть регистром, тогда как второй может быть регистром или константой. Кроме того, в МП ARM7 имеется многорегистровое устройство циклического сдвига, позволяющее при выполнении команды сдвигать значение второго операнда на величину до 32 битов. Бит S используется для управления флагами условий. Если этот бит установлен, флаги условий изменяются в соответствии с результатом выполнения команды. Если S сброшен, состояние флагов условий не изменяется.

Условное выполнение	Код операции	S	Операнды R1, R2, R3	Сдвиг
------------------------	-----------------	---	------------------------	-------

Рис. 4.2.2. Формат команд обработки данных

Команды передачи данных. МП ARM7 поддерживает команды загрузки/сохранения, позволяющие пересылать знаковые и беззнаковые числа разного размера (слово, полуслово, байт) в заданный регистр или из него.

Команды передачи блоков данных. С помощью команды LDM можно скопировать в память весь блок регистров или его часть, а с помощью команды STM — восстановить его содержимое.

Команда программного прерывания SWI генерирует исключительную ситуацию, в результате чего процессор переключается в режим Supervisor, а в счетчик команд заносится значение 0x08. Как и все остальные команды ARM, команда SWI содержит в четырех старших битах флаги условного выполнения, за которыми располагается код операции. Остальная часть слова команды остается свободной. Однако в этих неиспользуемых битах может храниться число. Данные биты можно проверять в начале подпрограммы обработки прерывания, чтобы определить, какую именно часть подпрограммы следует выполнять. Таким образом, с помощью команды SWI можно переключаться в защищенный режим для выполнения привилегированных участков программы или обработки системных вызовов.

Команды умножения. Наряду с многорегистровым устройством циклического сдвига в ядре ARM7 имеется встроенный модуль умножения с накоплением (MAC). Модуль MAC поддерживает умножение чисел типа integer и long integer. Команды умножения чисел типа integer выполняют умножение двух 32-битных регистров и помещают

результат в третий 32-битный регистр. Команда умножения с накоплением выполняет умножение и прибавляет произведение к промежуточной сумме. Команды умножения чисел типа long integer перемножают содержимое двух 32-битных регистров и помещают 64-битный результат в два регистра.

Несмотря на то, что ARM7 является 32-битным процессором, он поддерживает еще один набор команд (16-битный), называемый THUMB. На самом деле этот набор команд — сжатая форма набора команд ARM.

При выполнении программы сохраненные в 16-битном формате команды распаковываются в команды ARM, а затем выполняются. Хотя команды THUMB обеспечивают меньшую производительность по сравнению с командами ARM, благодаря им достигается более высокая плотность кода. Таким образом, для создания компактных программ, размещаемых в небольших однокристальных микроконтроллерах, код программ необходимо компилировать в виде совокупности функций THUMB и ARM. Этот процесс называется *interworking*. При компиляции с использованием набора команд THUMB можно получить 30%-ную экономию памяти программ, в то время как этот же код, скомпилированный с использованием команд ARM, будет выполняться приблизительно на 40% быстрее.

Набор команд THUMB похож на наборы команд традиционных микроконтроллеров. В отличие от команд ARM, команды THUMB не поддерживают условного выполнения (за исключением команд условных переходов). Команды обработки данных имеют двухадресный формат, причем в качестве регистра результата используется один из регистров-источников.

Команды THUMB не имеют полного доступа ко всем регистрам общего назначения. С регистрами R0–R7 (младшие регистры) могут работать все команды обработки данных, а к регистрам R8–R12 (старшие регистры) могут обращаться только некоторые из них.

Рассмотренная архитектура ARM7 является основой широко распространенного семейства 32-разрядных микроконтроллеров и отображает ключевые принципы построения, присущие контроллерам других семейств на базе ядра ARM.

Контрольные вопросы

1. В чем отличие микропроцессора от микроконтроллера?
2. Поясните основные особенности системной шины микроконтроллеров на базе процессорного ядра ARM7.
3. Объясните использование трехступенчатого конвейера в процессорном ядре ARM7.
4. Приведите шесть ключевых групп команд процессора ARM.
5. Перечислите достоинства и недостатки использования команд THUMB.

Литература

1. **Бойко В.И.** Схемотехника электронных систем. Микропроцессоры и микроконтроллеры. — СПб.: БХВ-Петербург, 2004. — 464 с.
2. **Тревор М.** Микроконтроллеры ARM7 семейств LPC2300/2400. Вводный курс разработчика: *пер. с англ. А.В. Евстифеева*. — М.: Додэка-XXI, 2010. — 336 с.

Лекция 4.3. Процессоры цифровой обработки сигналов: принципы организации, обобщенная структура, примеры реализации

Процессор цифровой обработки сигналов (ПЦОС) (Digital Signal Processor — DSP) — это специализированный микропроцессор, обладающий архитектурными особенностями, которые востребованы в цифровой обработке сигналов. К таким особенностям относятся способ организации памяти, система команд, векторная и конвейерная обработка данных.

Отметим, что МП других типов, например общего назначения, также могут обладать архитектурными особенностями, обеспечивающими эффективную реализацию алгоритмов цифровой обработки сигналов (в частности, в систему команд могут входить специальные инструкции), либо иметь одинаковую или даже большую по сравнению с ПЦОС производительность некоторых типов операций (например, процессор Intel Pentium III, работающий на частоте 1,1 ГГц, обладает в два раза большей производительностью операций с плавающей точкой, чем ПЦОС Texas Instruments TMS320C55х, работающий на частоте 300 МГц), тем не менее, ПЦОС имеют ряд преимуществ перед процессорами общего назначения: меньшее энергопотребление, стоимость, а благодаря архитектурным особенностям и наличию встроенной быстродействующей памяти возможность выполнять обработку радио-, аудио- и видеосигналов в режиме реального времени.

На основе ПЦОС создаются устройства, в которых требуется режим реального времени выполнения практически любых вычислительных задач. Можно выделить ряд типовых задач по обработке сигналов, решаемых с применением ПЦОС: фильтрация и свертка, вычисление значений авто- и кросс-корреляционной функции, усиление, нормализация, прямое и обратное дискретное преобразование Фурье.

Первый ПЦОС, получивший широкое распространение, — TMS320C10 — был выпущен компанией Texas Instruments в 1982 году. Он использовался в системах связи, системах автоматизации и в военной технике.

В настоящее время ПЦОС применяются во многих сферах деятельности человека, таких как: телекоммуникации, приборостроение, автоматизация в промышленности, управление техническими системами, автомобилестроение, медицинская техника, обработка звука и речи, обработка изображений и графические станции, бытовые приборы, носимая электроника, оборонная техника.

Отличительными особенностями задач цифровой обработки сигналов являются:

- высокая скорость поступления входных и выдачи выходных данных;
- широкий динамический диапазон данных;
- большое количество логических и арифметических операций;
- обеспечение гибкости и перестройки цифровых систем обработки сигналов;
- возможность параллельного выполнения алгоритмов.

Высокая скорость поступления входных и выдачи выходных данных обусловлена необходимостью обработки поступающей информации в режиме реального времени.

Динамический диапазон данных определяется разрядностью АЦП/ЦАП, которая на современном этапе технологии составляет 20 – 24 разряда, и собственными шумами схемы. Динамический диапазон определяет, какой тип арифметики будет использоваться: с фиксированной или плавающей точкой.

Большое количество операций сложения, умножения и сдвига определяется спецификой решения задач цифровой обработки сигналов. Умножение векторов является одной из самых распространенных операций в приложениях, связанных с цифровой обработкой сигналов. Эта операция используется, например, при вычислении свертки сигналов и корреляционной функции.

Обеспечение гибкости и перестройки цифровых систем обработки сигналов связано с изменением различных параметров, коэффициентов и данных в регулируемых и адаптивных системах.

Параллелизм алгоритмов проявляется в том, что для каждого набора входных данных выполняются такие действия, которые могут совмещаться во времени.

Обобщенная структура современных ПЦОС (рис. 4.3.1) основана на VLIW-архитектуре системы команд и модифицированной гарвардской архитектуре памяти.

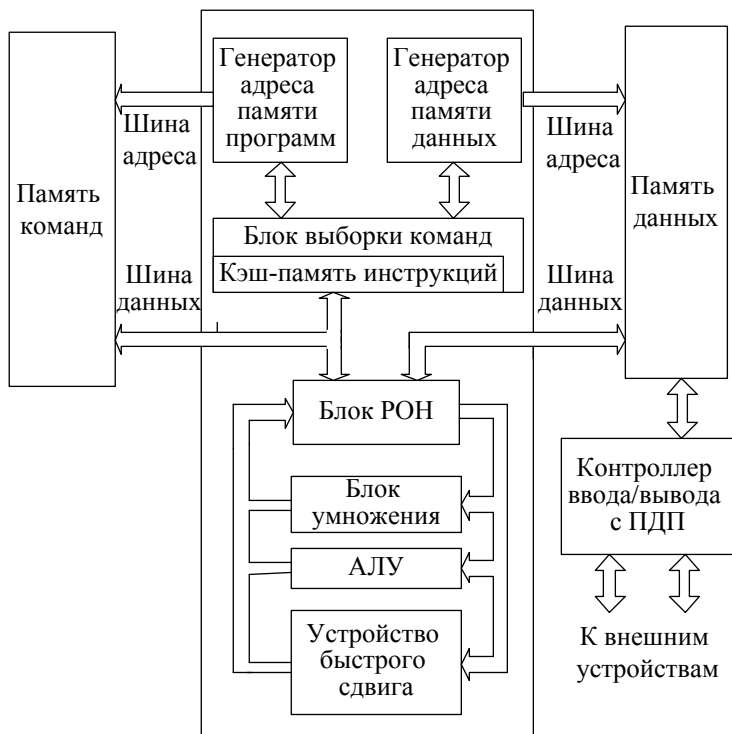


Рис. 4.3.1. Обобщенная структура ПЦОС

Архитектура VLIW реализует распараллеливание обработки данных на этапе компиляции и позволяет не вводить в состав процессора дополнительный блок, отвечающий за обеспечение параллельно работающих исполнительных устройств полезной нагрузкой, что является ее достоинством. Однако такой подход усложняет компилятор языка высокого уровня, делает миграцию имеющегося программного обеспечения затрудненной или вовсе невозможной.

Гарвардская архитектура применяется для повышения производительности и гибкости работы ПЦОС. Она подразумевает размещение программы и данных в отдельных запоминающих устройствах и их передачу по отдельным шинам, позволяет совмещать во вре-

мени выборку и выполнение команд. Модифицированная гарвардская архитектура допускает обмен содержимым между памятью программ и памятью данных, что расширяет возможности устройства.

В состав обобщенной структуры ПЦОС входят:

- генераторы адресов памяти программ и данных;
- разделенные блоки памяти команд и данных;
- блок выборки команд с кэш-памятью инструкций;
- блок РОН;
- блок умножения;
- арифметико-логическое устройство;
- контроллер ввода/вывода с ПДП;
- устройство быстрого сдвига.

Генераторы адресов памяти программ и данных отвечают за чтение и запись необходимых данных/команд. В более простых процессорах эти функции выполняются блоком выборки команд, что делает обмен процессора с памятью более прозрачным для программиста. Однако ПЦОС предназначены для работы с циклическими буферами, которые позволяют реализовывать эффективное управление памятью за счет дополнительной аппаратуры. Это дает возможность ускорить работу с памятью, поскольку не требуется дополнительных тактов для обмена с ней. Например, в процессорах архитектуры SHARC каждый из двух генераторов адреса может управлять восемью циклическими буферами. Это означает, что каждый генератор адреса может хранить до 32 переменных (по четыре на каждый буфер).

Блоки генерации адреса предназначены для поддержки эффективного выполнения быстрого преобразования Фурье. В этом режиме циклические буферы в блоках генерации адреса обеспечивают побитово-инверсную косвенную адресацию, необходимую для реализации быстрого преобразования Фурье. Кроме того, множество круговых буферов значительно упрощают составление кода как для программиста, так и для компиляторов языков высокого уровня, например, С и С++.

Блок выборки команд обеспечивает формирование последовательности микрокоманд. В его состав входит кэш-память инструкций, которая хранит около 30 последних использовавшихся инструкций. Эта особенность дает выигрыш при выполнении циклов за счет меньшего количества обращений к памяти программ.

Контроллер ввода/вывода отвечает за обмен данными между процессором и внешними устройствами и, как правило, включает в себя контроллер ПДП, обеспечивающий передачу данных от внешних устройств в память без использования МП, который в это время может производить обработку данных.

Блок РОН имеет традиционную структуру и представляет собой статическое СОЗУ. Например, блок РОН в процессорах фирмы Analog Devices 2106х с архитектурой SHARC содержит шестнадцать 40-битных регистров. Они могут хранить промежуточные результаты вычислений и данные для математического процессора, играть роль буфера для передачи данных, хранить флаги управления программой и т. д. В случае необходимости эти регистры могут использоваться также для реализации счетчиков циклов, однако, например, ПЦОС с архитектурой SHARC имеют в своем составе дополнительные аппаратные регистры, выполняющие эти функции.

Важной архитектурной особенностью ПЦОС является применение дублирующих (теневых) регистров для всех ключевых регистров центрального процессорного устройства. Эти регистры используются для быстрого выполнения прерываний. В традиционных микропроцессорах для выполнения прерываний требуется сохранение всех внутренних данных процессора. Обычно это реализуется путем записи содержимого всех регистров в стек, причем на запись содержимого каждого регистра в стек требуется один машинный цикл. В ПЦОС с архитектурой SHARC прерывание выполняется после записи содержимого всех регистров процессора в течение одного машинного цикла.

Блок умножения, АЛУ и устройство быстрого сдвига образуют математический процессор.

Блок умножения используется для реализации операции умножения с накоплением (Multiplication with Accumulation — MAC). Как было отмечено ранее, эта операция широко применяется в цифровой обработке сигналов, например, для перемножения векторов при реализации операции свертки.

Одним из наиболее важных требований к умножителям является способность функционировать в режиме реального времени, т. е. перемножение должно быть выполнено до того, как на входе массива появится следующий отсчет обрабатываемого сигнала. Для этого требуется аппаратная реализация и умножителя, и сумматора. Существу-

ет два основных подхода к решению данной задачи. Первый подход — это интеграция умножителя и сумматора в один функциональный блок. Такая реализация представлена, например, в ПЦОС Motorola DSP5600х. Второй подход — физическое разделение умножителя и сумматора. Например, в процессорах Analog Devices 2106х и Texas Instruments TMS320C5х умножитель считывает данные из двух РОН, перемножает их и записывает результат в третий РОН. Содержимое этого регистра, в свою очередь, может быть прибавлено к содержимому регистра-аккумулятора АСС с помощью АЛУ. Это же АЛУ можно задействовать и для выполнения других арифметико-логических операций. В обоих случаях операция умножения с накоплением будет выполнена за один машинный цикл. Характерной особенностью такой реализации является то, что умножитель и АЛУ могут быть доступны одновременно. Так, в архитектуре семейства SHARC данные из регистров 0–7 могут быть поданы на умножитель, а данные из регистров 8–15 могут быть переданы в АЛУ, при этом за один машинный цикл будут выполнены две команды и результаты могут быть сохранены в любых двух из 16 регистров.

АЛУ отвечает за выполнение арифметических и логических операций, кроме того, оно выполняет преобразование чисел с фиксированной или плавающей точкой.

Устройства быстрого сдвига выполняют простые побитовые операции, такие как сдвиг, переворот, считывание и запись части битов в регистре.

Приведенная на рис. 4.3.1 обобщенная структура ПЦОС в зависимости от решаемой задачи может быть дополнена различными модулями, такими как: АЦП, ЦАП, контроллеры интерфейсов, контроллеры ПДП, таймеры, блоки фазовой автоподстройки частоты.

Важной характеристикой ПЦОС является то, в каком формате данных производятся вычисления.

ПЦОС различных компаний-производителей образуют два класса, существенно различающихся по структуре: ПЦОС обработки данных в формате с фиксированной точкой и ПЦОС, аппаратно поддерживающие операции над данными в формате с плавающей точкой.

Использование данных в формате с плавающей точкой обусловлено несколькими причинами. Для многих задач, связанных с выполнением интегральных и дифференциальных преобразований, особую значимость имеет точность вычислений, обеспечить которую позволя-

ет экспоненциальный формат представления данных. Алгоритмы компрессии, декомпрессии, адаптивной фильтрации в цифровой обработке сигналов связаны с определением логарифмических зависимостей и весьма чувствительны к точности представления данных в широком динамическом диапазоне. Обработка данных в формате с плавающей точкой существенно ускоряет выполнение, поскольку не требует выполнения операций округления и нормализации данных, отслеживания ситуаций потери значимости и переполнения.

ПЦОС с плавающей точкой отличаются: высокая сложность функциональных устройств, выполняющих обработку данных в формате с плавающей точкой, необходимость использования более сложных технологий производства микросхем, больший процент отбраковки изделий и, как следствие, большая стоимость по сравнению с ПЦОС с фиксированной точкой.

Современные ПЦОС поддерживают выполнение обработки данных как в формате с фиксированной точкой, так и с плавающей.

В числе наиболее распространенных ПЦОС можно назвать следующие: TMS320 (Texas Instruments), 21xx, SHARC, Blackfin (Analog Devices) и MSC815x, MSC825x (Freescall). Выбор того или иного ПЦОС определяется задачей, стоящей перед разработчиком. Каждый из перечисленных производителей имеет в своем модельном ряду процессоры различной производительности. Если основным критерием является малое энергопотребление, как, например, в переносных устройствах, то стоит обратить внимание на процессоры меньшей производительности, но и с меньшим энергопотреблением. Рассмотрим многообразие ПЦОС на примере микросхем фирмы Texas Instruments.

ПЦОС с фиксированной точкой компании Texas Instruments представлены сериями TMS320C1x, TMS320C2x, TMS320C5x и TMS320C62x. Класс ПЦОС с плавающей точкой включает серии TMS320C3x, TMS320C4x и TMS320C67x. ПЦОС TMS320C8x также поддерживает операции с плавающей точкой и представляет собой мультипроцессорную систему, выполненную на одном кристалле.

Три серии ПЦОС — TMS320C2000, TMS320C5000, а также TMS320C6000 — предоставляют выбор разработчику по критерию «производительность/стоимость/потребляемая мощность».

Серия ПЦОС TMS320C2000 предназначена для решения задач встроенных применений и управления. Эти процессоры характеризуются развитой периферией и низкой стоимостью. Данную серию

представляют универсальные ПЦОС, например TMS320C20х, и специализированные, например TMS320C24х, для цифрового управления электродвигателями.

Серия ПЦОС TMS320C5000 ориентирована на рынок малопотребляющих портативных устройств и мобильной связи. ПЦОС серии TMS320C54хх оптимизированы по быстродействию (до 200 MIPS) и минимальному энергопотреблению (до 32 мА/MIPS).

Серия ПЦОС TMS320C6000 характеризуется максимальной производительностью для применений, требующих предельных скоростей вычислений как с фиксированной, так и с плавающей точкой. TMS320C62х (ПЦОС с фиксированной точкой и быстродействием 1600 MIPS) и TMS320C67х (ПЦОС с плавающей точкой и производительностью от 1 GFLOPS) программно совместимы. Типовые области применений ПЦОС серии TMS320C6000 — многоканальные модемы, базовые станции, устройства обработки изображений и др.

ПЦОС всех трех серий могут комплектоваться современными средствами разработки и отладки программ, объединенных единым пользовательским интерфейсом на базе программных средств Code Explorer и Code Composer Studio.

Обзор архитектур ПЦОС проведем на примере универсального ПЦОС серии TMS320C2х.

ПЦОС серии TMS320C2х имеют архитектуру, аналогичную архитектуре ПЦОС первого поколения TMS320C1х, но обладают повышенной производительностью и более широкими функциональными возможностями. Все ПЦОС серии TMS320C2х могут использовать по 64К слов памяти программ и данных, имеют по шестнадцать 16-разрядных портов ввода/вывода и последовательный порт. ПЦОС серии TMS320C2х имеют возможность использования внешнего контроллера ПДП. Умножитель этих МП, помимо операций умножения, позволяет выполнять за один такт возведение в квадрат.

В процессоры данной серии включена аппаратная поддержка циклического выполнения команд, реализован режим побитово-инверсной косвенной адресации, предназначенный для эффективной реализации быстрого преобразования Фурье.

Основные отличия архитектуры ПЦОС серии TMS320C2х от архитектуры ПЦОС первого поколения TMS320C1х:

- выполнение умножения и сохранение результатов в ПЦОС TMS320C2х осуществляются за один командный цикл;

- наличие команд, поддерживающих вычисления с плавающей точкой;
- микросхема ПЦОС включает внутреннее маскируемое ПЗУ программ размером 4К слов для TMS320C25;
- выполнение программ осуществляется из памяти программ (RAM), расположенной на кристалле;
- объем памяти программ — 544 слова, из которых 256 могут быть использованы как память данных;
- расширяемая внешняя память может иметь объем 128К слов (64К слов — память программ, 64К — память данных);
- ПЦОС TMS320C2х содержит внешний интерфейс для организации многопроцессорных связей и средства синхронизации для доступа к разделяемой памяти;
- реализована возможность перемещения содержимого памяти данных и памяти программ блоками;
- реализована возможность организации циклов ожидания при доступе к медленной внешней памяти или медленным периферийным устройствам;
- ПЦОС TMS320C2х содержит на кристалле таймер и последовательный порт;
- микросхема ПЦОС включает пять (для TMS320C20) или восемь (для TMS320C25) вспомогательных регистров и специальное арифметическое устройство для них;
- микросхема ПЦОС включает аппаратный стек размером четыре слова (для TMS320C20) или восемь слов (для TMS320C25) и возможность программного расширения стека в памяти данных;
- наличие команд обработки битовых данных;
- наличие трех маскируемых пользователем прерываний;
- наличие режима ПДП (только для TMS320C25).

Рассмотренная обобщенная структура и ряд частных особенностей присущи всем современным ПЦОС. Тем не менее, данная область микропроцессорной техники динамично развивается. Сформулируем основные тенденции ее развития:

- совершенствование конвейерной обработки и повышение тактовой частоты;
- увеличение количества вычислительных ядер;
- размещение МП общего назначения и ПЦОС на одном кристалле;

- интеграция ПЦОС внутрь ПЛИС в виде IP-ядер для большей программной гибкости вычислительных систем;
- реализация большого количества высокоскоростных интерфейсов на кристалле ПЦОС, в том числе и для интеграции в многопроцессорные вычислительные системы;
- уменьшение отношения потребляемой мощности к производительности.

Контрольные вопросы

1. Что означает термин «система, работающая в режиме реального времени»?
2. В каких областях деятельности человека применяются ПЦОС?
3. Какими особенностями обладают ПЦОС по сравнению с процессорами общего назначения?
4. Расскажите о составе и назначении блоков в обобщенной структуре ПЦОС.
5. Каковы основные направления развития ПЦОС?

Литература

1. **Солонина А.И., Улахович Д.А., Яковлев Л.А.** Алгоритмы и процессоры цифровой обработки сигналов. — СПб.: БХВ-Петербург, 2002. — 464 с.
2. **Корнеев В.В., Киселев А.В.** Современные микропроцессоры. — 3-е изд., перераб. и доп. — СПб.: БХВ-Петербург, 2003. — 448 с.
3. **Бойко В.И.** Схемотехника электронных систем. Микропроцессоры и микроконтроллеры. — СПб.: БХВ-Петербург, 2004. — 464 с.
4. **Smith S.W.** The Scientist and Engineer's Guide to Digital Signal Processing. Chapter 28. — 1999.

Лекция 4.4. Особенности построения микропроцессоров общего назначения на примере архитектуры Intel P6

Микропроцессоры общего назначения ориентированы на решение широкого круга задач цифровой обработки информации. Их основной областью применения являются персональные компьютеры, рабочие станции, серверы и другие системы массового применения. К этому классу относятся современные процессоры семейства Core компании Intel, Bulldozer компании AMD, PowerPC и Xenon компаний Motorola и IBM, SPARC компании Sun Microsystems и ряд других. Расширение области применения таких микропроцессоров в основном достигается путем повышения их производительности и снижения энергопотребления.

На настоящий момент около 80% процессоров общего назначения приходится на процессоры с архитектурой системы команд, совместимой с x86/IA-32 (Intel Architecture 32-bit). В связи с этим рассмотрим историю данной архитектуры с момента появления до настоящего времени.

Архитектура системы команд x86 впервые была представлена процессором Intel 8086, выпущенным в 1978 году. Этот 16-разрядный микропроцессор мог выполнять 98 инструкций и имел 7 режимов адресации.

Развитием Intel 8086 стал МП Intel 80186, в котором появились новые инструкции и новые элементы, такие как контроллеры ПДП, дешифраторы адреса, таймеры, контроллер прерываний. Это позволило значительно сократить количество дополнительных микросхем и интегрировать их функции в кристалл процессора.

В следующем процессоре Intel 80286 шины адреса и данных были физически разделены, шина адреса расширена до 24 битов по сравнению с 20 битами у процессора Intel 8086, увеличено количество регистров в связи с появлением нового режима работы МП — защищенного. *Защищенный режим*, или *режим защищенной виртуальной адресации* необходим для реализации многозадачности и расширения адресного пространства.

В 1985 году был выпущен 32-разрядный МП Intel 80386, в котором благодаря страничной организации стала возможной адресация до 4 ГБ памяти. Была усовершенствована работа с прерываниями, улучшена многозадачность с помощью системы приоритетного исполнения.

В 1989 году был представлен микропроцессор Intel 80486, который получил встроенный математический сопроцессор. Была реализована аппаратная поддержка объединения нескольких процессоров в один вычислитель, расширена очередь предварительной выборки команд, система команд увеличена до 150, улучшена защищенность страниц памяти. В связи с огромной популярностью было выпущено большое количество копий данного процессора, в том числе компаниями AMD, Siemens AG, IBM, Texas Instruments. Intel 80486 использовались во встраиваемых системах ввиду их малого энергопотребления и выпускались до 2007 года.

В 1993 году был выпущен процессор с архитектурой Intel P5 или Pentium. Это был первый CISC-процессор с суперскалярной архитектурой. В нем было два конвейера, что позволило выполнять две инструкции за один такт, шина данных стала 64-разрядной, кэш-память команд и кэш-память данных были разделены, появился механизм предсказания условных переходов, усовершенствован встроенный математический сопроцессор.

В 1995 году был представлен Pentium Pro — первый процессор архитектуры Intel P6, нацеленный на рынок рабочих станций и серверов. Intel P6 — суперскалярная суперконвейерная архитектура, лежащая в основе микропроцессоров Pentium Pro, Pentium II, Pentium III, Celeron и Xeon. В отличие от x86-совместимых процессоров предыдущих поколений с CISC-ядром, процессоры архитектуры P6 имели RISC-ядро, исполняющее сложные инструкции x86 не напрямую, а предварительно декодируя их в простые внутренние микрооперации.

В 1997 году был выпущен процессор Intel Pentium MMX, предназначенный для персональных компьютеров. Этот процессор отличался от Pentium новым блоком целочисленных матричных вычислений MMX (MultiMedia eXtension) и увеличенным до 32 КБ объемом кэш-памяти первого уровня.

Архитектура P6 для персональных компьютеров была представлена процессором Intel Pentium II, выпущенным в мае 1997 года и предназначавшимся для замены процессоров архитектуры P5.

Архитектура современных x86-совместимых процессоров Intel представляет собой усовершенствованную и дополненную функциональными устройствами архитектуру Intel P6. В связи с этим рассмотрим особенности построения современных микропроцессоров общего назначения на примере архитектуры Intel P6.

Первые процессоры архитектуры P6 имели существенные отличия от известных архитектур того времени. Процессор Pentium Pro отличали применение технологии динамического исполнения (изменения порядка исполнения инструкций), а также архитектура двойной независимой шины (dual independent bus), благодаря чему были сняты многие ограничения на пропускную способность памяти, характерные для предшественников и конкурентов.

Тактовая частота первого процессора архитектуры P6 составляла 150 МГц, а последние представители этой архитектуры имели тактовую частоту 1,4 ГГц. Процессоры архитектуры P6 имели 36-разрядную шину адреса, что позволило им адресовать до 64 ГБ памяти.

Основные характеристики данной архитектуры:

- суперскалярный механизм исполнения инструкций с изменением их последовательности;
- суперконвейеризация;
- двухуровневый адаптивный статистический алгоритм предсказания ветвлений;
- двойная независимая шина.

Принципиальным отличием архитектуры P6 от предшественников является RISC-ядро, работающее не с инструкциями x86, а с простыми внутренними микрооперациями. Это позволяет снять множество ограничений набора команд x86, таких как нерегулярное кодирование команд, переменная длина операндов и операции целочисленных пересылок «регистр-память». Кроме того, микрооперации исполняются не в той последовательности, которая предусмотрена программой, а в оптимальной с точки зрения производительности, применение трехконвейерной обработки дает возможность выполнять несколько инструкций за один такт.

Процессоры архитектуры P6 имеют конвейер длиной 12 ступеней. Это позволяет достигать более высоких тактовых частот по сравнению с процессорами, имеющими более короткий конвейер при одинаковой технологии производства. Так, максимальная тактовая частота

та процессоров AMD на ядре K6 (длина конвейера — 6 ступеней, технология — 180 нм) составляет 550 МГц, а процессоры Pentium III на ядре Coppermine способны работать на частоте, превышающей 1 ГГц.

Для того чтобы предотвратить ситуацию ожидания исполнения инструкции в архитектуре и Pentium, и Intel P6 используется структура, называемая *буфером предсказания переходов* (Branch Target Buffer — BTB), и оба процессора применяют методы предсказания, основанные на предыстории ветвлений. В Intel P6 использован двухуровневый адаптивный статистический алгоритм, который регистрирует предысторию и предсказывает переходы. Успешное предсказание ветвлений в Intel P6 в два раза больше по сравнению с процессорами Pentium и достигает 90%.

С целью увеличения пропускной способности подсистемы памяти в процессорах архитектуры P6 применяется двойная независимая шина. В отличие от предшествующих процессоров, системная шина которых была общей для нескольких устройств, процессоры архитектуры P6 имеют две отдельные шины: внутреннюю шину (Back Side Bus — BSB), соединяющую процессор с кэш-памятью второго уровня, и внешнюю системную шину (Front Side Bus — FSB), соединяющую процессор с северным мостом набора микросхем.

Процессоры архитектуры P6 состоят из четырех основных подсистем (рис. 4.4.1):

- подсистемы упорядоченной предварительной обработки;
- ядра исполнения с изменением последовательности;
- подсистемы упорядоченного завершения;
- подсистемы памяти.

Подсистема упорядоченной предварительной обработки

Подсистема упорядоченной предварительной обработки отвечает за выборку и декодирование инструкций в порядке, предусмотренном программой, и выполняет предсказание переходов.

К устройствам этой подсистемы относятся:

- модуль—буфер предсказания переходов BTB (Branch Target Buffer);
- декодер инструкций (instruction decoder);
- планировщик последовательности микроопераций (microcode sequencer);

- блок вычисления адреса следующей инструкции (next instruction pointer unit);

- блок выборки инструкций IFU (Instruction Fetch Unit).

Модуль—буфер предсказания переходов отвечают за предсказание переходов и хранение таблицы истории переходов. Таблица истории переходов содержит младшие значимые биты адреса инструкции и соответствующую им вероятность условного перехода: «скорее всего, будет выполнен», «возможно, будет выполнен», «возможно, не будет выполнен», «скорее всего, не будет выполнен» и обновляется после каждого выполненного перехода.

Декодер инструкций преобразует CISC-инструкции x86 в последовательность RISC-микроопераций, исполняемых процессором. Этот блок включает два декодера простых инструкций, обрабатывающих команды, которые могут быть выполнены одной микрооперацией, и декодер сложных инструкций, обрабатывающий команды, для которых нужно несколько (до четырех) микроопераций.

Планировщик последовательности микроопераций хранит последовательность микроопераций, полученных при декодировании сложных инструкций x86, требующих более четырех микроопераций.

Блок вычисления адреса следующей инструкции вычисляет адрес инструкции, которая должна быть обработана следующей, на основании таблицы переходов и информации о прерываниях.

Блок выборки инструкций реализует выборку инструкций из памяти по адресам, подготовленным блоком вычисления адреса следующей инструкции. Процессоры на ядре Tualatin дополнительно содержат блок предвыборки инструкций, который осуществляет предварительную выборку инструкций на основании таблицы переходов.

Исполнительное ядро с изменением последовательности операций

Исполнительное ядро с изменением последовательности операций отвечает за выполнение микроопераций в оптимальном порядке и организует взаимодействие ОУ. Исполнение с изменением последовательности, при котором меняется очередность выполнения инструкций так, чтобы это не приводило к изменению результата, позволяет ускорить работу за счет оптимального распределения загрузки и минимизации времени простоя вспомогательных блоков.

К устройствам организации исполнения с изменением последовательности относятся:

- таблица переименования регистров (register alias table);
- буфер переупорядочивания микроопераций (reorder buffer);
- станции резервирования (reservation stations).

Таблица переименования регистров устанавливает соответствие между регистрами архитектуры x86 и внутренними регистрами, используемыми при исполнении микроопераций.

Буфер переупорядочивания микроопераций обеспечивает выполнение микроопераций в оптимальной с точки зрения производительности последовательности.

Станции резервирования используются в алгоритме Томасуло для динамического планирования последовательности инструкций. Алгоритм Томасуло, разработанный в 1967 году инженером компании IBM Робертом Томасуло, позволяет процессору выполнять инструкции в таком порядке, чтобы те инструкции, которые имеют зависимости по данным, не вызывали остановку конвейера. Впервые этот алгоритм был реализован в математическом сопроцессоре IBM 360/91.

Станции резервирования обеспечивают возможность процессору выполнять выборку и использовать данные, которые являются результатом предыдущей операции, без необходимости записи этих данных в регистр и последующего чтения из него. Если несколько инструкций претендуют на запись в один регистр, то они будут выполнены логически последовательно, а в регистр будет записан только результат последней инструкции.

Станции резервирования устраняют конфликты по данным вида «чтение-после-записи» (Read After Write — RAW) путем проверки доступности операндов и структурные конфликты путем проверки доступности операционных устройств. Инструкции хранятся вместе с доступными параметрами и удаляются по мере готовности. Результаты определяются блоками, которые выполняют соответствующую инструкцию.

Неявное переименование регистров позволяет избегать конфликтов по данным вида «запись-после-записи» (Write After Write — WAW) и «чтение-после-записи» (Write After Read — WAR). Согласно алгоритму Томасуло инструкции буферизируются в порядке поступления их операндов. Если же операнд какой-либо инструкции не доступен, то соответствующая станция резервирования ожидает поступле-

ния операнда по общей шине, к которой подключены исполнительные устройства, и только после этого помещает инструкцию в очередь исполнения. Как правило, каждый исполнительный блок имеет свое подключение к общей шине и свою станцию резервирования.

К исполнительным устройствам ядра относятся:

- арифметико-логические устройства (АЛУ);
- блок операций с плавающей точкой FPU (Floating Point Unit);
- блок генерации адресов.

Арифметико-логические устройства выполняют арифметические и логические операции над целыми числами.

Блок операций с плавающей точкой реализует операции над вещественными числами. Процессоры Pentium III и последующие также имеют блок, осуществляющий исполнение инструкций SSE (SIMD FPU).

Блок генерации адресов вычисляет адреса данных, используемых инструкциями, и формирует запросы к кэш-памяти для загрузки/выгрузки этих данных.

Подсистема упорядоченного завершения

Подсистема упорядоченного завершения выдает результаты выполнения инструкций в порядке, предусмотренном программой. Эта система включает:

- регистровый файл (register file);
- буфер переупорядочивания памяти (memory reorder buffer);
- блок завершения (retirement unit).

Регистровый файл представляет собой массив регистров общего назначения процессора и является многопортовой памятью типа SRAM.

Буфер переупорядочивания памяти управляет порядком записи данных в память для предотвращения записи неверных данных из-за изменения порядка выполнения инструкций. Обычно каждая инструкция проходит три стадии: получение, выполнение, размещение результата. В алгоритме Томасуло каждая инструкция проходит дополнительную стадию — фиксацию результата, в то время как на стадии размещения результата результат выполнения инструкции попадает в буфер переупорядочивания памяти. Кроме того, буфер переупорядочивания памяти позволяет обрабатывать исключения и управлять

возвратом в случае ошибочного перехода. Буфер переупорядочивания памяти хранит инструкции в том порядке, в каком они были выбраны. Каждая станция резервирования имеет параметр, который позволяет получить доступ к определенной инструкции в буфере. В случае возникновения ошибки предсказания перехода или в случае необрабатываемого исключения все инструкции, хранящиеся в буфере, удаляются, а станция резервирования инициализируется повторно.

Блок завершения выдает результаты исполнения инструкций в той последовательности, в которой они поступили на исполнение.

Подсистема памяти

Подсистема памяти осуществляет взаимодействие с оперативной памятью. К этой подсистеме относятся:

- кэш-память данных первого уровня (level 1 data cache);
- кэш-память инструкций первого уровня (level 1 instruction cache);
- кэш-память второго уровня (level 2 cache);
- блок интерфейсов шины (bus interface unit).

Кэш-память данных первого уровня — память с малым временем доступа объемом 8 (для Pentium Pro) или 16 (для более новых процессоров) КБ, предназначенная для хранения данных.

Кэш-память инструкций первого уровня — память с малым временем доступа объемом 8 (для Pentium Pro) или 16 КБ, предназначенная для хранения инструкций.

Кэш-память второго уровня — память с малым временем доступа объемом 128, 256, 512, 1024 или 2048 КБ в зависимости от модели процессора. Ширина шины BSB составляет 64 или 256 битов (для процессоров на ядре Core2 и последующих). Процессоры Celeron на ядре Covington кэш-памяти второго уровня не имеют.

Блок интерфейсов шины управляет обменом командами и данными по системной шине.

Исполнение инструкции

Исполнение инструкции начинается с ее выборки и декодирования. Для этого из кэш-памяти инструкций первого уровня по адресу из буфера предсказания переходов выбирается 64 байта (две строки). Из них 16 байтов, начиная с адреса из блока вычисления ад-

реса следующей инструкции, выравниваются и передаются в декодер инструкций, преобразующий инструкции x86 в RISC-микрооперации. Если инструкции соответствует одна микрооперация, декодирование проводит один из декодеров простых инструкций. Если инструкции соответствует две, три или четыре микрооперации, декодирование проводит декодер сложных инструкций. Если инструкции соответствует большее число микроопераций, то они формируются планировщиком последовательностей микроопераций.

После декодирования инструкций проводится переименование регистров, а микрооперации и данные помещаются в буфер — станцию резервирования, откуда в соответствии с оптимальным порядком исполнения и при условии доступности необходимых для исполнения операндов направляются на исполнительные блоки (максимум пять инструкций за такт). Статус исполнения микроопераций и его результаты хранятся в буфере переупорядочивания микроопераций, а так как результаты исполнения одних микроопераций могут служить операндами других, они также помещаются и в станцию резервирования.

По результатам исполнения микроопераций определяется их готовность к завершению. В случае готовности происходит их завершение в порядке, предусмотренном программой, во время которой осуществляется обновление состояния логических регистров, а также отложенная фиксация результатов в памяти, управление порядком которой осуществляет буфер переупорядочивания памяти.

Конвейер микропроцессора Intel Pentium Pro имеет 12 ступеней (рис. 4.4.2):

- BTB0, BTB1 (Branch Target Buffer) — модуль и буфер предсказания переходов;
- IFU0, IFU1, IFU2 (Instruction Fetch Unit) — модуль выборки инструкций;
- ID0, ID1 (Instruction Decoder) — декодер инструкций;
- RAT (Register Alias Table) — таблица переименования регистров;
- ROB Rd (Reorder Buffer Read) — чтение из буфера переупорядочивания микроопераций;
- RS (Reservation Station) — станция резервирования;
- ROB WB (Reorder Buffer Writeback) — запись в буфер переупорядочивания микроопераций;
- RRF (Register Retirement File) — размещение результатов выполнения инструкции в регистр.

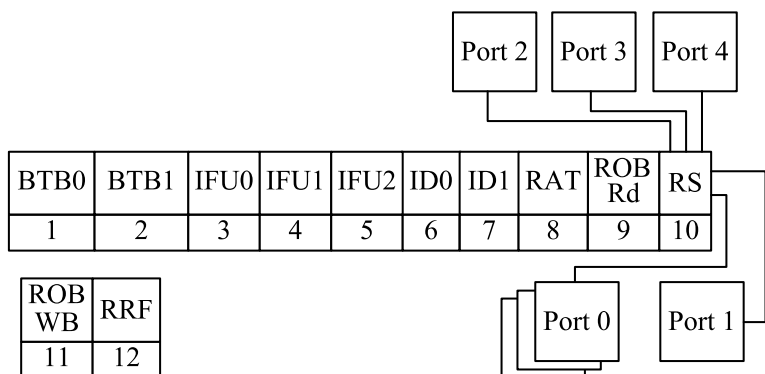


Рис. 4.4.2. Структура конвейера микропроцессора Intel Pentium Pro

На первых двух ступенях конвейера выполняется предсказание условного перехода, модуль предсказания переходов сообщает блоку выборки инструкций, где находится следующая инструкция.

На следующих трех ступенях конвейера реализуется выборка инструкций. Выбранных макроинструкций может быть не более трех за один машинный цикл, и длина каждой из них должна быть меньше 7 байтов.

На шестой и седьмой ступенях конвейера происходит декодирование инструкций. Декодер сложных инструкций может декодировать одну макроинструкцию, состоящую из четырех и менее микроопераций, за один машинный цикл. Оставшиеся два декодера простых инструкций могут декодировать инструкции, состоящие из одной микрооперации, по одной за каждый такт.

В табл. 4.4.1 показано, сколько микроопераций в общем случае требуется выполнить для реализации инструкции того или иного типа. Выполнение сложных инструкций, которые состоят из четырех и более микроопераций, потребует нескольких циклов декодирования.

На восьмой ступени конвейера происходит переименование регистров в таблице переименования регистров и размещение инструкции в буфере переупорядочивания микроопераций.

На девятой и десятой ступенях конвейера микрооперации читаются из буфера переупорядочивания микроопераций по соответствующему порту и попадают в станцию резервирования.

Таблица 4.4.1**Количество микроопераций в зависимости от типа инструкции**

Тип инструкции	Количество микроопераций
Простые инструкции пересылки «регистр-регистр»	1
Чтение данных из памяти в регистр общего назначения по выровненному адресу	1
Сохранение данных из регистра общего назначения в память по выровненному адресу	2
Простые инструкции «чтение-запись»	2
Простые операции пересылки «регистр-память»	2–3
Простые инструкции «чтение-изменение-запись»	4

На одиннадцатой ступени конвейера микрооперации могут быть записаны обратно в буфер переупорядочивания микроопераций, где будут храниться до тех пор, пока их выполнение не сможет завершиться.

На двенадцатой ступени конвейера происходит фиксация результатов в регистрах процессора в случае завершения микрооперации.

Процессоры архитектуры P6 имеют конвейеризованный математический сопроцессор, позволяющий достичь превосходства над предшественниками и конкурентами в скорости вычислений с вещественными числами. Встроенный математический сопроцессор процессоров архитектуры Intel P6 оставался лучшим среди конкурентов до появления в 1999 году процессора AMD Athlon. Кроме того, процессоры архитектуры Intel P6 имели превосходство над конкурентами и в скорости работы с кэш-памятью второго уровня. Pentium Pro и Pentium II имели двойную независимую шину, в то время как конкурирующие процессоры (AMD K5, K6, Cyrix 6x86) — традиционную системную шину, к которой подключалась в том числе и кэш-память второго уровня.

С появлением процессоров AMD Athlon, также использующих архитектуру с двойной независимой шиной, разрыв в производительности сократился, но 256-разрядная BSB процессоров Pentium III (начиная с ядра Coppermine) позволяла удерживать преимущество в скорости работы с кэш-памятью второго уровня над процессорами архитектуры K7, имевшими 64-разрядную BSB. Однако устаревшая на тот момент системная шина процессоров архитектуры Intel P6 в сочетании с большим объемом кэш-памяти первого уровня у процессоров архитектуры AMD K7 не позволяла получить преимущество в пропускной способности памяти.

Основным недостатком первых процессоров архитектуры P6 была низкая производительность при работе с широко распространенным в то время 16-разрядным программным обеспечением. Это было связано с тем, что при работе с такими приложениями внеочередное исполнение инструкций было затруднено. Так, процессор Pentium Pro не мог выполнить чтение из 32-битного регистра, если до этого была выполнена запись в его 16-битную младшую часть, а команда, выполнявшая запись, не была завершена. В процессоре Pentium II этот недостаток был устранен, что привело к увеличению производительности при работе с 16-разрядными программами более чем на треть.

Процессоры архитектуры P6 поддерживают работу в многопроцессорных системах, однако при этом используется разделяемая системная шина, что позволяет упростить трассировку системных плат, однако отрицательно сказывается на производительности подсистемы процессор-память и ограничивает максимальное количество процессоров в системе.

В 2000 году на смену архитектуре P6 на рынке настольных и серверных процессоров пришла архитектура NetBurst, представленная процессорами Pentium 4, Pentium D, Celeron, Celeron D, Xeon. Архитектура NetBurst является суперскалярной и суперконвейерной, она позволила достичь больших тактовых частот (3 ГГц и даже выше) благодаря использованию 24-стадийного конвейера. Кроме того, была применена технология Hyperthreading, которая в ряде задач могла увеличить быстродействие, а в других могла уменьшить его из-за перезагрузок многостадийного конвейера. По этой причине, а также вследствие высокого энергопотребления процессоров на этой архитектуре Intel приняла решение вернуться к архитектуре P6. Сменилось направление развития: от сложных одноядерных структур вида

NetBurst к упрощенным многоядерным с использованием последних достижений в области процессоров с RISC-архитектурой.

В 2006 году на смену процессорам архитектуры NetBurst пришли процессоры семейства Core 2 Duo, архитектура которых также представляет собой развитие архитектуры P6. Следующие архитектуры Intel, в том числе Nehalem, Sandy Bridge, базируются на архитектуре Core, которая является модифицированной архитектурой Intel P6.

Контрольные вопросы

1. В чем основное отличие архитектуры Intel P6 от предыдущих архитектур процессоров общего назначения?
2. Из каких подсистем состоит микропроцессор архитектуры P6?
3. Для чего используется механизм предсказания ветвлений?
4. Из каких ступеней состоит конвейер процессора Intel Pro?

Литература

1. **Бойко В.И.** Схемотехника электронных систем. Микропроцессоры и микроконтроллеры. — СПб.: БХВ-Петербург, 2004. — 464 с.
2. **Антошина И.В., Котов Ю.Т.** Микропроцессоры и микропроцессорные системы (аналитический обзор): учеб. пособие. — М.: МГУЛ, 2005. — 432 с.
3. Intel Architecture Optimization Manual. — 1997. — <http://download.intel.com/design/pentiumii/manuals/24281603.pdf>.
4. **Корнеев В.В., Киселев А.В.** Современные микропроцессоры. — 3-е изд., перераб. и доп. — СПб.: БХВ-Петербург, 2003. — 448 с.
5. Intel P6 Family of Processors. Hardware Developer's Manual. — 1998. — <http://download.intel.com/design/PentiumII/manuals/24400101.pdf>.

Раздел 5. Архитектуры и структуры параллельных вычислительных систем

Лекция 5.1. Назначение, область применения и классификация архитектур параллельных вычислительных систем

Сфера применения параллельных вычислительных систем (ПВС) непрерывно расширяется, начиная с середины XX века, это обусловлено тем, что данные системы предлагают пользователю высокую концентрацию вычислительных ресурсов, ресурсов памяти, высокоскоростных каналов ввода/вывода для решения важных задач цифровой обработки информации. Такая неформальная характеристика ПВС объединяет суперкомпьютеры с сотнями и тысячами процессоров, рабочие станции, объединенные в сеть, многопроцессорные рабочие станции и встроенные системы. Общей особенностью этих систем является использование параллельной работы различных устройств для ускорения выполнения операций.

Традиционно ПВС применялись в основном в научной сфере для решения задач, требующих мощных вычислительных ресурсов. Со временем совокупность таких научно-технических задач объединили понятием «grand challenges»; типовыми примерами являются задачи предсказания погоды, построения полупроводниковых приборов, астрономические и генетические задачи и т. д.

В связи с тем, что компьютерные технологии и электронный документооборот используются коммерческими организациями все шире, непрерывно растет потребность в построении централизованных вычислительных систем для критически важных приложений, связанных с обработкой транзакций, управлением базами данных и обслуживанием телекоммуникаций. Такие системы решают задачи глобальных корпоративных вычислений и отличаются повышенной надежностью, масштабируемостью, минимально допустимым временем простоя.

Таким образом, по области применения ПВС можно разделить на системы, ориентированные на решение сложных научно-технических задач, и системы, направленные на решение глобальных корпоративных задач. В свою очередь системы второго класса имеют две области применения: обработка транзакций в реальном времени и организация систем хранения и управления данными.

По специфическим особенностям назначения все ПВС можно разделить на три класса:

- системы высокой надежности;
- системы высокопроизводительных вычислений;
- многопоточные системы.

Благодаря тому, что ПВС, как правило, состоят из множества функционально заменяемых модулей, их часто используют для повышения надежности информационно-вычислительных систем. Замена физических модулей происходит незаметно для конечного пользователя, что обеспечивает непрерывную и безотказную работу.

Системы высокопроизводительных вычислений ориентированы на параллельное использование ресурсов с целью одновременного выполнения большого количества операций.

Многопоточные системы применяются для обеспечения единого интерфейса к ряду ресурсов, которые могут со временем произвольно наращиваться или сокращаться. Типовым примером такой задачи является совместная работа группы web-серверов.

Отметим, что приведенная классификация в некоторой степени условна: часто системы обладают свойствами и функциями, которые выходят за рамки рассмотренных выше классов. Более того, понятие архитектуры ПВС является достаточно широким, поскольку может включать в себя и способ параллельной обработки данных, и организацию памяти, и топологию связей между процессорными элементами, и способ использования арифметических операций.

На протяжении всей истории развития вычислительной техники предпринимались попытки найти какую-то общую классификацию, в которую попадали бы все возможные направления развития компьютерных архитектур. Однако ни одна из таких классификаций не могла охватить все разнообразие разрабатываемых архитектурных решений и не выдерживала испытания временем. Тем не менее, ряд классификаций широко применяются специалистами, пользователями и встречаются в литературе, например, предложенная в

1966 году М. Флинном классификация архитектур вычислительных систем.

В основу данной классификации положено понятие потока, под которым понимается последовательность команд или данных, обрабатываемых системой. В соответствии с таким подходом вычислительные системы подразделяются на четыре класса (рис. 5.1.1):

- 1) одиночный поток команд, одиночный поток данных (Single Instruction Single Data — SISD);
- 2) одиночный поток команд, множественный поток данных (Single Instruction Multiple Data — SIMD);
- 3) множественный поток команд, одиночный поток данных (Multiple Instruction Single Data — MISD);

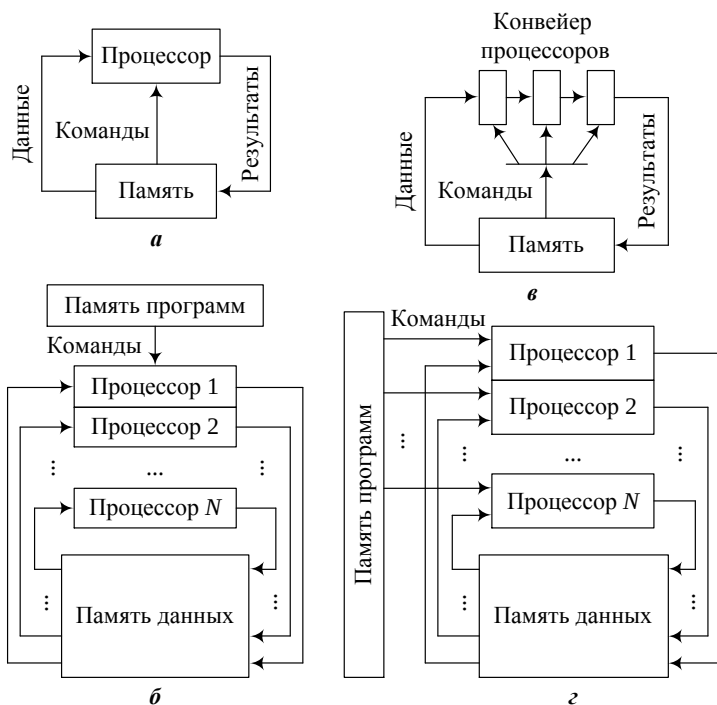


Рис. 5.1.1. Архитектуры вычислительных систем по Флинну: а — SISD; б — SIMD; в — MISD; г — MIMD

4) множественный поток команд, множественный поток данных (Multiple Instruction Multiple Data — MIMD).

SISD-системы относятся к традиционным вычислительным системам, в частности, систему на рис. 5.1.1, а можно отнести к архитектуре фон Неймана, SIMD-, MISD- и MIMD-системы на рис. 5.1.1, б, в, г принадлежат к параллельным вычислительным системам.

Классификация Флинна, как и многие другие, несовершенна: существуют системы, прямо в нее не попадающие, имеются важные признаки, которые в этой классификации не учтены. Так, к машинам типа SIMD часто относят векторные процессоры, хотя их высокая производительность зависит от другой формы параллелизма — конвейерной организации машин. Многопроцессорные векторные системы типа Cray состоят из нескольких векторных процессоров и поэтому могут быть названы Multiple SIMD (MSIMD). Классификация Флинна не делает различия по другим важным для вычислительных систем характеристикам, например, по уровню «зернистости» параллельных вычислений и методам синхронизации.

Можно выделить четыре основных типа архитектуры ПВС:

- системы с конвейерной и векторной обработкой;
- SIMD-системы;
- MIMD-системы;
- многопроцессорные системы с SIMD-процессорами.

Системы с конвейерной и векторной обработкой

Системы с конвейерной обработкой основаны на раздельном выполнении некоторой операции в несколько этапов (за несколько ступеней) с передачей данных от одного этапа к следующему. Производительность при этом возрастает благодаря тому, что одновременно на различных ступенях конвейера выполняются несколько операций. Конвейеризация эффективна только тогда, когда загрузка конвейера близка к полной, а скорость подачи новых операндов соответствует максимальной производительности конвейера. Если происходит задержка, то параллельно будет выполняться меньше операций и суммарная производительность снизится. Векторные операции обеспечивают идеальную возможность полной загрузки вычислительного конвейера.

При выполнении векторной команды одна и та же операция применяется ко всем элементам вектора или чаще всего к соответствующим элементам пары векторов. Для настройки конвейера на выполнение конкретной операции может потребоваться некоторое установочное время, однако затем операнды могут поступать в конвейер с максимальной скоростью, допускаемой возможностями памяти. При этом не возникает пауз ни в связи с выборкой новой команды, ни в связи с определением ветви вычислений при условном переходе. Таким образом, главный принцип вычислений на векторной машине состоит в выполнении некоторой элементарной операции или комбинации из нескольких элементарных операций, которые должны повторно применяться к некоторому блоку данных. Таким операциям в исходной программе соответствуют небольшие компактные циклы.

SIMD-системы

SIMD-системы состоят из большого числа идентичных процессорных элементов, имеющих собственную память. Все процессорные элементы в такой машине выполняют одну и ту же программу. Очевидно, что такая машина, составленная из большого числа процессоров, может обеспечить очень высокую производительность только на тех задачах, при решении которых все процессоры могут делать одну и ту же работу. Модель вычислений для машины SIMD очень похожа на модель вычислений для векторного процессора: одиночная операция выполняется над большим блоком данных.

В отличие от ограниченного конвейерного функционирования векторного процессора, SIMD-система может быть значительно более гибкой. В большинстве случаев термин «матричная вычислительная система» является синонимом термина «SIMD-система». Обработывающие элементы таких систем — это универсальные программируемые вычислительные машины, поэтому задача, решаемая параллельно, может быть достаточно сложной и содержать ветвления. Обычное проявление этой вычислительной модели в исходной программе примерно такое же, как и в случае векторных операций: циклы на элементах массива, в которых значения, вырабатываемые на одной итерации цикла, не используются на другой итерации цикла. Модели вычислений на векторных и матричных системах настолько схожи, что их часто рассматривают как эквивалентные.

MIMD-системы

Аналогично тому, как термин «матричная вычислительная система» применяется к SIMD-системам, термин «мультипроцессорная система» объединяет большинство машин типа MIMD и часто используется в качестве синонима. В мультипроцессорной системе каждый процессорный элемент (ПЭ) выполняет свою программу независимо от других ПЭ. В то же время ПЭ имеют связи друг с другом, что делает необходимой более подробную классификацию машин типа MIMD.

В мультипроцессорных системах с общей памятью или сильно-связанных мультипроцессорах имеется память данных и команд, доступная всем ПЭ. С общей памятью ПЭ связываются с помощью общей шины или коммутационной среды. В противоположность этому варианту в слабосвязанных многопроцессорных системах (с локальной памятью) вся память делится между процессорными элементами и каждый блок памяти доступен только связанному с ним процессору. Коммутационная среда соединяет ПЭ друг с другом.

Базовой моделью вычислений MIMD-системы является совокупность независимых процессов, эпизодически обращающихся к разделяемым данным. Существует большое количество вариантов этой модели: на одном конце спектра — модель распределенных вычислений, в которой программа делится на довольно большое число параллельных задач, состоящих из множества подпрограмм; на другом конце спектра — модель потоковых вычислений, в которых каждая операция в программе может рассматриваться как отдельный процесс. Такая операция ждет своих операндов, которые должны быть переданы ей другими процессами. После их получения операция выполняется, и полученное значение передается тем процессам, которые в нем нуждаются. В потоковых моделях вычислений с большим и средним уровнем гранулярности процессы содержат большое число операций и выполняются в потоковой манере.

Многопроцессорные системы с SIMD-процессорами

Многие современные суперЭВМ представляют собой многопроцессорные системы, в которых в качестве процессоров используются векторные процессоры или процессоры типа SIMD. Такие машины относятся к машинам класса MSIMD.

Языки программирования и соответствующие компиляторы для машин типа MSIMD обычно обеспечивают языковые конструкции, которые позволяют программисту описывать «крупнозернистый» параллелизм. В пределах каждой задачи компилятор автоматически векторизует подходящие циклы. Машины типа MSIMD, как можно себе представить, дают возможность использовать лучший из этих двух принципов декомпозиции: векторные операции («мелкозернистый» параллелизм) для тех частей программы, которые подходят для этого, и гибкие возможности MIMD-архитектуры для других частей программы.

Исторически первой стала осваиваться технология SIMD. Однако в настоящее время наблюдается повышенный интерес к архитектурам MIMD. Этот интерес главным образом определяется двумя факторами:

1) архитектура MIMD дает большую гибкость, при наличии адекватной поддержки со стороны аппаратных средств и программного обеспечения MIMD может работать и как однопользовательская система, обеспечивая высокопроизводительную обработку данных для одной прикладной задачи, и как многопрограммная машина, выполняющая множество задач параллельно, и как некоторая комбинация этих возможностей;

2) архитектура MIMD может использовать все преимущества современной микропроцессорной технологии на основе строгого учета отношения стоимость/производительность; в действительности практически все современные многопроцессорные системы строятся на тех же микропроцессорах, которые можно найти в персональных компьютерах, рабочих станциях и небольших однопроцессорных серверах.

Одной из отличительных особенностей многопроцессорной вычислительной системы является коммутационная среда, с помощью которой процессоры соединяются друг с другом или с памятью. Модель обмена настолько важна для такой системы, что многие характеристики производительности и другие оценки выражаются отношением времени обработки к времени обмена, соответствующим решаемым задачам. Существуют две основные модели межпроцессорного обмена: одна основана на передаче сообщений, другая — на использовании общей памяти.

В многопроцессорной системе с общей памятью один процессор осуществляет запись в конкретную ячейку, а другой процессор про-

изводит считывание из этой ячейки памяти. Чтобы обеспечить согласованность данных и синхронизацию процессов, обмен часто реализуется по принципу взаимоисключающего доступа к общей памяти.

В архитектурах с локальной памятью непосредственное разделение памяти невозможно. Вместо этого процессоры получают доступ к совместно используемым данным посредством передачи сообщений через коммутационную среду. Эффективность схемы коммуникаций зависит от протоколов обмена, основных сетей обмена и пропускной способности памяти и каналов обмена.

Часто в системах с общей памятью затраты на обмен не учитываются, так как проблемы обмена в значительной степени скрыты от программиста. Однако накладные расходы на обмен в этих системах имеются и определяются конфликтами шин, памяти и процессоров. Чем больше процессоров добавляется в систему, тем больше процессов соперничают при использовании одних и тех же данных и шины, что приводит к состоянию насыщения. Модель системы с общей памятью очень удобна для программирования и иногда рассматривается как высокоуровневое средство оценки влияния обмена на работу системы, даже если основная система в действительности реализована с применением локальной памяти и принципа передачи сообщений.

В сетях с коммутацией каналов и в сетях с коммутацией пакетов по мере возрастания требований к обмену следует учитывать возможность перегрузки сети. Здесь межпроцессорный обмен связывает сетевые ресурсы: каналы, процессоры, буферы сообщений. Объем передаваемой информации может быть сокращен с помощью тщательной функциональной декомпозиции задачи и диспетчеризации выполняемых функций.

Оценка производительности

Различают пиковую и реальную производительности. Пиковая производительность является теоретической величиной, при ее расчете предполагается, что все блоки системы функционируют в максимально производительном режиме. Эта характеристика является базовой при сравнении высокопроизводительных вычислительных систем. Чем больше пиковая производительность, тем теоретически быстрее пользователь сможет решить задачу.

На практике пиковая производительность недостижима, и речь идет о реальной производительности, получаемой при выполнении конкретного приложения. На выполнение конкретной программы в реальных условиях влияют такие особенности системы, как структура процессорных элементов, система команд, состав функциональных устройств, реализация ввода/вывода, эффективность работы компилятора и т. д.

Известно два способа оценки пиковой производительности. Первый способ состоит в определении количества команд, выполняемых системой за единицу времени. Единицей измерения, как правило, является *миллион операций в секунду* — MIPS (Million Instructions Per Second). Производительность, выраженная в MIPS, говорит о скорости выполнения системой своих же инструкций. При этом неизвестно, в какое количество таких инструкций отобразится конкретная программа, поэтому пиковая производительность, выраженная в MIPS, дает только общее представление о производительности системы.

Второй способ оценки пиковой производительности заключается в определении количества операций с вещественными операндами, выполняемых системой за единицу времени. Единицей измерения является *количество операций над числами с плавающей точкой в секунду* — FLOPS (Floating Point Operations Per Second). Этот способ более информативен для конечного пользователя, поскольку ему, как правило, известна вычислительная сложность программы.

Для оценки эффективности работы системы при выполнении реальных задач разработаны фиксированные наборы тестов. Наиболее известным набором является пакет LINPACK, основанный на решении системы линейных алгебраических уравнений. Данный пакет используется при формировании списка Top 500 — пятисот наиболее мощных вычислительных систем в мире. Недостатком такого подхода является естественный параллелизм задачи, решаемой LINPACK, поэтому невозможно оценить эффективность функционирования коммутационной среды системы.

Большое распространение получили тестовые программы, основанные на использовании алгоритмов из различных предметных областей и представляющие собой либо модельные, либо реальные промышленные приложения. Такой подход позволяет получить представление о фактической производительности системы при выполнении реальных задач.

Несмотря на разнообразие тестовых программ, задача определения реальной производительности остается нерешенной, поскольку тесты не дают представления о функционировании системы в различных режимах.

Контрольные вопросы

1. Дайте определение параллельной вычислительной системы.
2. Перечислите классы архитектур вычислительных систем по Флинну.
3. Охарактеризуйте SIMD-системы.
4. Чем отличаются пиковая и реальная производительности параллельной вычислительной системы?
5. Какие особенности вычислительной системы влияют на снижение ее производительности при выполнении конкретной программы?

Литература

1. Архитектуры и топологии многопроцессорных вычислительных систем / **А.В. Богданов, В.В. Корхов, В.В. Мареев, Е.Н. Станкова.** — М.: ИНТУИТ.РУ, 2004. — 176 с.
2. Микропроцессорные системы: учеб. пособие для вузов / **Е.К. Александров, Р.И. Грушвицкий, М.С. Куприянов и др.; под ред. Д.В. Пузанкова.** — СПб.: Политехника, 2002. — 935 с.

Лекция 5.2. Параллельные вычислительные системы с общей разделяемой и распределенной памятью.

Обобщенная масштабируемая структура параллельных вычислительных систем

Экспериментальные разработки по созданию многопроцессорных вычислительных систем начались в 1960-х годах. Одной из первых таких систем стала система ILLIAC-IV (Illinois Automatic Computer), которая включала 64 процессорных элемента, работающих по единой программе, применяемой к содержимому собственной оперативной памяти каждого ПЭ. Обмен данными между процессорами осуществлялся через специальную матрицу коммуникационных каналов. Стоимость первых матричных вычислительных систем (МВС) была чрезвычайно высока, и выпускались они буквально поштучно. Серийные образцы подобных систем появились значительно позже, однако не получили широкого распространения ввиду сложности программирования МВС с одним потоком управления.

Первые промышленные образцы многопроцессорных систем появились на базе векторно-конвейерных компьютеров в середине 1980-х годов. Наиболее распространенными системами такого типа были суперкомпьютеры фирмы Cray. Однако такие системы были чрезвычайно дорогими и производились небольшими партиями. Как правило, в подобных компьютерах объединялись от 2 до 16 процессоров, которые имели равноправный или симметричный доступ к общей оперативной памяти. В связи с этим они получили название симметричных мультипроцессорных систем (Symmetric Multiprocessing — SMP).

Альтернативой дорогим мультипроцессорным системам на базе векторно-конвейерных процессоров стали эквивалентные по мощности многопроцессорные системы из большого числа дешевых серийно выпускаемых микропроцессоров. Однако очень скоро обнаружилось, что SMP-архитектура обладает весьма ограниченными возможностями по наращиванию числа процессоров в системе из-за резкого увеличения числа конфликтов при обращении к общей шине памяти. Поэтому было предложено снабжать каждый процессор собственной

оперативной памятью, превращая параллельную систему в объединение независимых вычислительных узлов. Такой подход значительно увеличил степень масштабируемости многопроцессорных систем, но в свою очередь потребовал разработки специального способа обмена данными между вычислительными узлами, реализуемого обычно в виде механизма передачи сообщений, или Message Passing. Системы с такой архитектурой положили начало целому направлению ПВС — МРА-систем (Message Passing Architecture). В настоящее время эти два направления или их комбинации являются доминирующими в развитии суперкомпьютерных технологий.

Нечто среднее между SMP- и МРА-системами представляют собой системы с неоднородным доступом к памяти, или NUMA-системы (Non Uniform Memory Access), в которых память физически разделена, но логически общедоступна. При этом время доступа к различным блокам памяти становится неодинаковым. В одной из первых систем этого типа Cray T3D время доступа к памяти другого процессора было в 6 раз больше, чем к собственной.

Рассмотрим подробнее особенности систем каждого из трех перечисленных типов.

ПВС с общей разделяемой памятью (SMP-архитектура)

Основной особенностью SMP-систем является наличие общей физической памяти, разделяемой всеми процессорными элементами. Обычно такая система включает в себя несколько десятков ПЭ. Сравнительно небольшое количество процессоров в таких машинах позволяет иметь одну централизованную общую память и объединить процессоры и память с помощью одной шины. При наличии у процессоров кэш-памяти достаточного объема высокопроизводительная шина и общая память могут удовлетворить обращения к памяти, поступающие от нескольких процессоров. Поскольку имеется единственная память с одним и тем же временем доступа, эти системы иногда называют UMA (Uniform Memory Access). Типовая структура SMP-системы представлена на рис. 5.2.1.

Для соединения с модулями ввода/вывода используются более медленные магистрали, чем основная системная. Обычно SMP-система работает под управлением единой операционной системы, которая распределяет вычислительные процессы по ПЭ. Традиционно приме-

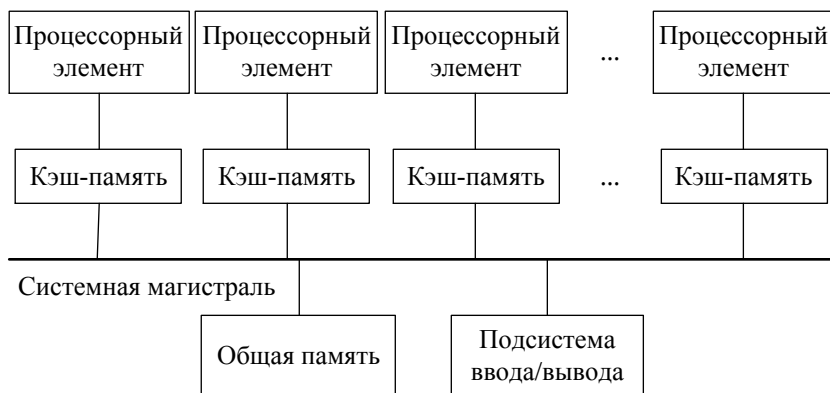


Рис. 5.2.1. Типовая архитектура мультипроцессорной системы с общей разделяемой памятью

няют UNIX-подобные операционные системы, однако для платформ на основе процессоров Intel реализуют поддержку Windows.

Архитектура SMP не накладывает ограничений на модель программирования, используемую при создании приложений. Обычно применяют модель параллельных процессов, при этом все процессорные элементы функционируют параллельно и независимо друг от друга. В то же время возможна организация вычислений с использованием межпроцессорного обмена. Применение общей памяти ускоряет процесс такого обмена, причем пользователь имеет доступ ко всему объему памяти в системе. Помимо этого для SMP-систем созданы достаточно эффективные средства автоматического распараллеливания задач.

Таким образом, основными достоинствами SMP-систем являются:

- простота и универсальность программирования;
- простота эксплуатации;
- относительно невысокая стоимость.

Недостаток SMP-систем — плохая масштабируемость, обусловленная конечной пропускной способностью системной магистрали, а также тем, что в каждый момент времени магистраль способна обслужить только одно активное устройство. Из-за наличия множества процессорных элементов, потенциально запрашивающих разрешения

на обмен информацией, могут возникать конфликты при обращениях к одним и тем же физическим областям памяти. Такие конфликты появляются уже при наличии восьми процессорных элементов и существенно снижают производительность системы в целом.

Несмотря на то, что общая магистраль является наиболее простой и распространенной топологией SMP-систем, можно выделить несколько основных типов коммуникационных сред, используемых в архитектурах с общей разделяемой памятью.

Для удовлетворения требований по загрузке система может иметь несколько каналов ввода/вывода, которые обеспечивают прямой доступ к каждому модулю памяти. Такого типа системы имеют организацию перекрестного соединения процессоров, каналов ввода/вывода и модулей памяти (рис. 5.2.2, а).

Недостатком перекрестного соединения является высокая стоимость, что в большинстве случаев приводит к появлению многоуровневой иерархической структуры коммутационной среды (рис. 5.2.2, б), при этом стоимость растет медленнее, чем количество портов. Экономия приводит к увеличению времени ожидания соединения и уменьшению полосы пропускания на порт. Прямой доступ ко всей памяти из любого процессора имеет несколько достоинств: любой процессор может запустить любые процессы или обратиться к любому событию ввода/вывода, а структуры данных могут быть разделены внутри операционной системы.

ПВС с общей распределенной памятью (МРА-архитектура)

Для того чтобы поддерживать большое количество процессоров, приходится физически распределять основную память между ними, в противном случае полосы пропускания памяти может не хватить для удовлетворения запросов, поступающих от большого числа процессоров. Кроме того, требуется реализовать связь процессоров между собой. Решением этой задачи является построение МРА-системы, основной особенностью которой является физически разделенная память. Такая система состоит из модулей, содержащих процессорный элемент, кэш-память, модуль локальной памяти, устройства ввода/вывода (рис. 5.2.3). Вычислительные модули объединены коммуникационной средой, обеспечивающей взаимодействие процессоров

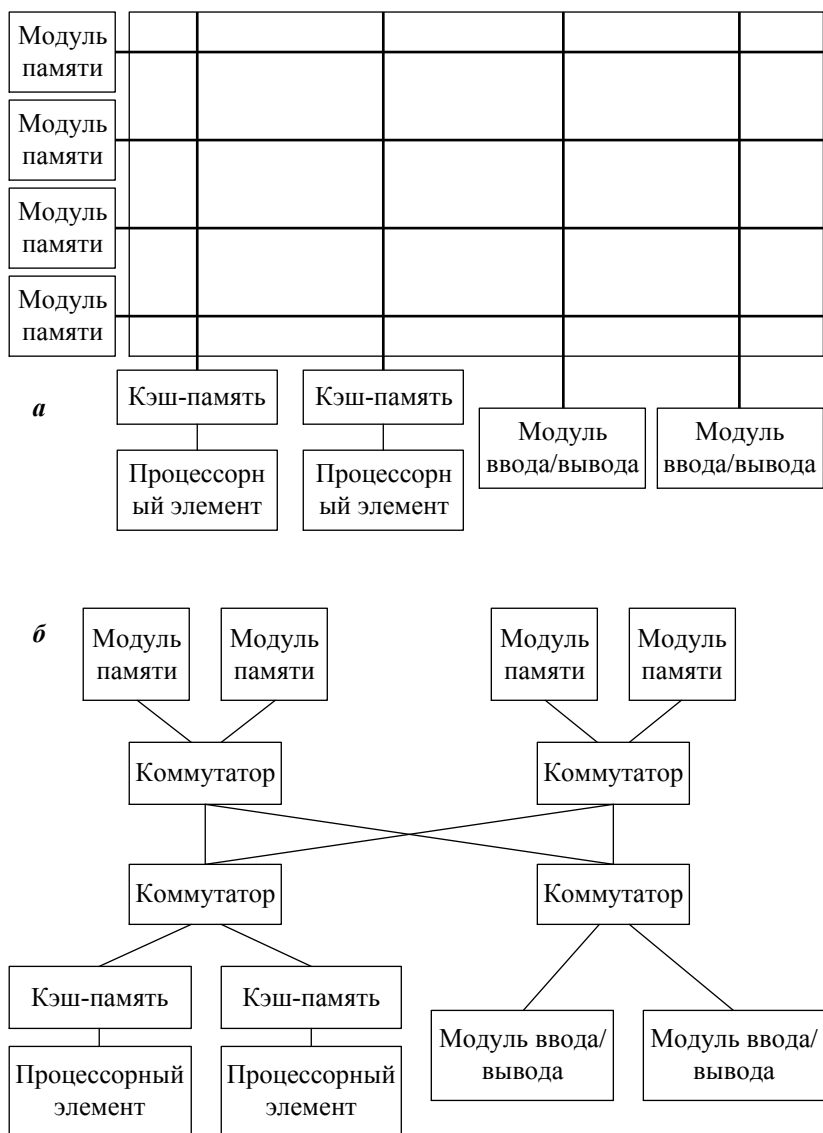


Рис. 5.2.2. Типовые структуры коммутационных сред в SMP-системах

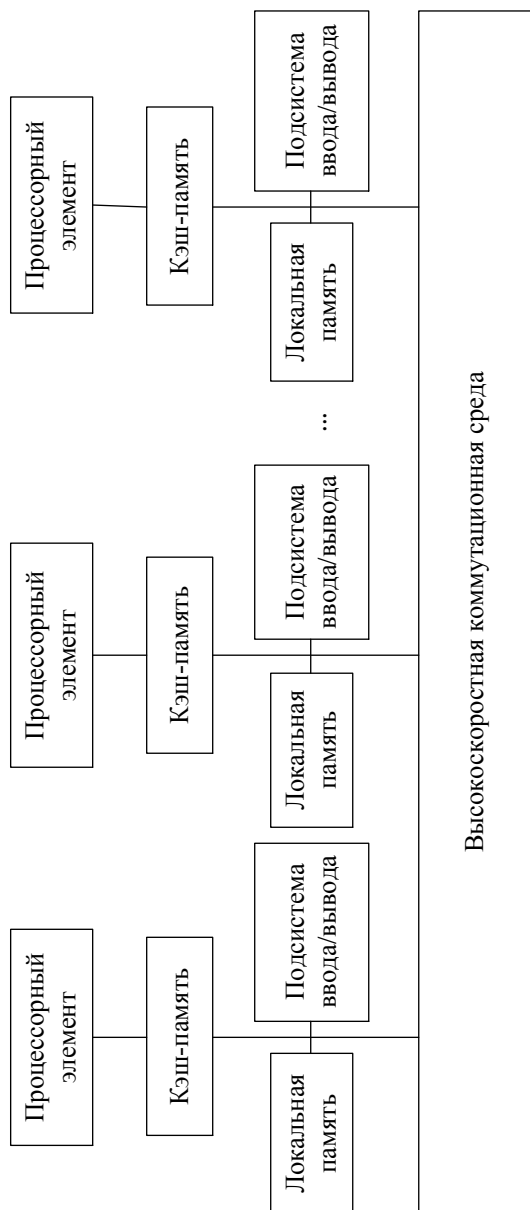


Рис. 5.2.3. Типовая структура мультипроцессорной системы с общей распределенной памятью

посредством простых операций ввода/вывода. Таким образом, в этих системах коммуникации интегрированы в уровень ввода/вывода, а не в систему доступа к памяти.

Распределение памяти между отдельными вычислительными узлами системы имеет три значимых достоинства: повышается масштабируемость системы, увеличивается полоса пропускания памяти, уменьшается время доступа к локальной памяти.

МРА-системы не лишены и ряда недостатков. Отсутствие общей памяти заметно снижает скорость межпроцессорного обмена данными, поскольку нет общего ресурса для хранения информации. При этом каждый ПЭ может использовать ограниченный объем модуля локальной памяти.

Обычно устройства ввода/вывода, так же как и память, распределяются по вычислительным узлам, и в действительности узлы могут состоять из небольшого числа (2–8) процессоров, соединенных между собой другим способом. Хотя такая кластеризация нескольких процессоров с памятью и сетевым интерфейсом может быть достаточно полезной с точки зрения эффективности в стоимостном выражении, она не очень значима для понимания того, как такая машина работает, поэтому мы остановимся на системах с одним процессором на узел. Основное различие в архитектуре, которое следует выделить в машинах с распределенной памятью, заключается в том, как осуществляется связь и какова логическая модель памяти.

ПВС с неоднородным доступом к памяти (NUMA-архитектура)

Особенностью гибридной NUMA-архитектуры является неоднородный доступ к памяти. Такая архитектура совмещает достоинства систем с общей разделяемой памятью с относительной дешевизной систем с общей распределенной памятью.

Идея NUMA-систем состоит в физическом распределении памяти по различным частям при том, что логически вся память является общей, т.е. пользователю доступно одно единое адресное пространство. NUMA-система построена из однотипных вычислительных модулей, каждый из которых содержит небольшое количество ПЭ, кэш-память и модуль памяти. Вычислительные модули объединены высокоскоростной коммутационной средой. Доступ к локальной

памяти осуществляется быстрее, чем к удаленной. По сути, NUMA-архитектура представляет собой МРА-систему, где в качестве вычислительных узлов использованы SMP-подсистемы.

На рис. 5.2.4 приведена структура NUMA-системы, в которой вычислительный узел состоит из четырех ПЭ, объединенных общей локальной памятью. Обмен через локальную память происходит быстро, с другими вычислительными узлами обмен также осуществляется, но с помощью более сложной системы адресации.

Обобщенная масштабируемая структура ПВС

Анализ основных параллельных архитектур показывает, что они имеют типовую параллельную организацию, которая представлена на рис. 5.2.5.

Типовая ПВС включает в себя множество законченных вычислительных узлов, содержащих один или несколько ПЭ с кэш-памятью и оперативной памятью, соединенных между собой через масштабируемый коммутатор. Управление процессом формирования выходных сообщений или приемом входных сообщений осуществляется устройством управления коммутатором, или communication assist-контроллером.

Все многообразие описанных выше архитектур и способов организации ПВС реализуется с помощью устройства управления коммутатором, а точнее, посредством закладываемых в него функциональных возможностей, которые определяются типом интерфейса между процессорными элементами, системой памяти и коммутационной средой.

В случае систем с общей разделяемой памятью устройство управления интегрируется с системой памяти для того, чтобы управлять событиями в памяти, которые могут потребовать взаимодействия с другими узлами системы. Помимо этого, устройство управления должно управлять приемом сообщений и доступом к памяти.

В случае систем с общей распределенной памятью коммутация инициализируется на системном или пользовательском уровне, т.е. не требуется «наблюдаемости» системных событий в памяти. Вместо этого необходимо наличие возможности быстрой отправки сообщения и формирования ответа на входящее сообщение. Можно потребовать, чтобы под ответом подразумевалось выполнение операции сравнения

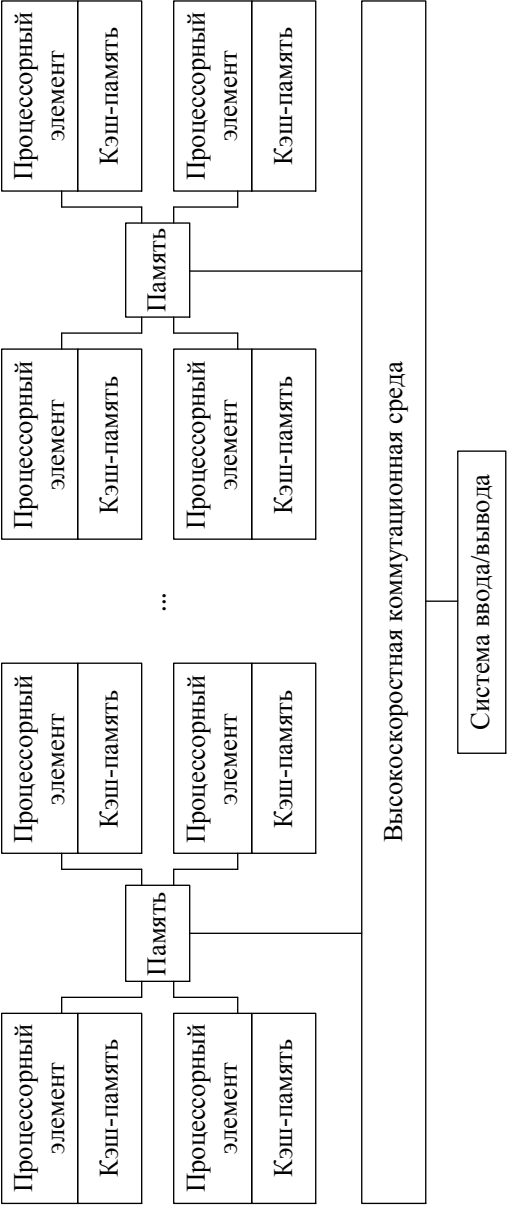


Рис. 5.2.4. Типовая структура многопроцессорной системы с неоднородным доступом к памяти

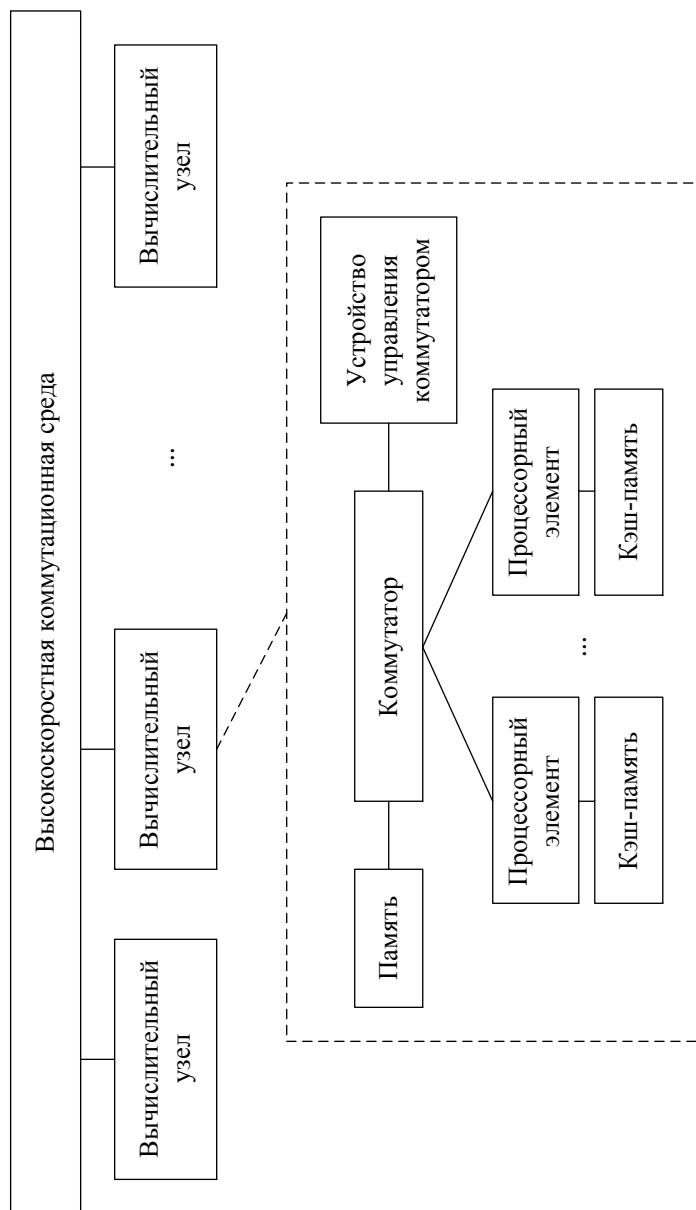


Рис. 5.2.5. Обобщенная масштабируемая структура ПВС

тегов на совпадение, чтобы буфера были локализованы, и передача данных комментировалась.

Систолические подходы делают акцент на быстрой глобальной синхронизации, которая может быть прямо поддержана сетевыми механизмами или устройством управления.

Контрольные вопросы

1. Охарактеризуйте ПВС с общей разделяемой памятью, укажите ее достоинства и недостатки.
2. Охарактеризуйте ПВС с общей распределенной памятью, укажите ее достоинства и недостатки.
3. Охарактеризуйте ПВС с неоднородным доступом к памяти, укажите ее достоинства и недостатки.
4. Поясните, каким образом обобщенная структура ПВС объединяет в себе как системы с общей разделяемой памятью, так и системы с распределенной памятью.

Литература

1. Архитектуры и топологии многопроцессорных вычислительных систем / **А.В. Богданов, В.В. Корхов, В.В. Мареев, Е.Н. Станкова.** — М.: ИНТУИТ.РУ, 2004. — 176 с.
2. Микропроцессорные системы: учеб. пособие для вузов / **Е.К. Александров, Р.И. Грушвицкий, М.С. Куприянов и др.; под ред. Д.В. Пузанкова.** — СПб.: Политехника, 2002. — 935 с.

Лекция 5.3. Матричные вычислительные системы

Экспериментальные разработки по созданию многопроцессорных вычислительных систем начались в 1960-х годах. Одной из первых таких систем стала МВС ILLIAC-IV (Illinois Automatic Computer), которая включала 8 кластеров по 64 ПЭ, работающих по единой программе, применяемой к содержимому собственной оперативной памяти каждого ПЭ.

Отличительной особенностью МВС является жестко синхронизируемая множественная обработка данных. Высокая производительность достигается с помощью большого количества простых по структуре процессоров, каждый из которых изготавливается в виде большой интегральной схемы и называется процессорным элементом ввиду ограниченного круга выполняемых операций.

Основная проблема, стоящая перед разработчиками МВС, — организация межпроцессорных связей. Устранение этой проблемы существенно упрощается для матрицы процессоров, работающих в синхронном режиме. Помимо технических причин такой организации матричных систем существует и экономическая: построить N процессоров с одним устройством управления дешевле, чем N аналогичных вычислительных машин.

Определим ключевые компоненты МВС и установим взаимосвязи между ними:

- набор процессорных элементов;
- набор банков памяти;
- коммутационная среда;
- блоки локального управления;
- блок общего управления.

Наиболее общая структура МВС содержит наборы идентичных ПЭ и банков памяти. Каждый ПЭ может непосредственно соединяться со своей секцией памяти. Связь между ПЭ и присвоенным ему банком памяти осуществляется посредством коммутационной среды, которая объединяет все ПЭ. Такая организация системы приведена на рис. 5.3.1, где коммутационная среда изображена как отдельный компонент, подключаемый к ПЭ с помощью входов и выходов. В общем



Рис. 5.3.1. Типовая структура процессорной матрицы с одинаковым числом процессоров и банков памяти

смысле при такой организации коммутирующая сеть обычно распределена между различными процессорами.

В другой часто встречающейся конфигурации используется более сложная коммутационная среда: коммутатор устанавливается между процессорами и банками памяти (рис. 5.3.2). Такая конфигурация устраняет ограничения на связь ПЭ только с одним банком памяти и позволяет объединить в матрицу различное число банков памяти и ПЭ.

Управление работой системы осуществляется устройством управления или управляющим процессором (УП), который характеризуется высоким быстродействием и имеет многомодульную оперативную память большого объема. Высокое быстродействие УП обеспечивает непрерывность потока команд, направляемых для выполнения в матрицу ПЭ. Повышенные требования к характеристикам оперативной памяти обусловлены спецификой применения матричных систем, рассчитанных на выполнение трудоемких программ, перерабатывающих наборы данных большого объема. Кроме программ, исходных наборов данных, промежуточных и выходных результатов в оперативной памяти размещаются резидентные программы операционной системы, организующей и контролирующей ход вычислительных процессов.

При работе системы УП выбирает команды из оперативной памяти, декодирует их с целью выявления команд обработки данных



Рис. 5.3.2. Типовая структура процессорной матрицы, предусматривающая различное число банков памяти и процессоров

и направляет последние в матрицу ПЭ поочередно. Команды переходов выполняются самим процессором управления с учетом состояния ПЭ на момент завершения предшествующей команды. Команды ввода/вывода передаются для выполнения в процессор ввода/вывода.

ПЭ однородны по составу и представляют собой относительно простые АЛУ, обладающие высокой скоростью обработки данных. Блоки управления ПЭ содержат средства дешифрации арифметико-логических операций, управления их реализацией, формирования признаков по результатам операций и обращения к локальной сверхоперативной памяти. Все ПЭ, кроме блокируемых, в один и тот же момент времени выполняют одну и ту же операцию, жестко задаваемую УП, над различными данными, размещаемыми в локальной сверхоперативной памяти. ПЭ может пропустить команду, т. е. блокировать ее, если на его входе нет данных, но выполнять команду, отличную от заданной для всех, не может. Асинхронное выполнение различных команд в различных ПЭ в одном такте потребовало бы существенно более сложной организации УП и усложнения структуры системы в целом.

Несмотря на то, что идея регулярной матрицы процессоров, выполняющих одну и ту же операцию над различными данными, является основой канонической МВС, на практике часто требуется некоторая форма автономного управления отдельным ПЭ для реализации независимых разветвлений данных в пределах потока команд.

В простейшем случае такой подход реализуется с помощью передачи одного одноразрядного флага в каждый ПЭ для маскирования выполнения команды. Таким образом, процесс управления представляет собой трансляцию команд в матрицу, выполняемую устройством управления, а затем каждый элементарный процессор делает выбор — выполнять или не выполнять команду в зависимости от того, установлен или не установлен флаг. Действия процессора обычно сводятся к тому, разрешить ли на основе состояния данного флага циклы записи в память или регистры.

Известно несколько альтернативных вариантов организации системы управления матрицей процессоров, предусматривающих декодирование некоторых команд в ПЭ. В зависимости от объема управления, возлагаемого на элементарный процессор, такие системы можно разбить на два типа: системы с полной процессорной независимостью и системы с микропрограммным управлением.

В *системах с полной процессорной независимостью* каждый процессор матрицы имеет локальный блок управления, который независимо от других процессоров и устройства управления декодирует собственную программу. Однако при любой управляющей структуре должна существовать некоторая форма глобального управления, которая инициирует, останавливает и синхронизирует элементарные процессоры. Заметим, что согласно используемой нами терминологии такие системы следует рассматривать как матрицу компьютеров.

Достоинство систем с полной независимостью процессоров заключается в том, что каждый процессор следит за своей магистралью в соответствии с индивидуальной программой выполняет операции над своими данными. Синхронизация требуется только при или после обновления глобальной базы данных и выполняется с помощью флага, устанавливаемого для каждого элементарного процессора или компьютера матрицы.

Простейшим примером *МВС с микропрограммным управлением* является двухуровневая управляющая структура. Первый уровень — общий, на нем производятся декодирование команд и транслирование их в матрицу, на этом уровне матрица синхронизируется. На втором (более низком) уровне элементарный процессор интерпретирует команду и генерирует микрокоманды. Важно отметить, что задаваемая команда может быть интерпретирована в микропроцессоре различными способами. Таким образом, общее управление является глобаль-

ным, а его интерпретация — локальной. Поскольку последовательности микрокоманд могут быть различной длины, синхронизация достигается с помощью системы флагов или, что предпочтительнее, посредством опережающего просмотра последовательностей более длинных микрокоманд в любой заданной команде.

Одной из характерных особенностей архитектуры МВС является организация памяти, определяющая способы ее адресации и подключения к элементарным процессорам.

В случае структуры, представленной на рис. 5.3.1, где каждый процессор подключается к своему блоку памяти, остается определить, как происходит адресация памяти. Команды, выдаваемые устройством управления, обычно содержат одинаковый адрес для всех процессоров. Вопрос состоит в том, разрешить ли индивидуальному процессору обрабатывать этот адрес с помощью своего индексного регистра.

В случае структуры, приведенной на рис. 5.3.2, используется более сложная коммутационная среда, обеспечивающая связь каждого ПЭ с любым блоком памяти. При этом адресация к каждому банку памяти осуществляется независимо.

Конфликты на уровне банков памяти — один из основных недостатков любой параллельной организации памяти. В качестве примера на рис. 5.3.3, а представлена матрица 4×4 , хранящаяся в четырех банках памяти. Можно видеть, что обращение к строкам матрицы А можно осуществлять без конфликтов, но все данные столбцов матрицы А располагаются в одном и том же банке памяти, поэтому выборку этих данных нельзя выполнять параллельно.

При наличии сети коммутации, которая может подключать все банки памяти к одному процессору, желательно иметь структуру памяти или данных, при которой обращение к строкам, столбцам и другим основным подструктурам в виде матриц можно осуществлять без конфликтов. Одним из основных способов достижения этой цели является применение асимметричных схем хранения. На рис. 5.3.3, б показан пример такой схемы для матрицы 4×4 . Она обеспечивает бесконфликтный доступ к строкам и столбцам, хотя для диагоналей матрицы А конфликты сохраняются.

Фирма Burroughs в проекте BSP предложила структуру памяти на основе асимметричных схем. Уникальной особенностью системы BSP является то, что каждое АЛУ обеспечивается операндами за один цикл обращения к памяти. Это означает, что расстояние в па-

а						б					
Адрес	3	$A_{4,1}$	$A_{4,2}$	$A_{4,3}$	$A_{4,4}$	Адрес	3	$A_{4,2}$	$A_{4,3}$	$A_{4,4}$	$A_{4,1}$
	2	$A_{3,1}$	$A_{3,2}$	$A_{3,3}$	$A_{3,4}$		2	$A_{3,3}$	$A_{3,4}$	$A_{3,1}$	$A_{3,2}$
	1	$A_{2,1}$	$A_{2,2}$	$A_{2,3}$	$A_{2,4}$		1	$A_{2,4}$	$A_{2,1}$	$A_{2,2}$	$A_{2,3}$
	0	$A_{1,1}$	$A_{1,2}$	$A_{1,3}$	$A_{1,4}$		0	$A_{1,1}$	$A_{1,2}$	$A_{1,3}$	$A_{1,4}$
		0	1	2	3			0	1	2	3
		Адрес банка						Адрес банка			

Рис. 5.3.3. Примеры использования схем хранения элементов матрицы, обеспечивающих бесконфликтный доступ к строкам матрицы A (а); к строкам и столбцам матрицы A (б)

мяти между элементами вектора необязательно должно быть единичным. Поэтому циклы могут выполняться с неединичным шагом, а программы могут получить доступ к строкам, столбцам или диагоналям матриц без возникновения конфликтов.

Следует отметить, что до появления асимметричной схемы BSP память суперкомпьютеров, основанная на относительно дешевых компонентах, не имела такого свойства. Разработчики либо использовали дешевые компоненты и вводили существенные ограничения на доступ к отдельным ячейкам памяти, либо применяли дорогую быстродействующую компонентную базу и пытались сократить количество конфликтов при обращениях к памяти повышением пропускной способности.

В основе предложенного способа организации памяти лежат: простое число портов памяти, полный матричный коммутатор между портами памяти АЛУ и специальные формирователи индекса ячеек памяти и тегов портов коммутатора.

Формирователи индексов и тегов вычисляют реальные адреса для конкретного способа адресации элементов массива, который основан на принципе, присущем последовательным системам: каждый последующий адрес ссылается на следующее слово в памяти. Таким

образом была обеспечена совместимость с последовательными языками программирования.

Индекс определяется выражением

$$i = [A/N_{\text{АЛУ}}],$$

где A — адрес, $N_{\text{АЛУ}}$ — количество АЛУ в системе. Таким образом, индекс остается неизменным для цикла с числом итераций, равным $N_{\text{АЛУ}}$, после чего он увеличивается на единицу. Тег, или номер банка памяти, в котором хранится значение, связанное с адресом A , определяется выражением

$$\mu = A \bmod N_{\text{Б}},$$

где $N_{\text{Б}}$ — количество банков памяти в системе (простое число, большее либо равное $N_{\text{АЛУ}}$). При этом значения тегов формируются таким образом, что к каждому АЛУ подключается свой банк памяти, обеспечивая независимые циклы чтения или записи для каждого АЛУ.

На рис. 5.3.4 представлен пример размещения в памяти матрицы 4×5 (четыре строки, пять столбцов) ПВС, содержащей шесть АЛУ

		μ						
		0	1	2	3	4	5	6
i	0	0 $A_{1,1}$	1 $A_{2,1}$	2 $A_{3,1}$	3 $A_{4,1}$	4 $A_{1,2}$	5 $A_{2,2}$	
	1	7 $A_{4,2}$	8 $A_{1,3}$	9 $A_{2,3}$	10 $A_{3,3}$	11 $A_{4,3}$		6 $A_{3,2}$
	2	14 $A_{3,4}$	15 $A_{4,4}$	16 $A_{1,5}$	17 $A_{2,5}$		12 $A_{1,4}$	13 $A_{2,4}$
	3	21	22	23		18 $A_{3,5}$	19 $A_{4,5}$	20
	4	28	29		24	25	26	27
	5	35		30	31	32	33	34

Рис. 5.3.4. Пример использования схемы хранения с числом банков памяти, задаваемым простым числом

и семь банков памяти. Расчеты индексов и тегов для заполнения памяти и обращения к элементам второго столбца матрицы приведены в табл. 5.3.1 и 5.3.2 соответственно. При обращении к строкам матрицы шаг приращения адреса равен числу строк, при обращении к столбцам — единице, а при обращении к диагоналям — числу строк плюс один. Неиспользованные ячейки памяти являются следствием того, что число АЛУ в системе на единицу меньше, чем число банков памяти. Заметим, что конфликт имеет место, если адреса разделены блоками, размер которых кратен числу банков памяти.

Таблица 5.3.1

Пример формирования индексов и тегов

Элемент массива	A	i	μ	Элемент массива	A	i	μ
$A_{1,1}$	0	0	0	$A_{3,3}$	10	1	3
$A_{2,1}$	1	0	1	$A_{4,3}$	11	1	4
$A_{3,1}$	2	0	2	$A_{1,4}$	12	2	5
$A_{4,1}$	3	0	3	$A_{2,4}$	13	2	6
$A_{1,2}$	4	0	4	$A_{3,4}$	14	2	0
$A_{2,2}$	5	0	5	$A_{4,4}$	15	2	1
$A_{3,2}$	6	1	6	$A_{1,5}$	16	2	2
$A_{4,2}$	7	1	0	$A_{2,5}$	17	2	3
$A_{1,3}$	8	1	1	$A_{3,5}$	18	3	4
$A_{2,3}$	9	1	2	$A_{4,5}$	19	3	5

Таблица 5.3.2

Пример вычисления индексов и тегов для доступа ко второму столбцу матрицы (начальный адрес — 1, шаг — 4)

Элемент массива	A	i	μ
$A_{2,1}$	1	$[1/6] = 0$	$1 \bmod 7 = 1$
$A_{2,2}$	5	$[5/6] = 0$	$5 \bmod 7 = 5$
$A_{2,3}$	9	$[9/6] = 1$	$9 \bmod 7 = 2$
$A_{2,4}$	13	$[13/6] = 2$	$13 \bmod 7 = 6$
$A_{2,5}$	17	$[17/6] = 2$	$17 \bmod 7 = 3$

Коммутационная среда является основой структуры МВС и продуманное ее построение важно для эффективного использования системы в целом.

Для обмена промежуточными результатами ПЭ могут быть связаны между собой как с применением межпроцессорного коммутатора, так и без него.

В системе без коммутатора наиболее распространенным является размещение ПЭ в узлах квадратной решетки, размерность которой $N = n \times n$. Пример матрицы из 16 ПЭ представлен на рис. 5.3.5. Каждый ПЭ связан с четырьмя другими по правилу $(i \pm 1) \bmod 16$ или $(i \pm 4) \bmod 16$. В схемах соединения этого вида передача данных от одного ПЭ к другому требует не более $(\log_2 N - 1)$ шагов. Возможны и другие способы соединения ПЭ, основанные на использовании решеток в виде шестиугольников, восьмиугольников и многомерных покрытий.

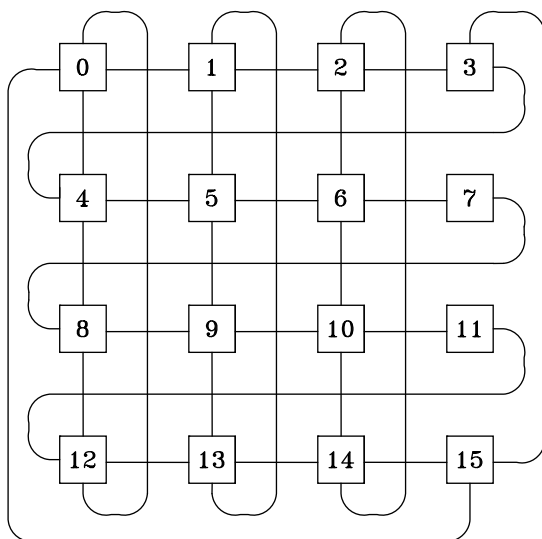


Рис. 5.3.5. Матрица процессорных элементов

Традиционным примером МВС является ILLIAC-IV. Проект этой системы рассчитан на 256 ПЭ, разбитых на четыре матрицы размером

8 × 8 каждая. Система должна была состоять из четырех квадрантов, каждый из которых включал 64 ПЭ и 64 модуля памяти, объединенных коммутатором на базе сети типа гиперкуба. Все ПЭ квадранта обрабатывают векторную инструкцию, которую им направляет процессор команд, причем каждый выполняет одну элементарную операцию вектора, данные для которой сохраняются в связанном с этим ПЭ модуле памяти. Таким образом, один квадрант ILLIAC-IV способен одновременно обрабатывать 64 элемента вектора, а вся система из четырех квадрантов — 256 элементов.

Результаты эксплуатации системы получились неоднозначными: с одной стороны, использование суперкомпьютера позволило решить ряд сложнейших задач аэродинамики, с которыми не могли справиться другие системы; с другой стороны, на практике разработчики ILLIAC-IV ограничились лишь одним квадрантом. Причинами явились не столько технические сложности в наращивании числа ПЭ системы, сколько проблемы, связанные с программированием обмена данными между ПЭ через коммутатор модулей памяти. Все попытки решить эту задачу с помощью системного программного обеспечения потерпели неудачу, в результате каждое приложение потребовало ручного программирования передач коммутатора, что и породило неудовлетворительные отзывы пользователей.

Несмотря на то, что МВС не нашли широкого применения, разработка суперкомпьютеров с матричной архитектурой имела огромное значение для дальнейшего развития параллельных вычислительных систем в целом. Во-первых, удалось доказать возможность практической реализации быстродействующей параллельной обработки информации. Во-вторых, на волне интереса к матричным структурам были сформулированы теоретические основы построения коммутационных сред, объединяющих множество ПЭ. В-третьих, в прикладной математике сформировалось самостоятельное направление по параллельным вычислениям.

Контрольные вопросы

1. К какому классу по Флинну относятся матричные вычислительные системы?
2. Перечислите основные компоненты МВС.

3. Почему первые МВС имели синхронную матрицу процессорных элементов?

4. Каковы особенности локального и глобального управлений в МВС?

5. Поясните принцип работы асимметричной схемы хранения данных в памяти.

6. Поясните принцип работы асимметричной схемы хранения с числом банков памяти, задаваемым простым числом.

Литература

1. Архитектуры и топологии многопроцессорных вычислительных систем / ***А.В. Богданов, В.В. Корхов, В.В. Мареев, Е.Н. Станкова.*** — М.: ИНТУИТ.РУ, 2004. — 176 с.

2. Микропроцессорные системы: учеб. пособие для вузов / ***Е.К. Александров, Р.И. Грушвицкий, М.С. Куприянов и др.; под ред. Д.В. Пузанкова.*** — СПб.: Политехника, 2002. — 935 с.

Лекция 5.4. Векторно-конвейерные вычислительные системы

Производительность большинства суперкомпьютеров достигается благодаря применению принципов векторно-конвейерной обработки данных. Следует отметить, что в настоящее время эти принципы широко используются и во встраиваемых процессорах, например, сигнальный процессор TigerSharc фирмы Analog Devices поддерживает векторные команды, существенно повышающие его быстродействие при выполнении одной и той же операции над векторами данных. Действительно, при решении научно-технических задач часто возникает необходимость вычислять значения одной и той же функции для группы данных, расположенных в ячейках памяти или регистрах с упорядоченными адресами. Такие наборы данных и называют *векторами*.

В целях повышения производительности вычислительной системы в состав ее процессора включают средства векторной обработки данных или в состав системы вводят специализированный векторный сопроцессор. Как правило, в системе, поддерживающей векторные команды, выполняются также и скалярные команды, т. е. команды над одиночными данными. Повышение быстродействия векторной системы происходит за счет уменьшения потерь времени на организацию вычислительного цикла. Обычно в целях достижения повышенного быстродействия выполнение самих векторных операций конвейеризируется, причем может быть несколько арифметических конвейеров, а отдельные устройства конвейерных линий могут, в свою очередь, содержать конвейеры для выполнения возложенных на них подфункций.

Векторные команды имеют ряд особенностей. Их код операции не только задает подлежащую выполнению операцию, но и определяет нужную конфигурацию активных частей конвейера для данной операции. Команда указывает начало вектора в памяти, длину вектора, размер и тип элементов вектора, определяет местоположение вектора-результата.

Основной потенциал по распараллеливанию задач на векторно-конвейерной системе заключается в распараллеливании вычислитель-

ных циклов. Известный специалист в области программирования Д. Кнут показал, что циклы DO занимают менее 4% кода на языке FORTRAN, но требуют более половины счетного времени задачи. Аналогичные рассуждения справедливы и для других языков программирования высокого уровня.

Простейшим примером вычислительного цикла может служить операция сложения двух одноименных элементов векторов A и B:

```
for (i = 0; i < N; i++)  
    C[i] = A[i] + B[i];
```

По классификации М. Флинна большинство компьютеров — от ПК до высокопроизводительных рабочих станций — относятся к SISD-системам, в которых программа выполняется последовательно, причем каждая команда формирует только один числовой результат. Очевидно, что для выполнения цикла, приведенного в качестве примера, компьютер класса SISD должен произвести как минимум N операций сложения элементов векторов A и B, не считая 2N команд приращения индекса и условного перехода.

Идея векторной обработки циклов такого рода заключается в том, что в систему команд компьютера вводится векторная операция сложения $\langle A + B \rangle$, которая задает сложение всех элементов векторов-операндов. При этом реализуются сразу две возможности ускорения вычислений:

1) сокращается число выполняемых процессором команд объектного кода, поскольку отпадает необходимость в пересчете индексов и организации условного перехода;

2) все операции сложения элементов векторов-операндов могут быть выполнены одновременно в силу параллелизма обработки.

Действительно, в нашем примере результат любой i-й операции цикла не влияет на ход остальных $(N - 1)$ операций. Если представить себе систему, способную выполнить все N элементарных операций цикла одновременно, то выполнение векторной команды $\langle A + B \rangle$ потребует в N раз меньше времени, чем последовательная обработка всех операций $A[i] + B[i]$ исходного цикла в компьютере класса SISD.

Таким образом, векторная команда, наряду с обеспечением содержательной операции над элементами векторов, берет на себя управление вычислительным циклом.

Важно отметить еще одну особенность векторной обработки, связанную с количеством элементарных операций цикла: чем больше параллельных операций входит в векторизуемый цикл, тем ощутимее выигрыш в скорости выполнения вычислений, так как сокращается доля непроизводительных временных затрат на выборку, дешифрацию и запуск на выполнение векторной команды. Поэтому при программировании векторных вычислений предметом особого внимания должно быть увеличение длины векторов, т. е. числа элементарных операций, составляющих векторную команду.

Характерная особенность векторной архитектуры заключается в том, что одна и та же операция одновременно формирует множество результатов. Это позволило Флинну выделить компьютеры с векторной архитектурой в самостоятельный класс SIMD.

Отличие векторно-конвейерных и матричных вычислительных систем состоит в том, что вместо множества процессорных элементов, выполняющих одну и ту же команду над элементами вектора, применяется единственный конвейер операций. При этом существенно, что векторно-конвейерные системы значительно дешевле матричных.

Еще одной весомой особенностью векторно-конвейерной архитектуры является то, что конвейер операций имеет всего один вход, по которому поступают операнды, и один выход результата, тогда как в матричных системах существует множество входов по данным в процессорные элементы и множество выходов из них. Другими словами, в компьютерах с конвейерной обработкой данные всех параллельно выполняемых операций записываются в единую память, поэтому отпадает необходимость в коммутаторе процессорных элементов, реализация которого сложна и стала одной из серьезных проблем при проектировании матричных систем.

Организация памяти векторно-конвейерных систем

Быстродействие обработки данных в векторно-конвейерных вычислительных системах обычно выше быстродействия оперативной памяти, поэтому для обеспечения непрерывности конвейерной обработки оперативная память строится по многомодульному принципу с расслоением обращений. При модульном построении памяти операции чтения/записи выполняются независимо в каждом из модулей, что позволяет совместить их работу во времени. Расслоение обра-

щений к памяти обеспечивает непрерывность совмещения операций чтения/записи в модулях и заключается в закреплении соседних адресов ячеек в модулях с соседними номерами, в результате чего реализуется совмещенная работа модулей памяти и увеличивается число обслуживаемых памятью обращений в единицу времени. На рис. 5.4.1 представлен пример схемы памяти с расслоением обращений по четырем модулям.

Модуль 0	Модуль 1	Модуль 2	Модуль 3
0	1	2	3
4	5	6	7
...	...	...	...
2^n-4	2^n-3	2^n-2	2^n-1

Рис. 5.4.1. Схема памяти с расслоением обращений

Число модулей определяет коэффициент расслоения памяти. Чем выше этот коэффициент, тем выше производительность памяти. Вместе с тем производительность существенно зависит от порядка адресации команд и операндов, определяемого структурой программ. Наибольшая производительность памяти достигается при выполнении программ линейной структуры, в которых обращение к командам осуществляется в естественном порядке. В этом случае время между моментами считывания команд из оперативной памяти уменьшается в k раз, где k — коэффициент расслоения.

Информация, считываемая из оперативной памяти, на время обработки направляется в буферы команд и операндов. Каждый из буферов представляет собой цепочку регистров, связанных с операционными устройствами и между собой. При продвижении информации в буфере команд осуществляется поэтапная обработка команд. При этом команды частично декодируются с целью определения типа операции, места ее выполнения, адресов операндов. На основе данных декодирования устройство управления вырабатывает запросы к оперативной памяти для чтения и записи операндов, а также управляет работой всего конвейера, определяя источники предшественников приемников данных в процессе выполнения операций.

Структура векторно-конвейерной системы на примере CRAY-1

Считается, что основоположником суперкомпьютеров общего назначения, построенных на основе векторно-конвейерной архитектуры, является Сеймур Крэй (Seymour Cray). При его участии был создан ряд передовых систем в компании CDC, например, в 1964 году был разработан суперкомпьютер CDC 6600, обладающий производительностью 3 млн оп./с и являющийся первым суперкомпьютером с VLIW-архитектурой.

В 1972 году Крэй основал компанию Cray Research, в рамках которой создал самые быстрые суперкомпьютеры общего назначения. В 1974 году было объявлено о выпуске CRAY-1 — векторно-конвейерной суперЭВМ, ставшей эпохальным событием в мире вычислительной техники. По меркам того времени эта машина считалась малогабаритной, ее высота немного превосходила средний человеческий рост, а занимаемая площадь чуть больше 2,5 кв. м. Производительность системы составила 160 MFLOPS, емкость оперативной памяти — 64 МБ.

На рис. 5.4.2 представлена упрощенная структура конвейерно-векторной системы CRAY-1. Поскольку реальные программы, как правило, требуют выполнения как векторных, так и скалярных операций, система имеет командные и аппаратные средства для реализации операций обоих типов. В системе реализованы конвейер команд и многочисленные конвейеры для различных арифметических и логических операций, используемых при скалярной и векторной обработке 64-разрядных слов с плавающей точкой. Числа с плавающей точкой имеют 15-разрядный порядок и 49-разрядную мантиссу. Числа с фиксированной точкой могут быть как 64-, так и 24-разрядными. Формат представления адресов — 22 разряда.

Операции выполняются двенадцатью конвейеризированными операционными устройствами, которые могут работать параллельно во времени. При этом возможно образование последовательных соединений конвейеров, в том числе так называемое *зацепление векторных операций*. Конвейеры имеют относительно малое число ступеней (на рисунке указано в скобках) с тем, чтобы уменьшить потери производительности, связанные со временем разогрева конвейера.

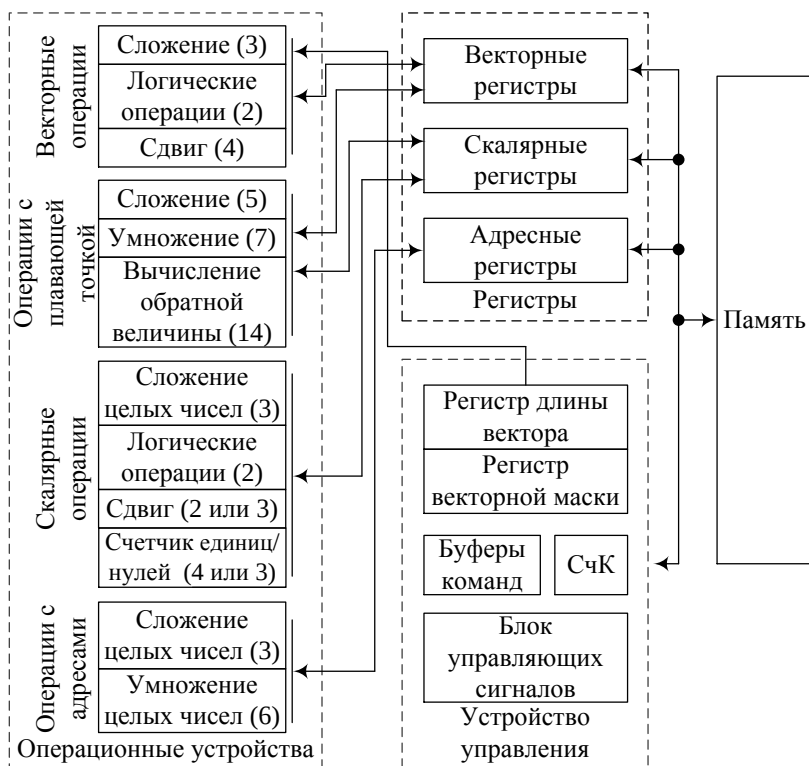


Рис. 5.4.2. Упрощенная структура конвейерно-векторной системы CRAY-1

Для поддержки конвейерно-векторной обработки требуется быстрая регистровая память, освобождающая от необходимости обращения к памяти при выполнении векторных операций и обеспечивающая высокие темпы загрузки конвейеров. Система CRAY-1 содержит:

- восемь векторных *V*-регистров, каждый из которых может хранить шестьдесят четыре 64-разрядных слова;
- восемь 24-разрядных адресных регистров;
- восемь скалярных 64-разрядных *S*-регистров.

Управляющее устройство, помимо традиционных блоков, содержит четыре буфера команд (на 64 командных слова каждый) и специальные регистры: регистр длины вектора и регистр векторной маски.

Регистр длины вектора фиксирует длину обрабатываемого вектора $S \leq 64$. Содержащий 64 разряда регистр векторной маски значением своего i -го разряда определяет, из каких участвующих в операции регистров i -й элемент выдается в регистр результата.

Формат команды — 16 или 32 разряда. 16-разрядные команды основных, в том числе векторных, операций имеют 7-разрядное поле кода операции и три 3-разрядных поля для номеров регистра операнда и результата. В 32-разрядной команде под адрес основной памяти или непосредственный операнд отведено 22 разряда. В памяти системы, имеющей емкость от 1 до 4 мегаслов (каждое слово состоит из 64 разрядов данных и 9 контрольных разрядов), применено 16-кратное чередование адресов.

Несмотря на перечисленные характеристики системы CRAY-1, достаточно неожиданным стал тот факт, что с большинством задач данная система справлялась быстрее, чем значительно превосходящая ее по габаритам и пиковой производительности система CDC CYBER-205. Этот эффект объясняется законом Амдала (G. Amdahl), который известный архитектор системы IBM/360 сформулировал в 1967 году: производительность вычислительной системы определяется самым медленным ее компонентом.

Любая программа, выполняемая векторно-конвейерной системой, состоит из двух взаимосвязанных частей — векторных команд, сгенерированных компилятором при векторизации исходной программы, и скалярных операций, которые компилятор не сумел перевести в векторную форму. На практике полностью параллельных задач не существует хотя бы потому, что скалярные операции, по определению, не являются параллельными и выполняются медленнее векторных. Поэтому любая векторно-конвейерная система показывает на задачах с высокой степенью векторизации существенно большую производительность, чем на слабо векторизированных задачах. Например, CRAY-1 на тесте LINPACK при малой длине векторов ($n \leq 100$) имеет производительность 27 MFLOPS, а при большой длине векторов ($n \leq 1000$) — 110 MFLOPS. Но CRAY-1 при цикле 12,5 нс обладает большим быстродействием скалярной обработки по сравнению с компьютером CYBER-205, у которого цикл равен 20 нс.

В целом следует выделить три основные причины существенного преимущества системы CRAY-1 перед аналогами:

- 1) высокая скорость выполнения скалярных операций;

2) использование принципа «регистр-регистр» при выполнении любых операций;

3) использование специализированного компилятора, векторизирующего программы, написанные на последовательных языках программирования.

В CRAY-1 впервые при выполнении всех операций был использован принцип «регистр-регистр». Все операции обработки данных, которые выполняет центральный процессор этой машины, выбирают операнды и записывают результаты вычислений, используя не оперативную память, как это было сделано в CYBER-205 и более ранних суперкомпьютерах, а специально предназначенные для этой цели программно-адресуемые регистры. Для реализации этой архитектуры в CRAY-1 было введено принципиальное новшество — векторные регистры, которые адресуется командами центрального процессора подобно обычным регистрам данных, но могут хранить до шестидесяти четырех 64-разрядных элементов каждый. Естественно, что по сравнению с CYBER-205 каждая отдельная векторная команда в CRAY-1 выполнялась быстрее, поскольку операции типа «регистр-регистр» отличаются от операций типа «память-память» меньшими временными затратами на чтение операндов и запись результата.

Преимущества CRAY-1 перед матричными суперкомпьютерами определяются не только лучшей сбалансированностью показателей производительности, но и простотой и удобством эксплуатации, а также высоким качеством системного программного обеспечения. При разработке системы CRAY-1 было учтено, что большинство существующих пакетов прикладных программ написано на языках последовательного типа, чаще всего на языке FORTRAN, поэтому в состав программного обеспечения был включен интеллектуальный FORTRAN-компилятор CTF, способный обнаруживать параллелизм в программах последовательного действия и преобразовывать их в векторизованный код. Таким образом, все имеющееся у пользователя программное обеспечение с незначительными доработками могло быть перенесено на базу суперЭВМ без привлечения специализированных программистов.

Основным компонентом разработанного компилятора стал блок векторизатора, преобразующий последовательные операторы программы пользователя в объектный код, выполняемый векторными вычислительными средствами. Векторизатор анализирует циклы программы

и подвергает их локальной оптимизации с целью максимального перевода объектного кода в векторную форму. Проблема заключается в том, что не все циклы допускают векторизацию, иногда циклы имеют слишком малое или вообще неизвестное число итераций, и их непосредственная векторизация может оказаться нецелесообразной. Поэтому векторизирующий компилятор использует такие методы локальной оптимизации, как удлинение самого внутреннего цикла DO, введение временных переменных, линейаризованных многомерных индексов, переупорядочивание операторов, сегментация цикла, упрощение индексов, обратная прокрутка, распределение цикла, преобразование условий, увеличение плотности вектора и т. д.

Кроме векторизации циклических конструкций, в компиляторе SFT было введено еще одно принципиальное новшество — автоматический поиск в исходной программе и построение многоконвейерных цепочек. Суть его заключается в следующем: если в программе встречаются две связанные векторные операции (т.е. результат первой служит операндом второй), то в отличие от случая связанных скалярных операций, когда выполнение второй операции начинается только после завершения первой, обе векторные команды могут обрабатываться практически параллельно, что удваивает производительность системы.

Например, программист создал следующий цикл:

```
for (i = 0; i < N; i++)
{
    C[i] = A[i] + B[i];
    E[i] = C[i] * D[i];
}
```

После векторизации компилятор организует из этого цикла две векторные команды:

$$C = A + B,$$

$$E = C \times D.$$

Формально выполнение второй векторной операции может начаться только после окончательного вычисления вектора-результата первой операции C, однако структура центрального процессора CRAY-1 позволяет запустить связанную операцию, как только сфор-

мируется первый элемент вектора C . В результате суммарное время выполнения исходного цикла ненамного превосходит этот показатель для одной векторной команды (рис. 5.4.3).

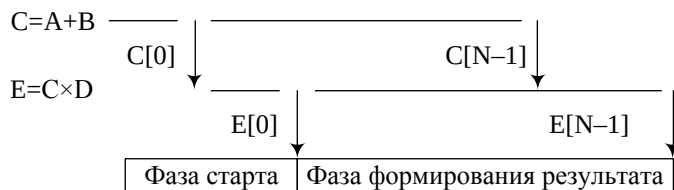


Рис. 5.4.3. Зацепление векторных операций

С появлением CRAY-1 впервые удалось создать цепочки из разных операций без привлечения специальных команд управления. Этот механизм оказался настолько эффективным, что при высокой степени векторизации практически 100% векторных команд выполнялись в режиме зацепления. Более того, указанный режим стал обязательным для всех последующих векторных суперЭВМ, а японские специалисты фирмы Hitachi в моделях серии S-810 довели число параллельно выполняемых векторных команд до шести, что, впрочем, не стало столь же заметным явлением в мире суперЭВМ, как, например, цепочка из двух команд в CRAY-1.

В целом система CRAY-1 является примером удачного сочетания простоты и эффективности технических решений в области аппаратного и программного обеспечения суперкомпьютера с простотой и удобством использования системы в первую очередь за счет программирования в привычной для большинства разработчиков среде на традиционных последовательных языках типа FORTRAN.

Контрольные вопросы

1. В чем заключается основной потенциал по распараллеливанию программ при выполнении их в векторно-конвейерной системе?
2. Поясните принцип работы памяти с расслоением.
3. Поясните принцип выполнения векторных операций с зацеплением.

4. Перечислите основные преимущества системы CRAY-1 перед аналогами, позволившие данной системе достичь более высокого быстродействия.

Литература

1. Архитектуры и топологии многопроцессорных вычислительных систем / ***А.В. Богданов, В.В. Корхов, В.В. Мареев, Е.Н. Станкова.*** — М.: ИНТУИТ.РУ, 2004. — 176 с.

2. Микропроцессорные системы: учеб. пособие для вузов / ***Е.К. Александров, Р.И. Грушвицкий, М.С. Куприянов и др.; под ред. Д.В. Пузанкова.*** — СПб.: Политехника, 2002. — 935 с.

Лекция 5.5. Кластерные вычислительные системы

Кластерные вычислительные системы стали продолжением развития идей, заложенных в архитектуре МРА-систем. Если в МРА-системе в качестве законченного вычислительного узла выступает процессорный модуль, то в кластерных системах в качестве таких вычислительных узлов используют серийно выпускаемые компьютеры. Развитие коммуникационных технологий, а именно, появление высокоскоростного сетевого оборудования и специальных программных библиотек, например, MPI (Message Passing Interface), реализующих механизм передачи сообщений по стандартным сетевым протоколам, сделали кластерные технологии общедоступными. В настоящее время создается множество небольших кластерных систем посредством объединения вычислительных мощностей компьютеров лаборатории или учебного класса.

Привлекательной чертой кластерных технологий является то, что для достижения необходимой производительности они позволяют строить гетерогенные системы, т. е. объединять в единые вычислительные системы компьютеры самого разного типа, начиная от персональных компьютеров и заканчивая мощными суперкомпьютерами. Широкое распространение кластерные технологии получили как средство создания систем суперкомпьютерного класса из составных частей массового производства, что значительно удешевляет стоимость вычислительной системы. В частности, одним из первых в 1998 году был реализован проект The COst effective COmputing Array (COCOА), в котором на базе 25 двухпроцессорных персональных компьютеров общей стоимостью порядка 100000 долларов была создана система с производительностью, эквивалентной 48-процессорному Cray T3D стоимостью несколько миллионов долларов.

Лайал Лонг (Lyle Long), профессор аэрокосмической инженерии в университете штата Пенсильвания (Penn State University), считает, что относительно дешевые кластерные вычислительные системы вполне могут служить альтернативой дорогим суперкомпьютерам в научных организациях. Под его руководством в университете был построен кластер СОСОА. В рамках данного проекта объединены 25 ра-

бочих станций от DELL, каждая из которых включает два процессора Pentium II/400 МГц, 512 МБ оперативной памяти, 4-гигабайтный жесткий диск SCSI и сетевой адаптер Fast Ethernet. Для связи узлов используется 24-портовый коммутатор Baynetworks 450T с одним модулем расширения. Установленное программное обеспечение включает операционную систему RedHat Linux, компиляторы Fortran 90 и HPF от Portland Group, свободно распространяемую реализацию MPI — Message Passing Interface Chameleon (MPICH) и систему поддержки очередей DQS.

В работе, представленной на 38-й конференции Aerospace Science Meeting and Exhibit, Лонг описывает параллельную версию расчетной программы с автоматическим распределением вычислительной нагрузки, используемой для предсказания уровня шума от вертолетов в различных точках. Для сравнения данная расчетная программа была запущена на трех различных 48-процессорных компьютерах для расчета шума в 512 точках. На системе Cray T3E расчет занял 177 секунд, на системе SGI Origin2000 — 95 секунд, а на кластере COCOA — 127 секунд. Таким образом, кластеры являются очень эффективной вычислительной платформой для задач такого класса.

Преимущество кластерных систем перед суперкомпьютерами состоит еще и в том, что их владельцам не приходится делить процессорное время с другими пользователями, как в крупных суперкомпьютерных центрах. В частности, COCOA обеспечивает более 400 тысяч часов процессорного времени в год, тогда как в суперкомпьютерных центрах бывает трудно получить 50 тысяч часов.

Конечно, о полной эквивалентности этих систем говорить не приходится. Как известно, производительность систем с распределенной памятью очень сильно зависит от производительности коммутационной среды, которую можно охарактеризовать двумя параметрами: латентностью — временем задержки при посылке сообщения, и пропускной способностью — скоростью передачи информации. Например, для компьютера Cray T3D эти параметры составляют соответственно 1 мкс и 480 Мб/с, а для кластера, в котором в качестве коммутационной среды использована сеть Fast Ethernet, — 100 мкс и 10 Мб/с. Это отчасти объясняет очень высокую стоимость суперкомпьютеров. При таких параметрах, как у рассматриваемого кластера, найдется не так много задач, которые могут эффективно решаться на достаточно большом числе процессоров.

На основе вышеизложенного дадим определение: *кластер* — это связанный набор полноценных компьютеров, используемый в качестве единого вычислительного ресурса. В качестве узлов кластеров могут использоваться как одинаковые (гомогенные кластеры), так и разные (гетерогенные кластеры) вычислительные машины. По своей архитектуре кластерная вычислительная система является слабосвязанной. Для создания кластеров обычно используются либо простые однопроцессорные персональные компьютеры, либо двух- или четырехпроцессорные SMP-серверы. При этом не накладывается никаких ограничений на состав и архитектуру узлов. Каждый из узлов может функционировать под управлением своей собственной операционной системы. Чаще всего используются стандартные операционные системы Linux, FreeBSD, Solaris, Tru64 Unix, Windows NT.

В литературе отмечают четыре преимущества, достигаемые с помощью кластеризации вычислительной системы:

- абсолютная масштабируемость;
- наращиваемая масштабируемость;
- высокий коэффициент готовности;
- соотношение цена/производительность.

Поясним каждую из перечисленных выше особенностей кластерной вычислительной системы.

Свойство *абсолютной масштабируемости* означает, что возможно создание больших кластеров, превосходящих по вычислительной мощности даже самые производительные одиночные вычислительные машины. Кластер может содержать десятки узлов, каждый из которых представляет собой мультипроцессор.

Свойство *наращиваемой масштабируемости* означает, что кластер можно наращивать, добавляя новые узлы небольшими порциями. Таким образом, пользователь может начать с малой системы, расширяя ее по мере необходимости.

Поскольку каждый узел кластера — самостоятельная вычислительная машина или система, отказ одного из узлов не приводит к потере работоспособности кластера. Во многих системах отказоустойчивость автоматически поддерживается программным обеспечением.

И наконец, кластерные системы обеспечивают недостижимое для суперкомпьютеров *соотношение цена/качество*. Кластеры любой производительности можно создать, используя стандартные «строительные блоки», при этом стоимость кластера будет ниже, чем оди-

ночной вычислительной машины с эквивалентной вычислительной мощностью.

Таким образом, на аппаратном уровне *кластер* — совокупность независимых вычислительных систем, объединенных сетью. Решения могут быть простыми, основывающимися на аппаратуре Ethernet, или сложными с высокоскоростными сетями с пропускной способностью в сотни мегабайтов в секунду.

Неотъемлемая часть кластера — специализированное ПО, на которое возлагается задача поддержания вычислений при отказе одного или нескольких узлов. Такое ПО производит перераспределение вычислительной нагрузки при отказе одного или нескольких узлов кластера, а также восстановление вычислений при сбое в узле. Кроме того, при наличии в кластере совместно используемых дисков кластерное ПО поддерживает единую файловую систему.

Классификация архитектур кластерных систем

В литературе описываются различные способы классификации кластерных систем. Простейшая классификация основана на способе использования дисковых массивов: совместно либо раздельно.

На рис. 5.5.1 и 5.5.2 приведены структуры кластеров из двух узлов, координация работы которых обеспечивается высокоскоростной линией, используемой для обмена сообщениями. Это может быть локальная сеть, применяемая также и не входящими в кластер компьютерами, либо выделенная линия. В случае выделенной линии один или несколько узлов кластера будут иметь выход на локальную или глобальную сеть, благодаря чему обеспечивается связь между серверным кластером и удаленными клиентскими системами.

Различие между представленными кластерами заключается в том, что в случае локальной сети узлы используют локальные дисковые массивы, а в случае выделенной линии узлы совместно используют один избыточный массив независимых жестких дисков или так называемый RAID (Redundant Array of Independent Disks). RAID состоит из нескольких дисков, управляемых контроллером, взаимосвязанных скоростными каналами и воспринимаемых внешней системой как единое целое. В зависимости от типа используемого массива могут обеспечиваться различные степени отказоустойчивости и быстродействия.

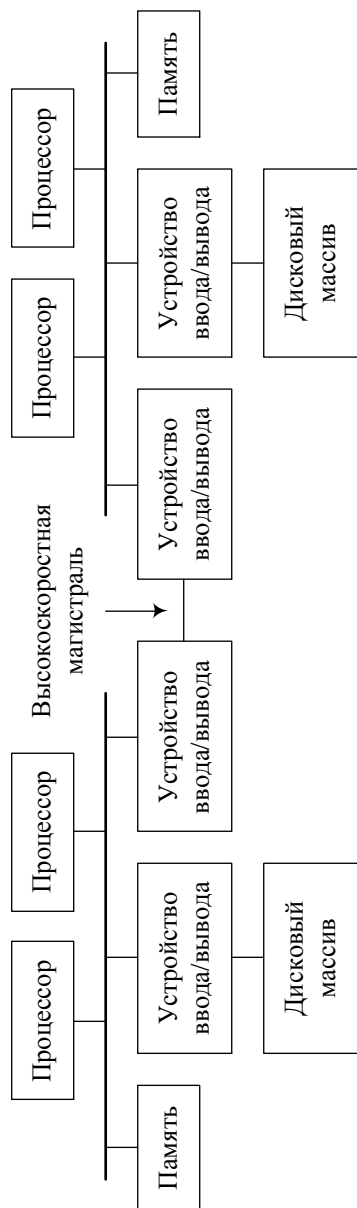


Рис. 5.5.1. Конфигурация кластера без совместно используемых дисков

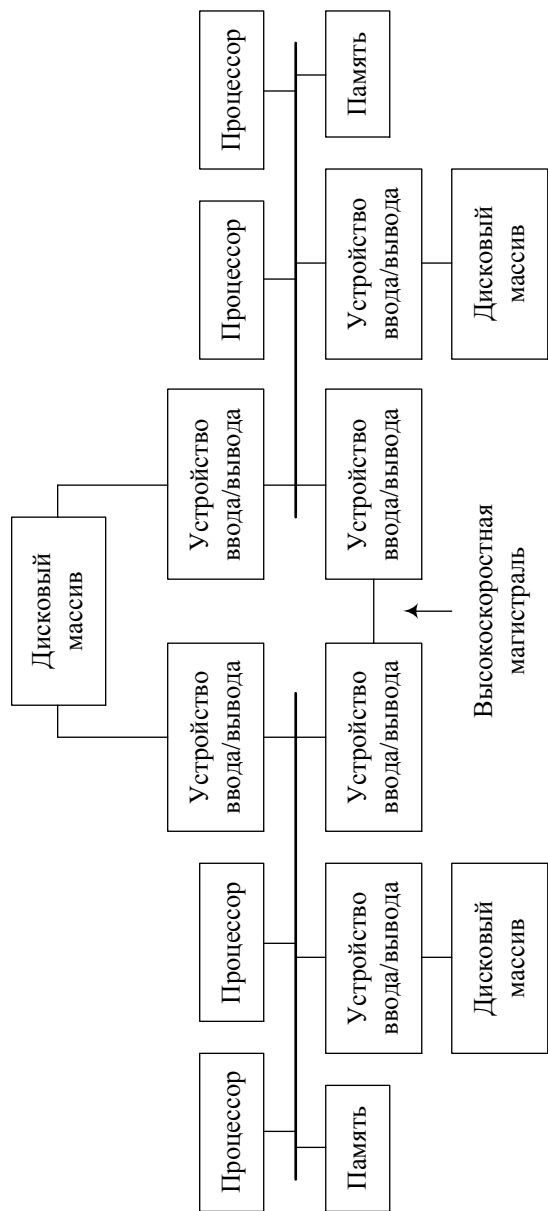


Рис. 5.5.2. Конфигурация кластера с совместно используемыми дисками

Рассмотрим наиболее распространенные типы дисковых массивов.

RAID0 (striping — чередование) — дисковый массив из двух или более жестких дисков с отсутствием резервирования. Информация разбивается на блоки данных и записывается на оба (несколько) дисков одновременно. Достоинство — существенное повышение производительности. Недостаток — надежность RAID0 заведомо ниже надежности любого из дисков в отдельности и снижается с увеличением количества входящих в RAID0 дисков, так как отказ любого из дисков приводит к неработоспособности всего массива.

RAID1 (mirroring — зеркалирование) — массив, состоящий как минимум из двух дисков. Достоинствами являются приемлемая скорость записи и выигрыш в скорости чтения при распараллеливании запросов, а также высокая надежность: работает до тех пор, пока функционирует хотя бы один диск в массиве. Вероятность выхода из строя сразу двух дисков равна произведению вероятностей отказа каждого диска. На практике при выходе из строя одного из дисков следует срочно принимать меры: вновь восстанавливать избыточность. Для этого с любым уровнем RAID (кроме нулевого) рекомендуют использовать диски горячего резерва. Достоинство такого подхода — поддержание постоянной доступности. Недостаток заключается в том, что приходится оплачивать стоимость двух жестких дисков, получая полезный объем одного жесткого диска.

RAID10 — зеркалированный массив, данные в котором записываются последовательно на несколько дисков, как в RAID0. Эта архитектура представляет собой массив типа RAID0, сегментами которого вместо отдельных дисков являются массивы RAID1. Соответственно, массив этого уровня должен содержать как минимум четыре диска. RAID10 сочетает высокую отказоустойчивость и производительность.

Более полное представление о кластерных вычислительных системах дает классификация кластеров по используемым методам кластеризации, которые определяют основные функциональные особенности системы:

- кластеризация с пассивным резервированием;
- кластеризация с активным резервированием;
- самостоятельные серверы;
- серверы с подключением ко всем дискам;
- серверы с совместно используемыми дисками.

Кластеризация с резервированием — наиболее старый и универсальный метод. Один из серверов берет на себя всю вычислительную нагрузку, в то время как другой остается неактивным, но готовым принять вычисления при отказе основного сервера. Активный (или первичный) сервер периодически посылает резервному (вторичному) серверу тактирующее сообщение. При отсутствии тактирующих сообщений, что рассматривается как отказ первичного сервера, вторичный сервер берет управление на себя. Такой подход повышает коэффициент готовности, но не улучшает производительности. Более того, если единственный вид общения между узлами — обмен сообщениями, и если оба сервера кластера не используют диски совместно, то резервный сервер не имеет доступа к базам данных, управляемым первичным сервером.

Пассивное резервирование для кластеров нехарактерно. Термин «кластер» относят ко множеству взаимосвязанных узлов, активно участвующих в вычислительном процессе и совместно создающих иллюзию одной мощной вычислительной машины. К такой конфигурации обычно применяют понятие системы с активным вторичным сервером, и здесь выделяют три метода кластеризации: самостоятельные серверы, серверы без совместного использования дисков и серверы с совместным использованием дисков.

В первом методе каждый узел кластера рассматривается как самостоятельный сервер с собственными дисками, причем ни один из дисков в системе не является совместно используемым. Схема обеспечивает высокую производительность и высокий коэффициент готовности, однако требует специального ПО для планирования распределения запросов клиентов по серверам так, чтобы добиться сбалансированного и эффективного использования всех серверов. Необходимо, чтобы при отказе одного из узлов в процессе выполнения какого-либо приложения другой узел кластера мог перехватить и завершить это приложение. Для этого данные в системе должны постоянно копироваться, чтобы каждый сервер имел доступ ко всем наиболее свежим данным в системе. Из-за этих издержек высокий коэффициент готовности обеспечивается лишь за счет потери производительности.

Для сокращения коммуникационных издержек большинство кластеров в настоящее время состоят из серверов, подключенных к общим дискам, обычно представленных дисковым массивом RAID (см. рис. 5.5.2).

Один из вариантов такого подхода предполагает, что совместное использование дисков не применяется. Общие диски разбиваются на разделы, и каждому узлу кластера выделяется свой раздел. Если один из узлов отказывает, кластер может быть реконфигурирован так, что права доступа к его разделу общего диска передаются другому узлу.

При другом варианте множество серверов разделяют во времени доступ к общим дискам, так что любой узел имеет доступ ко всем разделам всех общих дисков. Такой подход требует наличия каких-либо средств блокировки, гарантирующих, что в любой момент времени доступ к данным будет иметь только один из серверов.

Кластеры обеспечивают высокий уровень доступности — в них отсутствуют единая операционная система и совместно используемая память, т. е. нет проблемы когерентности кэш-памяти. Кроме того, специальное ПО в каждом узле постоянно контролирует работоспособность всех остальных узлов. Этот контроль основан на периодической рассылке каждым узлом сигнала «Я еще бодрствую». Если сигнал от некоторого узла не поступает, то такой узел считается вышедшим из строя; ему не предоставляется возможность выполнять ввод/вывод, его диски и другие ресурсы (включая сетевые адреса) переназначаются другим узлам, а выполнявшиеся в нем программы перезапускаются в других узлах.

Производительность кластеров хорошо масштабируется при добавлении узлов. В кластере может выполняться несколько отдельных приложений, но для масштабирования отдельного приложения требуется, чтобы его части взаимодействовали путем обмена сообщениями. Однако нельзя не учитывать, что взаимодействия между узлами кластера занимают гораздо больше времени, чем в традиционных вычислительных системах. Возможность практически неограниченного наращивания числа узлов и отсутствие единой операционной системы делают кластерные архитектуры исключительно хорошо масштабируемыми. Успешно используются системы с сотнями и тысячами узлов.

При разработке кластеров можно выделить два подхода. Первый подход состоит в создании небольших кластерных систем. В кластер объединяются полнофункциональные компьютеры, которые продолжают работать как самостоятельные единицы, например, компьютеры учебного класса или рабочие станции лаборатории. Второй подход заключается в целенаправленном создании мощных вычислительных ресурсов. Системные блоки компьютеров компактно размещают-

ся в специальных стойках, а для управления системой и запуска задач выделяется один или несколько полнофункциональных компьютеров, называемых хост-компьютерами. В этом случае нет необходимости снабжать компьютеры вычислительных узлов графическими картами, мониторами, дисковыми накопителями и другим периферийным оборудованием, что значительно удешевляет стоимость системы.

Разработано множество технологий объединения компьютеров в кластер. Наиболее широко в данное время применяется технология Ethernet, что обусловлено простотой ее использования и низкой стоимостью коммуникационного оборудования. Однако за это приходится расплачиваться заведомо недостаточной скоростью обменов.

Разработчики пакета подпрограмм ScaLAPACK, предназначенного для решения задач линейной алгебры на многопроцессорных системах, в которых велика доля коммуникационных операций, формулируют требование к многопроцессорной системе следующим образом: «Скорость межпроцессорных обменов между двумя узлами, измеренная в МБ/с, должна быть не менее 1/10 пиковой производительности вычислительного узла, измеренной в MFLOPS».

Топологии кластеров

Рассмотрим топологии, характерные для так называемых «малых» кластеров, состоящих из двух — четырех узлов.

Топология кластерных пар используется при организации двух- или четырехузловых кластеров (рис. 5.5.3). Узлы группируются попарно, дисковые массивы присоединяются к обоим узлам, входящим в состав пары, причем каждый узел пары имеет доступ ко всем дисковым массивам данной пары. Один из узлов пары используется как резервный для другого.

Четырехузловая кластерная пара представляет собой простое расширение двухузловой топологии. Обе кластерные пары с точки зрения администрирования и настройки рассматриваются как единое целое.

Данная топология может быть применена для организации кластеров с высокой готовностью данных, но отказоустойчивость реализуется только в пределах пары, так как принадлежащие паре устройства хранения информации не имеют физического соединения с другой парой.

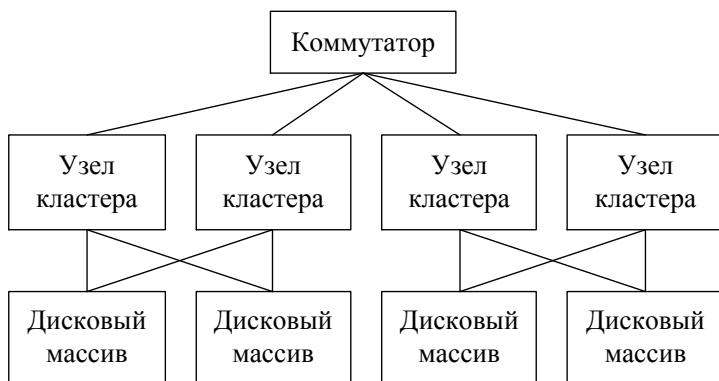


Рис. 5.5.3. Топология кластерных пар

Топология $N + 1$ позволяет создавать кластеры из двух, трех и четырех узлов (рис. 5.5.4). Каждый дисковый массив подключается только к двум узлам кластера. Дисковые массивы организованы по схеме RAID1 (mirroring). Один сервер имеет соединение со всеми дисковыми массивами и служит в качестве резервного для всех остальных (основных или активных) узлов. Резервный сервер может использоваться для обеспечения высокой степени готовности в паре с любым из активных узлов.

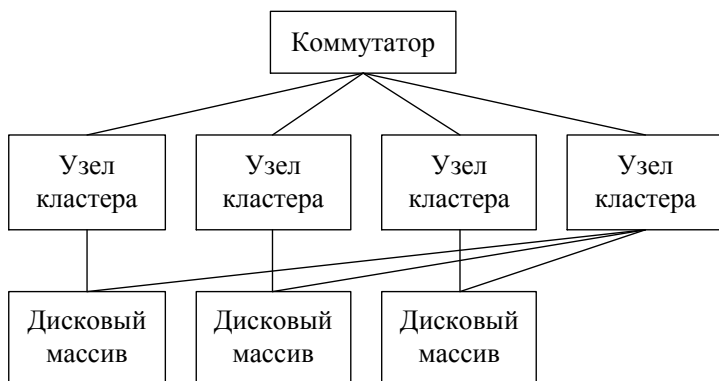


Рис. 5.5.4. Топология $N + 1$

Топология рекомендуется для организации кластеров с высокой готовностью данных. В тех конфигурациях, где имеется возможность выделить один узел для резервирования, эта топология позволяет уменьшить нагрузку на активные узлы и гарантировать, что нагрузка вышедшего из строя узла будет воспроизведена на резервном узле без потери производительности. Отказоустойчивость обеспечивается между любым из основных узлов и резервным узлом. В то же время топология не позволяет реализовать глобальную отказоустойчивость, поскольку основные узлы кластера и их системы хранения информации не связаны друг с другом.

Топология $N \times N$ аналогично топологии $N + 1$ позволяет создавать кластеры из двух, трех и четырех узлов, но в отличие от нее обладает большей гибкостью и масштабируемостью (рис. 5.5.5).

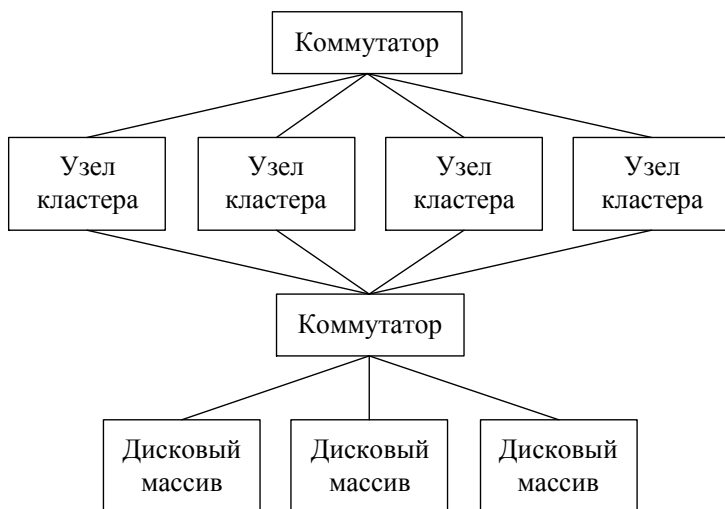


Рис. 5.5.5. Топология $N \times N$

Только в этой топологии все узлы кластера имеют доступ ко всем дисковым массивам, которые, в свою очередь, строятся по схеме RAID1 (mirroring). Масштабируемость топологии проявляется в простоте добавления к кластеру дополнительных узлов и дисковых массивов без изменения соединений в системе.

Топология позволяет организовать каскадную систему отказоустойчивости, при которой обработка переносится с неисправного узла на резервный, а в случае его выхода из строя на следующий резервный узел и т. д. В целом топология обладает лучшей отказоустойчивостью и гибкостью по сравнению с другими топологиями.



Рис. 5.5.6. Топология с полностью раздельным доступом

Топология с полностью раздельным доступом допускает соединение каждого дискового массива только с одним узлом кластера (рис. 5.5.6). Рекомендуется лишь для тех приложений, для которых характерна архитектура полностью раздельного доступа.

Контрольные вопросы

1. Дайте определение кластерной вычислительной системы.
2. Назовите основные достоинства и недостатки кластерных вычислительных систем.
3. Какие классификации кластерных вычислительных систем вы знаете?
4. Какие топологии кластерных систем вам известны? Назовите их достоинства и недостатки.

Литература

1. Архитектуры и топологии многопроцессорных вычислительных систем / ***А.В. Богданов, В.В. Корхов, В.В. Мареев, Е.Н. Станкова.*** — М.: ИНТУИТ.РУ, 2004. — 176 с.

2. Микропроцессорные системы: учеб. пособие для вузов / ***Е.К. Александров, Р.И. Грушвицкий, М.С. Куприянов и др.; под ред. Д.В. Пузанкова.*** — СПб.: Политехника, 2002. — 935 с.

Лекция 5.6. Реконфигурируемые и систолические вычислительные системы. Архитектура систем, управляемых потоками данных

Вычислительные системы с реконфигурируемой структурой. Однородные вычислительные системы

Вычислительные системы с реконфигурируемой структурой строятся на основе программируемых логических интегральных схем (ПЛИС), содержащих одну или несколько матриц вентиля и позволяющих программно скомпоновать из этих вентиля в одном корпусе электронную схему, эквивалентную схеме, включающей от нескольких десятков до сотен тысяч интегральных схем стандартной логики.

Основная идея подобных вычислительных систем состоит в том, чтобы программно настроить схему, реализующую требуемое преобразование данных. Достоинство такого подхода можно оценить на примере сравнения реализации цифрового фильтра с помощью процессора Alpha 21164 и ПЛИС Xilinx XC4085XL-09. Микропроцессор Alpha 21164 выполняет две 64-разрядных операции за такт при тактовой частоте 433 МГц, что эквивалентно производительности 55,7 бит/нс. ПЛИС XC4085XL-09 имеет 3136 программируемых логических блоков и минимальную длительность такта 4,6 нс. Для корректного сравнения можно принять, что один программируемый логический блок реализует одноканальную арифметическую операцию. При этом производительность ПЛИС равна 682 бит/нс. Таким образом, при реализации фильтра на ПЛИС достигается производительность, в 12 раз большая, чем на микропроцессоре Alpha 21164.

Отметим, что в настоящее время появились более совершенные и производительные процессоры цифровой обработки, в то же время увеличились максимальная тактовая частота и количество логических вентиля ПЛИС, поэтому приведенный пример по-прежнему является показательным.

Основная трудность в использовании реконфигурируемых вычислений заключается в подготовке для заданного алгоритма настроечной информации для создания в ПЛИС схемы, реализующей этот ал-

горитм. Поскольку ПЛИС разрабатывались для построения электронной аппаратуры, то изготовители в первую очередь обеспечили системы подготовки настроечной информации на высокоуровневых языках электронного проектирования, таких как Verilog HDL и VHDL. Однако эти языки непривычны для разработчиков алгоритмов.

Компания Star Bridge Systems производит семейство программно-реконфигурируемых вычислителей Hypercomputer System HC-X и предлагает комплексное решение для организации реконфигурируемых вычислений (рис. 5.6.1). Старшая модель семейства — суперкомпьютер HC-98m — состоит из управляющего компьютера и двухплатного программно-реконфигурируемого вычислителя, включающего 14 ПЛИС Virtex-II серии 6000 и 4 ПЛИС Virtex-II серии 4000, что в совокупности составляет 98 млн вентилей. После включения питания первая ПЛИС программируется из постоянного запоминающего устройства на плате. В этой ПЛИС формируется порт шины PCI-X, через который управляющий компьютер, используя среду разработки программ Viva, будет программировать остальные ПЛИС.

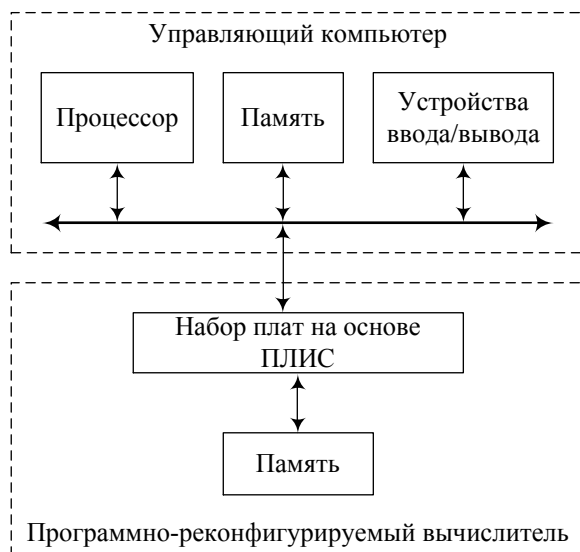


Рис. 5.6.1. Структура универсальной вычислительной системы на основе ПЛИС

Обладая способностью реализовать любую цифровую схему, ПЛИС имеют по сравнению с жесткой логикой большую площадь в пересчете на используемый вентиль. Одним из решений этой проблемы, позволяющим сохранить программируемость при уменьшении затрат оборудования по сравнению с ПЛИС, является создание однородных вычислительных сред из одного типа ячеек с учетом проблемной ориентации некоторой специфичной ячейки при ее реализации на жесткой логике и программируемости связей между множеством таких ячеек.

Идея однородных вычислительных сред была сформулирована в начале 1960-х годов сотрудником Института математики СО АН СССР Э.В. Евреиновым. Была показана возможность снятия ограничения на рост тактовой частоты в однородных средах за счет коротких связей между соседними ячейками среды и реализации взаимодействия между удаленными ячейками по принципу близкодействия через цепочку ячеек, расположенных между ними. В основе предложенного подхода к построению вычислительных системы лежат три принципа:

- параллельность операций;
- переменность логической структуры;
- конструктивная однородность элементов и связей между ними.

Первый принцип базируется на аксиоме параллельности задач и алгоритмов: всякая сложная задача может быть представлена в виде связанных между собой простых подзадач и для любой сложной задачи может быть предложен параллельный алгоритм, допускающий ее эффективное решение. Аксиома параллельности, таким образом, обеспечивает достижение высокой производительности за счет параллельной работы большого числа обрабатывающих устройств или элементов.

Второй принцип основан на аксиоме переменности логической структуры: процесс решения сложной задачи может быть представлен некоторой структурной моделью, включающей в себя подзадачи и связи между ними. Это означает, что для каждой сложной задачи можно предложить соответствующую структуру из обрабатывающих элементов, связанных между собой определенным образом.

Третий принцип базируется на аксиоме конструктивной однородности элементов и связей: все простые задачи получаются путем деления сложной задачи на части, а поэтому все эти простые задачи

примерно одинаковы по объему вычислений и связаны между собой одинаковыми схемами обмена. Это означает, что система для решения сложной задачи может быть построена из одинаковых обрабатывающих элементов, связанных между собой одинаковым образом.

Таким образом, при соблюдении трех принципов вычислительная система может быть представлена как совокупность неограниченного числа одинаковых обрабатывающих устройств, однотипно связанных между собой. Эта совокупность перед решением задачи настраивается соответствующим образом. Такая однородная вычислительная система должна решать задачи неограниченной сложности и объема при высокой надежности и готовности, что обеспечивается избыточностью обрабатывающих устройств, унифицированностью связей между ними и легкостью перестройки системы.

В рассмотренных ранее комплексах и системах в той или иной степени используются указанные принципы: почти везде возможна параллельная обработка задач; во многих случаях возможна реконфигурация и иногда прямая настройка системы на решение определенной задачи; наблюдается стремление унифицировать обрабатывающие средства, т. е. создавать по возможности однородные системы.

Описываемый подход к разработке вычислительных систем в виде однородных перестраиваемых структур отличается тем, что используются все три принципа, что, по мнению автора подхода, и должно обеспечить максимальный эффект.

Хотя эти идеи были выдвинуты более сорока лет назад, достаточно активная их реализация началась сравнительно недавно. Это и понятно, так как создание однородных систем из большого числа ЭВМ или процессоров второго и даже третьего поколения было вовсе не простой задачей: слишком громоздкими получались такие системы. Ясно, что эффективная реализация возможна лишь при весьма эффективной элементной базе. С развитием микропроцессоров и микроЭВМ на их основе такая база создана, и стала возможной реализация этих идей.

Отметим, что реализация однородных систем — непростая проблема, а эффективность таких систем не является безусловной. Необходимо учитывать, что аппаратные затраты на реализацию отдельных связей между модулями достаточно велики, причем чем элементарнее функции вычислительного модуля, тем больший удельный вес приобретают затраты на связи.

В силу возможности различного направления специализации ячеек и установления различной пропорции между реализацией на жесткой логике и программируемостью сегодня предложено много вариантов построения специализированных систем с программируемой структурой. К этому классу вычислительных систем относятся также систолические и волновые процессоры, программируемые (raw) процессоры, ассоциативные процессоры с SIMD-архитектурой, примером которых может служить проект CAM2000 и др.

Систолические вычислительные системы

Систолические вычислительные системы сочетают достоинства конвейерной и матричной обработок информации. Несмотря на относительную дороговизну таких систем, обусловленную применением специализированных аппаратных решений, систолы являются одним из самых эффективных методов решения специализированных задач, так как часто обеспечивают на конкретной задаче быстроедействие, недостижимое для большинства микропроцессорных архитектур. Отметим, что удешевление ПЛИС сводит на нет экономическую составляющую вопроса и пробуждает интерес исследователей к применению систол в сложных вычислительных задачах.

Авторы идеи систолической матрицы Кунг и Лейзерсон предложили организовать вычисления таким образом, чтобы данные, считанные из памяти, прошли как можно больше этапов обработки до попадания обратно в память. Систолические системы являются специализированными вычислителями и производятся в основном под конкретную задачу. Фактически задача построения систолического вычислителя сводится к построению аппаратного конвейера, имеющего достаточно большое время получения результата, но при этом сравнительно маленькое время между последовательной выдачей результатов, так как значительное количество промежуточных значений обрабатывается на разных ступенях конвейера.

Происхождение названия систолических вычислительных систем можно объяснить, сравнив их устройство со структурой живого организма. Систола — сокращение предсердий и желудочков сердца, при котором кровь нагнетается в артерии, т.е. вычислительные систолы напоминают систему сосудов и сердце, которое постоянно посылает кровь во все артерии, сосуды и капилляры тела.

Таким образом, *систолическая структура* — это однородная вычислительная среда из процессорных элементов, сочетающая свойства конвейерной и матричной обработки и обладающая следующими особенностями:

- вычислительный процесс представляет собой непрерывную и регулярную передачу данных от одного ПЭ к другому без сохранения промежуточных результатов вычислений;
- каждый элемент входных данных выбирается из памяти однократно и используется столько раз, сколько необходимо по алгоритму, ввод данных осуществляется в крайние элементы матрицы;
- ПЭ однотипны и каждый из них может быть менее универсальным, чем процессоры обычных многопроцессорных систем;
- потоки данных и управляющих сигналов обладают регулярностью, что позволяет объединять ПЭ локальными связями минимальной длины;
- производительность системы можно линейно увеличивать добавлением в матрицу определенного количества ПЭ.

По степени гибкости систолические структуры можно разделить:

- на специализированные;
- алгоритмически ориентированные;
- программируемые.

Специализированные структуры предназначены для выполнения определенного алгоритма, что выражается не только в конкретной геометрии структуры, количестве ПЭ и статичности связей между ними, но и в выборе типов операций, выполняемых всеми ПЭ. Примерами таких структур являются системы, ориентированные на реализацию алгоритмов фильтрации, быстрого преобразования Фурье и т. п.

Алгоритмически ориентированные структуры позволяют выполнять некоторое множество алгоритмов, сводимых к однотипным операциям над векторами, матрицами и другими числовыми множествами. Это достигается за счет возможности программирования либо конфигурации связей в систолической матрице, либо самих ПЭ.

Программируемые систолические структуры имеют возможность программирования как самих ПЭ, так и связей между ними. При этом ПЭ могут иметь локальную память программ и выполнять в один и тот же момент времени различные операции из некоторого набора.

На рис. 5.6.2 представлена структура систолической системы, вычисляющей свертку во временной области, которая определяется выражением

$$y_i = \sum_{j=1}^n h_j \cdot x_{i-j},$$

где x_i , y_i — отсчеты входного и выходного сигналов соответственно; h_j — отсчеты опорной функции длины n . На каждом такте работы схемы входные данные сдвигаются вправо, умножаются на отсчеты опорной функции и аккумулируются на выходе в порядке следования.

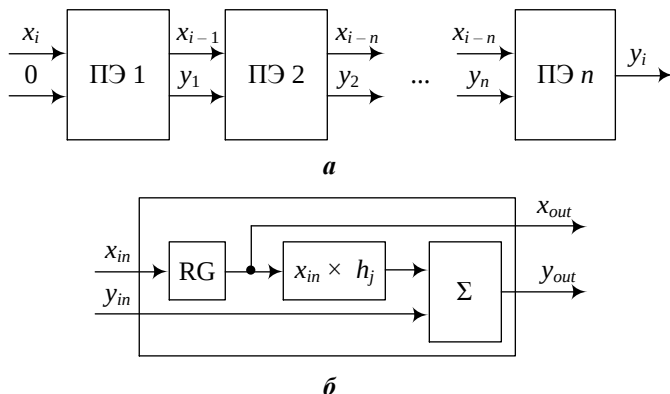


Рис. 5.6.2. Структура систолической системы, вычисляющей свертку во временной области (а); структура систолы (б)

Перечислим основные достоинства систолических вычислительных систем:

- минимизированы обращения к памяти, что позволяет согласовать скорость работы памяти со скоростью обработки;
- упрощено решение проблем ввода/вывода, поскольку сокращено количество конфликтов при обращении к памяти;
- эффективно используются технологические возможности СБИС за счет регулярности структуры.

Архитектура систем, управляемых потоками данных

В традиционных вычислительных машинах команды выполняются последовательно, после того как выполнена предшествующая команда. Последовательность выполнения команд определяется значением счетчика команд. Кроме этого широко распространенного способа управления известны еще два: управление потоками данных и управление по запросу. При управлении потоком данных команда выполняется, когда становятся доступны ее операнды, а при управлении по запросу команда выполняется, когда другим командам требуется результат ее выполнения. Рассмотрим организацию машин, управляемых потоком данных, подробнее.

Впервые идея вычислений, управляемых потоком данных, была предложена в 1960-х годах Карпом и Миллером. Основоположителем архитектуры, языка программирования и практической реализации машин, управляемых потоком данных, является Деннис. Его работы, выполненные в 1970-е годы, положили начало многим исследованиям в данном направлении.

Программа машины, управляемой потоком данных, представляет собой направленный граф. Этот граф состоит из вершин, отображающих операции, и ребер или дуг, по которым перемещаются данные. Узловые операции выполняются, когда по дугам в узел поступила вся необходимая информация. Обычно требуется один или два операнда, а для условных операций — один логический управляющий сигнал. По выполнению операции узел формирует один или два результата. Результаты передаются по дугам от узла к узлу до тех пор, пока не будет получен окончательный результат. Заметим, что одновременно могут быть инициированы несколько узлов, таким образом автоматически реализуется параллелизм вычислений.

На рис. 5.6.3 представлены вершины с дугами на различных стадиях выполнения операций. Наличие данных на дуге обозначают малым кругом, который называют *токеном*. Считается, что команда готова к выполнению, если данные присутствуют в каждом из входных портов и отсутствуют в выходном.

Поскольку практические вычисления требуют от узлов различных дополнительных возможностей, помимо непосредственно выполнения арифметических и логических операций, в потоковых графах предусмотрены узлы нескольких типов (рис. 5.6.4):

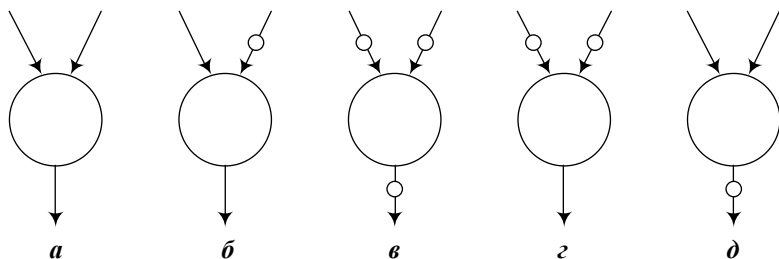


Рис. 5.6.3. Вершины на различных стадиях выполнения операции: команда не готова к выполнению (*а–в*); команда готова к выполнению (*г*); команда выполнена (*д*)

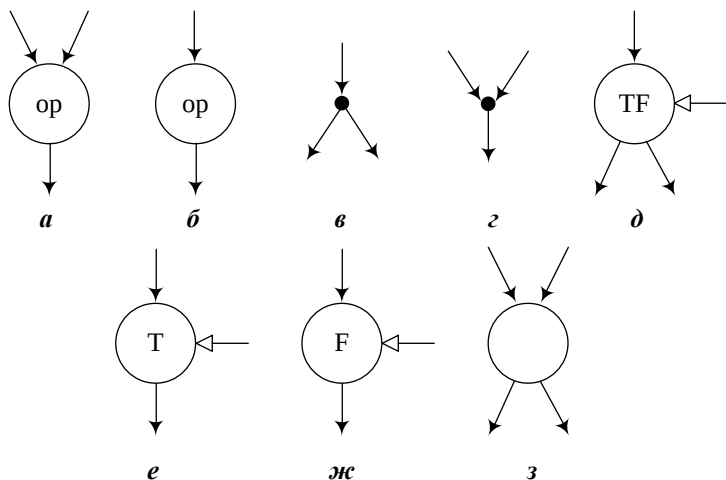


Рис. 5.6.4. Типовые вершины потоковых графов: двухвходовая операционная вершина (*а*); одновходовая операционная вершина (*б*); вершина ветвления (*в*); вершина слияния (*г*); TF-коммутатор (*д*); Т-коммутатор (*е*); F-коммутатор (*ж*); арбитр (*з*)

- двухвходовая операционная вершина — узел с двумя входами и одним выходом;
- одновходовая операционная вершина — узел с одним входом и одним выходом;
- вершина ветвления — узел с одним входом и двумя выходами,

осуществляет копирование входных данных и вывод их через две выходные дуги;

- вершина слияния — узел с двумя входами и одним выходом, данные поступают только по одной из входных дуг и без изменений подаются на выход;

- ТФ-коммутатор — узел с двумя входами и двумя выходами, верхний вход — дуга данных, правый — дуга управления; если значение правого входа истинно, то выходные данные выводятся через левый выход, а при ложном значении — через правый;

- арбитр — узел с двумя входами и двумя выходами, все дуги являются дугами данных, первые поступившие от двух входов данные следуют через левую дугу, а прибывшие позже — через правую.

В качестве примера рассмотрим решение квадратного уравнения

$$ax^2 + bx + c = 0, \quad \text{где } a \neq 0.$$

Количество корней определяется значением дискриминанта $D = b^2 - 4ac$: при $D > 0$ корней два, при $D = 0$ корень один, при $D < 0$ корней нет. Составим потоковый граф для $D > 0$, т.е. необходимо вычислить выражение

$$x_{1,2} = \frac{-b \pm \sqrt{D}}{2a}.$$

На рис. 5.6.5 горизонтальными штрихпунктирными линиями выделены группы операций, допускающих параллельное выполнение. Таким образом реализуется автоматический параллелизм в потоковых вычислениях.

На рис. 5.6.6 показан пример организации вычислений в цикле. Идея реализации циклического процесса аналогична типовым последовательным языкам программирования. Имеется переменная цикла i , которая увеличивается после каждого прохождения тела цикла, а промежуточные результаты подаются на входные узлы. Когда переменная цикла достигает определенного значения, циклический процесс завершается, и на выход подается результат вычислений.

Известные потоковые вычислительные системы могут быть отнесены к двум основным типам: статические и динамические системы. Статические системы работают по принципу «один токен на дугу», за счет чего достигается относительная простота реализации блока

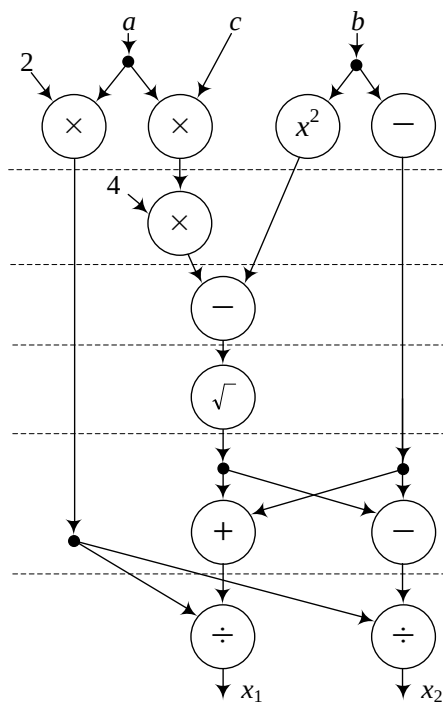


Рис. 5.6.5. Поточковый граф для решения квадратного уравнения

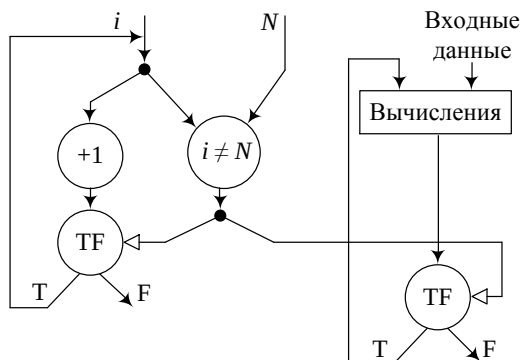


Рис. 5.6.6. Организация циклов при потоковой обработке

обнаружения активированных узлов. Преимуществом динамических потоковых вычислительных систем перед статическими системами является более высокая производительность, достигаемая за счет присутствия на дуге множества токенов. Трудностью в реализации данного подхода является эффективная реализация блока, собирающего на выполнение токены с одинаковыми тегами. Причем, как правило, количество токенов с совпадающими тегами достаточно велико.

Рассмотрим подробнее архитектуру статической потоковой вычислительной системы, предложенную Деннисом (рис. 5.6.7).

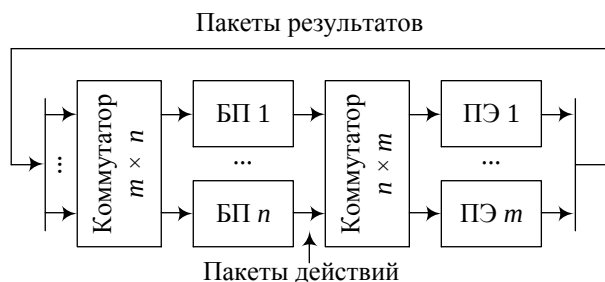


Рис. 5.6.7. Архитектура статической потоковой вычислительной системы, предложенная Деннисом

Система представляет собой кольцо ПЭ и блоков памяти, в котором информация передается в форме пакетов. ПЭ получают так называемые пакеты действий, содержащие код операции, операнды и информацию об адресате. В полях операндов непосредственно передаются числа, подлежащие обработке. Недостатком такого подхода является то, что в качестве операндов невозможно использовать структуры данных, даже такие простые, как векторы и массивы.

Пакеты результата содержат значение и информацию об адресате. Эти пакеты через коммутатор передаются в так называемые ячейки команды блока памяти. Когда получены все входные токены, ячейка порождает пакет операций, который через второй коммутатор поступает на вход одного из процессорных элементов. Если все ПЭ идентичны, то система считается гомогенной, и может быть выбран любой свободный ПЭ. В случае гетерогенной системы, состоящей из множества специализированных ПЭ, выбор нужного ПЭ производится по коду операции.

Память в вычислительных системах, управляемых потоком данных, является активной: непрерывно выполняется проверка, находится ли вся необходимая информация в ячейке команд, и в случае положительного результата память передает содержимое данной ячейки на вход коммутатора для выполнения операции.

Контрольные вопросы

1. Охарактеризуйте вычислительные системы с перестраиваемой структурой.
2. За счет чего разработчики вычислительных систем с перестраиваемой структурой компенсируют их аппаратную избыточность?
3. Перечислите особенности систолических вычислительных систем.
4. Каковы основные отличия машин, управляемых потоком данных, от традиционной машины класса SISD?
5. Чем определяется число параллельно выполняемых операций в машине, управляемой потоком данных?

Литература

1. **DeHon A.** The Density Advantage of Configurable Computing // Computer. — 2000. — N. 4.
2. **Евреинов Э.В., Косарев Ю.Г.** Однородные универсальные вычислительные системы высокой производительности. — Новосибирск: Наука, 1966.
3. **Богачев М.П.** Архитектура вычислительной системы с однородной структурой. — Львов: ФМИ АН УССР, 1981.
4. **Седов В.С.** Матрица одноразрядных процессоров. — Львов: НТЦ «Интеграл», 1991. — 47 с.
5. **Durbeck L., Macias N.** The Cell Matrix: An Architecture for Nanocomputing. — <http://www.cellmatrix.com>.
6. The Raw Microprocessor: A Computational Fabric for Software Circuits and General-Purpose Programs / **M. Taylor, J. Kim, J. Miller et al.** // IEEE Micro. — 2002. — Vol. 22, N. 2.
7. **Smith D., Hall J., Miyake K.** The CAM2000 Chip Architecture. — Rutgers University. — <http://www.cs.rutgers.edu/pub/technical-reports>.

8. **Кун С.** Матричные процессоры на СБИС. — М.: Мир, 1991. — 672 с.

9. Архитектуры и топологии многопроцессорных вычислительных систем / **А.В. Богданов, В.В. Корхов, В.В. Мареев, Е.Н. Станкова.** — М.: ИНТУИТ.РУ, 2004. — 176 с.

10. **Бойко В.И.** Схемотехника электронных систем. Микропроцессоры и микроконтроллеры. — СПб.: БХВ-Петербург, 2004. — 464 с.

11. Микропроцессорные системы: учеб. пособие для вузов / **Е.К. Александров, Р.И. Грушвицкий, М.С. Куприянов и др.; под ред. Д.В. Пузанкова.** — СПб.: Политехника, 2002. — 935 с.

Содержание

Список используемых аббревиатур	3
Предисловие	5
Раздел 1. Микропроцессор: ключевые понятия, классификация, структура, операционные устройства	7
Лекция 1.1. Микропроцессор: основные определения, классификация, закономерности развития, области применения.	7
Лекция 1.2. Обобщенная структура микропроцессора. Арифметико-логическое устройство — основа операционных устройств микропроцессора	17
Лекция 1.3. Организация цепей переноса в пределах секции АЛУ. Нарастивание разрядности обрабатываемых слов. Примеры использования АЛУ	25
Лекция 1.4. Структуры операционных устройств. Регистровое арифметико-логическое устройство	34
Лекция 1.5. Разрядно-модульные и однокристалльные регистровые арифметико-логические устройства	43
Раздел 2. Устройства управления. Конвейерный принцип выполнения команд. Основные режимы функционирования микропроцессора	53
Лекция 2.1. Устройство управления: структура, способы формирования управляющих сигналов и адресации микрокоманд	53
Лекция 2.2. Система команд и способы адресации операндов. Конвейерный принцип выполнения команд	63
Лекция 2.3. Структурные конфликты и конфликты по данным. Методы их минимизации	75

Лекция 2.4. Минимизация конфликтов по управлению. Сокращение потерь на выполнение команд условного перехода	87
Лекция 2.5. Режимы функционирования микропроцессорной системы: выполнение основной программы, вызов подпрограмм	98
Лекция 2.6. Обработка прерываний и исключений. Примеры построения систем прерывания	107
Раздел 3. Системы памяти. Обмен информацией в микропроцессорных системах	117
Лекция 3.1. Классификация систем памяти. Организация систем памяти в микропроцессорных системах	117
Лекция 3.2. Принципы организации кэш-памяти. Схема обнаружения и исправления ошибок	127
Лекция 3.3. Обмен информацией между микропроцессором и внешними устройствами. Арбитр магистрали	137
Лекция 3.4. Режим прямого доступа к памяти	152
Лекция 3.5. Виртуальная память. Устройство управления памятью	161
Раздел 4. Архитектуры и структуры микропроцессоров и систем на их основе	170
Лекция 4.1. Классификация архитектур современных микропроцессоров	170
Лекция 4.2. Структура современных 32-разрядных микроконтроллеров с RISC-архитектурой	180
Лекция 4.3. Процессоры цифровой обработки сигналов: принципы организации, обобщенная структура, примеры реализации	190
Лекция 4.4. Особенности построения микропроцессоров общего назначения на примере архитектуры Intel P6	200

Раздел 5. Архитектуры и структуры параллельных вычислительных систем	214
Лекция 5.1. Назначение, область применения и классификация архитектур параллельных вычислительных систем...	214
Лекция 5.2. Параллельные вычислительные системы с общей разделяемой и распределенной памятью. Обобщенная масштабируемая структура параллельных вычислительных систем	224
Лекция 5.3. Матричные вычислительные системы	235
Лекция 5.4. Векторно-конвейерные вычислительные системы	246
Лекция 5.5. Кластерные вычислительные системы	257
Лекция 5.6. Реконфигурируемые и систолические вычислительные системы. Архитектура систем, управляемых потоками данных	271

Учебное издание

Беклемишев Дмитрий Николаевич

Орлов Александр Николаевич

Переверзев Алексей Леонидович

Попов Михаил Геннадиевич

Горячев Александр Васильевич

Кононова Александра Игоревна

Микропроцессорные средства и системы

Редактор *Е.Г. Кузнецова*. Технический редактор *Л.Г. Лосякова*.

Корректор *Л.Г. Лосякова*. Верстка авторов.

Подписано в печать с оригинал-макета 12.12.2013. Формат 60х84 1/16. Печать офсетная. Бумага офсетная. Гарнитура литературная. Усл. печ. л. 16,7. Уч.-изд. л. 14,4. Тираж 350 экз. Заказ 64.

Отпечатано в типографии ИПК МИЭТ.

124498, Москва, Зеленоград, проезд 4806, д. 5, МИЭТ.